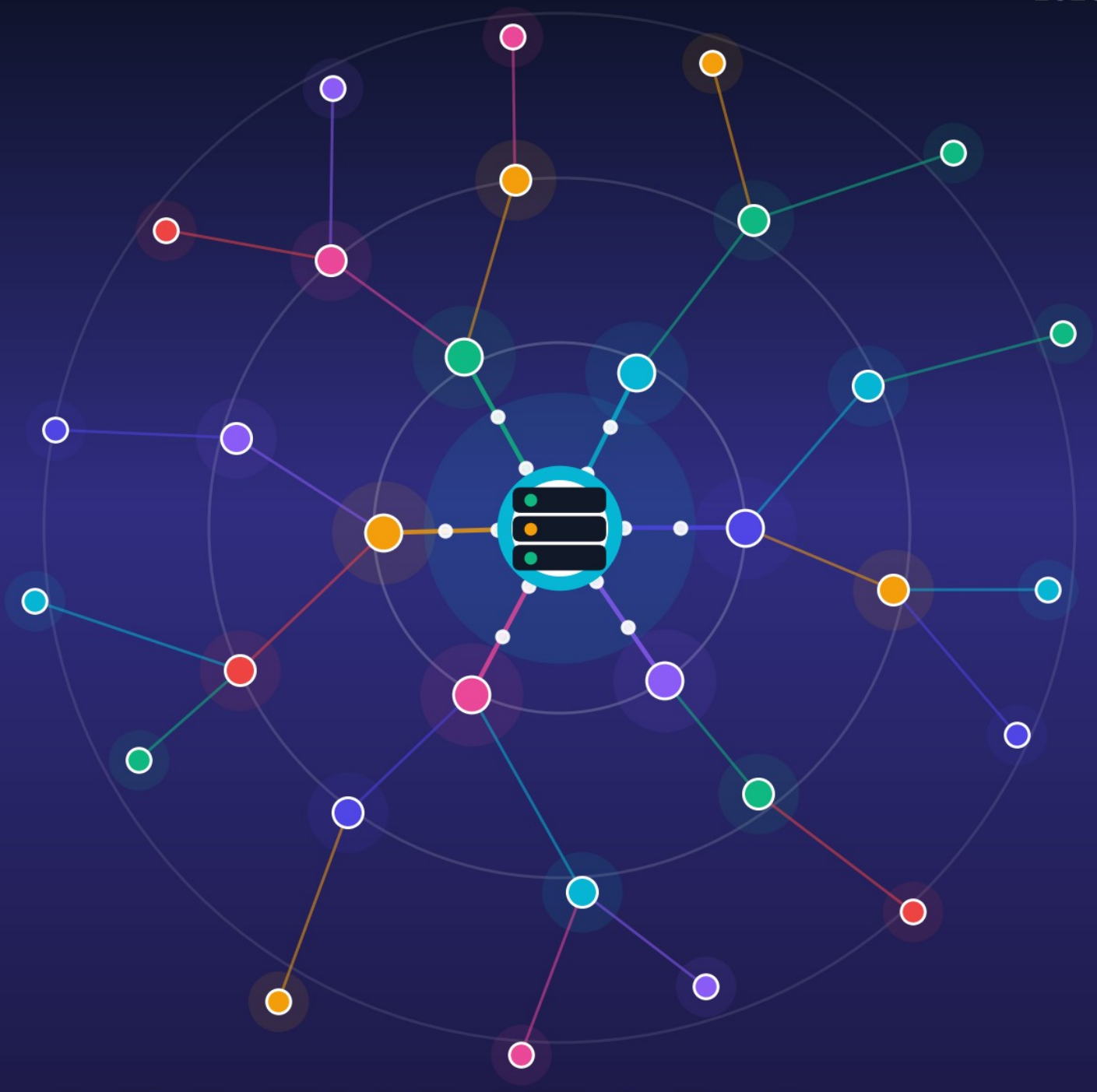
2026



APRENDIZADO FEDERADO

Fundamentos, Prática e Fronteiras

Araken de Medeiros Santos

APRENDIZADO FEDERADO

Fundamentos, Prática e Fronteiras

Araken de Medeiros Santos

2026

Sumário

Apresentação **13**
Para quem é este livro 13
Por que este tema importa 13
Como o livro está organizado 13
Como estudar com este livro 14
Caixas e símbolos do livro 14
Sobre os códigos 14
Agradecimentos e uso 15

Parte I — Fundamentos **16**

Capítulo 1 — Bem-vindo ao Aprendizado Federado **17**
1.1 Uma história para começar 17
1.2 O problema: dados espalhados e protegidos 18
1.2.1 Os dados nascem distribuídos 18
1.2.2 Por que não juntar tudo em um só lugar? 18
1.2.3 O papel das leis 18
1.3 O que é aprendizado federado 19
1.3.1 Uma definição mais precisa 19
1.4 Um pouco de história 20
1.5 Dois grandes cenários: cross-device e cross-silo 20
1.6 Onde o aprendizado federado já é usado 21
1.6.1 Saúde 21
1.6.2 Teclados e assistentes móveis 21
1.6.3 Finanças 21
1.6.4 Indústria e internet das coisas 21
1.7 Federado, distribuído, centralizado: qual a diferença? 22
1.8 Mitos e verdades 22
1.9 Vocabulário essencial 23
1.10 Prepare o ambiente 23
1.11 Sua primeira simulação federada 23
1.12 Roteiro do livro 25
1.13 Resumo do capítulo 25
Exercícios de múltipla escolha 26
Exercícios discursivos 30
Exercícios práticos em Python (scikit-learn) 31
Exercício 1 — Conhecendo os dados dos "hospitais" 31
Exercício 2 — Quanto o tamanho do hospital importa? 31
Exercício 3 — Repetindo para não se enganar com o acaso 31
Projeto integrador do capítulo 32
Rede de Clínicas: vale a pena colaborar? 32

Capítulo 2 — Preparando o Terreno: Dados, Modelos e Otimização **33**
2.1 Por que revisar isto agora? 33
2.2 Dados, modelo e erro 33
2.2.1 Os ingredientes 33
2.2.2 A função de perda 34
2.2.3 Risco empírico: a média das perdas 34
2.3 Gradiente descendente: descendo a montanha 34
2.3.1 A analogia da neblina 34
2.3.2 A taxa de aprendizado 35

2.3.3 Mini-lotes e o SGD....36
2.3.4 Um gradiente descendente do zero....36
2.4 Generalizar: o que importa é o dado novo....37
2.4.1 Sobreajuste....37
2.4.2 Treino, validação e teste....37
2.4.3 Regularização....38
2.5 Medindo a qualidade....38
2.5.1 Acurácia e seus limites....38
2.5.2 Matriz de confusão, precisão, revocação e F1....38
2.6 Os modelos usados neste livro....39
2.7 Treino que pausa e retoma: partial_fit e warm_start....39
2.8 Pré-processamento federado: a padronização....40
2.8.1 O problema....40
2.8.2 A solução: somas locais....40
2.9 Resumo do capítulo....41
Exercícios de múltipla escolha....42
Exercícios discursivos....46
Exercícios práticos em Python (scikit-learn)....47
Exercício 1 — Taxa de aprendizado em ação....47
Exercício 2 — Gradiente completo versus mini-lotes....47
Exercício 3 — Acurácia que engana....47
Exercício 4 — Padronização federada com clientes de tamanhos diferentes....47
Projeto integrador do capítulo....48
Triagem com poucos dados: do zero ao modelo....48
Capítulo 3 — Anatomia de um Sistema Federado: FedSGD e FedAvg....49
3.1 Da ideia ao algoritmo....49
3.2 Anatomia de uma rodada....49
3.2.1 Um exemplo numérico....50
3.3 Os quatro botões do FedAvg....51
3.3.1 FedSGD: o caso mais simples....51
3.3.2 FedAvg: fazer mais em casa....51
3.4 Pseudocódigo do FedAvg....52
3.5 Por que a média funciona? E quando não funciona?....52
3.6 A biblioteca fedlib.py....53
3.7 Rodando o FedAvg....55
3.8 FedSGD contra FedAvg....55
3.9 Efeito de C e E....56
3.10 O custo de comunicação....57
3.11 Variações do lado do servidor....58
3.12 Resumo do capítulo....58
Exercícios de múltipla escolha....59
Exercícios discursivos....63
Exercícios práticos em Python (scikit-learn)....64
Exercício 1 — Sua primeira agregação à mão....64
Exercício 2 — Reproduzindo a curva do FedAvg....64
Exercício 3 — Variando E e η....64
Exercício 4 — Calculando o custo de comunicação....64
Projeto integrador do capítulo....65
Rede de Hospitais: do central ao federado....65
Parte II — Desafios do Treino Federado....66
Capítulo 4 — Quando os Dados Não São Iguais: Heterogeneidade (Não-IID)....67
4.1 Cada cliente é de um jeito....67

4.2 O que é IID e não-IID ... 67
4.3 Os tipos de heterogeneidade ... 68
4.4 Como simular dados não-IID ... 68
4.4.1 Shards (fatias por classe) ... 68
4.4.2 Dirichlet ... 69
4.5 Por que o não-IID atrapalha? ... 69
4.5.1 Deriva dos clientes ... 69
4.5.2 Outros efeitos ... 70
4.6 Medindo a heterogeneidade ... 71
4.7 Algoritmos para dados heterogêneos ... 71
4.7.1 FedProx: uma "coleira" no cliente ... 71
4.7.2 SCAFFOLD: corrigir a direção ... 72
4.7.3 FedNova: normalizar o esforço ... 72
4.7.4 Outras ideias ... 72
4.8 Implementando o SCAFFOLD ... 72
4.9 Comparando FedAvg, FedProx e SCAFFOLD ... 73
4.10 Resumo do capítulo ... 74
Exercícios de múltipla escolha ... 75
Exercícios discursivos ... 79
Exercícios práticos em Python (scikit-learn) ... 80
Exercício 1 — Medindo a heterogeneidade de rótulos ... 80
Exercício 2 — Épocas locais e heterogeneidade ... 80
Exercício 3 — Heterogeneidade de quantidade: ponderar ou não? ... 80
Exercício 4 — Quanto vale a "coleira" do FedProx? ... 80
Projeto integrador do capítulo ... 81
Escolhendo o algoritmo: mapa de heterogeneidade ... 81
Capítulo 5 — Falando Menos: Eficiência de Comunicação ... 82
5.1 O gargalo da rede ... 82
5.2 Quanto tempo leva um treino federado? ... 83
5.2.1 Um exemplo ... 83
5.3 Mapa de estratégias ... 83
5.4 Quantização: menos bits por número ... 84
5.4.1 A ideia ... 84
5.4.2 Arredondamento estocástico ... 84
5.4.3 Quantos bytes economizamos? ... 84
5.5 Esparsificação: enviar só o que importa ... 85
5.5.1 O problema: o que foi descartado se perde ... 85
5.6 Implementando a compressão ... 85
5.7 O experimento: acurácia contra bytes ... 87
5.8 Outras ideias ... 88
5.9 Resumo do capítulo ... 89
Exercícios de múltipla escolha ... 90
Exercícios discursivos ... 94
Exercícios práticos em Python (scikit-learn) ... 95
Exercício 1 — Viés do arredondamento: determinístico contra estocástico ... 95
Exercício 2 — Calculadora de tempo por rodada ... 95
Exercício 3 — Esparsificação com e sem memória de erro ... 95
Exercício 4 — Quantização com dados muito heterogêneos ... 95
Projeto integrador do capítulo ... 96
Orçamento de comunicação: qual configuração escolher? ... 96
Capítulo 6 — Cada Um do Seu Jeito: Personalização ... 97
6.1 Roupa de prateleira ou sob medida? ... 97

6.2 Quando o modelo global não serve ... 97
6.2.1 Medir por cliente ... 97
6.2.2 Dois cenários em que o global falha ... 98
6.3 Mapa das estratégias de personalização ... 98
6.4 Ajuste fino local ... 99
6.5 Ditto: um modelo pessoal com coleira ... 99
6.6 Partição do modelo: corpo e cabeça ... 99
6.7 Agrupar clientes parecidos: IFCA ... 100
6.8 Meta-aprendizado: aprender a se adaptar ... 101
6.9 A biblioteca personalizacao.py ... 101
6.10 Experimento A: poucos rótulos por cliente ... 102
6.11 Experimento B: mudança de conceito ... 103
6.12 Custos e riscos da personalização ... 105
6.13 Resumo do capítulo ... 105
Exercícios de múltipla escolha ... 106
Exercícios discursivos ... 110
Exercícios práticos em Python (scikit-learn) ... 111
Exercício 1 — Avaliando o modelo global cliente a cliente ... 111
Exercício 2 — Quantas épocas de ajuste fino? ... 111
Exercício 3 — O botão λ do Ditto ... 111
Exercício 4 — IFCA com o número errado de grupos ... 111
Projeto integrador do capítulo ... 112
Chegou um cliente novo: o que fazer com poucos dados? ... 112

Parte III — Privacidade e Segurança ... 113

Capítulo 7 — O Que Pode Vazar: Ameaças à Privacidade ... 114
7.1 O segredo que escapa ... 114
7.2 Quem ataca e o que enxerga ... 115
7.2.1 Os adversários ... 115
7.2.2 O que trafega ... 115
7.3 Mapa dos ataques ... 116
7.4 Inversão de gradientes: o gradiente entrega a entrada ... 116
7.4.1 A ideia ... 116
7.4.2 Um caso em que a conta é exata ... 116
7.4.3 Pondo à prova ... 117
7.5 Inferência de pertencimento ... 118
7.5.1 A pergunta ... 118
7.5.2 O ataque mais simples: o limiar de perda ... 118
7.5.3 Pondo à prova ... 118
7.6 Inferência de propriedades ... 119
7.7 Memorização e extração ... 119
7.8 Ataques ativos de um servidor malicioso ... 120
7.9 Por que isso acontece? ... 120
7.10 O que ajuda, e o que não ajuda ... 120
7.11 Resumo do capítulo ... 120
Exercícios de múltipla escolha ... 121
Exercícios discursivos ... 125
Exercícios práticos em Python (scikit-learn) ... 126
Exercício 1 — Reconstruindo cinco dígitos, um por vez ... 126
Exercício 2 — Atualização com vários passos locais ... 126
Exercício 3 — Quanto tempo de treino é demais? ... 126
Exercício 4 — Escolhendo o limiar do atacante ... 126
Projeto integrador do capítulo ... 127

Auditoria de privacidade: qual configuração aprovar? 127
Capítulo 8 — Protegendo os Dados: Privacidade Diferencial, Agregação Segura e Criptografia 128
8.1 Camadas de defesa 128
8.2 Privacidade diferencial 129
8.2.1 A intuição: a pesquisa da moeda 129
8.2.2 A definição 130
8.2.3 Como se obtém DP: sensibilidade e ruído 130
8.2.4 Local, central e distribuída 131
8.2.5 DP em aprendizado federado 131
8.2.6 Contabilidade de privacidade 132
8.2.7 A biblioteca privacidade.py 132
8.2.8 O experimento: privacidade contra utilidade 133
8.2.9 DP contra os ataques do Capítulo 7 134
8.3 Agregação segura 135
8.3.1 A ideia: máscaras que se cancelam 135
8.3.2 Simulação 135
8.3.3 O que falta no protocolo real 136
8.4 Criptografia homomórfica e computação multipartidária 136
8.4.1 Calcular sem abrir o cofre 136
8.4.2 Um Paillier de brinquedo 137
8.4.3 Computação multipartidária 138
8.4.4 Ambientes de execução confiáveis 138
8.5 Comparando as defesas 138
8.6 Resumo do capítulo 139
Exercícios de múltipla escolha 140
Exercícios discursivos 144
Exercícios práticos em Python (scikit-learn) 145
Exercício 1 — Resposta aleatorizada: privacidade contra precisão 145
Exercício 2 — Qual ruído preciso para ε = 2? 145
Exercício 3 — Escolhendo a norma de corte S 145
Exercício 4 — Agregação segura e clientes que desistem 145
Projeto integrador do capítulo 146
Orçamento de privacidade da clínica 146
Capítulo 9 — Quando Alguém Trapaceia: Robustez e Ataques de Envenenamento 147
9.1 Quando alguém trapaceia 147
9.2 Quem trapaceia e o que quer 147
9.2.1 O adversário bizantino 147
9.2.2 Envenenamento de dados e de modelo 147
9.2.3 Qual o objetivo? 148
9.3 Por que a média é frágil 148
9.4 Agregadores robustos 149
9.4.1 Mediana coordenada a coordenada 149
9.4.2 Média aparada 149
9.4.3 Krum e Multi-Krum 149
9.4.4 Corte de norma 149
9.4.5 Outros 150
9.5 A biblioteca robustez.py 150
9.6 Ataques não direcionados 151
9.7 Backdoor 153
9.7.1 O ataque 153
9.8 O custo da robustez com dados não-IID 154
9.9 Outras defesas e limites 155

9.9.1 A tensão com a agregação segura....155
9.10 Resumo do capítulo....156
Exercícios de múltipla escolha....157
Exercícios discursivos....161
Exercícios práticos em Python (scikit-learn)....162
Exercício 1 — Ponto de ruptura na prática....162
Exercício 2 — Troca de rótulos: até onde vai a média?....162
Exercício 3 — Quanto aparar?....162
Exercício 4 — Quando a mediana também cai....162
Projeto integrador do capítulo....163
Parecer de segurança: qual agregador adotar?....163
Parte IV — Arquiteturas, Aplicações e Fronteiras....164
Capítulo 10 — Outros Desenhos: Federado Vertical, Split Learning, Descentralizado e por Transferência....165
10.1 Os dados podem ser repartidos de maneiras diferentes....165
10.2 Aprendizado federado vertical....166
10.2.1 Os desafios....166
10.2.2 Como treinar uma regressão logística vertical....166
10.2.3 A biblioteca arquiteturas.py....167
10.2.4 Experimento: três bancos de dados, uma pessoa....169
10.2.5 O perigo: o resíduo entrega o rótulo....169
10.3 Split learning....170
10.3.1 A ideia: partir a rede em duas....170
10.3.2 A matemática é a mesma....171
10.4 Aprendizado descentralizado: sem servidor....171
10.4.1 E se não houver servidor?....171
10.4.2 A matriz de mistura e a topologia....171
10.4.3 Vantagens e desafios....172
10.5 Aprendizado federado por transferência....172
10.6 Comparação das arquiteturas....172
10.7 Resumo do capítulo....173
Exercícios de múltipla escolha....174
Exercícios discursivos....178
Exercícios práticos em Python (scikit-learn)....179
Exercício 1 — Quantos participantes?....179
Exercício 2 — Ruído no resíduo....179
Exercício 3 — Comunicação: split learning contra FedAvg....179
Exercício 4 — Velocidade do consenso no anel....179
Projeto integrador do capítulo....180
Banco e loja: quando o aprendizado vertical compensa?....180
Capítulo 11 — Fronteiras e Prática: Não Supervisionado, Modelos de Fundação, Ferramentas, Aplicações e Ética....181
11.1 Além da classificação....181
11.2 K-means federado....181
11.3 PCA federado....182
11.4 A biblioteca fronteiras.py....182
11.5 Experimento: agrupar e comprimir sem juntar os dados....184
11.6 Modelos de fundação e modelos de linguagem....185
11.6.1 O cenário....185
11.6.2 A saída: ajuste eficiente em parâmetros....185
11.6.3 Um experimento em miniatura....186
11.7 Ferramentas e frameworks....187

11.8 Aplicações reais ... 188
11.8.1 Saúde ... 188
11.8.2 Dispositivos móveis ... 189
11.8.3 Internet das coisas e redes ... 189
11.8.4 Finanças e seguros ... 189
11.9 Regulação, ética e responsabilidade ... 189
11.9.1 Aprendizado federado não é anonimização ... 189
11.9.2 Princípios de proteção de dados ... 189
11.9.3 Justiça (fairness) ... 189
11.9.4 Incentivos e governança ... 190
11.10 Lista de verificação de um projeto federado ... 190
11.11 Problemas em aberto ... 190
11.12 Resumo do capítulo ... 191
Exercícios de múltipla escolha ... 192
Exercícios discursivos ... 196
Exercícios práticos em Python (scikit-learn) ... 197
Exercício 1 — Escolhendo k no K-means federado ... 197
Exercício 2 — PCA federado nos dígitos ... 197
Exercício 3 — Regressão linear federada exata ... 197
Exercício 4 — Quanto posto o adaptador precisa? ... 197
Projeto integrador do capítulo ... 198
Perfil de clientes sem juntar os cadastros ... 198

Parte V — Projeto Final ... 199

Capítulo 12 — Projeto Integrador Final: do Problema ao Relatório ... 200
12.1 O desafio ... 200
12.2 Os dados e o cenário ... 201
12.3 O motor de simulação projeto.py ... 201
12.4 As nove etapas ... 202
12.5 Modelo de relatório ... 203
12.6 Critérios de avaliação ... 203
12.6.1 Para comparar depois ... 203
12.7 O que você aprendeu ... 204
12.8 Para continuar estudando ... 204
Exercícios de múltipla escolha ... 205
Exercícios discursivos ... 209
Projeto integrador final ... 210
Federa Saúde: a decisão final ... 210

Apêndice A — Gabarito dos exercícios ... 211

Capítulo 1 — Bem-vindo ao Aprendizado Federado ... 212
Múltipla escolha ... 212
Discursivas (respostas-modelo) ... 212

Capítulo 2 — Preparando o Terreno ... 215
Múltipla escolha ... 215
Discursivas (respostas-modelo) ... 215

Capítulo 3 — Anatomia de um Sistema Federado ... 217
Múltipla escolha ... 217
Discursivas (respostas-modelo) ... 217

Capítulo 4 — Quando os Dados Não São Iguais ... 219
Múltipla escolha ... 219
Discursivas (respostas-modelo) ... 219

Capítulo 5 — Falando Menos 221
Múltipla escolha 221
Discursivas (respostas-modelo) 221
Capítulo 6 — Cada Um do Seu Jeito 223
Múltipla escolha 223
Discursivas (respostas-modelo) 223
Capítulo 7 — O Que Pode Vazar 225
Múltipla escolha 225
Discursivas (respostas-modelo) 225
Capítulo 8 — Protegendo os Dados 227
Múltipla escolha 227
Discursivas (respostas-modelo) 227
Capítulo 9 — Quando Alguém Trapaceia 229
Múltipla escolha 229
Discursivas (respostas-modelo) 229
Capítulo 10 — Outros Desenhos 231
Múltipla escolha 231
Discursivas (respostas-modelo) 231
Capítulo 11 — Fronteiras e Prática 233
Múltipla escolha 233
Discursivas (respostas-modelo) 233
Capítulo 12 — Projeto Integrador Final 235
Múltipla escolha 235
Discursivas (respostas-modelo) 235
Apêndice B — Glossário 237
Apêndice C — Guia rápido de algoritmos e códigos 241
C.1 Mapa de decisão 241
C.2 Arquivos de código do livro 241
C.3 Código completo da biblioteca fedlib.py 241
C.4 Modelo de código para um experimento federado 243
C.5 Lista de verificação final 244
Apêndice D — Soluções dos exercícios práticos 245
Capítulo 1 — Bem-vindo ao Aprendizado Federado 246
Exercício 1 — Conhecendo os dados dos "hospitais" 246
Exercício 2 — Quanto o tamanho do hospital importa? 246
Exercício 3 — Repetindo para não se enganar com o acaso 247
Capítulo 2 — Preparando o Terreno 249
Exercício 1 — Taxa de aprendizado em ação 249
Exercício 2 — Gradiente completo versus mini-lotes 249
Exercício 3 — Acurácia que engana 250
Exercício 4 — Padronização federada com clientes de tamanhos diferentes 251
Capítulo 3 — Anatomia de um Sistema Federado 253
Exercício 1 — Sua primeira agregação à mão 253
Exercício 2 — Reproduzindo a curva do FedAvg 253
Exercício 3 — Variando E e η 254
Exercício 4 — Calculando o custo de comunicação 254
Capítulo 4 — Quando os Dados Não São Iguais 256
Exercício 1 — Medindo a heterogeneidade de rótulos 256

Exercício 2 — Épocas locais e heterogeneidade ... 256
Exercício 3 — Heterogeneidade de quantidade: ponderar ou não? ... 257
Exercício 4 — Quanto vale a "coleira" do FedProx? ... 258
Capítulo 5 — Falando Menos ... 259
Exercício 1 — Viés do arredondamento: determinístico contra estocástico ... 259
Exercício 2 — Calculadora de tempo por rodada ... 259
Exercício 3 — Esparsificação com e sem memória de erro ... 260
Exercício 4 — Quantização com dados muito heterogêneos ... 260
Capítulo 6 — Cada Um do Seu Jeito ... 262
Exercício 1 — Avaliando o modelo global cliente a cliente ... 262
Exercício 2 — Quantas épocas de ajuste fino? ... 262
Exercício 3 — O botão λ do Ditto ... 263
Exercício 4 — IFCA com o número errado de grupos ... 264
Capítulo 7 — O Que Pode Vazar ... 265
Exercício 1 — Reconstruindo cinco dígitos, um por vez ... 265
Exercício 2 — Atualização com vários passos locais ... 265
Exercício 3 — Quanto tempo de treino é demais? ... 266
Exercício 4 — Escolhendo o limiar do atacante ... 267
Capítulo 8 — Protegendo os Dados ... 269
Exercício 1 — Resposta aleatorizada: privacidade contra precisão ... 269
Exercício 2 — Qual ruído preciso para ε = 2? ... 269
Exercício 3 — Escolhendo a norma de corte S ... 270
Exercício 4 — Agregação segura e clientes que desistem ... 271
Capítulo 9 — Quando Alguém Trapaceia ... 272
Exercício 1 — Ponto de ruptura na prática ... 272
Exercício 2 — Troca de rótulos: até onde vai a média? ... 272
Exercício 3 — Quanto aparar? ... 273
Exercício 4 — Quando a mediana também cai ... 274
Capítulo 10 — Outros Desenhos ... 275
Exercício 1 — Quantos participantes? ... 275
Exercício 2 — Ruído no resíduo ... 275
Exercício 3 — Comunicação: split learning contra FedAvg ... 276
Exercício 4 — Velocidade do consenso no anel ... 277
Capítulo 11 — Fronteiras e Prática ... 279
Exercício 1 — Escolhendo k no K-means federado ... 279
Exercício 2 — PCA federado nos dígitos ... 279
Exercício 3 — Regressão linear federada exata ... 280
Exercício 4 — Quanto posto o adaptador precisa? ... 281
Apêndice E — Soluções de referência dos projetos integradores ... 283
Capítulo 1 — Bem-vindo ao Aprendizado Federado ... 284
Rede de Clínicas: vale a pena colaborar? ... 284
Capítulo 2 — Preparando o Terreno ... 286
Triagem com poucos dados: do zero ao modelo ... 286
Capítulo 3 — Anatomia de um Sistema Federado ... 288
Rede de Hospitais: do central ao federado ... 288
Capítulo 4 — Quando os Dados Não São Iguais ... 289
Escolhendo o algoritmo: mapa de heterogeneidade ... 289
Capítulo 5 — Falando Menos ... 291
Orçamento de comunicação: qual configuração escolher? ... 291

Capítulo 6 — Cada Um do Seu Jeito **293**
Chegou um cliente novo: o que fazer com poucos dados? 293
Capítulo 7 — O Que Pode Vazar **294**
Auditoria de privacidade: qual configuração aprovar? 294
Capítulo 8 — Protegendo os Dados **296**
Orçamento de privacidade da clínica 296
Capítulo 9 — Quando Alguém Trapaceia **298**
Parecer de segurança: qual agregador adotar? 298
Capítulo 10 — Outros Desenhos **300**
Banco e loja: quando o aprendizado vertical compensa? 300
Capítulo 11 — Fronteiras e Prática **302**
Perfil de clientes sem juntar os cadastros 302
Capítulo 12 — Projeto Integrador Final **304**
Federa Saúde: a decisão final 304
Referências **306**

Apresentação

Para quem é este livro

Este livro foi escrito para um **público misto**: estudantes de graduação e pós-graduação, profissionais de dados e curiosos que querem entender como as máquinas aprendem **em equipe, sem que ninguém precise entregar seus dados**.

Você **não precisa** de base avançada em matemática ou programação. Cada conceito técnico essencial é explicado no momento em que aparece. Quando uma fórmula surgir, ela virá com uma explicação "em palavras" e, quase sempre, com um exemplo numérico pequeno.

Ao mesmo tempo, o livro avança. Os capítulos começam no nível básico, passam pelo intermediário e chegam a temas avançados, como privacidade diferencial, agregação segura, ataques de inversão de gradientes, aprendizado federado vertical e modelos de fundação federados.

Por que este tema importa

Dados pessoais, de saúde e financeiros são valiosos e sensíveis. Ao mesmo tempo, os modelos de inteligência artificial precisam de muitos dados para funcionar bem. O **aprendizado federado** propõe uma saída: **levar o modelo até os dados**, em vez de levar os dados até o modelo.

Este livro explica essa ideia do começo ao fim: como funciona, onde falha, como se proteger e como praticar.

Como o livro está organizado

O livro tem **cinco partes** e **doze capítulos**.

- **Parte I — Fundamentos:** o que é aprendizado federado, as bases de treino de modelos e o algoritmo FedAvg.
- **Parte II — Desafios do treino federado:** dados diferentes entre clientes (não-IID), economia de comunicação e personalização.
- **Parte III — Privacidade e segurança:** ameaças à privacidade, privacidade diferencial, agregação segura e robustez contra participantes mal-intencionados.
- **Parte IV — Arquiteturas, aplicações e fronteiras:** aprendizado federado vertical, descentralizado e *split learning*; aprendizado não supervisionado federado, modelos de fundação, ferramentas, aplicações, regulação e ética.
- **Parte V — Projeto final:** um estudo completo que integra todas as técnicas.

Cada capítulo segue o mesmo roteiro:

1. **Objetivos de aprendizagem**, no início.
2. **Conteúdo** com analogias, figuras, exemplos e códigos em Python.
3. **Resumo** em cartões de memorização.
4. **30 questões de múltipla escolha** e **20 questões discursivas**.
5. **Exercícios práticos em Python** com a biblioteca scikit-learn, em ordem crescente de dificuldade.
6. **Projeto integrador do capítulo**, com contexto, tarefas, entregáveis e critérios de avaliação.
7. Um quadro **"Para ir além"**, com conjuntos de dados públicos da UCI e do Kaggle para praticar.

O **gabarito de todas as questões** está no Apêndice A. O **glossário** está no Apêndice B e o **guia rápido**, com o código da biblioteca `fedlib.py`, no Apêndice C. As **soluções dos exercícios práticos** estão no Apêndice D, e a **solução de referência dos projetos integradores**, no Apêndice E. Tente resolver sozinho antes de consultá-las.

Como estudar com este livro

- **Estude em blocos curtos.** Sessões de 25 a 40 minutos, com pausas, ajudam a manter a atenção.
- **Pare e explique.** Após cada seção, tente explicar a ideia, em voz alta, como se falasse com um amigo.
- **Rode os códigos.** Altere um parâmetro por vez e observe o que muda.
- **Responda antes de olhar o gabarito.** Tentar lembrar fixa mais do que reler.
- **Volte aos cartões de memorização** dias depois. Revisões espaçadas ajudam a memória de longo prazo.

Caixas e símbolos do livro

> **Dica**
>
> Sugestões práticas para facilitar seu trabalho.

> **Atenção**
>
> Cuidados e erros comuns. Leia com calma.

> **No seu dia a dia**
>
> Exemplos do cotidiano que aproximam o conceito da sua realidade.

> **Saiba mais**
>
> Curiosidades e aprofundamentos para quem quer ir além.

> **Exemplo ou exercício guiado**
>
> Pequenas atividades para você experimentar.

> **Memorize**
>
> O essencial do assunto, em poucas linhas.

Sobre os códigos

Todos os códigos foram escritos em **Python 3** e testados com **NumPy**, **scikit-learn** e **matplotlib**. Para a maior parte do livro, basta instalar:

```bash
pip install numpy pandas matplotlib scikit-learn
```

Os experimentos usam conjuntos de dados da **UCI Machine Learning Repository** (Dua; Graff, 2019) que já vêm no scikit-learn (dígitos manuscritos, câncer de mama de Wisconsin, vinhos, íris). Os quadros "Para ir além"

sugerem outras bases da UCI e do Kaggle, que **não foram usadas nem testadas** nos códigos do livro: confira descrição, licença e colunas antes de utilizá-las.

A biblioteca didática `fedlib.py`, apresentada no Capítulo 3, simula clientes e servidor em um só computador. Salve-a na pasta dos seus exercícios.

> **Atenção**
>
> As simulações do livro são **didáticas**. Os números exatos podem variar com versões diferentes das bibliotecas. Em um sistema real, há também rede, autenticação, falhas e requisitos legais que a simulação não mostra.

Agradecimentos e uso

Bons estudos! Se você encontrar erros ou tiver sugestões, registre-as: elas ajudam a melhorar o material nas próximas edições.

Araken de Medeiros Santos

PARTE I

Parte I — Fundamentos

Por que treinar modelos sem juntar os dados, como o treino de modelos funciona por dentro e como nasce o algoritmo FedAvg, a base de quase tudo o que vem depois.

Nesta parte:

- Cap. 1 — Bem-vindo ao Aprendizado Federado
- Cap. 2 — Preparando o Terreno: Dados, Modelos e Otimização
- Cap. 3 — Anatomia de um Sistema Federado: FedSGD e FedAvg



Capítulo 1 — Bem-vindo ao Aprendizado Federado

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar, com suas palavras, o que é aprendizado federado e por que ele existe.
- Descrever os problemas de privacidade, custo e legislação que motivam treinar modelos sem juntar os dados.
- Diferenciar aprendizado centralizado, distribuído em data center e federado.
- Distinguir os cenários cross-device e cross-silo.
- Reconhecer aplicações reais e desfazer mitos comuns sobre privacidade.
- Executar sua primeira simulação federada em Python com scikit-learn.

1.1 Uma história para começar

Imagine cinco amigas que adoram cozinhar. Cada uma guarda, em um caderno, **receitas de família**. Os cadernos são secretos. Nenhuma quer entregar o seu.

Mesmo assim, elas sonham em criar um **livro de receitas melhor** do que qualquer caderno individual.

Como fazer isso sem mostrar os cadernos?

Elas combinam um jogo. Uma delas escreve uma "receita-base" e envia uma cópia para todas. Cada amiga **testa e ajusta** a receita em casa, usando o que sabe. Depois, cada uma devolve **apenas os ajustes**: "menos sal", "mais tempo no forno". Ninguém mostra o caderno.

A coordenadora junta os ajustes, melhora a receita-base e começa uma nova rodada.

Depois de algumas rodadas, a receita ficou excelente. **Ninguém revelou os cadernos.**

Isso é, em essência, o **aprendizado federado** (*federated learning*).

NO SEU DIA A DIA **Onde isso aparece no seu dia a dia**

O teclado do seu celular sugere a próxima palavra e aprende com o seu jeito de escrever. Em alguns sistemas, esse aprendizado acontece no próprio aparelho, e só pequenas atualizações de modelo são enviadas. Suas mensagens não precisam sair do celular.

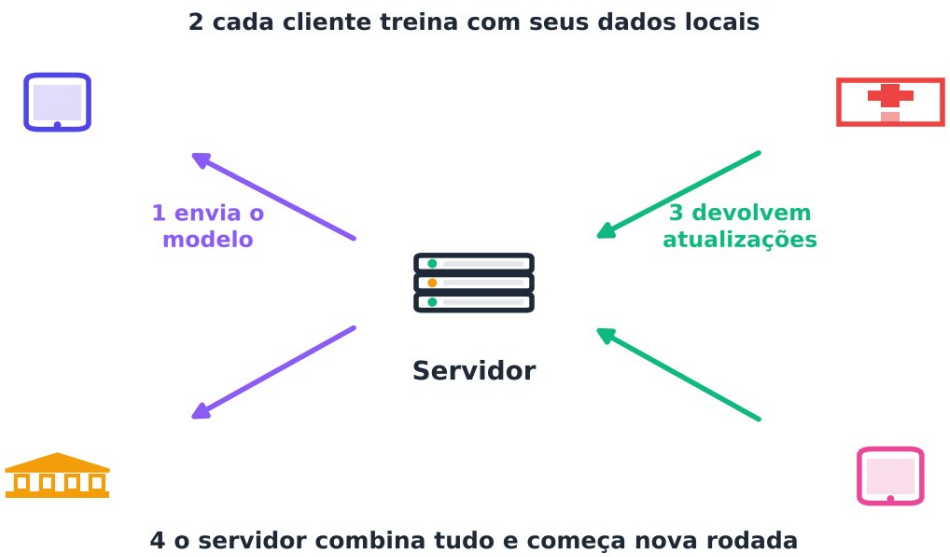


Figura 1.1 — A ideia central em quatro passos: o servidor envia o modelo, cada cliente treina com seus dados, devolve só a atualização, e o servidor combina tudo.

1.2 O problema: dados espalhados e protegidos

Antes de definir o aprendizado federado, vamos entender o problema que ele resolve.

1.2.1 Os dados nascem distribuídos

Os dados raramente nascem em um só lugar. Eles surgem em:

- **celulares**, com mensagens, fotos e hábitos de uso;
- **hospitais**, com exames e prontuários;
- **bancos**, com transações e perfis de crédito;
- **fábricas e carros**, com sensores que medem tudo o tempo todo.

Cada um desses lugares é um **silo de dados**: um "armazém" fechado, controlado por um dono. Em geral, o dono não pode ou não quer abrir o armazém para os outros.

1.2.2 Por que não juntar tudo em um só lugar?

A resposta tradicional em aprendizado de máquina é: "junte todos os dados e treine". Essa abordagem se chama **aprendizado centralizado**. Ela funciona bem quando os dados são seus e cabem em um lugar. Mas há quatro obstáculos:

1. **Privacidade.** Dados de saúde, finanças e conversas são sensíveis. Copiá-los aumenta o risco de vazamento.
2. **Leis e regras.** Regulamentos limitam a coleta e o uso de dados pessoais.
3. **Custo e tamanho.** Mover petabytes de sensores de milhões de aparelhos é caro e lento.
4. **Confiança e concorrência.** Dois bancos concorrentes raramente entregam seus dados a um terceiro.

1.2.3 O papel das leis

Duas leis ajudam a entender o contexto:

- O **GDPR** (*General Data Protection Regulation*), regulamento da União Europeia, em vigor desde 2018.

- A **LGPD** (Lei Geral de Proteção de Dados, Lei nº 13.709/2018), no Brasil.

Ambas exigem, entre outros pontos, **finalidade clara**, **minimização de dados** (coletar só o necessário) e **segurança**. O aprendizado federado se alinha bem à ideia de minimização, porque evita copiar os dados brutos. Mas ele **não substitui** a análise jurídica: cada projeto precisa ser avaliado com especialistas.

> **Atenção**
>
> Este livro explica técnicas. Ele **não é aconselhamento jurídico**. Antes de usar dados reais de pessoas, consulte o encarregado de dados e o time jurídico da sua organização.

1.3 O que é aprendizado federado

Vamos à definição.

> ***Aprendizado federado*** *é uma forma de treinar um modelo de aprendizado de máquina em que* ***vários participantes colaboram****, cada um com seus próprios dados,* ***sem enviar os dados brutos*** *a um servidor central. O que trafega são* ***parâmetros do modelo*** *ou* ***atualizações*** *(McMahan et al., 2017; Kairouz et al., 2021).*

Em uma frase: **levamos o modelo até os dados, e não os dados até o modelo.**

Três termos aparecerão em todo o livro. Vamos explicá-los já.

- **Cliente:** quem possui dados e treina localmente. Pode ser um celular, um hospital, um banco.
- **Servidor** (ou coordenador): quem organiza o treino e **combina** as contribuições dos clientes.
- **Rodada de comunicação:** um ciclo completo em que o servidor envia o modelo, os clientes treinam e devolvem atualizações, e o servidor agrega.



Figura 1.2 — No aprendizado centralizado, os dados brutos viajam até o servidor. No federado, os dados ficam com os donos e só os modelos viajam.

1.3.1 Uma definição mais precisa

Imagine K clientes. O cliente k possui um conjunto de dados local com n_k exemplos. O total é $n = n_1 + n_2 + \ldots + n_K$. Queremos encontrar parâmetros w que minimizem o erro médio sobre **todos** os dados, mesmo que ninguém os veja juntos:

$$f(w) = \Sigma_k \, (n_k / n) \cdot F_k(w)$$

Em palavras: o erro global é a **média ponderada** dos erros locais. *F_k(w)* é o erro do modelo *w* nos dados do cliente *k*. Clientes com mais dados pesam mais. Esta é a fórmula que sustenta o livro inteiro. Você a verá de novo no Capítulo 3.

Memorize

- Cliente = quem tem os dados. Servidor = quem coordena.
- Os dados brutos ficam no cliente.
- O servidor recebe atualizações de modelo e as agrega.
- O objetivo é o mesmo do centralizado: minimizar o erro **global**.

1.4 Um pouco de história

O termo "aprendizado federado" foi popularizado por pesquisadores do Google, que propuseram o algoritmo **FedAvg** (*Federated Averaging*) para treinar redes neurais em celulares (McMahan et al., 2017). No mesmo ano, surgiu um protocolo de **agregação segura**, que impede o servidor de ver a atualização de cada cliente isolada (Bonawitz et al., 2017).

Depois vieram visões gerais (Yang et al., 2019; Li et al., 2020b), a descrição de um sistema em escala de produção (Bonawitz et al., 2019) e uma grande revisão com dezenas de problemas em aberto (Kairouz et al., 2021). Mais recentemente, o tema avançou para modelos de linguagem muito grandes.

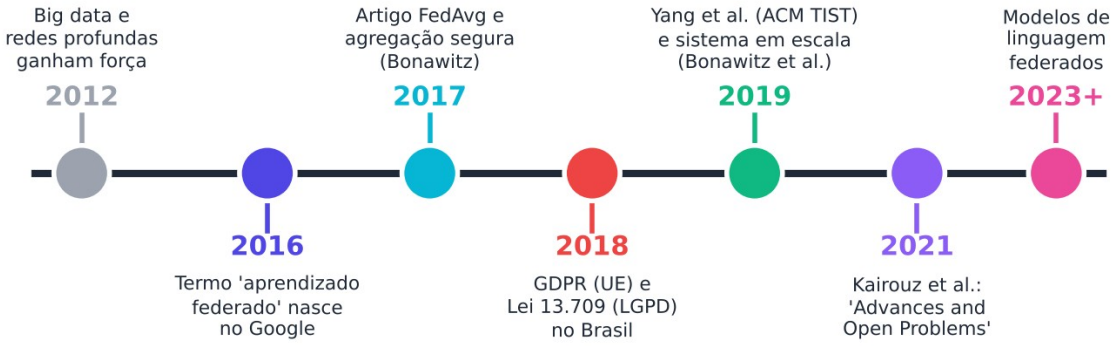
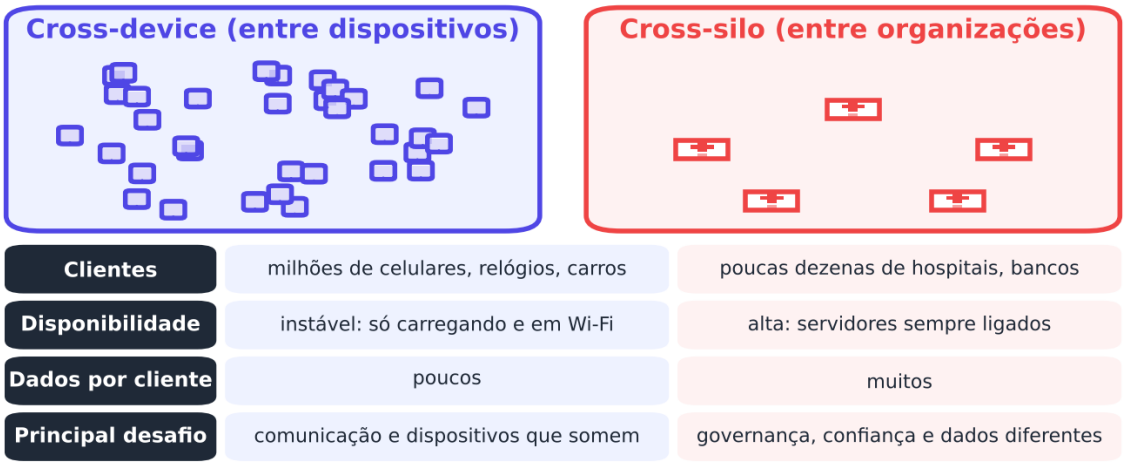


Figura 1.3 — Linha do tempo resumida do aprendizado federado.

1.5 Dois grandes cenários: cross-device e cross-silo

Nem todo sistema federado se parece. Duas famílias aparecem o tempo todo (Kairouz et al., 2021).

- **Cross-device** (entre dispositivos): milhões de clientes pequenos, como celulares. Cada um tem poucos dados. Eles ficam disponíveis de forma instável. Só uma fração participa de cada rodada.
- **Cross-silo** (entre organizações): poucos clientes grandes e confiáveis, como hospitais e bancos. Cada um tem muitos dados e boa infraestrutura. Em geral, todos participam de todas as rodadas.



	Cross-device (entre dispositivos)	Cross-silo (entre organizações)
Clientes	milhões de celulares, relógios, carros	poucas dezenas de hospitais, bancos
Disponibilidade	instável: só carregando e em Wi-Fi	alta: servidores sempre ligados
Dados por cliente	poucos	muitos
Principal desafio	comunicação e dispositivos que somem	governança, confiança e dados diferentes

Figura 1.4 — Cross-device e cross-silo diferem em número de clientes, disponibilidade, volume de dados e desafios.

> **Dica**
>
> Quando você ler um artigo novo, pergunte primeiro: **é cross-device ou cross-silo?** Muitas decisões de projeto dependem da resposta.

1.6 Onde o aprendizado federado já é usado

1.6.1 Saúde

Hospitais têm dados valiosos e protegidos. Estudos mostram que treinar de forma federada permite colaboração entre instituições sem compartilhar prontuários. Exemplos:

- Segmentação de tumores cerebrais em imagens, com vários centros médicos (Sheller et al., 2020).
- Previsão de necessidades de oxigênio em pacientes com COVID-19, com dados de vinte hospitais em vários continentes (Dayan et al., 2021).
- Um panorama do tema para a saúde digital (Rieke et al., 2020).

1.6.2 Teclados e assistentes móveis

O texto digitado é muito pessoal. Treinar no aparelho reduz a necessidade de enviar o que você escreve. Este foi um dos casos motivadores originais (McMahan et al., 2017; Bonawitz et al., 2019).

1.6.3 Finanças

Bancos podem melhorar a detecção de fraudes combinando conhecimento sem trocar dados de clientes. O desafio é maior, pois fraudes são raras e os dados de cada banco são diferentes.

1.6.4 Indústria e internet das coisas

Fábricas, carros e redes elétricas geram dados em borda (*edge*), perto de onde são produzidos. Treinar ali economiza banda e reduz a latência.



Figura 1.5 — Três cenários típicos e o que fica local em cada um.

1.7 Federado, distribuído, centralizado: qual a diferença?

Muita gente confunde **aprendizado distribuído** com **federado**. No distribuído clássico, os dados ficam em um data center, e o objetivo é **acelerar** o treino dividindo o trabalho entre máquinas. Os dados são embaralhados para ficarem parecidos entre as máquinas.

No federado, o objetivo é **respeitar** onde os dados estão. Não podemos embaralhá-los nem controlar quem participa.

Aspecto	Centralizado	Distribuído (data center)	Federado
Onde estão os dados	Em um só lugar	Em um cluster controlado	Com cada cliente
Quem controla	O treinador	O treinador	Cada dono
Dados são parecidos entre nós?	Não se aplica	Sim, por embaralhamento	Em geral, não
Comunicação	Rápida	Rápida	Lenta e instável
Participantes	Um	Máquinas fixas	Podem entrar e sair
Objetivo principal	Qualidade	Velocidade	Privacidade e colaboração

> **Saiba mais**
>
> Os dados não serem parecidos entre clientes é chamado de dado **não-IID** (*non independent and identically distributed*, "não independentes e identicamente distribuídos"). É um dos desafios centrais do aprendizado federado. Estudaremos o tema no Capítulo 4.

1.8 Mitos e verdades

Vamos desfazer algumas ideias comuns.

Mito 1: "No federado, a privacidade é automática." Não é. Atualizações de modelo podem vazar informação sobre os dados. Há ataques que reconstroem imagens e descobrem se um exemplo participou do treino. Defesas existem, e o Capítulo 8 as explica.

Mito 2: "O federado é sempre pior do que o centralizado." Nem sempre. Em muitos casos, ele chega perto. E às vezes é a **única** forma de treinar, pois os dados não podem ser reunidos.

Mito 3: "Todos os clientes são confiáveis." Não. Clientes podem falhar ou trapacear. O Capítulo 9 trata da robustez.

Mito 4: "Basta ter servidor e clientes." Também é preciso lidar com comunicação lenta, dispositivos que somem e dados diferentes. Esses são os temas da Parte II.

> **Memorize**
>
> Federado **reduz** a exposição de dados, mas **não garante** privacidade sozinho. Para garantias formais, combine-o com privacidade diferencial e agregação segura.

1.9 Vocabulário essencial

Termo	Significado simples
Modelo	O "programa" que aprende; ele tem parâmetros (números) ajustáveis
Parâmetros (pesos)	Os números que o modelo ajusta ao aprender
Atualização	A mudança nos parâmetros após o treino local
Agregação	Combinar as atualizações de vários clientes em um único modelo
Rodada	Um ciclo: enviar, treinar localmente, devolver, agregar
Silo	Armazém de dados fechado, controlado por um dono
IID / não-IID	Dados parecidos entre clientes / dados diferentes entre clientes
Privacidade diferencial	Técnica que adiciona ruído para esconder indivíduos
Agregação segura	Protocolo que esconde cada atualização individual do servidor

1.10 Prepare o ambiente

Você usará **Python 3** e as bibliotecas **NumPy**, **pandas**, **matplotlib** e **scikit-learn** (Pedregosa et al., 2011). Instale com:

```bash
pip install numpy pandas matplotlib scikit-learn
```

Pode usar o Google Colab, sem instalar nada. Os conjuntos de dados desta unidade vêm **dentro do scikit-learn** e foram originalmente publicados no repositório **UCI Machine Learning**.

1.11 Sua primeira simulação federada

Vamos simular **três hospitais**. Cada um tem apenas **50 pacientes** do conjunto *Breast Cancer Wisconsin*, da UCI, disponível em `load_breast_cancer`. O objetivo é classificar tumores como malignos ou benignos.

Compararemos três estratégias:

1. cada hospital treina **sozinho**;
2. os hospitais **enviam só os modelos**, e o servidor tira a média;
3. os dados são **juntados** (centralizado, o "sonho" proibido).

```python
```

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
esc = StandardScaler().fit(Xtr)
Xtr, Xte = esc.transform(Xtr), esc.transform(Xte)

# três "hospitais" com poucos pacientes cada
rng = np.random.default_rng(0)
idx = rng.permutation(len(ytr))
hospitais = np.array_split(idx[:150], 3)   # 50 pacientes em cada
print("Pacientes por hospital:", [len(h) for h in hospitais])

# 1) cada hospital treina sozinho
locais = []
for i, h in enumerate(hospitais, 1):
    m = LogisticRegression(max_iter=1000).fit(Xtr[h], ytr[h])
    locais.append(m)
    print(f"Hospital {i} sozinho: acurácia = {m.score(Xte, yte):.3f}")

# 2) média dos modelos (um único encontro com o servidor)
media = LogisticRegression(max_iter=1000)
media.coef_ = np.mean([m.coef_ for m in locais], axis=0)
media.intercept_ = np.mean([m.intercept_ for m in locais], axis=0)
media.classes_ = locais[0].classes_
print(f"Média dos 3 modelos: acurácia = {media.score(Xte, yte):.3f}")

# 3) centralizado: juntaria os dados dos três hospitais
todos = np.concatenate(hospitais)
cen = LogisticRegression(max_iter=1000).fit(Xtr[todos], ytr[todos])
print(f"Dados juntos (centralizado): acurácia = {cen.score(Xte, yte):.3f}")
```

O que você deve notar:

- Cada hospital sozinho tem desempenho razoável, mas **ligeiramente menor**.
- A **média dos modelos** (sem nenhum dado trocado) alcança o mesmo valor do modelo centralizado neste exemplo.
- Foi uma única "rodada". Nos próximos capítulos, faremos várias.

> **Atenção**
>
> Este resultado é **otimista** por três razões. O problema é simples. Os dados dos hospitais são sorteados do mesmo conjunto (são IID). E há só uma rodada. Na vida real, os dados são diferentes entre os hospitais, e a média de modelos pode funcionar bem pior. Veremos isso no Capítulo 4.

> **Dica**
>
> Observe uma pequena "trapaça" do código: a padronização (`StandardScaler`) foi calculada com **todos** os dados de treino. Em um sistema federado de verdade, isso exigiria juntar os dados! No Capítulo 2, você aprenderá a calcular média e desvio padrão de forma federada.

1.12 Roteiro do livro

O livro tem cinco partes. Você começa pelos fundamentos, passa pelos desafios do treino, aprende a proteger os dados e a resistir a trapaças, conhece outros desenhos de aprendizado federado e fecha com um projeto completo.



Figura 1.6 — Mapa do livro: do básico ao avançado, sempre com prática.

1.13 Resumo do capítulo

MEMORIZE **O essencial do Capítulo 1**

- Aprendizado federado treina um modelo com vários donos de dados **sem juntar os dados brutos**.
- O ciclo básico: servidor envia modelo, clientes treinam localmente, devolvem atualizações, servidor agrega.
- Motivos: privacidade, leis (GDPR, LGPD), custo e confiança.
- Cenários: **cross-device** (muitos clientes pequenos) e **cross-silo** (poucos clientes grandes).
- Federado não é o mesmo que distribuído em data center, e **não garante privacidade sozinho**.
- O objetivo é minimizar o erro global: a média ponderada dos erros locais.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Qual é a ideia central do aprendizado federado?

a) Levar o modelo até os dados, em vez de levar os dados até o modelo.
b) Apagar os dados depois do treino.
c) Treinar sem usar nenhum dado.
d) Enviar todos os dados para um servidor poderoso e treinar lá.

Questão 2. Na analogia das cinco amigas e dos cadernos de receitas, o que cada amiga devolve para a coordenadora?

a) O caderno completo.
b) Apenas os ajustes feitos na receita-base.
c) O nome dos ingredientes secretos.
d) Uma foto de cada receita.

Questão 3. Em aprendizado federado, o termo "cliente" designa:

a) o usuário final que usa o aplicativo pronto.
b) quem possui dados e treina localmente.
c) o computador que armazena todos os dados.
d) o programador do servidor.

Questão 4. Qual é o papel do servidor (coordenador)?

a) Apagar os modelos antigos dos clientes.
b) Organizar o treino e combinar as contribuições dos clientes.
c) Guardar os dados brutos de todos os clientes.
d) Rotular os dados dos clientes.

Questão 5. O que é uma "rodada de comunicação"?

a) Uma reunião entre os donos dos dados.
b) O treino de um cliente isolado.
c) Um ciclo em que o servidor envia o modelo, clientes treinam e devolvem atualizações, e o servidor agrega.
d) O envio de todos os dados ao servidor.

Questão 6. Um "silo de dados" é:

a) um tipo de rede neural.
b) um tipo de algoritmo de agrupamento.
c) um armazém de dados fechado, controlado por um dono.
d) uma técnica de criptografia.

Questão 7. Qual destes NÃO é um motivo típico para adotar aprendizado federado?

a) Dados sensíveis, como prontuários.
b) Restrições de leis de proteção de dados.
c) Custo de mover enormes volumes de dados.
d) Desejo de deixar o modelo menos preciso de propósito.

Questão 8. A LGPD é:

a) uma biblioteca do scikit-learn.
b) a lei brasileira de proteção de dados pessoais (Lei nº 13.709/2018).
c) um algoritmo de aprendizado federado.
d) um protocolo de agregação segura.

Questão 9. O princípio de "minimização de dados" diz que se deve:

a) coletar apenas o necessário para a finalidade.
b) comprimir todos os dados em arquivos zip.
c) coletar o maior volume possível de dados.
d) guardar os dados para sempre.

Questão 10. Aprendizado centralizado é aquele em que:

a) os dados ficam com cada cliente.
b) o servidor nunca vê o modelo.
c) os dados são reunidos em um só lugar para o treino.
d) vários modelos são treinados sem comunicação.

Questão 11. Qual cenário descreve melhor o cross-device?

a) Poucos hospitais sempre conectados.
b) Milhões de celulares, cada um com poucos dados e disponibilidade instável.
c) Um único computador com vários núcleos.
d) Dois bancos com servidores potentes.

Questão 12. Qual cenário descreve melhor o cross-silo?

a) Dispositivos que participam uma única vez.
b) Milhões de relógios inteligentes.
c) Um servidor sem clientes.
d) Poucas organizações com muitos dados e boa infraestrutura.

Questão 13. Em geral, em cada rodada do cenário cross-device:

a) só o servidor treina.
b) todos os milhões de clientes participam.
c) apenas uma fração dos clientes participa.
d) nenhum cliente participa.

Questão 14. Na fórmula $f(w) = \Sigma\ (n_k / n) \cdot F_k(w)$, o termo n_k / n representa:

a) o número de rodadas.
b) a probabilidade de falha do cliente.
c) a taxa de aprendizado.
d) o peso do cliente k, proporcional ao tamanho do seu conjunto de dados.

Questão 15. Na mesma fórmula, $F_k(w)$ é:

a) o número de clientes.
b) o ruído adicionado ao modelo.
c) o tamanho do modelo.
d) o erro do modelo w nos dados do cliente k.

Questão 16. O artigo que apresentou o algoritmo FedAvg e popularizou o termo "aprendizado federado" é:

a) Hastie et al. (2009).
b) McMahan et al. (2017).
c) Dwork e Roth (2014).
d) Pedregosa et al. (2011).

Questão 17. A sigla FedAvg significa:

a) Fed Average Gradient.
b) Federated Averaging.
c) Federated Average Value.
d) Federal Averaging.

Questão 18. Qual a principal diferença entre aprendizado distribuído em data center e aprendizado federado?

a) O distribuído usa apenas celulares.
b) No data center os dados são controlados e embaralhados; no federado ficam com seus donos e são diferentes entre si.
c) O federado não usa modelos.
d) Não há diferença.

Questão 19. Dados "não-IID" significam que:

a) os dados foram criptografados.
b) os dados têm distribuições diferentes entre os clientes.
c) os dados são idênticos em todos os clientes.
d) os dados estão incompletos.

Questão 20. A afirmação "no federado a privacidade é automática" é:

a) falsa apenas para celulares.
b) verdadeira apenas para modelos lineares.
c) verdadeira, sempre.
d) falsa, pois atualizações de modelo podem vazar informação.

Questão 21. Qual técnica adiciona ruído para esconder a contribuição de indivíduos?

a) Normalização.
b) Agregação segura.
c) Validação cruzada.
d) Privacidade diferencial.

Questão 22. Qual proteção impede o servidor de ver a atualização individual de cada cliente, vendo só a soma?

a) Regularização L2.
b) Aumento de dados.
c) Agregação segura.
d) Padronização dos dados.

Questão 23. No exemplo dos três hospitais, a média dos modelos funcionou bem. Qual alerta o livro faz?

a) O resultado é otimista: dados IID, problema simples e uma só rodada.
b) O resultado só vale para celulares.
c) A média dos modelos nunca funciona.
d) O resultado vale sempre, para qualquer dado.

Questão 24. Qual conjunto de dados foi usado na primeira simulação do capítulo?

a) MNIST.
b) CIFAR-10.
c) Breast Cancer Wisconsin, da UCI, via load_breast_cancer.
d) ImageNet.

Questão 25. Qual das opções é um exemplo de uso real de aprendizado federado em saúde mencionado no capítulo?

a) Reconhecimento de placas de carro.
b) Tradução de textos jurídicos.
c) Recomendação de filmes.
d) Previsão de necessidade de oxigênio em pacientes com COVID-19 com dados de hospitais em vários continentes.

Questão 26. O que é uma "atualização" de modelo?

a) A troca de servidor.
b) A instalação de uma nova versão do Python.
c) O apagamento dos dados de treino.
d) A mudança nos parâmetros após o treino local.

Questão 27. O que é "agregação"?

a) Criptografar os dados.
b) Combinar as atualizações de vários clientes em um único modelo.
c) Dividir um modelo em partes.
d) Rotular exemplos.

Questão 28. Por que o StandardScaler calculado com todos os dados de treino é uma "trapaça" na simulação?

a) Porque deixa o modelo mais lento.
b) Porque exige reunir dados que, em um sistema federado real, não poderiam ser juntados.
c) Porque só funciona com imagens.
d) Porque remove o ruído.

Questão 29. Em um cenário em que dois bancos concorrentes querem melhorar a detecção de fraudes, o principal fator a favor do aprendizado federado é:

a) a vontade de compartilhar clientes entre si.
b) a impossibilidade ou indisposição de entregar os dados a terceiros.
c) a falta de computadores.
d) o excesso de dados rotulados.

Questão 30. Treinar na borda (edge), perto de onde os dados são produzidos, ajuda principalmente a:

a) aumentar o tamanho dos dados.
b) economizar banda e reduzir latência.
c) tornar o modelo menos preciso.
d) eliminar a necessidade de modelos.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique, com suas palavras e com uma analogia do cotidiano, o que é aprendizado federado.
2. Descreva os quatro passos de uma rodada de aprendizado federado.
3. Cite quatro obstáculos ao aprendizado centralizado quando os dados são sensíveis e espalhados.
4. Diferencie cross-device de cross-silo quanto a número de clientes, disponibilidade e volume de dados.
5. Explique a fórmula $f(w) = \Sigma\ (n_k / n) \cdot F_k(w)$ em palavras.
6. Por que aprendizado federado não é o mesmo que aprendizado distribuído em data center?
7. O que significa dizer que dados são não-IID? Dê um exemplo.
8. Por que o aprendizado federado não garante privacidade por si só?
9. Qual a relação entre aprendizado federado e o princípio de minimização de dados da LGPD e do GDPR?
10. Descreva o experimento dos três hospitais e o que ele mostrou.
11. Dê dois exemplos de aplicações de aprendizado federado e explique o que permanece local em cada uma.
12. Por que clientes com mais dados recebem peso maior na agregação?
13. Explique por que o StandardScaler da simulação do capítulo viola a lógica federada.
14. Quais são as vantagens e as limitações do aprendizado federado em relação ao centralizado?
15. O que é um servidor "honesto, mas curioso" e por que isso importa em aprendizado federado?
16. Explique o que significa "levar o modelo até os dados" e dê uma vantagem prática.
17. Cite duas razões pelas quais a comunicação é um gargalo em aprendizado federado.
18. Proponha um cenário no seu contexto (trabalho, estudo ou vida) em que o aprendizado federado seria útil. Identifique clientes, servidor e dados que ficam locais.
19. Por que a introdução de um conjunto de dados IID pode dar uma impressão otimista do desempenho da média de modelos?
20. Resuma em cinco linhas o que você aprendeu neste capítulo.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Conhecendo os dados dos "hospitais"

NÍVEL: BÁSICO

Carregue o conjunto **Breast Cancer Wisconsin** (`load_breast_cancer`). Mostre o formato de X, o nome das duas classes e quantos exemplos há em cada uma. Depois, sorteie **três hospitais** de 60 pacientes cada (use `np.random.default_rng(1)`) e conte quantos tumores benignos e malignos cada hospital possui.

Pergunta: os três hospitais têm proporções de classes parecidas? Por que isso importa no aprendizado federado?

Exercício 2 — Quanto o tamanho do hospital importa?

NÍVEL: BÁSICO

Repita o experimento do capítulo variando o número de pacientes por hospital: 10, 25, 50 e 100. Para cada tamanho, calcule a acurácia média dos três modelos locais e a acurácia do modelo obtido pela **média dos coeficientes**. Padronize os dados e use `random_state=0` na divisão.

Pergunta: quando os hospitais são muito pequenos, o que acontece com a diferença entre treinar sozinho e colaborar?

Exercício 3 — Repetindo para não se enganar com o acaso

NÍVEL: INTERMEDIÁRIO

Um único sorteio pode enganar. Repita o experimento com 50 pacientes por hospital em **30 sementes diferentes** (muda a divisão dos hospitais). Calcule a média e o desvio padrão das acurácias de: (a) modelo local, (b) média dos modelos e (c) modelo centralizado com os mesmos 150 pacientes.

Pergunta: a média dos modelos é, em média, melhor que o local? E em relação ao centralizado?

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** *Heart Disease*, *Diabetes (Pima Indians)* e *Adult*, ótimos para simular clínicas e bancos.
> - **Kaggle:** *Breast Cancer Wisconsin (Diagnostic)* e *Heart Failure Prediction*.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Rede de Clínicas: vale a pena colaborar?

Contexto. Cinco clínicas pequenas de uma rede (fictícia) querem um modelo de **apoio à triagem**. Como não temos dados reais, vamos simular com o conjunto **Wine** da UCI (`load_wine`, 178 amostras, 13 atributos, 3 classes), usando-o como substituto didático. Cada clínica possui poucos exemplos e não pode compartilhá-los. A diretoria pergunta: *"Vale a pena colaborar sem juntar os dados?"*

Tarefas.

1. Padronize os dados e separe um conjunto de teste de 30%.
2. Reparta o treino em cinco clínicas de tamanhos iguais.
3. Treine uma `LogisticRegression` em cada clínica e meça a acurácia no teste.
4. Faça a média dos modelos (coeficientes e interceptos) e meça a acurácia.
5. Compare com o modelo centralizado.
6. Repita com 20 sementes e relate média e desvio padrão.
7. Escreva um parágrafo de recomendação para a diretoria, citando **limites** do estudo.

Entregáveis. Código comentado, uma tabela de resultados e o parágrafo de recomendação.

A solução de referência está no Apêndice E.



Capítulo 2 — Preparando o Terreno: Dados, Modelos e Otimização

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar o que são exemplos, atributos, rótulos, modelo, parâmetros e função de perda.
- Descrever o gradiente descendente e o SGD com mini-lotes, e o papel da taxa de aprendizado.
- Reconhecer sobreajuste e usar treino, validação e teste.
- Calcular métricas de classificação, incluindo F1, e saber quando a acurácia engana.
- Calcular estatísticas globais, como média e desvio padrão, de forma federada.
- Usar `partial_fit` e `warm_start` do scikit-learn para pausar e retomar o treino.

2.1 Por que revisar isto agora?

Aprendizado federado é **aprendizado de máquina feito em equipe**. Para entender a equipe, você precisa conhecer bem o treino individual.

Neste capítulo, revisamos as peças essenciais. Se você já conhece o assunto, aproveite para fazer o resumo e os exercícios. Se não conhece, vá com calma: tudo é explicado do zero, com exemplos do dia a dia.

> **No seu dia a dia**
>
> Pense em aprender a cobrar pênaltis. Você chuta, vê onde a bola foi, corrige um pouco a mira e chuta de novo. Treinar um modelo é parecido: tentar, medir o erro e ajustar.

2.2 Dados, modelo e erro

2.2.1 Os ingredientes

No **aprendizado supervisionado**, temos exemplos com a resposta correta.

- **Exemplo:** uma linha da tabela de dados, como um paciente.
- **Atributo** (*feature*): uma coluna, como idade ou tamanho do tumor.
- **Rótulo** (*label*): a resposta correta, como "benigno" ou "maligno".

Chamamos de **X** a tabela de atributos e de **y** os rótulos.

O **modelo** é uma função com números ajustáveis, os **parâmetros** (ou pesos), que chamaremos de **w**. Dado um exemplo x, o modelo produz uma previsão. Aprender é encontrar bons valores para **w**.

2.2.2 A função de perda

Como saber se o modelo está bom? Medimos o erro com uma **função de perda** (*loss*). Ela dá uma nota para cada previsão: nota baixa é bom, nota alta é ruim.

Para classificação, uma perda muito usada é a **entropia cruzada**. Para um exemplo cuja classe correta é *c*, ela vale:

$$\text{perda} = -\log(\, p_c \,)$$

Em palavras: *p_c* é a probabilidade que o modelo deu à classe correta. Se ele deu 0,9, a perda é pequena (cerca de 0,11). Se deu 0,1, a perda é grande (cerca de 2,30). **Quanto mais confiante no acerto, menor a perda.**

2.2.3 Risco empírico: a média das perdas

O erro total é a média das perdas em todos os exemplos:

$$F(w) = (1/n) \cdot \Sigma_i \, \text{perda}(\, w \,;\, x_i \,,\, y_i \,)$$

Este número, chamado de **risco empírico**, é o que queremos reduzir. Treinar significa **minimizar F(w)**.

Observe a conexão com o Capítulo 1. Lá, o erro global era uma média ponderada dos erros de cada cliente. Cada *F_k(w)* é o risco empírico calculado só com os dados do cliente *k*.

2.3 Gradiente descendente: descendo a montanha

2.3.1 A analogia da neblina

Você está no alto de uma montanha, com neblina densa. Quer chegar ao vale, mas só enxerga o chão sob seus pés. O que fazer?

Sinta para onde o chão **desce mais** e dê um passo nessa direção. Repita.

Isso é o **gradiente descendente** (*gradient descent*). O **gradiente** é o vetor que aponta a direção em que o erro **mais sobe**. Andamos no sentido oposto:

$$w \leftarrow w - \eta \cdot \nabla F(w)$$

Aqui, ∇F(w) é o gradiente, e **η** (letra grega "eta") é a **taxa de aprendizado**: o tamanho do passo.

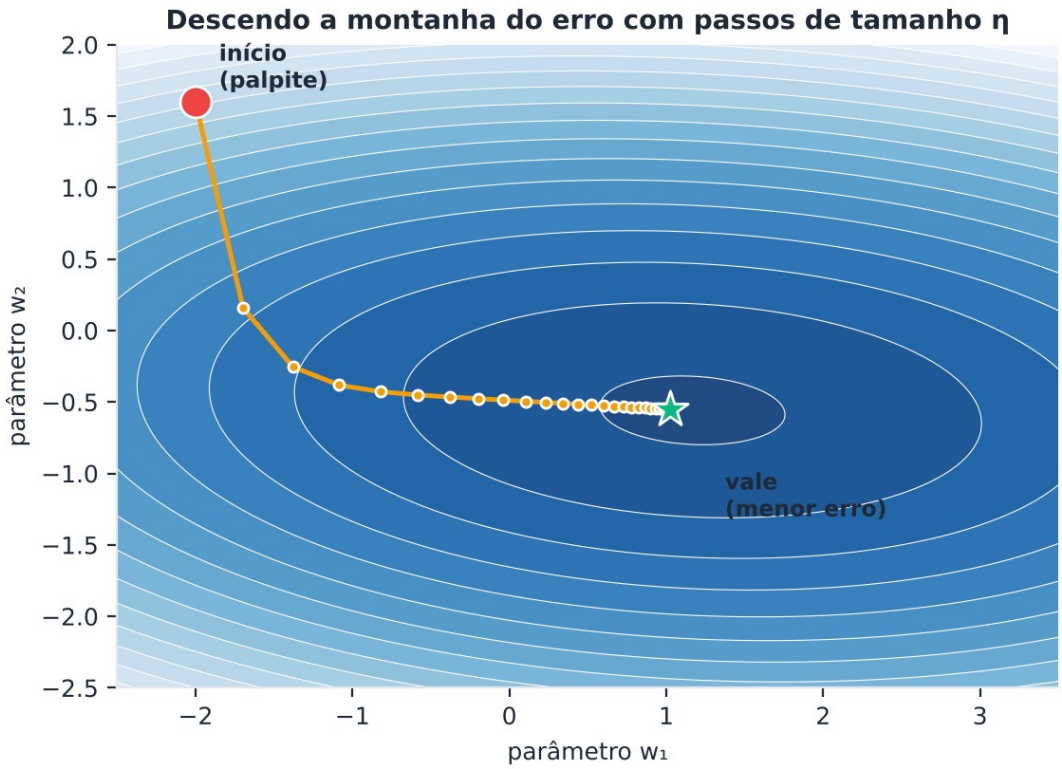


Figura 2.1 — O gradiente descendente dá passos em direção ao vale do erro. Cada ponto é um valor dos parâmetros.

2.3.2 A taxa de aprendizado

O tamanho do passo faz muita diferença.

- **Muito pequeno:** você chega ao vale, mas demora muito.
- **Adequado:** você chega rápido e para perto do vale.
- **Muito grande:** você passa por cima do vale e pode até se afastar dele (divergir).

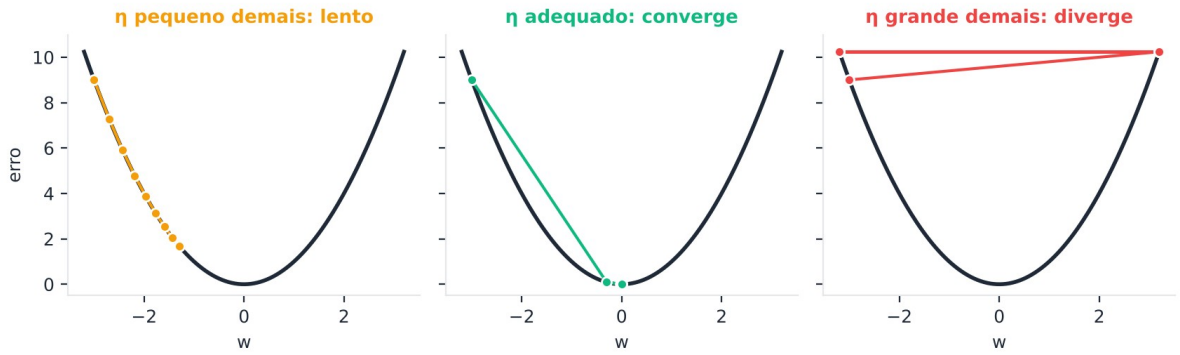


Figura 2.2 — Efeito da taxa de aprendizado em uma montanha simples de uma dimensão.

> **Dica**
>
> Se a perda **sobe** ou fica "louca" durante o treino, quase sempre a taxa de aprendizado está alta demais. Reduza-a pela metade e tente de novo.

2.3.3 Mini-lotes e o SGD

Calcular o gradiente com **todos** os exemplos é caro. A alternativa é usar um pequeno grupo, um **mini-lote** (*mini-batch*), de, digamos, 32 exemplos.

O gradiente fica um pouco "tremido", pois é uma estimativa. Mas é barato e, na média, aponta para o lugar certo. Esse método é o **gradiente descendente estocástico** (*stochastic gradient descent*, **SGD**). "Estocástico" quer dizer "com sorteio", pois os exemplos do lote são sorteados.

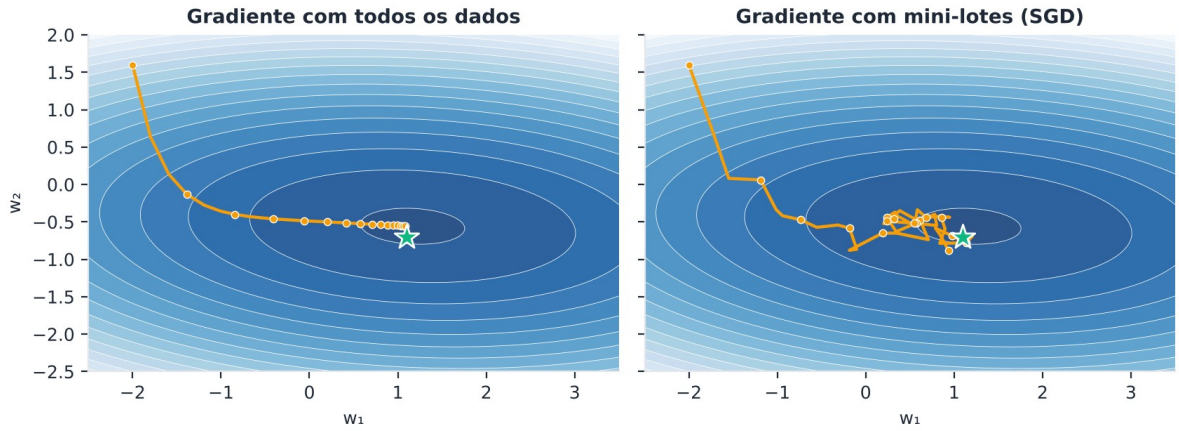


Figura 2.3 — Com todos os dados, o caminho é suave. Com mini-lotes, ele treme, mas chega à mesma região.

Dois termos aparecem o tempo todo no livro.

- **Época:** uma passada completa por todos os exemplos de treino.
- **Passo** (ou iteração): uma atualização dos parâmetros com um mini-lote.

Se há 1.000 exemplos e o lote tem 50, uma época tem **20 passos**.

> **Memorize**
>
> - Gradiente: direção em que o erro mais sobe. Andamos no sentido contrário.
> - η (taxa de aprendizado): tamanho do passo.
> - SGD: gradiente calculado em mini-lotes sorteados.
> - Época: uma passada por todos os dados. Passo: uma atualização.

2.3.4 Um gradiente descendente do zero

Vamos implementar o gradiente descendente para uma **regressão logística** em duas classes. Usaremos o conjunto *Breast Cancer Wisconsin*, da UCI.

A regressão logística calcula uma nota $z = x \cdot w + b$, e transforma-a em probabilidade com a função **sigmoide**, $p = 1 / (1 + e^{-z})$. Quanto maior z, mais próxima de 1 é a probabilidade.

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
esc = StandardScaler().fit(Xtr)
```

```
Xtr, Xte = esc.transform(Xtr), esc.transform(Xte)

def sigmoide(z):
    return 1 / (1 + np.exp(-z))

def perda_e_gradiente(w, b, X, y):
    p = sigmoide(X @ w + b)
    perda = -np.mean(y * np.log(p + 1e-12) + (1 - y) * np.log(1 - p + 1e-12))
    erro = p - y                        # diferença entre previsão e resposta
    return perda, X.T @ erro / len(y), erro.mean()

w, b, eta = np.zeros(X.shape[1]), 0.0, 0.1
for passo in range(201):
    perda, gw, gb = perda_e_gradiente(w, b, Xtr, ytr)
    w, b = w - eta * gw, b - eta * gb
    if passo % 50 == 0:
        acc = np.mean((sigmoide(Xte @ w + b) > 0.5) == yte)
        print(f"passo {passo:3d} | perda = {perda:.4f} | acurácia no teste = {acc:.3f}")
```

Repare que a perda **diminui** a cada passo. Esse é o sinal de que o treino está funcionando.

2.4 Generalizar: o que importa é o dado novo

2.4.1 Sobreajuste

Um aluno que decora as respostas da prova de treino tira 10, mas vai mal em uma prova nova. Em aprendizado de máquina, isso se chama **sobreajuste** (*overfitting*): o modelo "decora" os exemplos de treino e não **generaliza** para dados novos (Hastie; Tibshirani; Friedman, 2009).

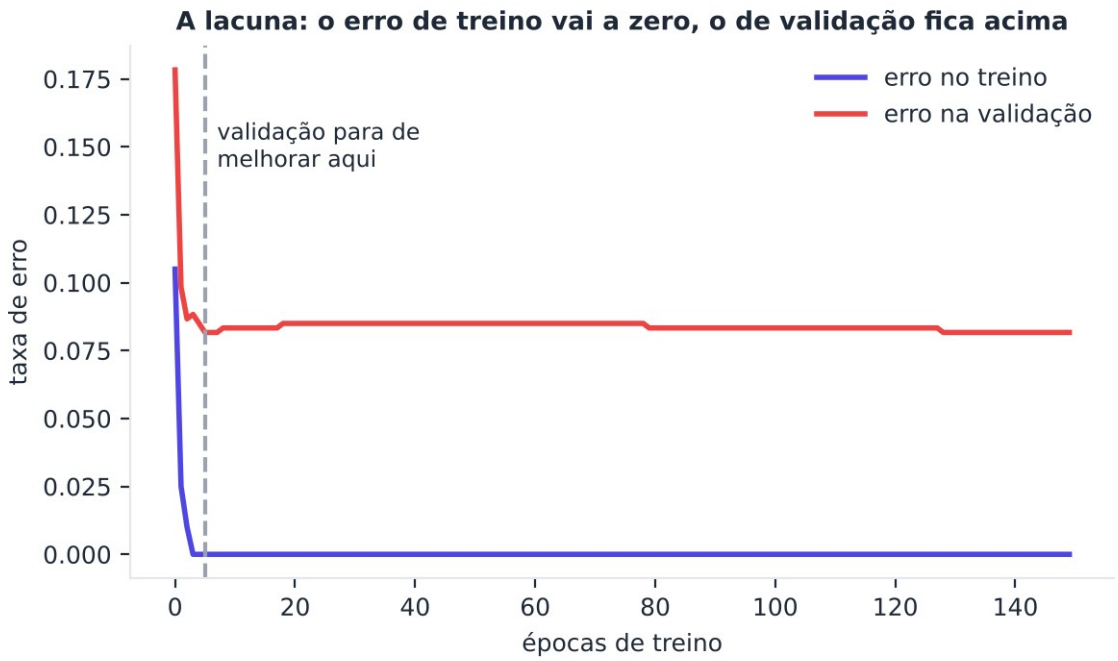


Figura 2.4 — O erro de treino chega a zero, mas o erro em dados novos (validação) fica acima. Essa lacuna indica sobreajuste.

2.4.2 Treino, validação e teste

Para medir de forma honesta, separamos os dados em três partes:

- **Treino:** ajusta os parâmetros.

- **Validação:** escolhe hiperparâmetros, como a taxa de aprendizado. **Hiperparâmetros** são as "regulagens" que não são aprendidas, mas escolhidas por nós.
- **Teste:** dá a nota final. Use **uma única vez**.

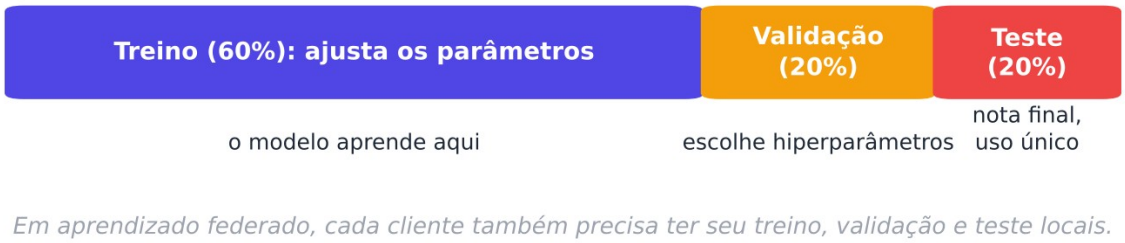


Figura 2.5 — Uma divisão típica dos dados. No federado, cada cliente guarda suas próprias partes.

> **Atenção**
>
> Nunca escolha hiperparâmetros olhando o conjunto de teste. Isso "vaza" informação do teste para o modelo e deixa o resultado otimista demais.

2.4.3 Regularização

Uma forma de combater o sobreajuste é a **regularização**: penalizar parâmetros muito grandes. Na **regularização L2**, somamos à perda um termo $\lambda \|w\|^2$. O hiperparâmetro λ (lambda) controla a força da penalização. Modelos mais "contidos" tendem a generalizar melhor.

2.5 Medindo a qualidade

2.5.1 Acurácia e seus limites

A **acurácia** é a fração de acertos. É simples, mas pode enganar.

Imagine um banco em que 99% das transações são legítimas. Um modelo que responde "legítima" sempre tem **99% de acurácia**, e é **inútil**: nunca acha uma fraude.

2.5.2 Matriz de confusão, precisão, revocação e F1

A **matriz de confusão** conta acertos e erros em cada classe. Chamando "fraude" de classe positiva:

	Previu fraude	Previu legítima
Era fraude	Verdadeiro positivo (VP)	Falso negativo (FN)
Era legítima	Falso positivo (FP)	Verdadeiro negativo (VN)

Daí saem três métricas:

- **Precisão:** dos que o modelo chamou de fraude, quantos eram fraude? = VP / (VP + FP).
- **Revocação** (*recall*): das fraudes que existiam, quantas o modelo achou? = VP / (VP + FN).
- **F1:** a média harmônica das duas. Só é alta se **ambas** forem altas.

$$F1 = 2 \cdot \text{precisão} \cdot \text{revocação} \,/\, (\,\text{precisão} + \text{revocação}\,)$$

> **Dica**
>
> Em problemas com classes desbalanceadas, olhe sempre **F1 ou revocação**, e não apenas a acurácia. Isso será essencial quando falarmos de clientes com dados diferentes.

2.6 Os modelos usados neste livro

Usaremos dois tipos de modelo.

1. **Regressão logística** (e sua versão para várias classes, a **softmax**). É simples, rápida e fácil de interpretar. O modelo inteiro cabe em uma tabela de números.
2. **Rede neural pequena** (*perceptron* multicamadas, `MLPClassifier`). Tem camadas de "neurônios" e aprende padrões mais complexos.

Em ambos, os parâmetros são listas de números. Isso é crucial: **um modelo federado nada mais é que um vetor de números**, que podemos enviar, receber, somar e dividir.

Modelo	Parâmetros (dígitos 8×8, 10 classes)	Observação
Softmax	64 × 10 + 10 = **650**	Convexo: fácil de otimizar
MLP com 32 neurônios	64 × 32 + 32 + 32 × 10 + 10 = **2.410**	Não convexo: mais flexível

2.7 Treino que pausa e retoma: partial_fit e warm_start

No aprendizado federado, o cliente **recebe** um modelo, treina um pouco e **devolve**. Depois recebe de novo. Precisamos, portanto, de treino que possa **ser retomado** de onde parou.

O scikit-learn oferece duas ferramentas:

- **`partial_fit`:** continua o treino com mais dados, sem reiniciar. Disponível em `SGDClassifier`, `MLPClassifier` e outros.
- **`warm_start=True`:** ao chamar `fit` de novo, parte dos parâmetros atuais.

```python
import numpy as np
from sklearn.datasets import load_digits
from sklearn.linear_model import SGDClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X, y = load_digits(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=0, stratify=y)
esc = StandardScaler().fit(Xtr)
Xtr, Xte = esc.transform(Xtr), esc.transform(Xte)

modelo = SGDClassifier(loss="log_loss", learning_rate="constant", eta0=0.01, random_state=0)
classes = np.arange(10)
for epoca in range(1, 11):
    modelo.partial_fit(Xtr, ytr, classes=classes)
    if epoca in (1, 2, 5, 10):
        print(f"após {epoca:2d} época(s): acurácia no teste = {modelo.score(Xte, yte):.3f}")

print("formato dos parâmetros:", modelo.coef_.shape, "+", modelo.intercept_.shape)
```

Veja como o modelo é só um conjunto de números: `coef_` (pesos) e `intercept_` (vieses). **É isso que o servidor vai enviar e receber.**

2.8 Pré-processamento federado: a padronização

2.8.1 O problema

Muitos modelos funcionam melhor quando os atributos têm escala parecida. A **padronização** subtrai a média e divide pelo desvio padrão de cada atributo.

Mas, em um sistema federado, **ninguém pode olhar todos os dados** para calcular essa média e esse desvio. O que fazer? Lembre da trapaça apontada no Capítulo 1.

2.8.2 A solução: somas locais

A média e a variância globais podem ser calculadas **a partir de somas parciais**. Cada cliente envia apenas três quantidades por atributo:

- n_k: quantos exemplos tem;
- $S1_k = \Sigma\, x$: a soma dos valores;
- $S2_k = \Sigma\, x^2$: a soma dos quadrados.

O servidor soma tudo e calcula:

$$\mu = S1 / n \qquad \sigma^2 = S2 / n - \mu^2$$

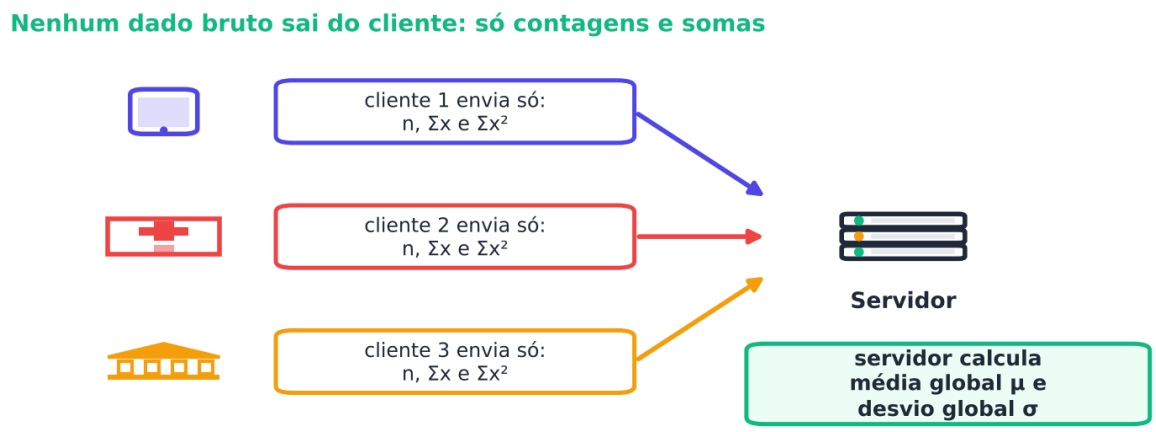


Figura 2.6 — Padronização federada: os clientes enviam só contagens e somas, e o servidor devolve a média e o desvio globais.

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)

# quatro clientes, cada um com um pedaço dos dados de treino
rng = np.random.default_rng(0)
```

```
clientes = np.array_split(rng.permutation(len(Xtr)), 4)

# cada cliente calcula e envia apenas: n, soma e soma dos quadrados
resumos = [(len(c), Xtr[c].sum(axis=0), (Xtr[c] ** 2).sum(axis=0)) for c in clientes]

# o servidor combina os resumos
n = sum(r[0] for r in resumos)
mu = sum(r[1] for r in resumos) / n
sigma = np.sqrt(sum(r[2] for r in resumos) / n - mu ** 2)

# conferência: igual ao StandardScaler que olhou todos os dados?
ref = StandardScaler().fit(Xtr)
print("diferença máxima na média :", np.abs(mu - ref.mean_).max())
print("diferença máxima no desvio:", np.abs(sigma - ref.scale_).max())
```

O resultado é **idêntico** (a menos de erros de arredondamento). Ou seja: conseguimos a padronização global **sem que nenhum dado bruto saísse do cliente**.

Atenção

Esta fórmula simples pode perder precisão numérica quando as médias são muito grandes. Em sistemas reais, usa-se a combinação estável de médias e variâncias parciais (algoritmo de Chan). Além disso, mesmo médias e somas podem revelar informações em grupos muito pequenos. Veremos formas de proteger isso nos Capítulos 8 e 9.

Saiba mais

Esse truque funciona porque média e variância são **estatísticas suficientes** que se combinam por soma. Muitas outras operações do aprendizado federado seguem a mesma lógica: o cliente calcula algo local e o servidor **soma**.

2.9 Resumo do capítulo

MEMORIZE **O essencial do Capítulo 2**

- Um modelo é uma função com parâmetros **w**. Aprender é reduzir a perda média F(w).
- Gradiente descendente: $w \leftarrow w - \eta \nabla F(w)$. **η** é o tamanho do passo.
- SGD usa mini-lotes. **Época** = uma passada pelos dados.
- Sobreajuste: boa nota no treino, má nota em dados novos. Use treino, validação e teste.
- Acurácia pode enganar em classes desbalanceadas. Use **precisão, revocação e F1**.
- Um modelo federado é um **vetor de números** que se envia, recebe e combina.
- `partial_fit` permite retomar o treino. Média e desvio globais saem de **somas locais**.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. No aprendizado supervisionado, os exemplos de treino contêm:

a) apenas recompensas.
b) apenas atributos.
c) apenas rótulos.
d) atributos e rótulos (a resposta correta).

Questão 2. Em uma tabela de dados, um atributo (feature) corresponde a:

a) uma linha.
b) o modelo inteiro.
c) a taxa de aprendizado.
d) uma coluna.

Questão 3. Os parâmetros (pesos) de um modelo são:

a) os exemplos de teste.
b) as métricas finais.
c) os números ajustáveis que o modelo aprende.
d) as regulagens escolhidas pelo programador, como a taxa de aprendizado.

Questão 4. Qual é a função de uma função de perda?

a) Embaralhar os dados.
b) Dividir os dados em treino e teste.
c) Criptografar os parâmetros.
d) Dar uma nota ao erro das previsões do modelo.

Questão 5. Na entropia cruzada, se o modelo dá probabilidade 0,9 à classe correta, a perda é:

a) pequena (cerca de 0,11).
b) negativa.
c) exatamente zero.
d) grande (cerca de 2,3).

Questão 6. Se o modelo dá probabilidade 0,1 à classe correta, a perda de entropia cruzada é:

a) igual a 1.
b) grande (cerca de 2,3).
c) pequena (cerca de 0,1).
d) zero.

Questão 7. O risco empírico F(w) é:

a) o número de épocas.
b) a soma dos parâmetros.
c) a perda do pior exemplo.
d) a média das perdas sobre os exemplos de treino.

Questão 8. O gradiente de F(w) aponta para:

a) a direção em que o erro mais diminui.
b) a direção em que o erro mais aumenta.
c) um ponto aleatório.
d) o valor ótimo dos parâmetros.

Questão 9. Na regra $w \leftarrow w - \eta \cdot \nabla F(w)$, o símbolo η representa:

a) o ruído.
b) o número de clientes.
c) o rótulo.
d) a taxa de aprendizado (tamanho do passo).

Questão 10. O que costuma acontecer quando a taxa de aprendizado é grande demais?

a) O modelo se torna mais privado.
b) Nada muda.
c) O treino pode oscilar ou divergir, com a perda subindo.
d) O treino converge mais devagar, mas sempre chega.

Questão 11. O que costuma acontecer quando a taxa de aprendizado é pequena demais?

a) O modelo esquece os dados.
b) O treino é muito lento.
c) O treino diverge.
d) A perda vira negativa.

Questão 12. O SGD (gradiente descendente estocástico) calcula o gradiente usando:

a) sempre todos os exemplos.
b) os parâmetros do servidor.
c) um mini-lote sorteado de exemplos.
d) apenas os dados de teste.

Questão 13. Se há 1.000 exemplos de treino e o mini-lote tem 50, quantos passos tem uma época?

a) 5.
b) 1.000.
c) 50.
d) 20.

Questão 14. Uma época é:

a) uma atualização de parâmetros.
b) um mini-lote.
c) uma passada completa pelos exemplos de treino.
d) uma rodada de comunicação.

Questão 15. O que é sobreajuste (overfitting)?

a) Os dados têm poucos atributos.
b) O modelo é rápido demais.
c) O modelo vai mal em todos os dados.
d) O modelo vai bem no treino e mal em dados novos.

Questão 16. Para que serve o conjunto de validação?

a) Ajustar os parâmetros do modelo.
b) Dar a nota final, uma única vez.
c) Escolher hiperparâmetros e acompanhar a generalização.
d) Substituir o conjunto de treino.

Questão 17. Por que não se deve escolher hiperparâmetros com base no conjunto de teste?

a) Porque o teste é muito grande.
b) Porque isso deixa o modelo mais lento.
c) Porque vazaria informação do teste, deixando o resultado otimista.
d) Porque o teste é criptografado.

Questão 18. A regularização L2 consiste em:

a) somar à perda uma penalização proporcional ao quadrado dos pesos.
b) aumentar a taxa de aprendizado.
c) usar mais clientes.
d) apagar atributos aleatoriamente.

Questão 19. Um banco tem 99% de transações legítimas. Um modelo que sempre responde "legítima" tem:

a) 100% de revocação para fraudes.
b) 99% de acurácia e nunca detecta fraudes.
c) 50% de acurácia.
d) F1 igual a 1.

Questão 20. A revocação (recall) mede:

a) dos positivos existentes, quantos o modelo encontrou.
b) o número de épocas.
c) a velocidade do modelo.
d) dos que o modelo chamou de positivos, quantos eram de fato positivos.

Questão 21. A precisão mede:

a) a taxa de aprendizado ideal.
b) dos que o modelo chamou de positivos, quantos eram de fato positivos.
c) dos positivos existentes, quantos foram achados.
d) o tamanho do lote.

Questão 22. O F1 é:

a) a média harmônica entre precisão e revocação.
b) a média aritmética entre acurácia e perda.
c) o menor valor de perda.
d) o número de falsos positivos.

Questão 23. Qual a principal vantagem da regressão logística/softmax neste livro?

a) É a mais precisa em qualquer tarefa.
b) Não usa parâmetros.
c) Dispensa dados.
d) É simples, rápida e seu problema de otimização é convexo.

Questão 24. Quantos parâmetros tem uma softmax para dígitos 8x8 (64 atributos) com 10 classes, contando vieses?

a) 2.410.
b) 64.
c) 650.
d) 10.

Questão 25. Por que "um modelo federado é um vetor de números" é uma ideia importante?

a) Porque podemos enviar, receber, somar e dividir modelos como se fossem vetores.
b) Porque modelos não têm parâmetros.
c) Porque vetores são criptografados por padrão.
d) Porque só modelos lineares existem.

Questão 26. Para que serve `partial_fit` no scikit-learn?

a) Dividir os dados em treino e teste.
b) Remover o ruído.
c) Reiniciar o modelo do zero a cada chamada.
d) Continuar o treino com mais dados, sem reiniciar.

Questão 27. O que faz `warm_start=True` em um modelo do scikit-learn?

a) Faz a próxima chamada de fit partir dos parâmetros atuais.
b) Remove o viés.
c) Aquece a CPU antes do treino.
d) Duplica os dados.

Questão 28. Quais quantidades cada cliente envia para calcular média e desvio globais de forma federada?

a) Todos os seus dados brutos.
b) Apenas os rótulos.
c) n, a soma dos valores e a soma dos quadrados.
d) Apenas o nome do cliente.

Questão 29. Na padronização federada, a variância global é calculada como:

a) $S2/n - \mu^2$, com $\mu = S1/n$.
b) A média das variâncias, sempre.
c) $S1 - S2$.
d) $n \cdot \mu$.

Questão 30. Por que a padronização com somas locais é considerada mais adequada ao federado do que a padronização centralizada?

a) Porque dispensa tratar escalas.
b) Porque elimina a necessidade de treino.
c) Porque nenhum dado bruto sai dos clientes.
d) Porque usa menos memória no servidor em qualquer caso.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique, com a analogia da montanha com neblina, o que é o gradiente descendente.
2. Defina função de perda e risco empírico e explique a relação entre eles.
3. Calcule a perda de entropia cruzada (−log) para probabilidades 0,5 e 0,99 atribuídas à classe correta e interprete.
4. Qual o papel da taxa de aprendizado? Descreva o que ocorre se for pequena demais e grande demais.
5. Diferencie SGD de gradiente descendente com todos os dados, citando a vantagem do primeiro.
6. Diferencie época e passo. Quantos passos tem uma época com 2.400 exemplos e lote de 32?
7. Explique o que é sobreajuste e como detectá-lo com os conjuntos de treino e validação.
8. Por que se separam treino, validação e teste? Qual o uso de cada um?
9. Explique a regularização L2 e o papel do hiperparâmetro λ.
10. Por que a acurácia pode enganar? Dê um exemplo e diga qual métrica usar.
11. Explique precisão e revocação com o exemplo de detecção de fraudes.
12. Monte a matriz de confusão para: 8 fraudes detectadas, 2 fraudes perdidas, 4 alarmes falsos e 86 legítimas corretas. Calcule precisão e revocação.
13. Por que se diz que "um modelo federado é um vetor de números"? Qual a consequência prática?
14. Compare softmax e rede neural pequena quanto a número de parâmetros e dificuldade de otimização.
15. Por que o treino federado precisa de modelos que possam ser retomados? Cite uma ferramenta do scikit-learn.
16. Explique como calcular a média global de um atributo sem juntar os dados.
17. Explique como obter o desvio padrão global de forma federada.
18. Mesmo em estatísticas simples como somas, quais riscos de privacidade podem existir?
19. Descreva uma situação cotidiana de "tentar, medir o erro e ajustar" e relacione-a com o treino de um modelo.
20. Resuma em cinco linhas as ferramentas deste capítulo que serão reutilizadas no aprendizado federado.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Taxa de aprendizado em ação

NÍVEL: BÁSICO

Use o código do gradiente descendente do capítulo (regressão logística do zero, conjunto *Breast Cancer Wisconsin*). Treine por 100 passos com $\eta \in \{0{,}001; 0{,}1; 10; 1000\}$ e imprima a perda final de cada um.

Pergunta: qual valor de η foi o pior? Por que valores grandes como 10 ainda funcionaram nesta base?

Exercício 2 — Gradiente completo versus mini-lotes

NÍVEL: BÁSICO

Implemente o SGD com mini-lotes de 16 exemplos para a mesma regressão logística. Compare, após **5 épocas**, a perda e a acurácia no teste do SGD ($\eta = 0{,}05$) com as do gradiente descendente com todos os dados (também $\eta = 0{,}05$, 5 passos).

Pergunta: por que o SGD avançou mais em 5 épocas?

Exercício 3 — Acurácia que engana

NÍVEL: INTERMEDIÁRIO

Crie um conjunto desbalanceado com `make_classification` (2.000 exemplos, 3% de classe positiva, `random_state=0`). Treine um modelo ingênuo (`DummyClassifier(strategy="most_frequent")`), uma `LogisticRegression` e uma `LogisticRegression(class_weight="balanced")`. Mostre acurácia, precisão, revocação e F1 de cada um, e a matriz de confusão da logística comum.

Pergunta: o modelo ingênuo parece bom pela acurácia? E pelo F1? Qual modelo você escolheria se não pudesse deixar passar casos positivos?

Exercício 4 — Padronização federada com clientes de tamanhos diferentes

NÍVEL: INTERMEDIÁRIO

Divida o treino do conjunto *Wine* (`load_wine`) em cinco clientes de **tamanhos diferentes** (por exemplo, usando `np.split` em pontos desiguais). Calcule média e desvio padrão globais usando apenas n, Σx e Σx^2 de cada cliente. Compare com o `StandardScaler` e mostre que o resultado é igual. Mostre também o que ocorre se você **errar** e calcular a média simples das médias locais.

Pergunta: por que a média das médias dá resultado errado quando os clientes têm tamanhos diferentes?

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** *Heart Disease*, *Pima Indians Diabetes* e *Wine Quality* (para treinar validação e métricas).
> - **Kaggle:** *Heart Failure Prediction* e *Credit Card Fraud Detection* (muito desbalanceado, ótimo para praticar F1).
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Triagem com poucos dados: do zero ao modelo

Contexto. Uma clínica (fictícia) tem poucos exemplos e quer montar um modelo de apoio à triagem. Antes de pensar em colaboração, você precisa de um **modelo local bem validado**. Para não depender de downloads, use o conjunto **Breast Cancer Wisconsin** da UCI (`load_breast_cancer`) como substituto didático.

Tarefas.

1. Divida os dados em treino (60%), validação (20%) e teste (20%), com estratificação.
2. Padronize usando **apenas o treino**.
3. Treine uma `SGDClassifier` (perda logística) com `partial_fit`, por 30 épocas, registrando a acurácia de treino e de validação a cada época.
4. Teste três valores de `eta0` (0,001; 0,01; 0,1) e escolha o melhor **pela validação**.
5. Avalie o modelo escolhido **uma única vez** no teste: acurácia, precisão, revocação, F1 e matriz de confusão.
6. Escreva um parágrafo explicando por que o teste foi usado só no final.

Entregáveis. Código, tabela com a validação de cada `eta0`, métricas finais e o parágrafo.

A solução de referência está no Apêndice E.



Capítulo 3 — Anatomia de um Sistema Federado: FedSGD e FedAvg

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Descrever uma rodada de aprendizado federado, passo a passo, com a notação do livro.
- Explicar a diferença entre FedSGD e FedAvg e o papel de C, E, B e η.
- Calcular a agregação por média ponderada.
- Entender por que todos os clientes devem partir do mesmo modelo inicial.
- Usar a biblioteca didática `fedlib.py` para simular clientes e servidor.
- Estimar o custo de comunicação de um treino federado.

3.1 Da ideia ao algoritmo

No Capítulo 1 você aprendeu **o que** é aprendizado federado. No Capítulo 2 revisou **como** um modelo aprende sozinho. Agora juntamos as duas coisas e construímos o algoritmo mais importante da área: o **FedAvg**, proposto por McMahan et al. (2017).

Lembre do objetivo global, a média ponderada dos erros locais:

$$f(w) = \Sigma_k \, (n_k / n) \cdot F_k(w)$$

Queremos achar **w** que minimize *f(w)*, mas ninguém pode somar os dados. O FedAvg resolve isso com a dança da receita de bolo do Capítulo 1: cada cliente melhora o modelo em casa, e o servidor tira a média.

> **No seu dia a dia**
>
> Pense em um trabalho em grupo no Google Docs com versões separadas. A coordenadora envia a mesma versão-base a todos. Cada pessoa edita a sua cópia por algum tempo. A coordenadora junta as edições, ponderando quem escreveu mais. Depois reenvia a nova versão-base. Repetir isso é o FedAvg.

3.2 Anatomia de uma rodada

Vamos observar uma rodada *t* com calma. Os símbolos são poucos.

- ***w_t***: o modelo global no início da rodada *t*.
- ***S_t***: o grupo de clientes sorteados na rodada *t*.
- ***n_k***: quantos exemplos o cliente *k* possui.

A rodada tem **quatro passos**.

1. **Seleção e envio.** O servidor sorteia um subconjunto S_t de clientes e envia w_t a cada um.
2. **Treino local.** Cada cliente k parte de w_t e treina com seus dados por E épocas, usando mini-lotes de tamanho B e taxa de aprendizado η. O resultado é w_{t+1}^k.
3. **Devolução.** Cada cliente devolve w_{t+1}^k (ou a diferença em relação a w_t).
4. **Agregação.** O servidor combina as respostas por **média ponderada**:

$$w_{t+1} = \sum_{k \in S_t} (n_k / n_S) \cdot w_{t+1}^k \qquad \text{com} \quad n_S = \sum_{k \in S_t} n_k$$

Em palavras: cada cliente contribui na proporção do tamanho dos seus dados.

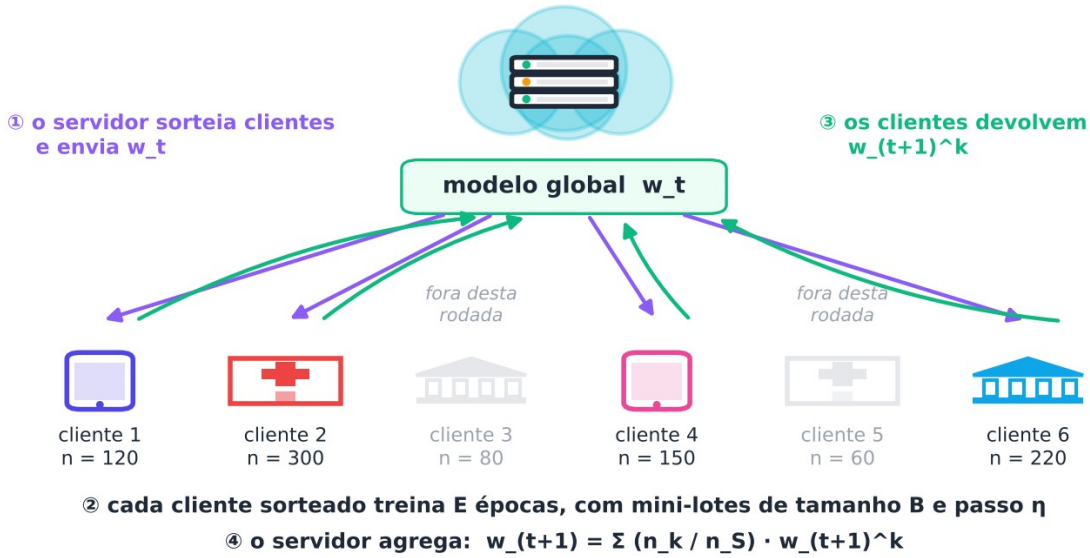


Figura 3.1 — Uma rodada do FedAvg. Os clientes 3 e 5 não foram sorteados e ficam de fora desta rodada.

3.2.1 Um exemplo numérico

Três clientes treinaram e devolveram, para um parâmetro, os valores abaixo.

Cliente	n_k	Valor devolvido
1	100	0,80
2	300	0,60
3	600	0,50

Total: $n_S = 1.000$. O novo valor é:

$w = 0{,}1 \times 0{,}80 + 0{,}3 \times 0{,}60 + 0{,}6 \times 0{,}50 = 0{,}08 + 0{,}18 + 0{,}30 = \mathbf{0{,}56}$

Observe que o resultado fica mais perto de 0,50, valor do cliente com mais dados. Uma média simples daria 0,63.

> **Memorize**
>
> A agregação é uma **média ponderada pelo tamanho dos dados**. O cliente que tem mais exemplos puxa mais o modelo global.

3.3 Os quatro botões do FedAvg

O algoritmo tem quatro hiperparâmetros principais (McMahan et al., 2017).

Símbolo	Nome	O que controla
C	Fração de clientes	Que fração dos clientes participa de cada rodada
E	Épocas locais	Quanto cada cliente treina antes de devolver
B	Tamanho do mini-lote	Tamanho do lote no SGD local
η	Taxa de aprendizado	Tamanho do passo no treino local

Um cálculo útil: quantos passos locais cada cliente dá por rodada? Se o cliente tem *n_k* exemplos, são:

$$u_k = E \cdot n_k / B$$

Com *n_k* = 200, *E* = 5 e *B* = 20, são **50 passos** por rodada.

3.3.1 FedSGD: o caso mais simples

Se cada cliente dá **um único passo** de gradiente (*E* = 1 e *B* igual ao conjunto inteiro), o algoritmo se chama **FedSGD** (*Federated SGD*). Ele é equivalente a um passo de gradiente descendente com todos os dados, e não há diferença para o centralizado.

O problema é o custo: **um passo por rodada**. Como cada rodada exige comunicação, o treino precisa de milhares de rodadas.

3.3.2 FedAvg: fazer mais em casa

O **FedAvg** permite **vários passos locais** antes de comunicar. Ele troca comunicação por computação local. Como o celular ou o hospital costuma ter capacidade de sobra, mas a rede é lenta, **vale a pena**. Os autores reportaram uma redução de ordens de grandeza no número de rodadas em alguns problemas (McMahan et al., 2017).

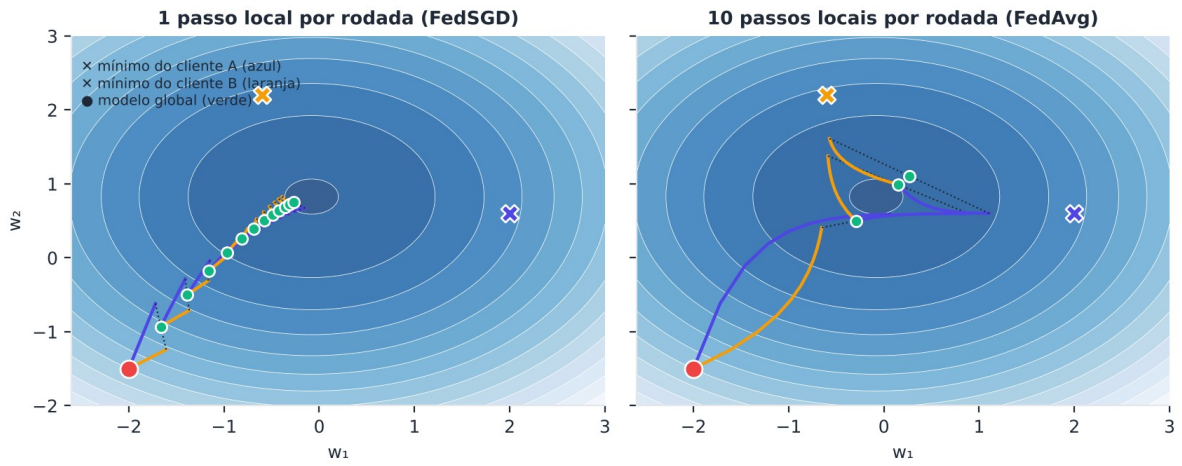


Figura 3.2 — Dois clientes com mínimos diferentes. No FedSGD, o modelo global dá um pequeno passo por rodada. No FedAvg, cada cliente anda bastante em direção ao seu mínimo e depois o servidor faz a média.

> **Saiba mais**
>
> Há um preço: com muitos passos locais, cada cliente caminha para o **seu** mínimo local, e a média pode ficar longe do mínimo global. Esse fenômeno, chamado de **deriva dos clientes** (*client drift*), é maior quando os dados são diferentes entre clientes. É o tema do Capítulo 4.

3.4 Pseudocódigo do FedAvg

PSEUDOCÓDIGO

```
Servidor:
    inicialize w_0
    para cada rodada t = 0, 1, 2, ...:
        S_t ← sorteie max(C·K, 1) clientes
        para cada cliente k em S_t, em paralelo:
            w_k ← TreinoLocal(k, w_t)
        w_(t+1) ← soma sobre k de (n_k / n_S) · w_k

TreinoLocal(k, w):
    divida os dados do cliente k em mini-lotes de tamanho B
    para cada época de 1 até E:
        para cada mini-lote b:
            w ← w − η · gradiente(w; b)
    devolva w
```

3.5 Por que a média funciona? E quando não funciona?

A média de dois modelos pode ser ótima ou péssima. Tudo depende de **de onde eles partiram**.

Se os dois clientes começam do **mesmo** modelo e treinam um pouco, eles ficam "na mesma vizinhança" do espaço de parâmetros. A média cai perto deles, em uma região boa.

Se começam de modelos **inicializados ao acaso, de forma diferente**, as redes neurais podem aprender os mesmos conceitos em "neurônios" trocados de lugar. A média mistura peças que não combinam, e o resultado é ruim. McMahan et al. (2017) observaram exatamente isso: a média só funciona bem quando os modelos compartilham a inicialização.

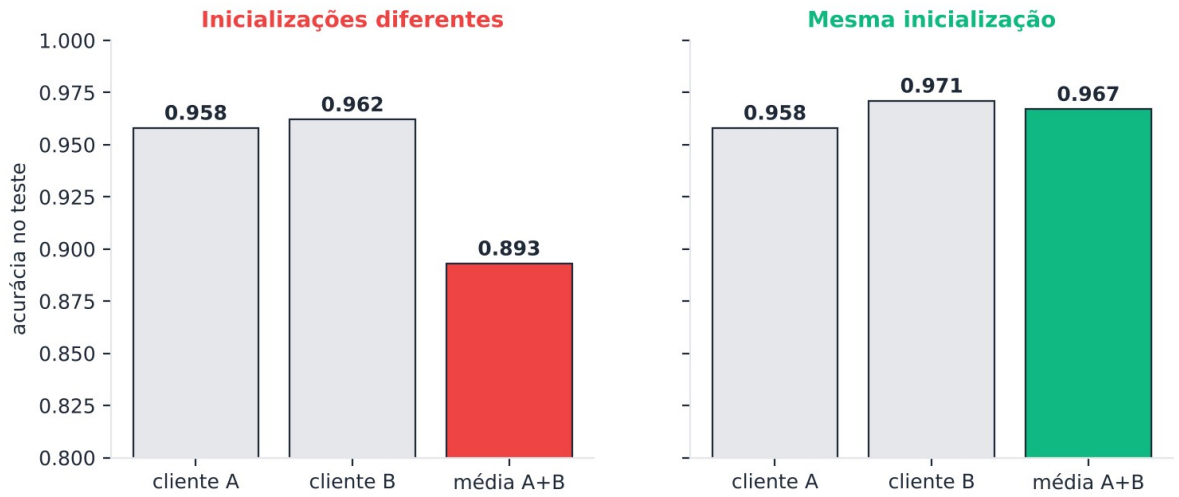


Figura 3.3 — Média de duas redes pequenas treinadas em metade dos dígitos cada. Com inicializações diferentes, a média perde cerca de 7 pontos. Com a mesma inicialização, ela fica no nível dos clientes.

> **Dica**
>
> **Regra de ouro:** em toda rodada, o servidor envia **o mesmo modelo** a todos. Na primeira rodada, o modelo inicial w_0 também deve ser único.

3.6 A biblioteca fedlib.py

Para o restante do livro, usaremos uma pequena biblioteca didática que simula clientes e servidor em **um só computador**. Ela contém:

- carregamento do conjunto de **dígitos manuscritos** da UCI (via scikit-learn), já padronizado;
- três formas de **repartir os dados** entre clientes (IID, por *shards* e por Dirichlet);
- o **treino local** (SGD de mini-lotes em uma regressão logística multiclasse);
- o **FedAvg** completo.

Salve o código abaixo em um arquivo chamado `fedlib.py`, na mesma pasta dos seus exercícios. Você o encontra também no Apêndice C.

```python
"""fedlib.py - biblioteca didática do livro 'Aprendizado Federado'.
Simula clientes e servidor em um único computador, usando NumPy e scikit-learn."""
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler


def carregar_digits(seed=0):
    """Dígitos manuscritos da UCI (via scikit-learn), padronizados, com coluna de viés."""
    X, y = load_digits(return_X_y=True)
    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=seed, stratify=y)
    esc = StandardScaler().fit(Xtr)
    mais_um = lambda A: np.hstack([esc.transform(A), np.ones((len(A), 1))])
    return mais_um(Xtr), mais_um(Xte), ytr, yte


def particao_iid(y, k, seed=0):
    """Embaralha tudo e reparte em k clientes de tamanho parecido."""
    idx = np.random.default_rng(seed).permutation(len(y))
    return [np.array(p) for p in np.array_split(idx, k)]


def particao_shards(y, k, por_cliente=2, seed=0):
    """Ordena por classe, corta em k*por_cliente pedaços e sorteia pedaços para cada cliente."""
    rng = np.random.default_rng(seed)
    pedacos = np.array_split(np.argsort(y, kind="stable"), k * por_cliente)
    ordem = rng.permutation(len(pedacos))
    return [np.concatenate([pedacos[ordem[i * por_cliente + j]] for j in range(por_cliente)]) for i in
range(k)]


def particao_dirichlet(y, k, alfa=0.5, seed=0):
    """Cada classe é repartida entre os clientes com proporções sorteadas de uma Dirichlet(alfa)."""
    rng = np.random.default_rng(seed)
    partes = [[] for _ in range(k)]
    for c in np.unique(y):
        ic = rng.permutation(np.where(y == c)[0])
        p = rng.dirichlet([alfa] * k)
        cortes = (np.cumsum(p) * len(ic)).astype(int)[:-1]
        for i, pedaco in enumerate(np.split(ic, cortes)):
            partes[i] += pedaco.tolist()
    return [np.array(p, dtype=int) for p in partes]
```

```
def softmax(Z):
    Z = Z - Z.max(axis=1, keepdims=True)
    E = np.exp(Z)
    return E / E.sum(axis=1, keepdims=True)


def gradiente(W, X, y, mu=0.0, W_ref=None):
    """Gradiente da entropia cruzada (regressão logística multiclasse) + termo proximal opcional."""
    P = softmax(X @ W)
    P[np.arange(len(y)), y] -= 1.0
    g = X.T @ P / len(y)
    if mu > 0 and W_ref is not None:
        g = g + mu * (W - W_ref)
    return g


def treino_local(W, X, y, epocas=1, eta=0.1, lote=32, mu=0.0, seed=0):
    """Treino local do cliente: mini-batch SGD a partir do modelo global W."""
    rng = np.random.default_rng(seed)
    W0, W = W, W.copy()
    for _ in range(epocas):
        ordem = rng.permutation(len(y))
        for i in range(0, len(y), lote):
            b = ordem[i:i + lote]
            W -= eta * gradiente(W, X[b], y[b], mu, W0)
    return W


def acuracia(W, X, y):
    return float(np.mean(np.argmax(X @ W, axis=1) == y))


def perda(W, X, y):
    P = softmax(X @ W)
    return float(-np.mean(np.log(P[np.arange(len(y)), y] + 1e-12)))


def fedavg(partes, X, y, Xte, yte, rodadas=30, epocas=1, eta=0.1, lote=32,
           frac=1.0, mu=0.0, seed=0, W0=None, classes=10):
    """FedAvg: em cada rodada, clientes sorteados treinam localmente e o servidor faz a média
ponderada."""
    rng = np.random.default_rng(seed)
    W = np.zeros((X.shape[1], classes)) if W0 is None else W0.copy()
    historico = []
    for r in range(rodadas):
        k = max(1, int(round(frac * len(partes))))
        escolhidos = rng.choice(len(partes), k, replace=False)
        locais, tamanhos = [], []
        for c in escolhidos:
            if len(partes[c]) == 0:
                continue
            locais.append(treino_local(W, X[partes[c]], y[partes[c]], epocas, eta, lote, mu, seed *
1000 + r * 50 + int(c)))
            tamanhos.append(len(partes[c]))
        p = np.array(tamanhos) / sum(tamanhos)
        W = sum(pi * Wi for pi, Wi in zip(p, locais))
```

```
        historico.append(acuracia(W, Xte, yte))
    return W, historico
```

Alguns detalhes de projeto merecem atenção.

- O modelo é a **regressão logística multiclasse** (softmax) escrita com NumPy. Ele tem apenas 65 × 10 = 650 parâmetros, incluindo o viés. Escolhemos o modelo simples de propósito: o foco é o **protocolo**, e não o modelo.
- `treino_local` recebe o modelo global `W`, faz uma cópia e treina. Isso garante que todos os clientes **partam do mesmo ponto**.
- `fedavg` repete: sortear clientes, treinar localmente, tirar a média ponderada por *n_k*, medir a acurácia no teste.
- A semente (`seed`) torna os experimentos **reprodutíveis**.

> **Atenção**
>
> Isto é uma **simulação**. Em um sistema real, cliente e servidor são programas em máquinas diferentes, que conversam pela rede, com autenticação, criptografia e tolerância a falhas. O Capítulo 11 apresenta frameworks para isso.

3.7 Rodando o FedAvg

Vamos treinar com **10 clientes** que recebem partes aleatórias dos dígitos (dados IID) e comparar com o treino centralizado.

PYTHON

```python
from fedlib import *
from sklearn.linear_model import LogisticRegression

Xtr, Xte, ytr, yte = carregar_digits()
partes = particao_iid(ytr, k=10, seed=0)
print("Exemplos por cliente:", [len(p) for p in partes])

# FedAvg: 5 épocas locais por rodada, 30 rodadas
W, hist = fedavg(partes, Xtr, ytr, Xte, yte, rodadas=30, epocas=5, eta=0.1, lote=32)
for r in (1, 5, 10, 20, 30):
    print(f"rodada {r:2d}: acurácia no teste = {hist[r - 1]:.3f}")

# Referência centralizada: scikit-learn com todos os dados juntos
cen = LogisticRegression(max_iter=3000).fit(Xtr[:, :-1], ytr)
print(f"centralizado (todos os dados): {cen.score(Xte[:, :-1], yte):.3f}")
```

Repare que, em poucas rodadas, o modelo federado chega perto do centralizado, **sem que nenhum dado saísse dos clientes**.

3.8 FedSGD contra FedAvg

Agora comparamos, **com o mesmo número de rodadas**, o FedSGD (um passo de gradiente completo por rodada) e o FedAvg (cinco épocas locais por rodada).

PYTHON

```python
from fedlib import *

Xtr, Xte, ytr, yte = carregar_digits()
partes = particao_iid(ytr, k=10, seed=0)

# FedSGD: uma época com lote gigante = um único passo de gradiente por rodada
_, h_sgd = fedavg(partes, Xtr, ytr, Xte, yte, rodadas=40, epocas=1, eta=0.5, lote=10**6)
# FedAvg: 5 épocas locais, lotes de 32
_, h_avg = fedavg(partes, Xtr, ytr, Xte, yte, rodadas=40, epocas=5, eta=0.1, lote=32)

print("rodada | FedSGD | FedAvg")
for r in (1, 5, 10, 20, 40):
    print(f"{r:6d} | {h_sgd[r - 1]:.3f}  | {h_avg[r - 1]:.3f}")
```

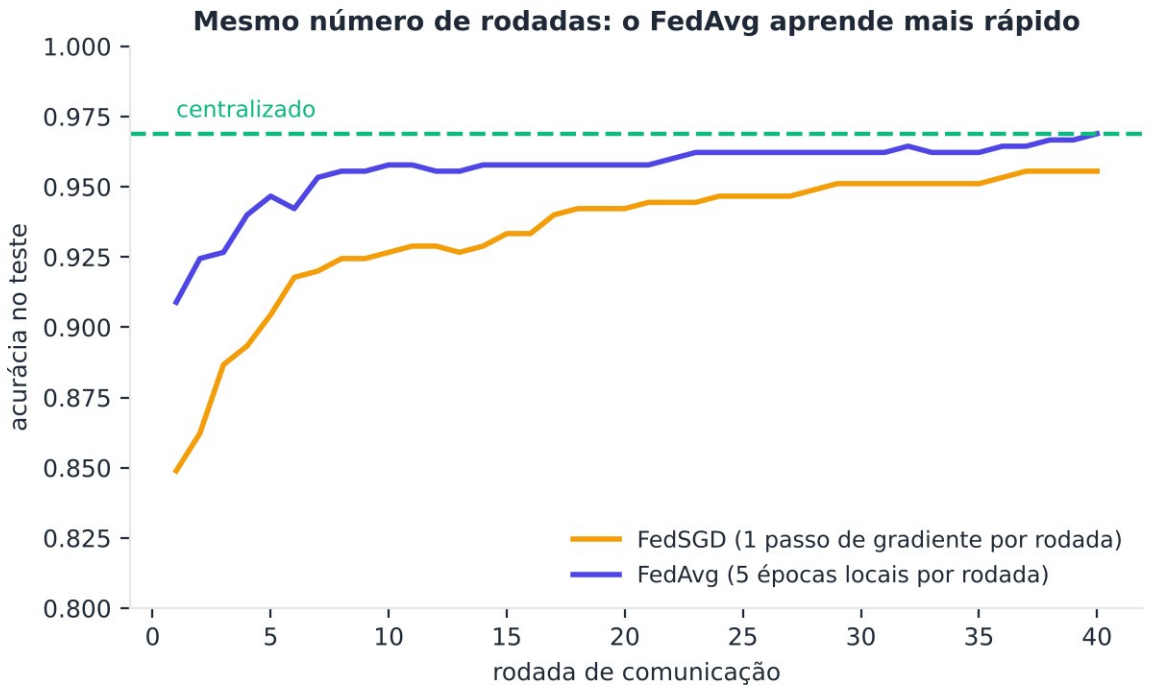


Figura 3.4 — Com o mesmo número de rodadas, o FedAvg aprende mais rápido, pois cada rodada aproveita muito mais cálculo local.

3.9 Efeito de C e E

Qual é o impacto de sortear poucos clientes por rodada (C) ou de treinar mais localmente (E)? Fizemos a experiência com **20 clientes IID**.

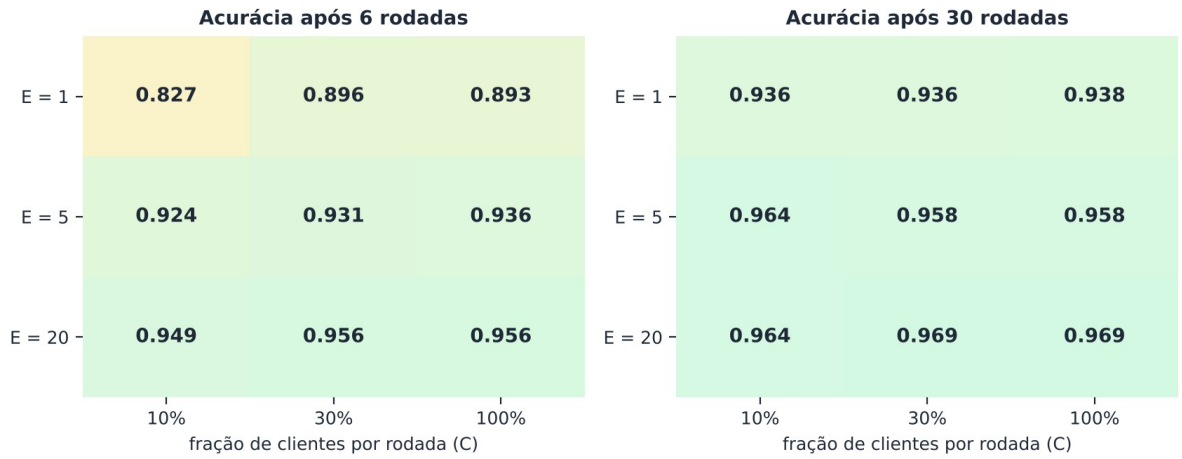


Figura 3.5 — Acurácia após 6 e 30 rodadas, para diferentes valores de E e C. Mais épocas locais aceleram o início do treino. Em dados IID, a fração C faz pouca diferença ao final.

Observações:

- **Aumentar E** acelera o início do treino: com $E = 20$ e 6 rodadas, a acurácia já passa de 0,94.
- **Reduzir C** atrasa um pouco o início quando E é pequeno, mas, ao final, a diferença é pequena quando os dados são IID, pois qualquer subconjunto de clientes é uma boa amostra do todo.
- Em dados **não-IID**, essa conclusão muda bastante. Veremos no Capítulo 4.

> **Dica**
>
> Ao ajustar um sistema federado, comece por **E** (quanto trabalho local) e **η** (passo). Depois ajuste **C**. Compare sempre pelo número de **rodadas**, pois é ele que custa comunicação.

3.10 O custo de comunicação

Cada rodada move dados pela rede. Para estimar o custo, conte os parâmetros.

Em ponto flutuante de 32 bits, cada parâmetro ocupa **4 bytes**. Por rodada, cada cliente sorteado **baixa** o modelo e **sobe** a atualização. Logo:

$$\text{bytes por rodada} \approx 2 \cdot 4 \cdot |w| \cdot (\text{número de clientes sorteados})$$

Para a nossa softmax (650 parâmetros) e 10 clientes:

$2 \times 4 \times 650 \times 10 =$ **52.000 bytes** (cerca de 51 KB) por rodada.

Parece pouco, mas modelos modernos têm milhões ou bilhões de parâmetros. Uma rede com 10 milhões de parâmetros ocupa 40 MB, e cada cliente move 80 MB por rodada. Com milhares de rodadas e milhões de clientes, o custo explode. Por isso o Capítulo 5 é dedicado a **economizar comunicação**.

Modelo	Parâmetros	Tamanho (32 bits)	Tráfego por cliente e rodada
Softmax dos dígitos	650	2,6 KB	5,2 KB
Rede com 1 milhão de parâmetros	1.000.000	4 MB	8 MB
Rede com 100 milhões de parâmetros	100.000.000	400 MB	800 MB

3.11 Variações do lado do servidor

A agregação por média simples não é a única. O servidor pode tratar a diferença média como um "gradiente" e aplicar **momento** ou **métodos adaptativos**. Reddi et al. (2021) propuseram variantes como **FedAdam**, **FedYogi** e **FedAdagrad**, que podem melhorar a convergência em problemas difíceis. O livro volta a elas no Capítulo 4.

3.12 Resumo do capítulo

MEMORIZE **O essencial do Capítulo 3**

- Uma rodada: **sortear → enviar → treinar localmente → devolver → agregar**.
- Agregação do FedAvg: média ponderada por n_k / n_S.
- **C** (fração de clientes), **E** (épocas locais), **B** (lote), **η** (passo).
- **FedSGD** = um passo por rodada. **FedAvg** = vários passos locais: menos rodadas.
- Em redes neurais, todos devem partir do **mesmo** modelo inicial.
- Custo por rodada ≈ 2 × 4 bytes × $|w|$ × clientes sorteados.
- `fedlib.py` simula tudo em um computador e será reutilizada ao longo do livro.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Qual algoritmo foi proposto por McMahan et al. (2017) e é a base do aprendizado federado moderno?

a) SecAgg.
b) DP-SGD.
c) FedAvg (Federated Averaging).
d) K-means federado.

Questão 2. Qual é o primeiro passo de uma rodada do FedAvg?

a) O servidor sorteia clientes e envia o modelo global.
b) O servidor treina com todos os dados.
c) Os clientes enviam seus dados ao servidor.
d) Os clientes apagam seus modelos.

Questão 3. No treino local, cada cliente parte de:

a) um modelo aleatório próprio.
b) o modelo da rodada anterior do próprio cliente, ignorando o servidor.
c) os dados dos outros clientes.
d) o modelo global w_t recebido do servidor.

Questão 4. O que o cliente devolve ao servidor em uma rodada típica?

a) Apenas os rótulos.
b) O modelo atualizado (ou a diferença em relação ao modelo recebido).
c) O nome das classes.
d) Os dados brutos.

Questão 5. Como o servidor combina os modelos dos clientes no FedAvg?

a) Soma os dados dos clientes.
b) Usa o modelo do primeiro cliente.
c) Escolhe o melhor modelo e descarta os demais.
d) Faz a média ponderada pelo número de exemplos de cada cliente.

Questão 6. Em uma rodada, três clientes têm n = 100, 300 e 600 e devolvem, para um parâmetro, 0,80, 0,60 e 0,50. Qual o valor agregado?

a) 0,63.
b) 0,50.
c) 0,56.
d) 0,70.

Questão 7. Se, no exemplo anterior, fosse usada média simples (sem pesos), o valor seria:

a) 0,50.
b) 0,80.
c) 0,63.
d) 0,56.

Questão 8. Dois clientes: o primeiro com 50 exemplos devolve o valor 1,0; o segundo, com 150 exemplos, devolve 0,0. Qual o valor agregado?

a) 0,50.
b) 0,75.
c) 0,25.
d) 0,33.

Questão 9. Em qual situação o FedAvg é equivalente ao FedSGD?

a) Quando há apenas um cliente.
b) Quando E é muito grande.
c) Quando E = 1 e o mini-lote é todo o conjunto local (um passo de gradiente por rodada).
d) Quando $\eta = 0$.

Questão 10. Qual é a principal desvantagem do FedSGD?

a) Exige muitas rodadas, pois dá um passo por rodada.
b) Envia dados brutos.
c) Não usa gradientes.
d) Não funciona com mais de um cliente.

Questão 11. O FedAvg reduz o número de rodadas principalmente porque:

a) elimina a necessidade de gradientes.
b) o servidor usa mais memória.
c) os clientes enviam menos dados.
d) cada cliente faz vários passos locais antes de comunicar.

Questão 12. O que significa o hiperparâmetro C no FedAvg?

a) O tamanho do mini-lote.
b) A fração de clientes sorteados em cada rodada.
c) A taxa de aprendizado.
d) O número de classes.

Questão 13. Em um sistema com K = 100 clientes e C = 0,1, quantos clientes são sorteados por rodada?

a) 100.
b) 1.
c) 10.
d) 50.

Questão 14. Com K = 50 e C = 0,3, quantos clientes participam de cada rodada?

a) 30.
b) 35.
c) 5.
d) 15.

Questão 15. O hiperparâmetro E representa:

a) a estimativa de ruído.
b) o erro do modelo.
c) o número de épocas de treino local por rodada.
d) o número de clientes.

Questão 16. O hiperparâmetro B representa:

a) o tamanho do mini-lote no treino local.
b) o viés do modelo.
c) o número de rodadas.
d) o número de classes.

Questão 17. Um cliente tem 200 exemplos, E = 5 e B = 20. Quantos passos locais ele dá por rodada?

a) 50.
b) 200.
c) 10.
d) 100.

Questão 18. Um cliente com 600 exemplos usa E = 2 e B = 30. Quantos passos locais por rodada?

a) 40.
b) 60.
c) 20.
d) 100.

Questão 19. O que é a "deriva dos clientes" (client drift)?

a) Cada cliente se afasta do mínimo global ao caminhar para o seu mínimo local, e a média fica prejudicada.
b) Os dados são apagados.
c) O servidor perde clientes.
d) Os clientes mudam de endereço IP.

Questão 20. Por que todos os clientes devem partir do mesmo modelo inicial em redes neurais?

a) Porque o servidor exige.
b) Porque isso economiza memória.
c) Porque, com inicializações diferentes, a média pode misturar neurônios que não se correspondem e piorar muito o modelo.
d) Porque a média só funciona com modelos lineares.

Questão 21. Na simulação de duas redes pequenas, a média de modelos com inicializações diferentes teve acurácia:

a) igual a zero.
b) idêntica à do centralizado.
c) acima dos dois clientes.
d) bem abaixo dos dois clientes (cerca de 0,89 contra cerca de 0,96).

Questão 22. A biblioteca fedlib.py, deste livro, é usada para:

a) conectar celulares reais ao servidor.
b) simular clientes e servidor em um único computador.
c) criptografar dados reais.
d) treinar modelos de linguagem.

Questão 23. Qual modelo a fedlib.py usa por padrão?

a) Rede convolucional profunda.
b) Árvore de decisão.
c) K-vizinhos.
d) Regressão logística multiclasse (softmax) escrita com NumPy.

Questão 24. Quantos parâmetros tem a softmax usada para os dígitos na fedlib.py (65 atributos incluindo viés, 10 classes)?

a) 2.410.
b) 6.500.
c) 64.
d) 650.

Questão 25. Em fedlib.py, por que treino_local faz uma cópia do modelo global (W.copy())?

a) Para economizar memória.
b) Para criptografar os pesos.
c) Para apagar o modelo global.
d) Para garantir que cada cliente parta do mesmo modelo global, sem alterá-lo.

Questão 26. Para que serve o parâmetro seed nas simulações?

a) Aumentar a privacidade.
b) Aumentar o tamanho dos dados.
c) Tornar os experimentos reprodutíveis.
d) Reduzir o número de clientes.

Questão 27. Para um modelo de 1.000 parâmetros em 32 bits e 8 clientes sorteados, o tráfego por rodada (subida + descida) é aproximadamente:

a) 64.000 bytes.
b) 8.000 bytes.
c) 32.000 bytes.
d) 128.000 bytes.

Questão 28. Qual das variações abaixo atua no lado do servidor, tratando a diferença média como um gradiente?

a) FedAdam.
b) Validação cruzada.
c) Padronização.
d) K-means.

Questão 29. Qual é a regra de ouro mencionada no capítulo sobre o modelo enviado aos clientes?

a) Enviar modelos diferentes para cada cliente.
b) Enviar o mesmo modelo a todos os clientes da rodada.
c) Nunca enviar modelo.
d) Enviar apenas os dados de teste.

Questão 30. Segundo o capítulo, ao ajustar um sistema federado, por onde convém começar?

a) Por remover clientes.
b) Por aumentar K.
c) Por mudar todos os parâmetros ao mesmo tempo.
d) Por E e η, depois C, comparando pelo número de rodadas.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Descreva os quatro passos de uma rodada do FedAvg usando a notação w_t, S_t e n_k.
2. Por que a agregação usa média ponderada por n_k em vez de média simples?
3. Calcule a agregação para três clientes com n = 20, 30 e 50 que devolvem 2,0, 4,0 e 1,0 para um parâmetro. Mostre as contas.
4. Explique a diferença entre FedSGD e FedAvg e diga por que o FedAvg usa menos rodadas.
5. Por que trocar comunicação por computação local é vantajoso em aprendizado federado?
6. Explique o que são C, E, B e η e dê um efeito esperado de aumentar E.
7. Quantos passos locais um cliente com 450 exemplos dá por rodada com E = 4 e B = 30? Mostre a conta.
8. O que é a deriva dos clientes e por que ela é maior quando os dados são diferentes entre clientes?
9. Explique por que, em redes neurais, a média de modelos com inicializações diferentes tende a falhar.
10. Descreva o experimento da Figura 3.3 e o que ele ensina.
11. Faça uma estimativa do tráfego por rodada para 20 clientes sorteados e um modelo de 500.000 parâmetros de 32 bits.
12. Como o tráfego mudaria no exercício anterior se o modelo tivesse 16 bits por parâmetro?
13. Explique o papel da semente aleatória (seed) em experimentos de simulação federada.
14. Por que a fedlib.py usa uma regressão logística em vez de uma rede profunda? Quais as vantagens e limitações dessa escolha?
15. Por que a comparação entre algoritmos federados deve ser feita por número de rodadas, e não só por épocas?
16. Descreva o efeito de reduzir a fração de clientes C quando os dados são IID.
17. Explique em palavras o objetivo global f(w) = Σ (n_k/n) F_k(w) e como o FedAvg o aproxima sem juntar dados.
18. Cite duas diferenças entre a simulação com fedlib.py e um sistema federado real.
19. Quais estratégias você consideraria para reduzir o custo de comunicação de um treino federado?
20. Resuma em cinco linhas o algoritmo FedAvg e suas limitações.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Sua primeira agregação à mão

NÍVEL: BÁSICO

Implemente a agregação do FedAvg **sem** usar a `fedlib`. Dados três vetores de parâmetros e os tamanhos dos clientes, calcule a média ponderada e compare com a média simples.

Use: `w1 = [0.8, -1.0]`, `w2 = [0.6, 0.0]`, `w3 = [0.5, 2.0]` e `n = [100, 300, 600]`.

Pergunta: de qual cliente o resultado ponderado mais se aproxima? Por quê?

Exercício 2 — Reproduzindo a curva do FedAvg

NÍVEL: BÁSICO

Usando `fedlib`, treine com **10 clientes IID**, 30 rodadas, `epocas=5`, `eta=0.1`, `lote=32`. Imprima a acurácia nas rodadas 1, 5, 10, 20 e 30. Depois repita com `seed=1` e `seed=2`. Calcule a média e o desvio das acurácias finais.

Pergunta: os resultados variam muito com a semente?

Exercício 3 — Variando E e η

NÍVEL: INTERMEDIÁRIO

Com 10 clientes IID e 15 rodadas, teste todas as combinações de `epocas` $\in \{1, 5, 20\}$ e `eta` $\in \{0{,}01; 0{,}1; 0{,}5\}$. Imprima uma tabela com a acurácia final. Depois responda: qual combinação é melhor? Há combinações que desperdiçam computação?

Dica: use `lote=32`.

Exercício 4 — Calculando o custo de comunicação

NÍVEL: INTERMEDIÁRIO

Escreva uma função `bytes_por_rodada(n_parametros, n_clientes, bits=32)` que devolva o tráfego total por rodada (descida + subida). Calcule, para a softmax dos dígitos e 10 clientes, o tráfego por rodada e o total após 30 rodadas. Depois estime quanto seria com 16 bits.

Pergunta: quantas rodadas cabem em 10 MB com 32 bits?

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** *Optical Recognition of Handwritten Digits*, *Letter Recognition* e *Statlog (Landsat Satellite)*.
> - **Kaggle:** *Digit Recognizer* (MNIST) e *Fashion-MNIST*, para testar modelos maiores.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Rede de Hospitais: do central ao federado

Contexto. Dez hospitais (fictícios) querem um modelo para reconhecer dígitos manuscritos em formulários (os dígitos fazem papel de "dados clínicos"). Cada hospital tem cerca de 135 imagens e não pode compartilhá-las. A diretoria quer saber: *"Quantas rodadas são necessárias e quanto vamos gastar em comunicação para chegar perto do modelo centralizado?"*

Tarefas.

1. Treine um modelo centralizado (`LogisticRegression`) como referência.
2. Treine com FedAvg (10 clientes IID, `epocas=5`, `eta=0.1`, `lote=32`) por 40 rodadas.
3. Encontre a **primeira rodada** em que o federado chega a 1 ponto percentual da referência.
4. Calcule o tráfego total até essa rodada (32 bits).
5. Compare com FedSGD (`epocas=1`, lote gigante, `eta=0.5`): em que rodada ele chega ao mesmo limiar, se chegar?
6. Escreva uma recomendação curta, em linguagem simples, para a diretoria.

Entregáveis. Código, tabela com rodada e tráfego para FedAvg e FedSGD, e a recomendação.

A solução de referência está no Apêndice E.

PARTE II

Parte II — Desafios do Treino Federado

O que muda quando os dados de cada participante são diferentes (não-IID), quando a rede é lenta e quando cada pessoa precisa de um modelo só seu.

Nesta parte:

- Cap. 4 — Quando os Dados Não São Iguais: Heterogeneidade (Não-IID)
- Cap. 5 — Falando Menos: Eficiência de Comunicação
- Cap. 6 — Cada Um do Seu Jeito: Personalização



Capítulo 4 — Quando os Dados Não São Iguais: Heterogeneidade (Não-IID)

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar o que significa dados IID e não-IID, com exemplos do cotidiano.
- Reconhecer os principais tipos de heterogeneidade: de rótulos, de atributos, de quantidade e de conceito.
- Simular dados não-IID por *shards* e pela distribuição de Dirichlet.
- Descrever a deriva dos clientes e seus efeitos na convergência.
- Explicar as ideias do FedProx, do SCAFFOLD e do FedNova.
- Comparar algoritmos federados em cenários heterogêneos e interpretar resultados com honestidade.

4.1 Cada cliente é de um jeito

Imagine uma rede de pizzarias. A unidade da praia vende muita pizza de camarão. A do centro financeiro vende mais pizza vegetariana. A do bairro universitário vende pizza grande e barata.

Se cada unidade ajusta a receita com base **só no seu movimento**, as receitas ficam bem diferentes. Se a matriz tirar a média das três, a pizza resultante pode agradar a **ninguém**.

É o problema dos **dados não-IID** (Kairouz et al., 2021).

> **No seu dia a dia**
>
> Seu celular e o do seu vizinho têm dados bem distintos: vocês escrevem de forma diferente, usam outros aplicativos e falam sobre outros assuntos. Um teclado inteligente que aprenda com todos precisa lidar com essas diferenças.

4.2 O que é IID e não-IID

IID significa "independentes e identicamente distribuídos". Em palavras simples: todos os exemplos vêm da **mesma fonte**, como se tivessem sido sorteados de uma única urna.

Em aprendizado centralizado, costumamos embaralhar os dados e assumir IID. No federado, isso quase nunca vale. Cada cliente tem sua própria "urna".

Formalmente, cada cliente k tem uma distribuição de probabilidade $P_k(x, y)$. Os dados são **IID entre clientes** se P_k é a mesma para todos. Caso contrário, são **não-IID**.

Uma distribuição conjunta pode ser lida de dois modos:

$$P(x, y) = P(y) \cdot P(x \mid y) = P(x) \cdot P(y \mid x)$$

Em palavras: *P(y)* diz quão comum é cada rótulo; *P(x | y)* diz como são os atributos de cada rótulo; *P(x)* diz como são os atributos em geral; *P(y | x)* diz qual rótulo costuma acompanhar cada atributo. Cada fatia pode variar entre clientes, e isso gera tipos diferentes de heterogeneidade.

4.3 Os tipos de heterogeneidade

Kairouz et al. (2021) e Li et al. (2022) organizam os casos mais comuns.

Tipo	O que muda entre clientes	Exemplo do dia a dia
Rótulos (*label skew*)	*P(y)*: a frequência de cada classe	Um hospital oncológico vê mais tumores do que uma clínica de rotina
Atributos (*feature skew*)	*P(x)*: como os dados "se parecem"	Fotos tiradas por câmeras diferentes, caligrafias diferentes
Quantidade (*quantity skew*)	*n_k*: tamanho do conjunto local	Um usuário muito ativo e outro que usa pouco o aplicativo
Conceito (*concept shift*)	*P(y | x)*: o mesmo dado, outra resposta	"Bom" para um cliente é "regular" para outro

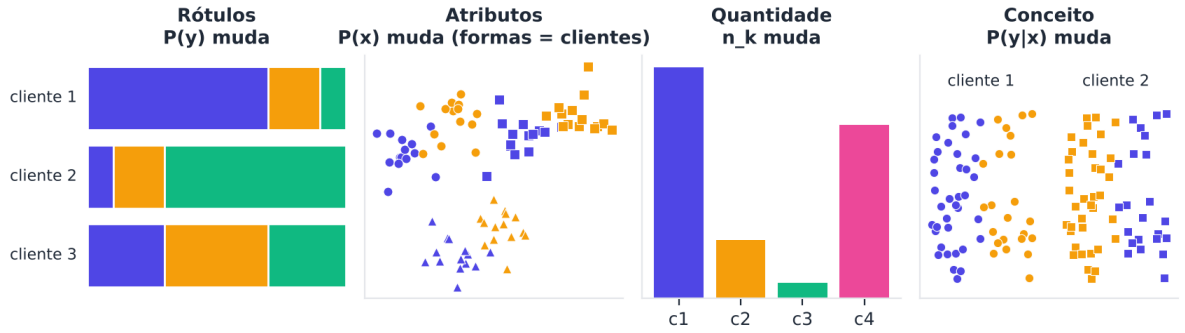


Figura 4.1 — Quatro tipos de heterogeneidade. Nas fatias de rótulos, atributos e quantidade, os clientes diferem em quem são e quantos são. No conceito, os mesmos pontos recebem rótulos diferentes.

> **Saiba mais**
>
> Na prática, os tipos se misturam. Um hospital pequeno de uma cidade do interior pode ter poucos dados (quantidade), pacientes mais velhos (atributos) e mais casos de certa doença (rótulos), tudo ao mesmo tempo.

4.4 Como simular dados não-IID

Para estudar o problema, precisamos de dados com heterogeneidade **controlada**. Há duas receitas muito usadas.

4.4.1 Shards (fatias por classe)

McMahan et al. (2017) ordenam os exemplos por classe, cortam em pedaços (*shards*) e entregam poucos pedaços a cada cliente. Se cada cliente recebe **um** pedaço, ele vê **uma única classe**: o caso extremo.

4.4.2 Dirichlet

A distribuição de **Dirichlet** sorteia proporções que somam 1. Para cada classe, sorteamos como ela será repartida entre os clientes. Um parâmetro **α** (alfa) controla o grau de heterogeneidade (Li et al., 2022):

- **α grande** (por exemplo, 100): repartição quase uniforme, ou seja, dados quase IID.
- **α = 1**: heterogeneidade moderada.
- **α pequeno** (por exemplo, 0,1): cada classe se concentra em poucos clientes, ou seja, dados muito não-IID.

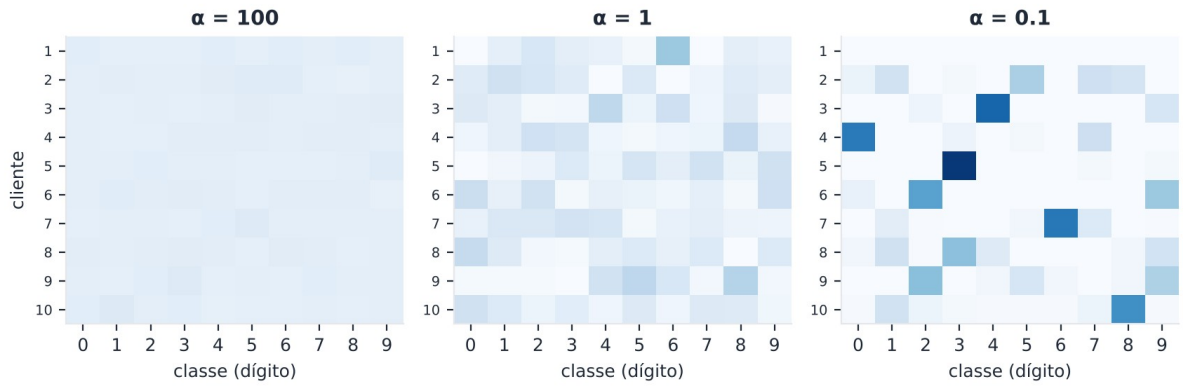


Figura 4.2 — Proporção de cada classe (colunas) em cada cliente (linhas), para três valores de α. Quanto menor o α, mais concentrada é a distribuição.

Vamos medir o efeito com a `fedlib`.

```python
import numpy as np
from fedlib import *

Xtr, Xte, ytr, yte = carregar_digits()
print(f"{'alfa':>6} | classe dominante média | classes por cliente (>= 5 exemplos) | tamanhos")
for alfa in (100, 1.0, 0.1):
    partes = particao_dirichlet(ytr, k=10, alfa=alfa, seed=0)
    dominante, n_classes = [], []
    for p in partes:
        cont = np.bincount(ytr[p], minlength=10)
        dominante.append(cont.max() / max(cont.sum(), 1))
        n_classes.append(int((cont >= 5).sum()))
    print(f"{alfa:>6} | {np.mean(dominante):>22.2f} | {np.mean(n_classes):>36.1f} | {sorted(len(p) for p in partes)}")
```

Observe: com α = 100, cada cliente vê quase todas as classes. Com α = 0,1, cada um vê poucas, e os tamanhos ficam muito desiguais.

4.5 Por que o não-IID atrapalha?

4.5.1 Deriva dos clientes

Cada cliente treina com seus dados e caminha para o **seu** mínimo local. Quanto mais passos locais, mais longe do mínimo global ele chega. Ao tirar a média, o modelo resultante pode ficar longe do ótimo (Karimireddy et al., 2020). Esse fenômeno é a **deriva dos clientes** (*client drift*).

Um exemplo extremo ajuda. Dois clientes, com mínimos locais diferentes. Se cada um treinasse **até convergir**, a média dos modelos seria a média dos dois mínimos locais, que **não é** o mínimo da função global. Wang et al.

(2020b) chamam isso de **inconsistência do objetivo**: com muitos passos locais e quantidades diferentes de passos, o FedAvg converge para um ponto que **não minimiza** o objetivo desejado.

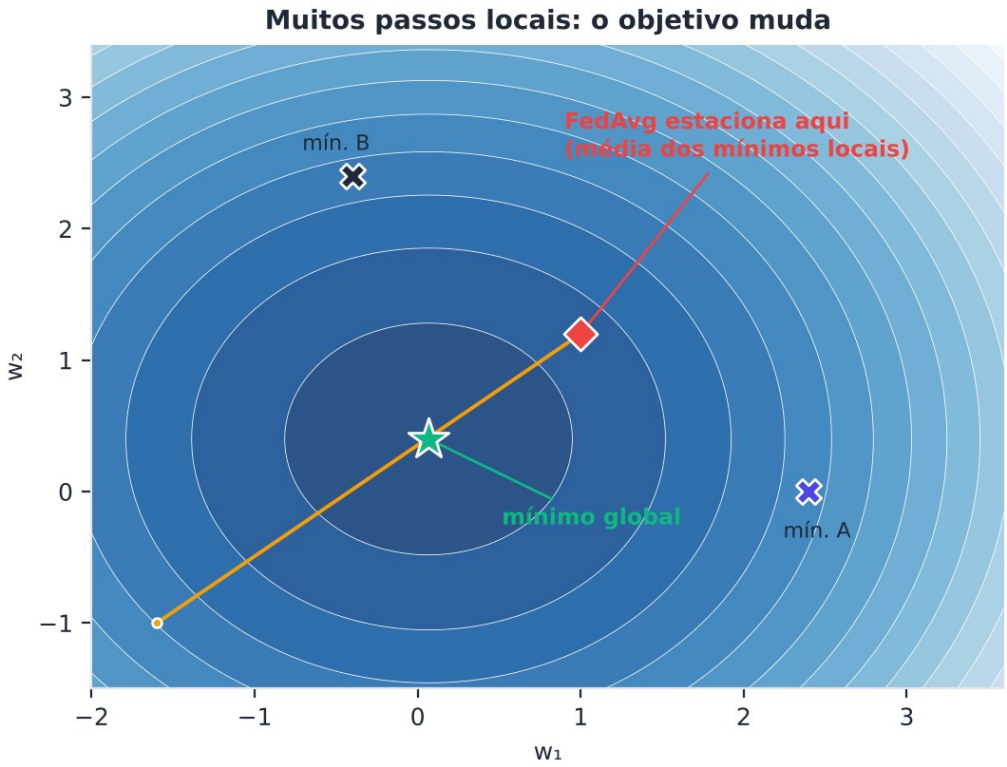


Figura 4.3 — Em um caso de brinquedo, com muitos passos locais, o FedAvg estaciona na média dos mínimos locais (losango), e não no mínimo global (estrela).

4.5.2 Outros efeitos

- **Convergência mais lenta e instável.** A curva de acurácia oscila, sobretudo quando poucos clientes participam de cada rodada.
- **Viés.** Clientes grandes dominam a média. Clientes pequenos ou "diferentes" ficam mal atendidos.
- **Difícil ajustar.** Hiperparâmetros que funcionam em IID podem falhar em não-IID.

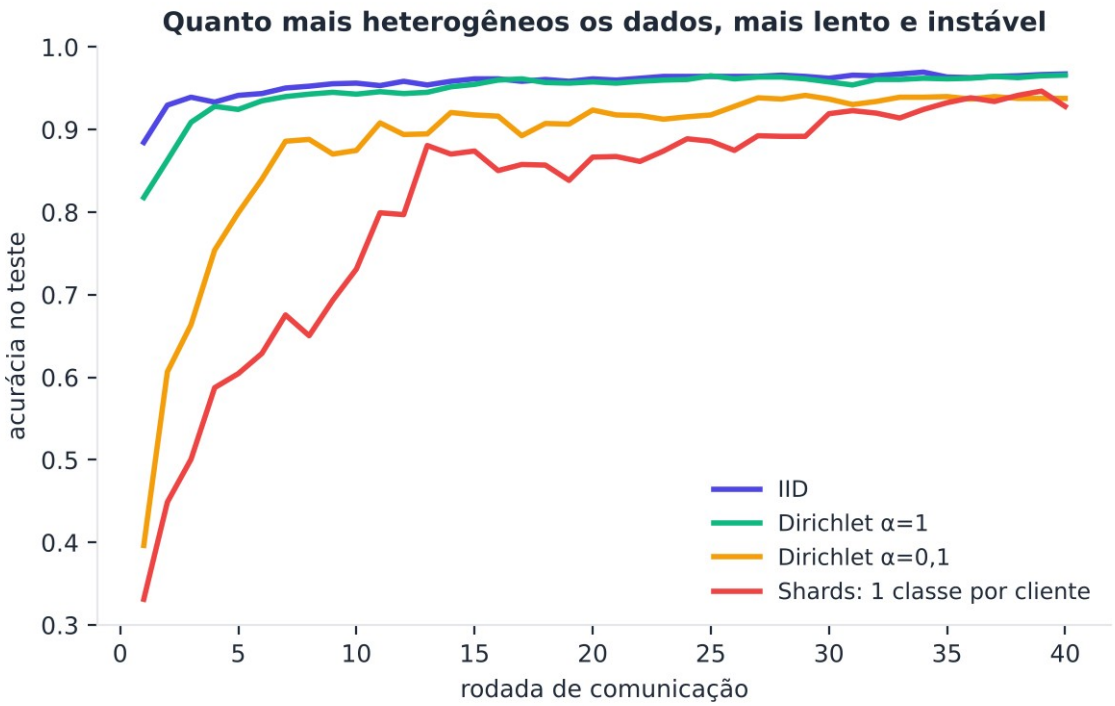


Figura 4.4 — FedAvg com 20 clientes, poucos participantes por rodada (10%), 20 épocas locais e η = 0,3. Quanto mais heterogêneos os dados, mais lenta e instável é a curva. Média de 3 sementes.

> **Atenção**
>
> Com modelos simples e convexos, como a regressão logística, o efeito final costuma ser **moderado**: o modelo chega perto do ideal, mas **demora mais e oscila**. Em redes neurais profundas, as perdas costumam ser maiores. Interprete cada resultado dentro do seu contexto.

4.6 Medindo a heterogeneidade

Antes de combater o problema, é bom medi-lo. Uma medida simples para rótulos é a **distância de variação total** entre a distribuição de classes de um cliente e a global:

$$\mathrm{TV}_k = (1/2) \cdot \Sigma_c \mid p_k(c) - p(c) \mid$$

Ela vai de 0 (idênticas) a 1 (totalmente diferentes). Em palavras: é a fração dos exemplos que "estaria na classe errada" se quiséssemos transformar uma distribuição na outra.

Médias altas de *TV_k* indicam forte heterogeneidade de rótulos.

4.7 Algoritmos para dados heterogêneos

Vários algoritmos foram criados para reduzir a deriva. Três ideias são essenciais.

4.7.1 FedProx: uma "coleira" no cliente

O **FedProx** (Li et al., 2020c) adiciona ao erro local um termo que **penaliza o afastamento** do modelo global:

$$F_k(w) + (\mu / 2) \cdot \| w - w_t \|^2$$

O hiperparâmetro μ ("mu") controla a força da coleira. Com $\mu = 0$, volta-se ao FedAvg. Com μ grande, o cliente quase não sai do lugar. O gradiente do termo extra é $\mu(w - w_t)$, e a `fedlib` já o suporta pelo argumento `mu`.

4.7.2 SCAFFOLD: corrigir a direção

O **SCAFFOLD** (Karimireddy et al., 2020) estima, para cada cliente, **quanto sua direção de descida difere da direção global**, e corrige cada passo local. Para isso, servidor e clientes mantêm **variáveis de controle**: vetores c (do servidor) e c_k (de cada cliente). O passo local vira:

$$w \leftarrow w - \eta \cdot (g(w) - c_k + c)$$

Em palavras: do gradiente do cliente, subtraímos sua tendência individual (c_k) e somamos a tendência global (c). Isso compensa a deriva, ao custo de **comunicar o dobro** de números por rodada, pois também se enviam as variáveis de controle.

4.7.3 FedNova: normalizar o esforço

Se um cliente dá 100 passos e outro dá 10, a média ponderada trata os dois como se tivessem contribuído igual. O **FedNova** (Wang et al., 2020b) **normaliza** cada atualização pelo número de passos locais antes de agregar, o que elimina a inconsistência do objetivo.

4.7.4 Outras ideias

- **Otimizadores no servidor:** FedAdam, FedYogi e FedAdagrad tratam a atualização média como um gradiente e usam métodos adaptativos (Reddi et al., 2021).
- **Personalização:** em vez de um modelo único, cada cliente fica com um modelo ajustado a si (Capítulo 6).
- **Agrupar clientes parecidos:** treinar um modelo por grupo (Capítulo 6).

Algoritmo	Ideia	Custo extra	Quando ajuda
FedProx	Penaliza afastar-se de w_t	Quase nenhum	Dispositivos com esforço variável
SCAFFOLD	Corrige a deriva com variáveis de controle	2× a comunicação; estado nos clientes	Forte heterogeneidade de rótulos
FedNova	Normaliza por número de passos	Quase nenhum	Quantidades de passos diferentes
FedAdam e afins	Otimizador adaptativo no servidor	Memória no servidor	Problemas mal condicionados

4.8 Implementando o SCAFFOLD

Abaixo, uma implementação do SCAFFOLD (variante com "opção II" de atualização dos controles) com a `fedlib`. Salve o código como **`scaffold.py`**, ao lado da `fedlib.py`. Atenção ao ponto crítico: **clientes não sorteados mantêm seu *c_k***.

```python
import numpy as np
from fedlib import *

def scaffold(partes, X, y, Xte, yte, rodadas=40, epocas=10, eta=0.1, lote=32,
             frac=0.25, seed=0, classes=10):
    rng = np.random.default_rng(seed)
```

```
    W = np.zeros((X.shape[1], classes))
    c = np.zeros_like(W)                          # controle do servidor
    ck = [np.zeros_like(W) for _ in partes]       # controle de cada cliente
    historico = []
    for r in range(rodadas):
        m = max(1, int(round(frac * len(partes))))
        escolhidos = rng.choice(len(partes), m, replace=False)
        dW, dC = [], []
        for k in escolhidos:
            idx = partes[k]
            if len(idx) == 0:
                continue
            Xk, yk = X[idx], y[idx]
            w, passos = W.copy(), 0
            rk = np.random.default_rng(seed * 1000 + r * 50 + int(k))
            for _ in range(epocas):
                ordem = rk.permutation(len(yk))
                for i in range(0, len(yk), lote):
                    b = ordem[i:i + lote]
                    w -= eta * (gradiente(w, Xk[b], yk[b]) - ck[k] + c)   # passo corrigido
                    passos += 1
            ck_novo = ck[k] - c + (W - w) / (passos * eta)               # novo controle local
            dW.append(w - W)
            dC.append(ck_novo - ck[k])
            ck[k] = ck_novo
        W = W + np.mean(dW, axis=0)
        c = c + (len(dW) / len(partes)) * np.mean(dC, axis=0)
        historico.append(acuracia(W, Xte, yte))
    return W, historico
```

4.9 Comparando FedAvg, FedProx e SCAFFOLD

Agora o teste mais duro: **20 clientes**, cada um com **uma única classe** (*shards*), **10% de participação** por rodada, 20 épocas locais e η = 0,3. Três sementes.

PYTHON

```
import numpy as np
from fedlib import *

from scaffold import scaffold      # o arquivo salvo na seção 4.8

Xtr, Xte, ytr, yte = carregar_digits()
cfg = dict(rodadas=40, epocas=20, eta=0.3, frac=0.1)
res = {"FedAvg": [], "FedProx (mu=0.1)": [], "SCAFFOLD": []}
for s in range(3):
    partes = particao_shards(ytr, k=20, por_cliente=1, seed=s)
    res["FedAvg"].append(fedavg(partes, Xtr, ytr, Xte, yte, seed=s, **cfg)[1])
    res["FedProx (mu=0.1)"].append(fedavg(partes, Xtr, ytr, Xte, yte, mu=0.1, seed=s, **cfg)[1])
    res["SCAFFOLD"].append(scaffold(partes, Xtr, ytr, Xte, yte, seed=s, **cfg)[1])

print(f"{'método':<18}{'rodada 10':>10}{'rodada 40':>10}{'oscilação (dp últ.10)':>24}")
for nome, hs in res.items():
    h = np.array(hs)
    print(f"{nome:<18}{h[:, 9].mean():10.3f}{h[:, -1].mean():10.3f}{h[:, -
10:].std(axis=1).mean():24.3f}")
```

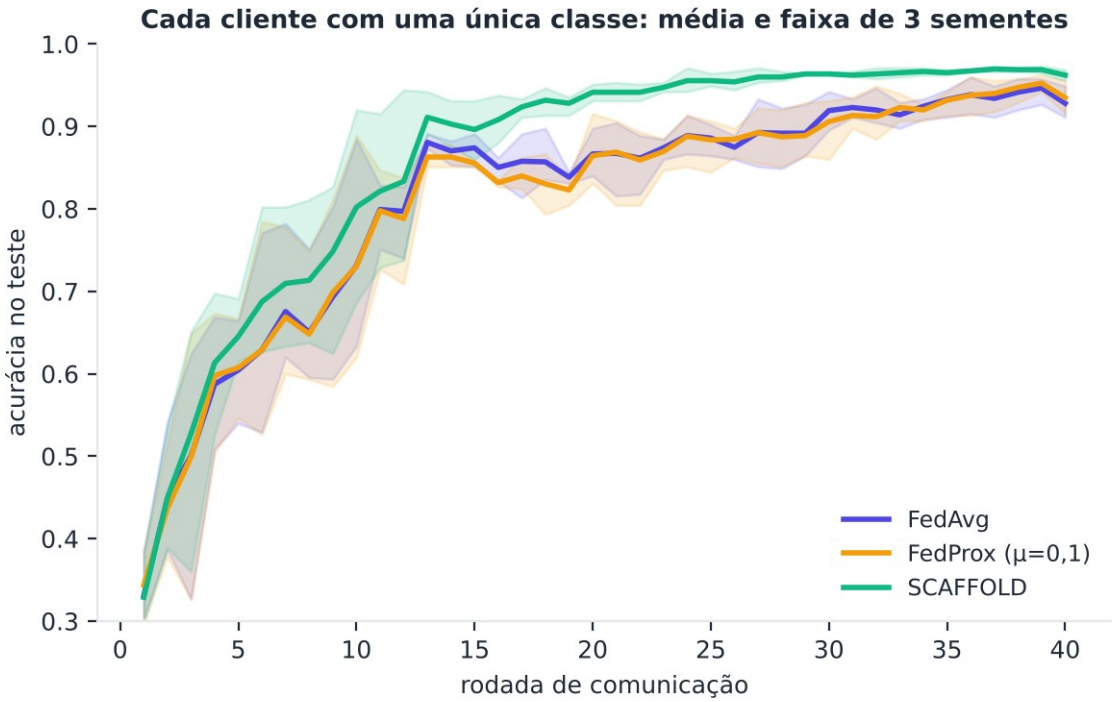


Figura 4.5 — Acurácia por rodada com 20 clientes de uma classe cada, 10% de participação, 20 épocas locais e η = 0,3. O SCAFFOLD é mais rápido e estável; o FedProx não ajudou neste cenário.

O que aprendemos:

- O **SCAFFOLD** foi claramente melhor: mais rápido e com menos oscilação.
- O **FedProx** com $\mu = 0{,}1$ praticamente **não mudou** o resultado. Em modelo convexo e simples, a coleira traz pouco. Com μ grande, atrapalha: mais rigidez, menos progresso.
- Nenhum algoritmo vence em tudo: **teste no seu problema**.

> **Dica**
>
> Sempre compare algoritmos com **várias sementes** e as mesmas condições (mesmos clientes, mesmo número de rodadas). Uma diferença de 0,3 ponto percentual com uma única semente não prova nada.

4.10 Resumo do capítulo

> MEMORIZE **O essencial do Capítulo 4**
>
> - **Não-IID**: cada cliente tem sua própria distribuição de dados.
> - Tipos: **rótulos, atributos, quantidade e conceito**.
> - Simulação: **shards** (por classe) e **Dirichlet(α)**, com α pequeno = muito heterogêneo.
> - Efeitos: **deriva dos clientes**, convergência lenta e instável, inconsistência do objetivo.
> - **FedProx**: coleira μ. **SCAFFOLD**: variáveis de controle (2× comunicação). **FedNova**: normaliza por passos.
> - Resultados dependem do problema: avalie com várias sementes.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. O que significa dizer que os dados dos clientes são IID?

a) Cada cliente tem classes diferentes.
b) Os dados estão criptografados.
c) Os dados têm o mesmo tamanho em todos os clientes, mas distribuições distintas.
d) Todos vêm da mesma distribuição, como se sorteados de uma única urna.

Questão 2. Dados não-IID entre clientes significam que:

a) os dados são idênticos em todos os clientes.
b) cada cliente tem sua própria distribuição de dados.
c) os dados estão incompletos.
d) o modelo é não linear.

Questão 3. No exemplo das pizzarias, o que simboliza o problema do não-IID?

a) As pizzarias não têm clientes.
b) Cada unidade ajusta a receita ao seu público, e a média pode não agradar a ninguém.
c) Todas vendem a mesma pizza.
d) A matriz não tem dados.

Questão 4. A heterogeneidade de rótulos (label skew) ocorre quando:

a) o tamanho do conjunto muda.
b) as fotos têm câmeras diferentes.
c) a frequência de cada classe muda entre clientes, P(y).
d) o mesmo dado recebe rótulos diferentes.

Questão 5. Fotos de um mesmo objeto tiradas por câmeras diferentes ilustram:

a) heterogeneidade de conceito.
b) heterogeneidade de rótulos.
c) dados IID.
d) heterogeneidade de atributos.

Questão 6. A heterogeneidade de quantidade (quantity skew) significa que:

a) os atributos variam.
b) os rótulos variam.
c) o modelo muda.
d) n_k, o tamanho do conjunto local, varia entre clientes.

Questão 7. A mudança de conceito (concept shift) ocorre quando:

a) as classes são iguais em todos.
b) a taxa de aprendizado muda.
c) o mesmo atributo recebe rótulos diferentes em clientes diferentes, P(y|x).
d) há mais exemplos em um cliente.

Questão 8. Na fórmula P(x, y) = P(y)·P(x|y), o termo P(y) descreve:

a) o tamanho do conjunto.
b) quão comum é cada rótulo.
c) o ruído.
d) como são os atributos de cada rótulo.

Questão 9. Na simulação por shards, se cada cliente recebe um único shard (pedaço ordenado por classe), ele vê:

a) uma única classe.
b) apenas dados de teste.
c) todas as classes igualmente.
d) nenhuma classe.

Questão 10. Na distribuição de Dirichlet usada para particionar dados, um α muito grande (por exemplo, 100) produz:

a) cada cliente com uma única classe.
b) uma repartição quase uniforme, ou seja, dados quase IID.
c) classes aleatórias com ruído.
d) clientes sem dados.

Questão 11. Com $\alpha = 0{,}1$, o que se espera dos clientes?

a) Todos têm todas as classes igualmente.
b) Todos têm o mesmo tamanho.
c) Cada classe se concentra em poucos clientes: dados muito heterogêneos.
d) Não há diferença em relação ao $\alpha = 100$.

Questão 12. Na tabela do capítulo ($\alpha = 100$), a classe dominante média de cada cliente foi cerca de:

a) 0,56.
b) 1,00.
c) 0,12.
d) 0,90.

Questão 13. Na tabela do capítulo ($\alpha = 0{,}1$), a classe dominante média foi cerca de:

a) 0,24.
b) 0,56.
c) 1,00.
d) 0,12.

Questão 14. A deriva dos clientes (client drift) é:

a) o afastamento do modelo local em direção ao seu mínimo local durante muitos passos de treino.
b) a perda de conexão do cliente.
c) a mudança de servidor.
d) a troca de rótulos.

Questão 15. A deriva dos clientes tende a ser maior quando:

a) o modelo tem zero parâmetros.
b) há muitos passos locais e dados diferentes entre clientes.
c) os dados são IID e há um passo local.
d) há apenas um cliente.

Questão 16. A inconsistência do objetivo (Wang et al., 2020) significa que:

a) o FedAvg pode convergir para um ponto que não minimiza o objetivo global desejado.
b) a perda é negativa.
c) o servidor não tem modelo.
d) os clientes não têm objetivo.

Questão 17. No exemplo de brinquedo da Figura 4.3, com muitos passos locais, o FedAvg estaciona:

a) no infinito.
b) exatamente no mínimo global.
c) na média dos mínimos locais.
d) no ponto inicial.

Questão 18. Quais efeitos o não-IID costuma causar?

a) Convergência mais lenta e instável, e viés a favor de clientes grandes.
b) Convergência mais rápida e estável.
c) Nenhum efeito.
d) Redução do número de parâmetros.

Questão 19. Com modelos convexos e simples, o efeito final do não-IID costuma ser:

a) moderado: o modelo demora mais e oscila, mas chega perto.
b) nulo em todos os casos.
c) sempre favorável.
d) catastrófico em todos os casos.

Questão 20. A distância de variação total TV_k entre a distribuição de classes do cliente e a global varia entre:

a) 0 e infinito.
b) 1 e 10.
c) -1 e 1.
d) 0 e 1.

Questão 21. O FedProx adiciona ao erro local:

a) um termo que aumenta o passo.
b) um termo que penaliza o afastamento do modelo global $(\mu/2)\|w - w_t\|^2$.
c) uma segunda rede neural.
d) ruído gaussiano.

Questão 22. No FedProx, quando $\mu = 0$:

a) o servidor desaparece.
b) o cliente fica parado.
c) o modelo diverge.
d) volta-se ao FedAvg.

Questão 23. No FedProx, μ muito grande faz com que:

a) o modelo fique melhor sempre.
b) o servidor ignore o cliente.
c) o cliente quase não saia do modelo global, progredindo pouco.
d) o cliente ande mais rápido.

Questão 24. O SCAFFOLD corrige a deriva usando:

a) variáveis de controle no servidor (c) e em cada cliente (c_k).
b) criptografia homomórfica.
c) apenas ruído.
d) mais clientes.

Questão 25. Qual o custo extra do SCAFFOLD?

a) Nenhum.
b) Cerca de o dobro de comunicação e estado armazenado nos clientes.
c) Ele reduz a comunicação pela metade.
d) Exige dados rotulados extras.

Questão 26. O FedNova se destaca por:

a) criptografar atualizações.
b) personalizar o modelo.
c) eliminar o servidor.
d) normalizar cada atualização local pelo número de passos antes de agregar.

Questão 27. Na comparação com 20 clientes de uma classe, 10% de participação, E = 20 e $\eta = 0{,}3$, o melhor algoritmo foi:

a) SCAFFOLD (mais rápido e com menos oscilação).
b) Nenhum aprendeu.
c) O FedSGD.
d) FedProx ($\mu = 0{,}1$), com folga.

Questão 28. Qual foi o efeito do FedProx ($\mu = 0{,}1$) naquele experimento?

a) Piorou muito.
b) Praticamente nenhum em relação ao FedAvg.
c) Melhorou muito.
d) Impediu o treino.

Questão 29. Por que comparar algoritmos com várias sementes?

a) Para reduzir o número de rodadas.
b) Para saber se uma diferença é maior do que a variação causada pelo acaso.
c) Para criptografar os resultados.
d) Para aumentar o número de clientes.

Questão 30. Qual dos otimizadores abaixo atua no servidor para lidar com heterogeneidade, tratando a atualização média como gradiente?

a) K-means.
b) FedAdam.
c) Validação cruzada.
d) PCA.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique o que são dados IID e não-IID usando uma analogia do cotidiano.
2. Descreva os quatro tipos de heterogeneidade apresentados e dê um exemplo de cada.
3. Explique como a distribuição de Dirichlet simula dados não-IID e o papel do parâmetro α.
4. Explique o procedimento de shards para criar dados não-IID.
5. O que é a deriva dos clientes e por que ela prejudica o FedAvg?
6. Explique a inconsistência do objetivo no FedAvg usando o exemplo de dois clientes com mínimos diferentes.
7. Cite três consequências do não-IID para o treino federado.
8. Calcule TV_k para um cliente com distribuição de classes (0,7; 0,2; 0,1) quando a global é (0,4; 0,3; 0,3).
9. Calcule TV_k para um cliente com apenas a classe 1 quando a distribuição global é (0,5; 0,3; 0,2).
10. Explique a ideia do FedProx e o papel de μ.
11. Explique a ideia do SCAFFOLD, citando as variáveis de controle e o custo adicional.
12. Explique o que o FedNova corrige e como.
13. Por que o FedProx pode não ajudar em modelos simples e convexos?
14. Descreva o experimento com 20 clientes de uma classe cada e as conclusões.
15. Por que clientes grandes podem dominar o modelo global e que consequência isso traz para clientes pequenos?
16. Como você verificaria, na prática, se seus clientes são heterogêneos antes de escolher o algoritmo?
17. Explique a diferença entre heterogeneidade de rótulos e de conceito.
18. Por que o número de passos locais (E, B) interage com a heterogeneidade?
19. Qual a importância de relatar média e desvio em vez de uma única execução em experimentos federados?
20. Resuma em cinco linhas o capítulo, destacando o que mais surpreendeu você.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Medindo a heterogeneidade de rótulos

NÍVEL: BÁSICO

Para os dígitos (`carregar_digits`), crie partições com `particao_dirichlet` (10 clientes, `seed=0`) para $\alpha \in \{100; 1; 0{,}1\}$. Para cada α, calcule a distância de variação total média entre a distribuição de classes de cada cliente e a global (ignore clientes sem dados).

Pergunta: como TV muda com α? Qual o valor aproximado para $\alpha = 100$?

Exercício 2 — Épocas locais e heterogeneidade

NÍVEL: BÁSICO

Particione os dígitos em 10 clientes com `particao_shards(ytr, 10, por_cliente=2)`. Treine com FedAvg (`frac=0.3`, `eta=0.1`, `lote=32`, 30 rodadas) para `epocas` $\in \{1, 5, 20\}$. Imprima a acurácia nas rodadas 5 e 30. Repita para dados IID e compare.

Pergunta: mais épocas locais ajudam ou atrapalham no não-IID?

Exercício 3 — Heterogeneidade de quantidade: ponderar ou não?

NÍVEL: INTERMEDIÁRIO

Crie 10 clientes com **tamanhos muito desiguais**, mas rótulos IID: sorteie tamanhos proporcionais a `rng.lognormal(0, 1)`. Implemente uma variante do FedAvg com **média simples** (sem pesos por n_k) usando `treino_local` da `fedlib`. Compare a acurácia final da média ponderada (`fedavg`) com a da média simples, em 3 sementes.

Pergunta: por que a média simples pode ser pior quando os tamanhos variam?

Exercício 4 — Quanto vale a "coleira" do FedProx?

NÍVEL: INTERMEDIÁRIO

Com 20 clientes e `particao_dirichlet(ytr, 20, alfa=0.1)`, 40 rodadas, `epocas=10`, `eta=0.1`, `frac=0.25`, varie `mu` $\in \{0; 0{,}01; 0{,}1; 1; 10\}$. Para cada valor, repita com 3 sementes e relate a acurácia média na rodada 10 e na rodada 40.

Pergunta: algum valor de μ melhorou o FedAvg? O que ocorre com $\mu = 10$?

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** *Adult* (particione por país ou ocupação para criar heterogeneidade natural), *Covertype* e *Bank Marketing*.
> - **Kaggle:** *Telco Customer Churn* (particione por tipo de contrato) e *Credit Card Fraud Detection*.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Escolhendo o algoritmo: mapa de heterogeneidade

Contexto. Uma empresa de telemedicina (fictícia) quer escolher um algoritmo federado, mas não sabe o quanto os dados das clínicas diferem. Você fará um **estudo de sensibilidade**: como o desempenho de FedAvg, FedProx e SCAFFOLD muda com a heterogeneidade?

Tarefas.

1. Para $\alpha \in \{1; 0{,}3; 0{,}1\}$, crie partições Dirichlet com 20 clientes. Calcule o TV médio de cada uma.
2. Para cada α, rode FedAvg, FedProx ($\mu = 0{,}1$) e SCAFFOLD, com `frac=0.25`, `epocas=10`, `eta=0.1`, 40 rodadas e 3 sementes.
3. Relate, em uma tabela, a acurácia média na rodada 10 e na rodada 40 e o desvio entre as sementes.
4. Escreva uma recomendação: em que nível de heterogeneidade vale pagar o custo extra de comunicação do SCAFFOLD?

Entregáveis. Código, tabela e recomendação com ao menos duas ressalvas.

A solução de referência está no Apêndice E.



Capítulo 5 — Falando Menos: Eficiência de Comunicação

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar por que a comunicação é o principal gargalo do aprendizado federado.
- Estimar o tempo total de um treino federado a partir de computação e comunicação.
- Aplicar quantização com arredondamento estocástico a atualizações de modelo.
- Aplicar esparsificação *top-k* e explicar o papel da memória de erro (*error feedback*).
- Comparar métodos pela relação entre acurácia e bytes enviados.
- Reconhecer outras estratégias: menos rodadas, menos clientes e modelos menores.

5.1 O gargalo da rede

Você já tentou enviar um vídeo grande pelo celular, em uma conexão fraca? É lento, instável e, às vezes, falha.

Agora imagine **milhões** de celulares, cada um enviando uma atualização de modelo, **centenas de vezes**. A rede vira o principal obstáculo do aprendizado federado (Kairouz et al., 2021).

Três fatos tornam o problema pior:

- **A subida é mais lenta que a descida.** Em casa, o upload costuma ser bem mais lento que o download.
- **Modelos são grandes.** Milhões de parâmetros viram dezenas de megabytes.
- **Os clientes são instáveis.** Celulares só participam carregando e em Wi-Fi, e podem sumir no meio da rodada.

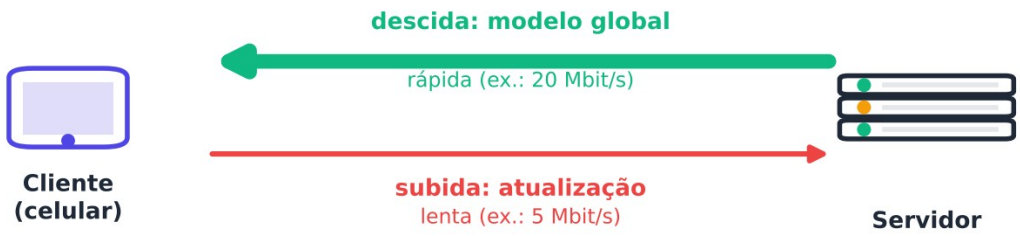


Figura 5.1 — Em cada rodada, o cliente baixa o modelo e sobe a atualização. A subida costuma ser o trecho mais lento e mais caro.

> **No seu dia a dia**
>
> Enviar uma foto em alta resolução e uma versão reduzida da mesma foto leva tempos muito diferentes. Comprimir a atualização do modelo é o mesmo raciocínio: mandar uma versão "mais leve" que preserve o que importa.

5.2 Quanto tempo leva um treino federado?

O tempo total é, em primeira aproximação, o número de rodadas vezes o tempo de cada rodada:

$$T \approx R \cdot (t_{comp} + t_{sub} + t_{desc})$$

Aqui, *R* é o número de rodadas, *t_comp* é o tempo de treino local do cliente mais lento, e *t_sub* e *t_desc* são os tempos de subida e de descida. Cada um é o tamanho em bytes dividido pela velocidade do enlace:

$$t_{sub} = \text{bytes}_{sub} / \text{velocidade}_{sub}$$

5.2.1 Um exemplo

Um modelo de 5 milhões de parâmetros (20 MB em 32 bits). A conexão tem 20 Mbit/s de descida e 5 Mbit/s de subida (1 byte = 8 bits). O cliente leva 30 s para treinar.

- Descida: 20 MB × 8 / 20 Mbit/s = **8 s**.
- Subida: 20 MB × 8 / 5 Mbit/s = **32 s**.
- Rodada: 30 + 8 + 32 = **70 s**. A subida foi quase metade do tempo.

Com 500 rodadas, o treino leva cerca de **9,7 horas**. Se comprimirmos a subida 8 vezes (para 4 s), a rodada cai para 42 s, e o total para cerca de **5,8 horas**.

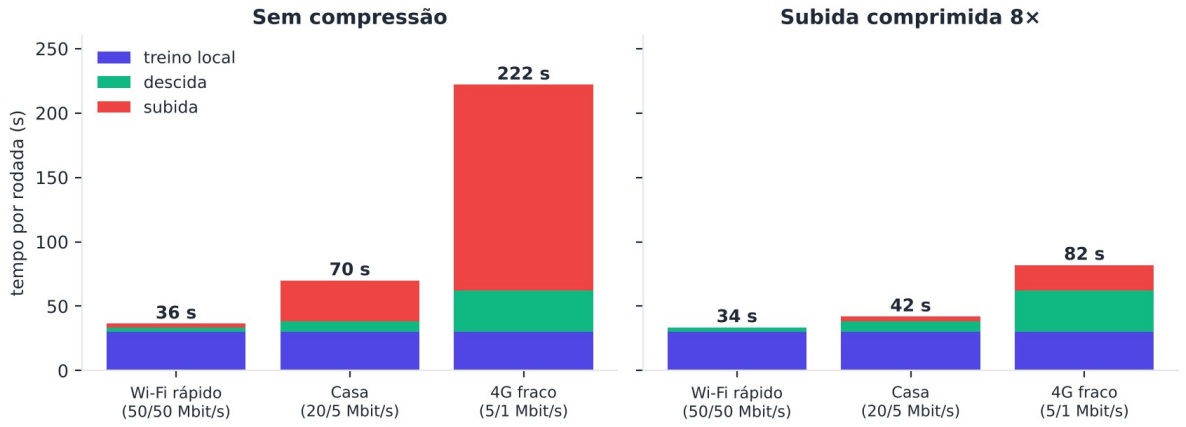


Figura 5.2 — Tempo por rodada em três cenários. Comprimir a subida muda muito o tempo total quando a rede é lenta.

5.3 Mapa de estratégias

Estratégia	Ideia	Exemplo
Menos rodadas	Mais cálculo local por rodada	FedAvg com *E* maior (Cap. 3)
Menos clientes	Sortear menos clientes por rodada	Reduzir *C*
Modelo menor	Menos parâmetros para enviar	Camadas menores, adaptadores
Quantização	Menos bits por parâmetro	8, 4, 2 ou 1 bit

Estratégia	Ideia	Exemplo
Esparsificação	Enviar só os valores mais importantes	*Top-k*
Combinações	Quantizar e esparsificar juntos	Sattler et al. (2020)

Neste capítulo, focamos nas duas técnicas de compressão mais usadas: quantização e esparsificação.

5.4 Quantização: menos bits por número

5.4.1 A ideia

Um parâmetro normalmente ocupa 32 bits. Muitas vezes, 8, 4 ou até 2 bits bastam para transmitir a **atualização** (o *delta*, ou seja, a diferença entre o modelo local e o global).

A **quantização uniforme** divide o intervalo entre o menor e o maior valor em *2^b − 1* degraus. Cada valor é "arredondado" para o degrau mais próximo.

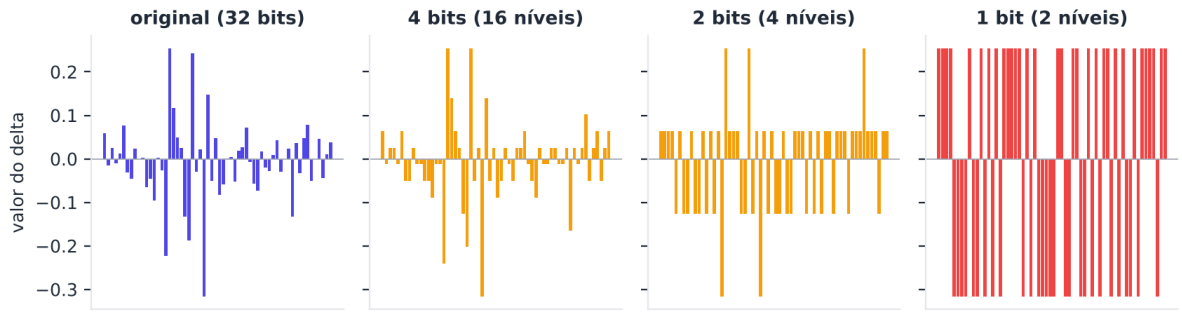


Figura 5.3 — Quantização uniforme de uma atualização: com 2 bits há só 4 níveis; com 4 bits, 16 níveis. Quanto menos bits, mais grosseira é a aproximação.

5.4.2 Arredondamento estocástico

Se arredondássemos sempre para o degrau mais próximo, teríamos um **viés**: erros sistemáticos que se acumulam. A solução (Alistarh et al., 2017) é o **arredondamento estocástico**: sobe-se para o degrau de cima com probabilidade igual à **parte fracionária**.

Exemplo: o valor cai em 2,3 degraus. Com probabilidade 0,3, arredonda-se para 3; com probabilidade 0,7, para 2. Em média, o resultado é 2,3. Dizemos que o quantizador é **não enviesado**: sua esperança é igual ao valor original.

> **Memorize**
>
> Arredondamento estocástico: o erro de cada número é aleatório, mas **não tem viés**. Ao somar muitos clientes, os erros tendem a se cancelar.

5.4.3 Quantos bytes economizamos?

Com *b* bits por parâmetro e *n* parâmetros, enviamos cerca de *n·b/8* bytes, mais 8 bytes para o menor e o maior valor. Para a softmax dos dígitos (650 parâmetros): 32 bits = 2.600 bytes; 8 bits = 658; 4 bits = 333; 2 bits = 171.

5.5 Esparsificação: enviar só o que importa

Em muitas atualizações, **poucos valores concentram a maior parte da mudança**. A **esparsificação** *top-k* envia só os *k* valores de maior módulo, e zera o resto. Para cada valor enviado, é preciso mandar também a **posição** (o índice).

Com *k* igual a 1% dos parâmetros, o ganho potencial é enorme (Lin et al., 2018).

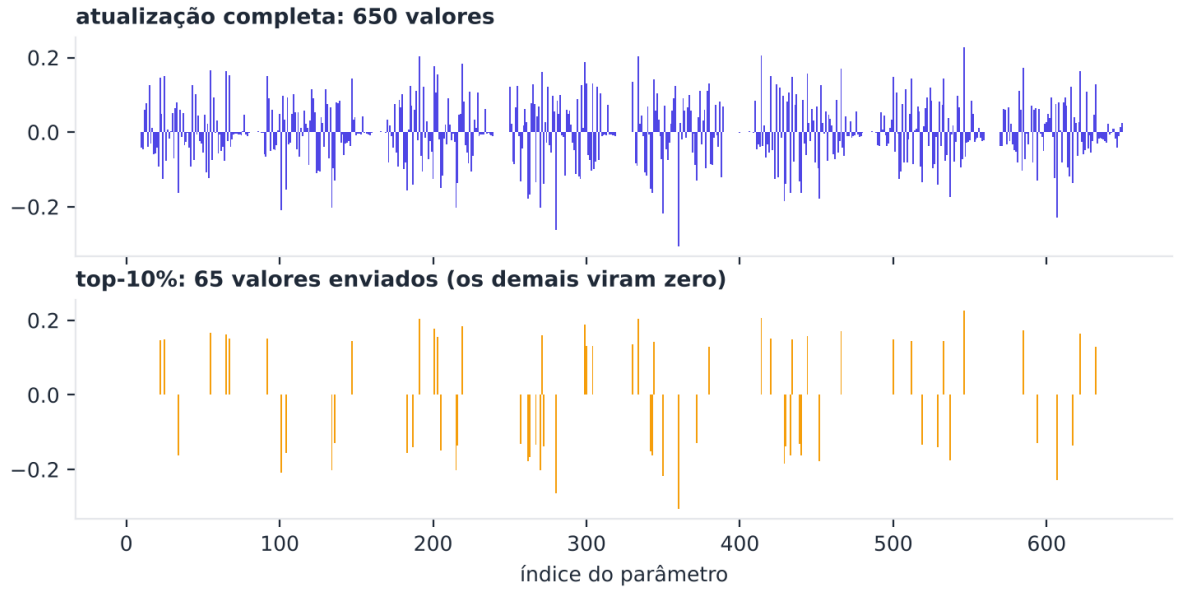


Figura 5.4 — Uma atualização com 650 valores e sua versão top-10%: só os valores mais intensos são mantidos.

5.5.1 O problema: o que foi descartado se perde

Se sempre descartarmos os valores pequenos, eles **nunca** são enviados, e o modelo deixa de aprender nessas direções. A solução é a **memória de erro** (*error feedback*): cada cliente **guarda o que não enviou** e **soma** esse resto à próxima atualização (Stich et al., 2018; Karimireddy et al., 2019).

Em palavras: o que foi cortado hoje não é jogado fora, e sim **adiado** para uma próxima rodada.

> **No seu dia a dia**
>
> É como pagar uma dívida em parcelas: o que você não paga hoje vira saldo e entra na conta de amanhã. Nada se perde, só se adia.

5.6 Implementando a compressão

Salve o código abaixo como `compressao.py`, ao lado da `fedlib.py`.

```python
import numpy as np
from fedlib import *


def quantizar(delta, bits, rng):
    """Quantização uniforme com arredondamento estocástico (não enviesada)."""
    lo, hi = delta.min(), delta.max()
    if hi == lo:
        return delta.copy()
    niveis = 2 ** bits - 1
```

```
    z = (delta - lo) / (hi - lo) * niveis            # leva os valores para [0, niveis]
    base = np.floor(z)
    z = base + (rng.random(z.shape) < (z - base))    # sobe com probabilidade igual à parte fracionária
    return lo + z / niveis * (hi - lo)


def top_k(delta, frac):
    """Mantém apenas os k maiores valores (em módulo); zera o resto."""
    k = max(1, int(round(frac * delta.size)))
    corte = np.sort(np.abs(delta).ravel())[-k]
    return np.where(np.abs(delta) >= corte, delta, 0.0)


def bytes_quantizado(n, bits):
    return int(np.ceil(n * bits / 8)) + 8            # +8: mínimo e máximo em 32 bits cada


def bytes_esparso(n, frac):
    k = max(1, int(round(frac * n)))
    return k * (4 + 2)                               # valor (4 bytes) + índice (2 bytes)


def fedavg_comprimido(partes, X, y, Xte, yte, metodo="nenhum", param=None, erro_acumulado=False,
                      rodadas=30, epocas=5, eta=0.1, lote=32, frac=0.5, seed=0, classes=10):
    """FedAvg em que cada cliente comprime a sua atualização (delta) antes de enviá-la."""
    rng = np.random.default_rng(seed)
    W = np.zeros((X.shape[1], classes))
    n = W.size
    memoria = [np.zeros_like(W) for _ in partes]     # erro acumulado de cada cliente
    total_subida, historico = 0, []
    for r in range(rodadas):
        m = max(1, int(round(frac * len(partes))))
        deltas, tam = [], []
        for c in rng.choice(len(partes), m, replace=False):
            if len(partes[c]) == 0:
                continue
            Wc = treino_local(W, X[partes[c]], y[partes[c]], epocas, eta, lote, 0.0, seed * 1000 + r *
50 + int(c))
            d = Wc - W
            if erro_acumulado:
                d = d + memoria[c]
            if metodo == "quantizar":
                dh, b = quantizar(d, param, rng), bytes_quantizado(n, param)
            elif metodo == "topk":
                dh, b = top_k(d, param), bytes_esparso(n, param)
            else:
                dh, b = d, 4 * n
            if erro_acumulado:
                memoria[c] = d - dh
            deltas.append(dh); tam.append(len(partes[c])); total_subida += b
        p = np.array(tam) / sum(tam)
        W = W + sum(pi * di for pi, di in zip(p, deltas))
        historico.append(acuracia(W, Xte, yte))
    return W, historico, total_subida
```

Pontos importantes:

- `quantizar` usa arredondamento estocástico.

- `top_k` guarda os maiores valores (em módulo).
- Em `fedavg_comprimido`, cada cliente calcula o *delta* (modelo local menos modelo global), soma a memória de erro (se `erro_acumulado=True`), comprime e guarda o resto.
- Contamos os **bytes de subida** de cada método, supondo valor de 4 bytes e índice de 2 bytes.

5.7 O experimento: acurácia contra bytes

Comparamos vários métodos com **20 clientes**, dados Dirichlet (α = 1), 50% de participação, 5 épocas locais e 30 rodadas. Médias de 3 sementes.

```python
import numpy as np
from compressao import *

Xtr, Xte, ytr, yte = carregar_digits()
metodos = {
    "float32":       dict(),
    "8 bits":        dict(metodo="quantizar", param=8),
    "4 bits":        dict(metodo="quantizar", param=4),
    "2 bits":        dict(metodo="quantizar", param=2),
    "1 bit":         dict(metodo="quantizar", param=1),
    "top-10%":       dict(metodo="topk", param=0.1),
    "top-10% + EF":  dict(metodo="topk", param=0.1, erro_acumulado=True),
    "top-1%":        dict(metodo="topk", param=0.01),
    "top-1% + EF":   dict(metodo="topk", param=0.01, erro_acumulado=True),
}
print(f"{'método':<14}{'acurácia':>9}{'subida (KB)':>13}{'economia':>10}")
base = None
for nome, kw in metodos.items():
    acc, tot = [], []
    for s in range(3):
        partes = particao_dirichlet(ytr, 20, 1.0, seed=s)
        _, h, t = fedavg_comprimido(partes, Xtr, ytr, Xte, yte, seed=s, **kw)
        acc.append(h[-1]); tot.append(t / 1024)
    base = base or np.mean(tot)
    print(f"{nome:<14}{np.mean(acc):9.3f}{np.mean(tot):13.0f}{base / np.mean(tot):9.1f}x")
```

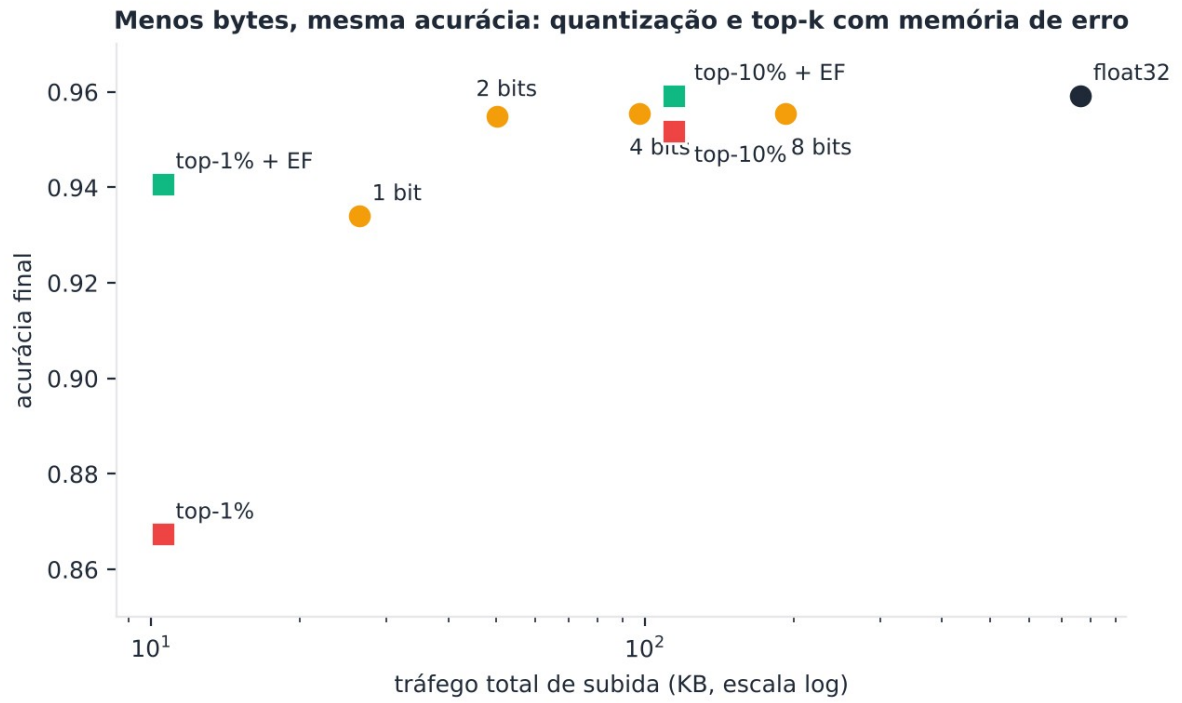


Figura 5.5 — Acurácia final contra tráfego de subida. Quanto mais à esquerda, menos bytes. O melhor método é o que fica no canto superior esquerdo.

O que aprendemos:

- **Quantização a 8, 4 e até 2 bits** praticamente não prejudicou a acurácia, com **4 a 15 vezes menos bytes** de subida.
- **1 bit** já machuca um pouco.
- **Top-10%** com memória de erro igualou o modelo sem compressão, usando cerca de **7 vezes menos bytes**.
- **Top-1% sem memória de erro** perdeu bastante acurácia; **com memória de erro**, recuperou grande parte, usando cerca de **70 vezes menos bytes**.

> **Dica**
>
> Se você usar esparsificação agressiva, **sempre** use memória de erro. É um dos "pulos do gato" mais importantes da compressão em aprendizado federado.

> **Atenção**
>
> A memória de erro exige que o cliente **guarde estado** entre rodadas. Em cenários cross-device, em que cada cliente participa poucas vezes, a memória pode ficar "velha" e ajudar menos. Em cross-silo, funciona muito bem.

5.8 Outras ideias

- **Compressão na descida também.** Enviar o modelo global comprimido economiza banda, mas exige cuidado, pois os clientes passam a treinar a partir de um modelo aproximado.
- **Sinal dos gradientes** (*signSGD*): enviar apenas o sinal (1 bit por valor) (Bernstein et al., 2018).
- **Parâmetros menos numerosos:** treinar só parte do modelo (adaptadores e camadas finais). No Capítulo 11, veremos isso em modelos de linguagem.

- **Periodicidade:** FedPAQ combina vários passos locais, quantização e participação parcial (Reisizadeh et al., 2020).
- **Compressão com garantias teóricas** para diversos compressores (Haddadpour et al., 2021).

> **Saiba mais**
>
> Compressão e privacidade conversam entre si. Esparsificar e quantizar reduzem a quantidade de informação enviada, o que pode dificultar alguns ataques. Mas isso **não é** uma garantia de privacidade. Para garantias formais, veja o Capítulo 8.

5.9 Resumo do capítulo

> MEMORIZE **O essencial do Capítulo 5**
>
> - A **subida** costuma ser o trecho mais caro: modelos grandes, redes lentas, clientes instáveis.
> - Tempo total ≈ rodadas × (computação + subida + descida).
> - **Quantização** reduz bits por parâmetro; o **arredondamento estocástico** evita viés.
> - **Esparsificação top-k** envia só os maiores valores; precisa enviar também os índices.
> - **Memória de erro** guarda o que foi cortado e o reinclui depois: essencial na esparsificação forte.
> - Compare métodos por **acurácia × bytes**, não só por acurácia.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Por que a comunicação é um gargalo no aprendizado federado?

a) Modelos são grandes, as redes são lentas (sobretudo na subida) e os clientes são instáveis.
b) Porque os dados são pequenos demais.
c) Porque o servidor não tem memória.
d) Porque os clientes têm CPUs lentas demais.

Questão 2. Qual é a característica típica das conexões domésticas relevante ao aprendizado federado?

a) A descida é sempre mais lenta que a subida.
b) A subida (upload) costuma ser mais lenta que a descida (download).
c) Não há conexão.
d) Não há diferença.

Questão 3. A fórmula $T \approx R \cdot (t_{comp} + t_{sub} + t_{desc})$ expressa:

a) o número de clientes.
b) a acurácia final.
c) a taxa de aprendizado.
d) o tempo total como número de rodadas vezes o tempo de cada rodada.

Questão 4. Um modelo ocupa 20 MB e a subida é de 5 Mbit/s. Qual o tempo de subida (1 byte = 8 bits)?

a) 100 s.
b) 4 s.
c) 32 s.
d) 8 s.

Questão 5. Com descida de 20 Mbit/s e modelo de 20 MB, o tempo de descida é:

a) 8 s.
b) 16 s.
c) 32 s.
d) 2 s.

Questão 6. Se o treino local leva 30 s, a descida 8 s e a subida 32 s, uma rodada leva:

a) 70 s.
b) 62 s.
c) 100 s.
d) 40 s.

Questão 7. No exemplo anterior, quanto levariam 500 rodadas?

a) Cerca de 9,7 horas.
b) Cerca de 100 horas.
c) Cerca de 20 minutos.
d) Cerca de 1 hora.

Questão 8. Se a subida for comprimida 8 vezes (de 32 s para 4 s), a rodada passa a levar:

a) 54 s.
b) 38 s.
c) 42 s.
d) 70 s.

Questão 9. O que é quantização de uma atualização?

a) Criptografar os dados.
b) Apagar as atualizações antigas.
c) Representá-la com menos bits por valor.
d) Aumentar o número de parâmetros.

Questão 10. Com b bits, quantos níveis a quantização uniforme tem?

a) 32.
b) b.
c) 2^b.
d) b^2.

Questão 11. Qual a vantagem do arredondamento estocástico sobre o determinístico?

a) Não introduz viés: a esperança do valor quantizado é igual ao valor original.
b) É sempre mais preciso em cada valor.
c) Usa mais bits.
d) Elimina a necessidade do servidor.

Questão 12. Se um valor cai em 2,3 degraus, no arredondamento estocástico ele vai para 3 com probabilidade:

a) 0,5.
b) 1,0.
c) 0,3.
d) 0,7.

Questão 13. Quantos bytes são necessários para enviar 650 parâmetros com 4 bits cada (sem o cabeçalho)?

a) 2.600 bytes.
b) 650 bytes.
c) 163 bytes.
d) 325 bytes.

Questão 14. Quantos bytes ocupam 650 parâmetros em 32 bits?

a) 650.
b) 2.600.
c) 5.200.
d) 1.300.

Questão 15. A esparsificação top-k:

a) envia sempre todos os valores.
b) elimina o servidor.
c) duplica o tamanho da atualização.
d) envia apenas os k valores de maior módulo e zera o restante.

Questão 16. Por que a esparsificação precisa enviar os índices?

a) Porque reduzem o ruído.
b) Porque é uma exigência legal.
c) Porque índices são criptografados.
d) Porque o servidor precisa saber a qual parâmetro cada valor se refere.

Questão 17. Quantos bytes custa enviar k valores, se cada valor ocupa 4 bytes e cada índice, 2 bytes?

a) 8k.
b) 4k.
c) 2k.
d) 6k.

Questão 18. Para 650 parâmetros e top-10%, quantos valores são enviados?

a) 65.
b) 6.
c) 130.
d) 325.

Questão 19. Qual o problema de descartar sempre os valores pequenos sem nenhuma compensação?

a) Eles aumentam a privacidade.
b) Eles nunca são enviados, e o modelo deixa de aprender nessas direções.
c) Eles quebram o servidor.
d) Eles aceleram o treino.

Questão 20. A memória de erro (error feedback) consiste em:

a) reduzir a taxa de aprendizado.
b) apagar os clientes que erram.
c) cada cliente guardar o que não enviou e somá-lo à atualização seguinte.
d) o servidor corrigir os erros de rótulo.

Questão 21. No experimento do capítulo, qual combinação teve acurácia igual à do modelo sem compressão com cerca de 7 vezes menos bytes?

a) Nenhuma.
b) 1 bit.
c) Top-10% com memória de erro.
d) Top-1% sem memória de erro.

Questão 22. No experimento, o top-1% sem memória de erro:

a) perdeu muita acurácia (cerca de 0,87).
b) não pôde ser executado.
c) foi o melhor de todos.
d) igualou o float32.

Questão 23. No experimento, o top-1% com memória de erro:

a) caiu para 0,10.
b) recuperou grande parte da acurácia (cerca de 0,94).
c) usou mais bytes que o float32.
d) ficou idêntico ao top-1% sem memória.

Questão 24. Quantos bits por parâmetro já começam a prejudicar a acurácia no experimento?

a) 4 bits.
b) Nenhum, nem 1 bit.
c) 8 bits.
d) 1 bit.

Questão 25. Qual economia aproximada de bytes de subida a 2 bits em relação ao float32?

a) Cerca de 100 vezes.
b) Cerca de 15 vezes.
c) Cerca de 2 vezes.
d) Nenhuma.

Questão 26. A técnica signSGD envia:

a) apenas o sinal de cada valor (1 bit).
b) somente os índices.
c) o valor completo em 64 bits.
d) apenas o maior valor.

Questão 27. Em que cenário a memória de erro pode ajudar menos?

a) Cross-silo com clientes sempre ativos.
b) Quando a rede é rápida.
c) Quando há um único cliente.
d) Cross-device, em que cada cliente participa poucas vezes e a memória fica velha.

Questão 28. Por que comparar métodos de compressão apenas pela acurácia é insuficiente?

a) Porque o objetivo é equilibrar acurácia e bytes enviados.
b) Porque bytes não importam.
c) Porque a acurácia não existe.
d) Porque só o tempo importa.

Questão 29. Qual das estratégias abaixo NÃO reduz a comunicação total?

a) Enviar só adaptadores em vez do modelo inteiro.
b) Quantizar as atualizações.
c) Aumentar as épocas locais para ter menos rodadas.
d) Aumentar o número de rodadas mantendo o mesmo tamanho de atualização.

Questão 30. Qual a principal vantagem de uma quantização de 8 bits em relação a 32 bits?

a) Torna o modelo mais privado.
b) Aumenta o número de parâmetros.
c) Reduz o tráfego de subida em cerca de 4 vezes, em geral com pouca perda de acurácia.
d) Elimina a necessidade de treino local.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique por que a subida costuma ser o gargalo e dê um exemplo numérico.
2. Mostre como estimar o tempo total de um treino federado e o que comprimir a subida muda.
3. Descreva a quantização uniforme e o papel do número de bits.
4. Explique o arredondamento estocástico e por que ele evita viés.
5. Calcule quantos bytes uma atualização de 1.000 parâmetros ocupa com 32, 8 e 2 bits (sem cabeçalho).
6. Explique a esparsificação top-k e o custo de enviar os índices.
7. Calcule o custo em bytes de enviar top-5% de um modelo de 10.000 parâmetros (valor de 4 bytes e índice de 2 bytes).
8. O que é a memória de erro e por que ela é importante na esparsificação agressiva?
9. Interprete os resultados do experimento top-1% com e sem memória de erro.
10. Compare quantização de 2 bits e top-10% com memória de erro quanto a bytes e acurácia no experimento.
11. Por que o cabeçalho de mínimo e máximo é necessário na quantização uniforme?
12. Quais são os riscos de comprimir também a descida (modelo global)?
13. Explique o que é o signSGD e uma limitação.
14. Por que aumentar o número de épocas locais também pode ser visto como compressão de comunicação?
15. Dê duas razões pelas quais clientes cross-device têm mais dificuldade que cross-silo quanto à comunicação.
16. Como a compressão se relaciona com a privacidade? Ela garante proteção?
17. Qual a diferença entre compressão com perdas e sem perdas? Qual delas se usa nas atualizações?
18. Se você tivesse um orçamento fixo de 100 KB de subida por rodada, como escolheria um método de compressão?
19. Quais são as limitações do experimento do capítulo (dados, modelo e repetições)?
20. Resuma em cinco linhas as técnicas de compressão estudadas e quando usar cada uma.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Viés do arredondamento: determinístico contra estocástico

NÍVEL: BÁSICO

Considere o valor 0,3 em um intervalo [0, 1] quantizado com 2 bits (4 níveis: 0, 1/3, 2/3, 1). Quantize 20.000 vezes o mesmo valor: (a) com arredondamento ao nível mais próximo, (b) com arredondamento estocástico (`quantizar` de `compressao.py`, aplicado a um vetor que contenha 0 e 1 para fixar o intervalo). Compare a média dos resultados com 0,3.

Pergunta: qual método é enviesado? Por que isso importa ao somar muitos clientes?

Exercício 2 — Calculadora de tempo por rodada

NÍVEL: BÁSICO

Escreva `tempo_rodada(n_parametros, bits_subida, desc_mbps, sub_mbps, t_comp)` que devolva o tempo total de uma rodada (descida em 32 bits, subida com `bits_subida`). Para um modelo de 5 milhões de parâmetros, tempo de treino de 30 s, descida de 20 Mbit/s e subida de 5 Mbit/s, calcule o tempo por rodada e o total de 500 rodadas para 32, 8, 4 e 2 bits de subida.

Pergunta: a partir de quantos bits o ganho adicional é pequeno? Por quê?

Exercício 3 — Esparsificação com e sem memória de erro

NÍVEL: INTERMEDIÁRIO

Com `fedavg_comprimido`, teste `top_k` com frações 0,5; 0,1; 0,05 e 0,01, **com e sem** memória de erro. Use 20 clientes Dirichlet (α = 1), 3 sementes, e mostre a acurácia final média e o tráfego de subida em KB.

Pergunta: a partir de que fração a memória de erro passa a fazer diferença?

Exercício 4 — Quantização com dados muito heterogêneos

NÍVEL: INTERMEDIÁRIO

Repita a quantização (32, 8, 4, 2 e 1 bit) com dados **muito não-IID**: `particao_shards(ytr, 20, por_cliente=1)`, `frac=0.25`, `epocas=5`, 40 rodadas, 3 sementes. Compare com a acurácia obtida em dados IID.

Pergunta: a quantização é mais ou menos prejudicial quando os dados são heterogêneos?

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** *Covertype*, *Letter Recognition* e *Isolet* (muitos atributos, bom para testar compressão).
> - **Kaggle:** *Fashion-MNIST* e *Forest Cover Type*, para modelos maiores em que a compressão importa mais.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Orçamento de comunicação: qual configuração escolher?

Contexto. Uma empresa de dispositivos médicos (fictícia) pode gastar, no máximo, uma quantidade fixa de tráfego de subida por treino federado. Ela quer saber **qual combinação de método de compressão e épocas locais** entrega a melhor acurácia dentro do orçamento.

Tarefas.

1. Defina dois orçamentos de subida: **100 KB** e **300 KB** (total do treino, somando todos os clientes).
2. Teste as configurações: float32, 8 bits, 2 bits, top-10% + EF e top-1% + EF, com `epocas` $\in \{1, 5\}$, 60 rodadas, 20 clientes Dirichlet ($\alpha = 1$), `frac=0.5`, 2 sementes.
3. Para cada configuração, calcule o tráfego por rodada e a rodada máxima que cabe em cada orçamento. Registre a acurácia nessa rodada.
4. Apresente uma tabela e indique a melhor configuração para cada orçamento.
5. Escreva uma recomendação de três a cinco linhas.

Entregáveis. Código, tabela e recomendação.

A solução de referência está no Apêndice E.



Capítulo 6 — Cada Um do Seu Jeito: Personalização

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar por que um único modelo global pode servir mal a certos clientes.
- Medir a qualidade por cliente, incluindo o pior cliente e o percentil 10.
- Descrever as grandes famílias de personalização: ajuste fino, regularização, partição do modelo, agrupamento e meta-aprendizado.
- Implementar ajuste fino local, o método Ditto e o agrupamento iterativo de clientes (IFCA).
- Interpretar experimentos em que o modelo global falha, como na mudança de conceito.

6.1 Roupa de prateleira ou sob medida?

Uma loja vende camisas em três tamanhos: P, M e G. Para a maioria, serve bem. Para quem tem ombros largos e braços curtos, nenhum tamanho serve de verdade.

O **modelo global** do aprendizado federado é essa camisa de prateleira: bom na média, mas pode vestir mal certos clientes. A **personalização** é o alfaiate: ajusta o modelo ao corpo de cada um.

> **No seu dia a dia**
>
> O teclado do seu celular aprende com milhões de pessoas, mas você escreve gírias, nomes e expressões que ninguém mais usa. Um bom teclado combina o que aprendeu com todos e o que aprende só de você.

6.2 Quando o modelo global não serve

Já vimos que dados não-IID atrapalham o treino. Agora vamos olhar para o **resultado**: como o modelo global se sai **em cada cliente**?

6.2.1 Medir por cliente

A acurácia média esconde injustiças. Por isso, medimos por cliente, em um **teste local**, e resumimos com três números:

- **Média** entre clientes.
- **Pior cliente**: o menos bem servido.
- **Percentil 10**: 90% dos clientes têm acurácia **pelo menos** igual a esse valor.

> **Atenção**
>
> Em cada cliente, o teste local precisa vir **dos dados daquele cliente**. Senão, a avaliação não reflete o que ele vai usar de fato.

6.2.2 Dois cenários em que o global falha

1. **Mudança de rótulos** (*label skew*): cada cliente vê poucas classes. O global é bom na média, mas imperfeito para quem tem distribuição diferente.
2. **Mudança de conceito** (*concept shift*): o mesmo dado significa coisas diferentes para grupos diferentes. Aqui, um único modelo **não pode** servir a todos.

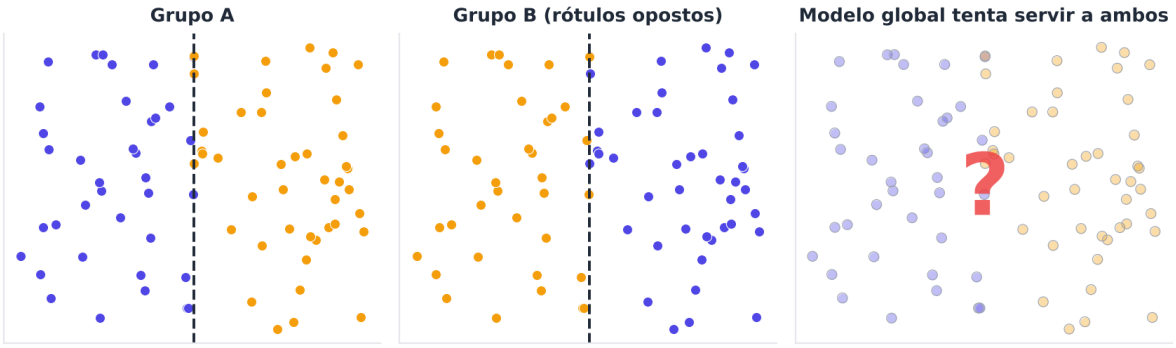


Figura 6.1 — Mudança de conceito: dois grupos de clientes têm a mesma região de pontos, mas rótulos opostos. Nenhum modelo único acerta para os dois.

6.3 Mapa das estratégias de personalização

Tan et al. (2022) organizam a personalização em duas grandes linhas: criar **um modelo global melhor de personalizar** e criar **modelos locais aprendidos em conjunto**. Na prática, as técnicas mais usadas são:

Família	Ideia	Exemplos
Ajuste fino local	Treinar o modelo global mais alguns passos com os dados do cliente	FedAvg + fine-tuning
Regularização	Cada cliente tem seu modelo, preso por uma "coleira" ao global	Ditto (Li et al., 2021), pFedMe (T Dinh et al., 2020)
Partição do modelo	Corpo compartilhado e "cabeça" pessoal	FedRep (Collins et al., 2021)
Agrupamento	Clientes parecidos compartilham um modelo	IFCA (Ghosh et al., 2020), CFL (Sattler et al., 2021)
Multitarefa	Cada cliente é uma tarefa relacionada	MOCHA (Smith et al., 2017)
Meta-aprendizado	Um modelo inicial que se adapta rápido	Per-FedAvg (Fallah et al., 2020)

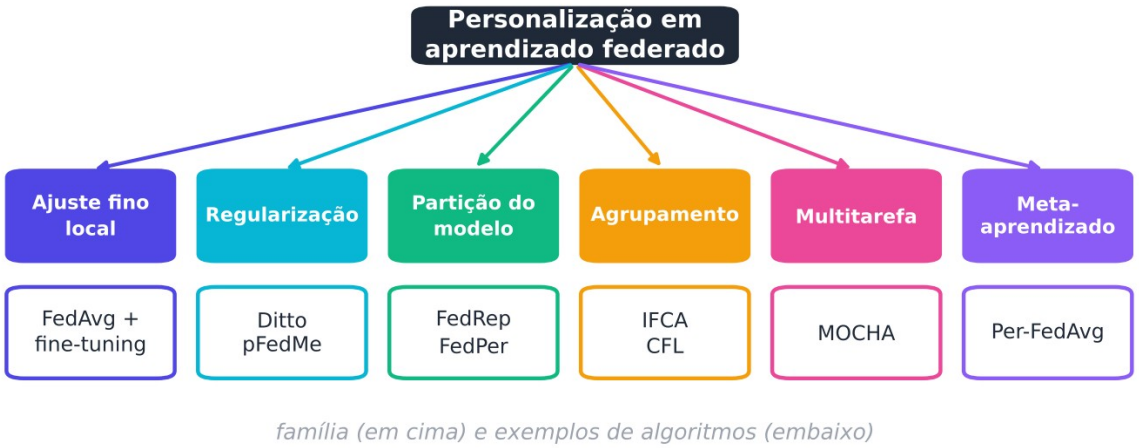


Figura 6.2 — As principais famílias de personalização em aprendizado federado.

6.4 Ajuste fino local

A ideia mais simples: o cliente recebe o modelo global e **continua treinando** um pouco com seus dados. É o **ajuste fino** (*fine-tuning*).

- **Vantagem:** simples, sem custo de comunicação extra.
- **Cuidado:** poucos passos ajustam pouco; muitos passos "esquecem" o que o global aprendeu e podem sobreajustar.

6.5 Ditto: um modelo pessoal com coleira

No **Ditto** (Li et al., 2021), cada cliente mantém, além do modelo global, um **modelo pessoal** v_k. O treino pessoal minimiza o erro local com uma penalização que o mantém **perto do global**:

$$\min_{v_k} F_k(v_k) + (\lambda / 2) \cdot \| v_k - w \|^2$$

O hiperparâmetro λ escolhe o ponto entre dois extremos:

- $\lambda = 0$: o modelo pessoal é treinado **só com o erro local**. Se ele parte do modelo global, isso equivale ao ajuste fino.
- λ muito grande: o modelo pessoal fica **igual ao global**.

Em palavras: λ é o quanto o cliente "confia" no coletivo.

Note a semelhança com o FedProx do Capítulo 4. No FedProx, a coleira estabiliza o **treino global**. No Ditto, ela define o **modelo pessoal**, e o global segue sendo treinado normalmente.

6.6 Partição do modelo: corpo e cabeça

Em redes neurais, as primeiras camadas aprendem **características gerais** (bordas, formas), e as últimas, **decisões específicas**. A ideia do **FedRep** (Collins et al., 2021) e de métodos similares é:

- o **corpo** (camadas iniciais) é treinado **em conjunto**;
- a **cabeça** (camada final) é **pessoal**, aprendida só localmente.

Assim, todos compartilham o conhecimento geral, e cada um tem a sua decisão final. Também economiza comunicação, pois a cabeça não é enviada.

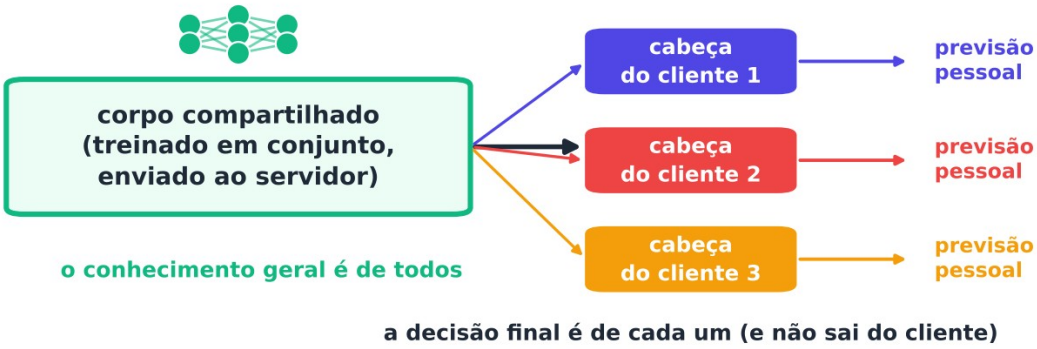


Figura 6.3 — Corpo compartilhado e cabeça pessoal: cada cliente tem sua própria cabeça de decisão.

6.7 Agrupar clientes parecidos: IFCA

Se há **grupos** de clientes com comportamentos distintos, a ideia natural é treinar **um modelo por grupo**. Mas ninguém sabe de antemão quais clientes pertencem a qual grupo.

O **IFCA** (*Iterative Federated Clustering Algorithm*, Ghosh et al., 2020) resolve isso de forma iterativa. O servidor mantém *K* modelos. Em cada rodada:

1. envia os *K* modelos a cada cliente;
2. cada cliente **escolhe o modelo de menor perda** nos seus dados (é o "grupo" dele);
3. treina esse modelo e devolve a atualização;
4. o servidor **faz a média dentro de cada grupo**.

Com o tempo, os clientes se agrupam espontaneamente.

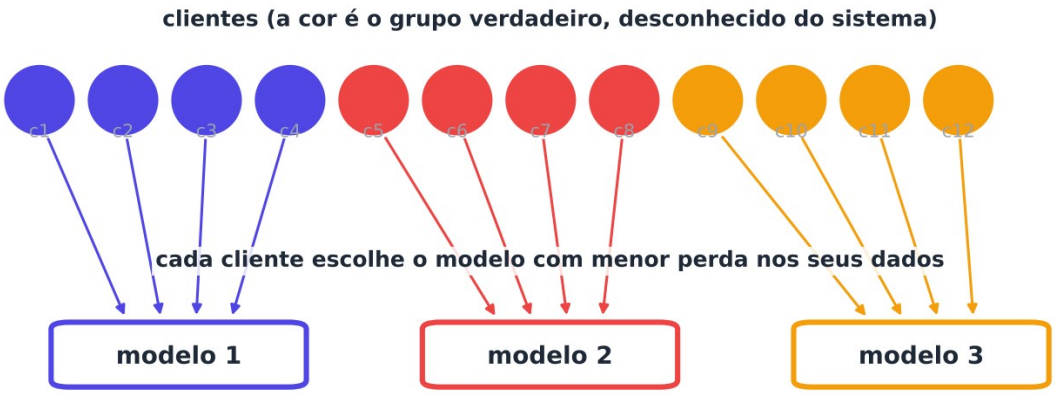


Figura 6.4 — Em cada rodada, cada cliente escolhe o modelo com menor perda. Clientes do mesmo grupo verdadeiro terminam no mesmo modelo.

6.8 Meta-aprendizado: aprender a se adaptar

Em vez de procurar o melhor modelo, o **Per-FedAvg** (Fallah et al., 2020) procura o melhor **ponto de partida**: um modelo global que, **após poucos passos** de ajuste local, fique bom para cada cliente. É o espírito do meta-aprendizado: "aprender a aprender rápido". A formulação exige derivadas de segunda ordem (ou aproximações), e por isso é mais custosa.

6.9 A biblioteca personalizacao.py

Salve o código abaixo como **`personalizacao.py`**, junto da `fedlib.py`.

```python
import numpy as np
from fedlib import *


def dividir_local(partes, frac_teste=0.3, seed=0):
    """Cada cliente separa uma parte dos seus próprios dados para teste local."""
    rng = np.random.default_rng(seed)
    treino, teste = [], []
    for p in partes:
        q = rng.permutation(p)
        k = int(len(q) * (1 - frac_teste))
        treino.append(q[:k]); teste.append(q[k:])
    return treino, teste


def acuracias_locais(modelos, teste, X, y):
    """Acurácia de cada cliente, usando o modelo que cabe a ele e o seu teste local."""
    return np.array([acuracia(W, X[t], y[t]) for W, t in zip(modelos, teste)])


def resumo(acc):
    return f"média = {acc.mean():.3f} | pior cliente = {acc.min():.3f} | percentil 10 =
{np.percentile(acc, 10):.3f}"


def ditto(treino, X, y, Wg, lam=0.5, epocas=5, eta=0.1, lote=32, seed=0):
    """Modelos pessoais v_k treinados com uma 'coleira' em direção ao modelo global Wg."""
    pessoais = []
    for k, p in enumerate(treino):
        v, rng = Wg.copy(), np.random.default_rng(seed + k)
        for _ in range(epocas):
            ordem = rng.permutation(len(p))
            for i in range(0, len(p), lote):
                b = p[ordem[i:i + lote]]
                v -= eta * gradiente(v, X[b], y[b], lam, Wg)
        pessoais.append(v)
    return pessoais


def particao_conceitos(y, n_clientes=12, grupos=3, seed=0):
    """Mudança de conceito: cada grupo de clientes usa uma troca diferente dos nomes das classes."""
    rng = np.random.default_rng(seed)
    perms = [np.arange(10)] + [rng.permutation(10) for _ in range(grupos - 1)]
    partes = particao_iid(y, n_clientes, seed)
    grupo = np.arange(n_clientes) % grupos
```

```
    y_novo = y.copy()
    for k, p in enumerate(partes):
        y_novo[p] = perms[grupo[k]][y[p]]
    return partes, y_novo, grupo


def ifca(treino, X, y, K=3, rodadas=30, epocas=5, eta=0.1, lote=32, seed=0):
    """Agrupamento iterativo: cada cliente escolhe o modelo de menor perda; o servidor faz a média por grupo."""
    rng = np.random.default_rng(seed)
    modelos = [rng.normal(0, 0.05, (X.shape[1], 10)) for _ in range(K)]
    for r in range(rodadas):
        locais, tamanhos = [[] for _ in range(K)], [[] for _ in range(K)]
        escolha = []
        for c, p in enumerate(treino):
            j = int(np.argmin([perda(W, X[p], y[p]) for W in modelos]))
            escolha.append(j)
            locais[j].append(treino_local(modelos[j], X[p], y[p], epocas, eta, lote, 0.0, seed * 1000 + r * 50 + c))
            tamanhos[j].append(len(p))
        for j in range(K):
            if locais[j]:
                pj = np.array(tamanhos[j]) / sum(tamanhos[j])
                modelos[j] = sum(a * b for a, b in zip(pj, locais[j]))
    return modelos, escolha
```

6.10 Experimento A: poucos rótulos por cliente

Dez clientes com dados Dirichlet (α = 0,1): cada um vê poucas classes. Comparamos: modelo **só local**, **global** (FedAvg), **global + ajuste fino** (5 épocas) e **Ditto** (λ = 0,5). Cada cliente tem um teste local. Resultados com 5 sementes (todos os clientes juntos).

PYTHON

```
import numpy as np
from personalizacao import *

Xtr, Xte, ytr, yte = carregar_digits()
R = {"só local": [], "global": [], "global + ajuste fino": [], "Ditto (λ=0,5)": []}
for s in range(5):
    partes = particao_dirichlet(ytr, 10, 0.1, s)
    treino, teste = dividir_local(partes, 0.3, s)
    ok = [i for i in range(10) if len(teste[i]) >= 5]       # ignora clientes quase sem dados
    treino = [treino[i] for i in ok]; teste = [teste[i] for i in ok]

    Wg, _ = fedavg(treino, Xtr, ytr, Xte, yte, rodadas=30, epocas=5, eta=0.1, seed=s)
    zero = np.zeros_like(Wg)
    R["só local"].append(acuracias_locais(
        [treino_local(zero, Xtr[p], ytr[p], 30, 0.1, 32, 0.0, s + i) for i, p in enumerate(treino)], teste, Xtr, ytr))
    R["global"].append(acuracias_locais([Wg] * len(treino), teste, Xtr, ytr))
    R["global + ajuste fino"].append(acuracias_locais(
        [treino_local(Wg, Xtr[p], ytr[p], 5, 0.1, 32, 0.0, s + i) for i, p in enumerate(treino)], teste, Xtr, ytr))
    R["Ditto (λ=0,5)"].append(acuracias_locais(ditto(treino, Xtr, ytr, Wg, 0.5, seed=s), teste, Xtr, ytr))
```

```
for nome, v in R.items():
    print(f"{nome:<22}", resumo(np.concatenate(v)))
```

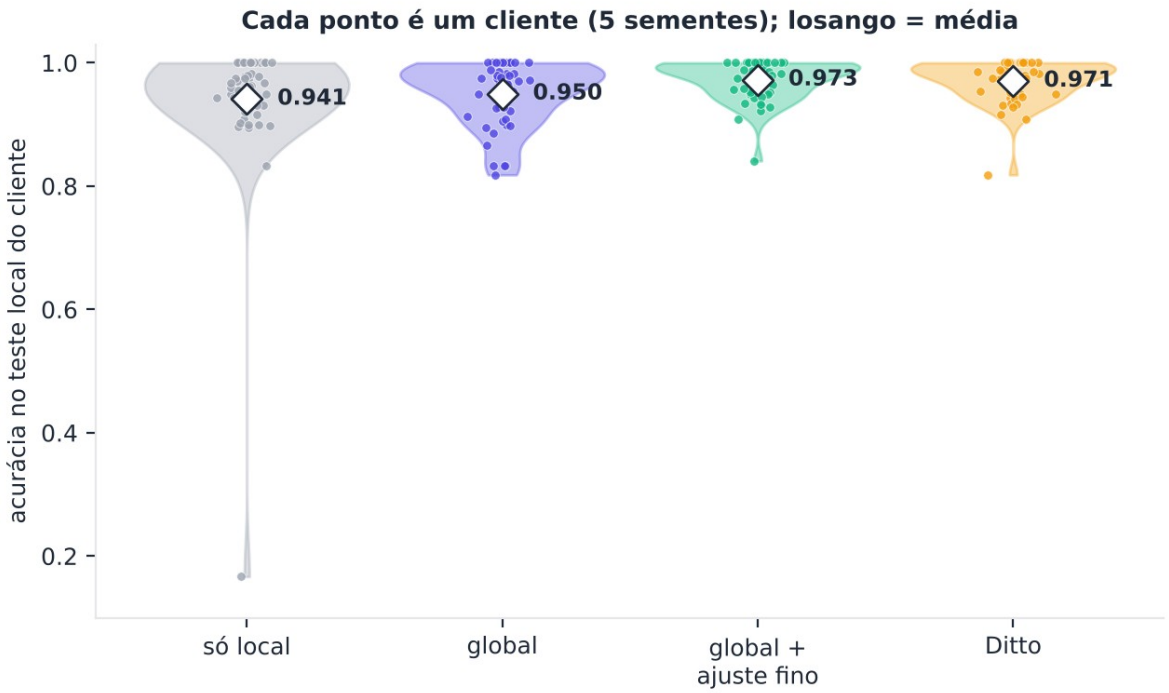


Figura 6.5 — Acurácia por cliente com dados muito heterogêneos nos rótulos (α = 0,1). A personalização melhora a média e o percentil 10; o modelo só local falha no cliente com poucos dados.

O que aprendemos:

- O **ajuste fino** e o **Ditto** elevaram a média de cerca de 0,95 para cerca de 0,97, e o percentil 10 de 0,88 para 0,93.
- O modelo **só local** tem boa média, mas **o pior cliente cai muito** (um cliente com pouquíssimos dados). Colaborar protege quem tem pouco dado.
- Ditto e ajuste fino ficaram parecidos aqui. A vantagem do Ditto aparece, em geral, quando o ajuste fino sobreajusta.

6.11 Experimento B: mudança de conceito

Agora o cenário difícil. Doze clientes, em **três grupos**. Cada grupo usa uma **troca diferente dos nomes das classes**: o que o grupo 1 chama de "3", o grupo 2 chama de "7". É uma mudança de conceito. Comparamos global, só local, ajuste fino e **IFCA** com $K = 3$.

```
PYTHON
import numpy as np
from personalizacao import *

Xtr, Xte, ytr0, yte0 = carregar_digits()
print(f"{'semente':>7}{'global':>9}{'só local':>10}{'ajuste fino':>13}{'IFCA':>8}  grupos
recuperados?")
for s in range(3):
    partes, y, grupo = particao_conceitos(ytr0, n_clientes=12, grupos=3, seed=s)
    treino, teste = dividir_local(partes, 0.3, s)
```

```
    Wg, _ = fedavg(treino, Xtr, y, Xte, yte0, rodadas=30, epocas=5, eta=0.1, seed=s)
    local = [treino_local(np.zeros_like(Wg), Xtr[p], y[p], 30, 0.1, 32, 0.0, i) for i, p in
enumerate(treino)]
    ajuste = [treino_local(Wg, Xtr[p], y[p], 5, 0.1, 32, 0.0, i) for i, p in enumerate(treino)]
    modelos, escolha = ifca(treino, Xtr, y, K=3, seed=s)
    clusters = [modelos[j] for j in escolha]

    # os grupos foram recuperados se clientes do mesmo grupo escolheram o mesmo modelo, e grupos
distintos, modelos distintos
    mapa = {g: {escolha[k] for k in range(12) if grupo[k] == g} for g in range(3)}
    ok = all(len(v) == 1 for v in mapa.values()) and len({next(iter(v)) for v in mapa.values()}) == 3
    a = lambda Ws: acuracias_locais(Ws, teste, Xtr, y).mean()
    print(f"{s:>7}{a([Wg] * 12):>9.3f}{a(local):>10.3f}{a(ajuste):>13.3f}{a(clusters):>8.3f}  {'sim' if
ok else 'não'}")
```

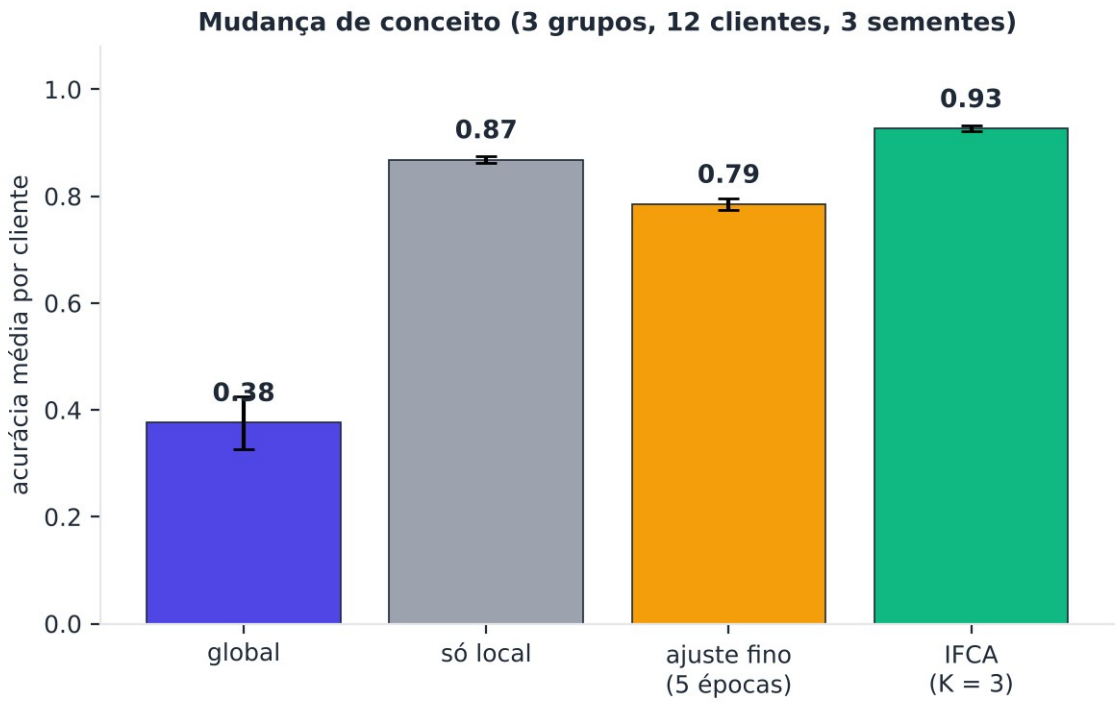


Figura 6.6 — Mudança de conceito: o modelo global falha (cerca de 0,35 a 0,45). O IFCA recupera os três grupos e atinge cerca de 0,93.

Interpretação:

- O **global** é desastroso: um só modelo não consegue servir a três "linguagens" diferentes.
- O **ajuste fino com 5 épocas** não basta para desfazer o que o global aprendeu.
- O **só local** vai bem (cada cliente aprende o seu conceito), mas usa só os dados do próprio cliente.
- O **IFCA** descobriu os três grupos sem que ninguém os informasse, e **superou todos**, porque compartilha dados (via modelos) **dentro do grupo certo**.

Memorize

Personalizar não é só "ajustar no final". Se os clientes têm **conceitos** diferentes, o melhor caminho pode ser **agrupar** primeiro, depois treinar em conjunto **dentro do grupo**.

Atenção

O IFCA exige escolher o número de grupos K e inicializar bem os modelos. Com inicializações ruins, dois grupos podem cair no mesmo modelo. Em casos reais, os grupos raramente são tão limpos quanto neste experimento de laboratório.

6.12 Custos e riscos da personalização

- **Comunicação:** o ajuste fino local não custa nada extra. O IFCA exige enviar K modelos por cliente.
- **Armazenamento:** modelos pessoais precisam de memória em cada cliente.
- **Privacidade:** modelos pessoais podem memorizar mais os dados do cliente. O Capítulo 7 discute ataques.
- **Novos clientes** (*cold start*): um cliente novo não tem dados para personalizar. Ele precisa começar com o global, ou ser associado a um grupo.

6.13 Resumo do capítulo

MEMORIZE **O essencial do Capítulo 6**

- O modelo global pode servir mal a certos clientes: meça **por cliente** (média, pior, percentil 10).
- **Ajuste fino:** continuar o treino com dados locais. **Ditto:** modelo pessoal com coleira λ ao global.
- **Partição:** corpo compartilhado, cabeça pessoal. **Agrupamento (IFCA):** um modelo por grupo.
- **Per-FedAvg:** um ponto de partida que se adapta rápido.
- Com **mudança de conceito**, o global falha e o agrupamento brilha.
- Modelos locais puros protegem a média, mas deixam mal quem tem pouco dado.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Qual é a principal motivação para personalizar modelos em aprendizado federado?

a) Aumentar o número de rodadas.
b) Reduzir o número de parâmetros.
c) O modelo global pode servir mal a certos clientes cujos dados são diferentes da média.
d) Eliminar o servidor.

Questão 2. Na analogia da camisa, o modelo global corresponde a:

a) a camisa de prateleira, boa para a média, mas que pode vestir mal alguns.
b) o alfaiate.
c) o tecido.
d) a loja fechada.

Questão 3. Por que a acurácia média pode esconder injustiças?

a) Porque a média só vale para IID.
b) Porque a média é sempre pior que a mediana.
c) Porque a média não é calculável.
d) Porque pode haver clientes com acurácia muito baixa compensados por outros com acurácia alta.

Questão 4. O que mede o percentil 10 da acurácia entre clientes?

a) O valor abaixo do qual estão os 10% piores clientes.
b) A acurácia do décimo cliente.
c) O número de clientes com 10 exemplos.
d) A média dos 10 melhores clientes.

Questão 5. Em que cada cliente deve ser testado para uma avaliação de personalização?

a) Em dados sintéticos aleatórios.
b) Em um teste global igual para todos.
c) Nos dados de outro cliente.
d) Em um teste local, feito com dados daquele cliente.

Questão 6. Em qual situação um único modelo global não pode servir a todos?

a) Quando o modelo é linear.
b) Quando todos os clientes têm os mesmos dados.
c) Em mudança de conceito, quando o mesmo x tem rótulos diferentes entre grupos.
d) Quando há um único cliente.

Questão 7. O ajuste fino local consiste em:

a) apagar o modelo global.
b) treinar do zero com dados globais.
c) enviar dados ao servidor.
d) continuar treinando o modelo global por alguns passos com dados locais.

Questão 8. Qual a vantagem do ajuste fino local em termos de comunicação?

a) Não custa comunicação extra.
b) Exige K modelos por cliente.
c) Dobra o tráfego.
d) Exige enviar todos os dados.

Questão 9. Qual o risco do ajuste fino com muitos passos?

a) O modelo fica mais genérico.
b) A taxa de aprendizado vira zero.
c) O servidor trava.
d) O cliente pode esquecer o que o global aprendeu e sobreajustar.

Questão 10. No Ditto, o modelo pessoal v_k é treinado para:

a) ignorar o dado local.
b) maximizar o erro local.
c) ser idêntico ao global.
d) minimizar o erro local com uma penalização que o mantém perto do modelo global.

Questão 11. No Ditto, $\lambda = 0$ corresponde a:

a) usar dados de todos os clientes.
b) usar apenas o modelo global.
c) eliminar o cliente.
d) treinar o modelo pessoal só com dados locais.

Questão 12. No Ditto, um λ muito grande faz com que:

a) o modelo pessoal fique igual ao local puro.
b) o servidor desapareça.
c) o modelo seja ignorado.
d) o modelo pessoal fique praticamente igual ao global.

Questão 13. Qual a diferença entre a coleira do FedProx e a do Ditto?

a) No FedProx, ela estabiliza o treino global; no Ditto, define o modelo pessoal, enquanto o global segue normal.
b) No FedProx a coleira é maior que no Ditto.
c) No Ditto não há coleira.
d) Não há diferença.

Questão 14. Na partição do modelo (FedRep), a "cabeça" é:

a) a camada final, pessoal e treinada apenas localmente.
b) a primeira camada, compartilhada.
c) o servidor.
d) o conjunto de teste.

Questão 15. Qual é uma vantagem adicional da partição corpo/cabeça?

a) Economizar comunicação, pois a cabeça não precisa ser enviada.
b) Aumentar o tamanho do modelo.
c) Eliminar a necessidade de dados.
d) Evitar o treino local.

Questão 16. O que o IFCA faz em cada rodada?

a) O servidor sorteia os grupos sem olhar os dados.
b) Todos os clientes usam o mesmo modelo.
c) Cada cliente escolhe o modelo de menor perda e o servidor faz a média dentro de cada grupo.
d) Os clientes enviam seus dados.

Questão 17. No IFCA, o que é preciso definir antes do treino?

a) O grupo verdadeiro de cada cliente.
b) A senha do servidor.
c) O número de grupos K.
d) O rótulo de cada cliente.

Questão 18. No experimento B (mudança de conceito), o modelo global teve acurácia média de cerca de:

a) 0,10.
b) 0,90.
c) 0,35 a 0,45.
d) 0,99.

Questão 19. No experimento B, o IFCA atingiu aproximadamente:

a) 0,10.
b) 0,40.
c) 0,60.
d) 0,93.

Questão 20. Por que o ajuste fino de 5 épocas foi pior que o modelo só local no experimento B?

a) Porque os dados eram IID.
b) Porque o local usa mais dados.
c) Porque o ajuste fino não funciona.
d) O modelo global partia de um ponto ruim para cada grupo, e 5 épocas não bastaram para corrigi-lo.

Questão 21. No experimento A ($\alpha = 0{,}1$), qual foi o ponto fraco do modelo só local?

a) Ele não pôde ser treinado.
b) Ele usou dados dos outros clientes.
c) A média foi 0,10.
d) O pior cliente caiu a cerca de 0,17, por ter pouquíssimos dados.

Questão 22. No experimento A, o ajuste fino e o Ditto aumentaram a média de cerca de 0,95 para:

a) cerca de 0,50.
b) 1,00 exato.
c) cerca de 0,97.
d) cerca de 0,60.

Questão 23. Qual a principal ideia do Per-FedAvg?

a) Encontrar um modelo inicial que se adapte rapidamente com poucos passos locais.
b) Quantizar atualizações.
c) Eliminar o servidor.
d) Aumentar o lote.

Questão 24. O que é o problema de cold start (cliente novo) na personalização?

a) O modelo some.
b) Os dados são apagados.
c) O servidor reinicia.
d) O cliente novo não tem dados para personalizar e precisa começar com o modelo global ou ser associado a um grupo.

Questão 25. Qual dos custos abaixo é típico de modelos pessoais?

a) Reduzir o número de clientes.
b) Dobrar o número de classes.
c) Memória adicional em cada cliente para guardar o modelo pessoal.
d) Aumentar a taxa de aprendizado.

Questão 26. Qual o risco de privacidade da personalização?

a) Modelos pessoais podem memorizar mais os dados do cliente.
b) Não há riscos.
c) Eles eliminam ataques.
d) Eles sempre aumentam a privacidade.

Questão 27. Qual família de personalização trata cada cliente como uma tarefa relacionada?

a) Esparsificação.
b) Multitarefa (por exemplo, MOCHA).
c) Quantização.
d) Padronização.

Questão 28. Quando o agrupamento de clientes é especialmente útil?

a) Quando não há dados.
b) Quando todos os clientes são idênticos.
c) Quando há só um cliente.
d) Quando há grupos de clientes com comportamentos ou conceitos distintos.

Questão 29. No experimento com cliente novo (projeto do capítulo), por que o modelo do grupo (IFCA) funcionou bem com apenas 5 exemplos?

a) Porque poucos exemplos bastam para identificar o grupo certo pela menor perda, e o modelo do grupo já está treinado.
b) Porque o global já era perfeito.
c) Porque o IFCA gera dados novos.
d) Porque o teste foi usado no treino.

Questão 30. Em que condição o ajuste fino com poucas épocas tende a falhar?

a) Quando o modelo global é um ponto de partida ruim para o cliente, como em mudança de conceito.
b) Quando o modelo é linear.
c) Quando há muitos dados locais.
d) Quando os dados são IID.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique, com a analogia da camisa, por que um único modelo global pode ser insuficiente.
2. Descreva as três medidas usadas para avaliar a qualidade por cliente e o que cada uma revela.
3. Diferencie mudança de rótulos e mudança de conceito e diga em qual delas o global falha mais.
4. Explique o ajuste fino local, citando uma vantagem e um risco.
5. Explique o papel do hiperparâmetro λ no Ditto e os dois extremos.
6. Compare Ditto e FedProx quanto ao papel do termo proximal.
7. Descreva a ideia de corpo compartilhado e cabeça pessoal e a vantagem em comunicação.
8. Explique o algoritmo IFCA em quatro passos.
9. Quais são as limitações do IFCA?
10. Interprete o resultado do experimento B para global, só local, ajuste fino e IFCA.
11. Interprete o experimento A: qual a vantagem de colaborar para clientes com poucos dados?
12. Por que o modelo só local pode ser ruim mesmo com boa média?
13. Explique o que é o Per-FedAvg e por que é mais custoso.
14. O que é cold start em aprendizado federado personalizado e como mitigá-lo?
15. Cite dois riscos ou custos da personalização.
16. Em que cenário você escolheria agrupamento em vez de ajuste fino?
17. Como você avaliaria um cliente novo em um sistema personalizado?
18. Por que é importante repetir os experimentos de personalização com várias sementes?
19. Resuma o capítulo em cinco linhas, destacando uma situação em que personalizar é indispensável.
20. Explique por que, no Ditto com $\lambda = 0$ partindo do modelo global, o resultado coincide com o ajuste fino local.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Avaliando o modelo global cliente a cliente

NÍVEL: BÁSICO

Crie 10 clientes com `particao_dirichlet(ytr, 10, alfa=0.3)`, divida cada um em treino e teste locais (`dividir_local`) e treine um modelo global com FedAvg (30 rodadas, `epocas=5`). Calcule a acurácia do global **em cada cliente** e use `resumo` para a média, o pior cliente e o percentil 10. Compare com a acurácia no teste global.

Pergunta: a acurácia global conta toda a história sobre o desempenho por cliente?

Exercício 2 — Quantas épocas de ajuste fino?

NÍVEL: BÁSICO

Com 10 clientes Dirichlet (α = 0,1) e o modelo global do FedAvg, aplique ajuste fino local com 0, 1, 3, 5, 10 e 30 épocas (`treino_local`, `eta=0.1`). Para cada valor, reporte a média, o pior cliente e o percentil 10 em 3 sementes.

Pergunta: mais épocas sempre ajudam?

Exercício 3 — O botão λ do Ditto

NÍVEL: INTERMEDIÁRIO

Para os mesmos 10 clientes Dirichlet (α = 0,1) e 3 sementes, varie $\lambda \in \{0; 0{,}01; 0{,}1; 0{,}5; 2; 10\}$ no Ditto. Compare a média, o pior cliente e o percentil 10 com os do global puro.

Pergunta: o que acontece com λ pequeno? E com λ grande?

Exercício 4 — IFCA com o número errado de grupos

NÍVEL: INTERMEDIÁRIO

Na mudança de conceito (12 clientes, 3 grupos verdadeiros), rode o `ifca` com $K \in \{2, 3, 5\}$. Para cada K, reporte a acurácia média por cliente (3 sementes) e quantos modelos distintos foram realmente escolhidos pelos clientes.

Pergunta: o que ocorre quando K é menor que o número real de grupos? E quando é maior?

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** *Adult* (agrupe por país ou ocupação para criar grupos naturais), *Human Activity Recognition Using Smartphones* (cada voluntário é um cliente, ótimo para personalização) e *Wine Quality*.
> - **Kaggle:** *Telco Customer Churn* (agrupe por tipo de contrato) e *Heart Failure Prediction*.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Chegou um cliente novo: o que fazer com poucos dados?

Contexto. A rede de clínicas (fictícia) tem 12 unidades, em 3 grupos com protocolos diferentes (mudança de conceito). Três unidades novas, uma de cada grupo, acabam de aderir, mas só têm **5, 20 ou 50** exemplos rotulados. A diretoria quer saber: *como atender essas unidades bem e rápido?*

Tarefas.

1. Use `particao_conceitos` (12 clientes, 3 grupos) e `dividir_local`. Reserve três clientes (um de cada grupo) como "novos", e treine global e IFCA (K = 3) **apenas com os outros nove**.
2. Para cada cliente novo, com apenas os `n` primeiros exemplos de treino ($n \in \{5, 20, 50\}$), avalie no seu teste local: (a) modelo global; (b) modelo treinado **só** com esses exemplos; (c) global com ajuste fino de 5 épocas; (d) o modelo do grupo (IFCA) escolhido pela menor perda nos `n` exemplos.
3. Reporte a acurácia média dos três clientes novos, em 3 sementes.
4. Escreva uma recomendação para a diretoria: como atender um cliente novo, sabendo que ele tem poucos dados?

Entregáveis. Código, tabela por `n` e recomendação com ressalvas.

A solução de referência está no Apêndice E.

PARTE III

Parte III — Privacidade e Segurança

O que ainda pode vazar mesmo sem enviar dados, como se proteger (privacidade diferencial, agregação segura, criptografia) e como resistir a participantes mal-intencionados.

Nesta parte:

- Cap. 7 — O Que Pode Vazar: Ameaças à Privacidade
- Cap. 8 — Protegendo os Dados: Privacidade Diferencial, Agregação Segura e Criptografia
- Cap. 9 — Quando Alguém Trapaceia: Robustez e Ataques de Envenenamento



Capítulo 7 — O Que Pode Vazar: Ameaças à Privacidade

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Descrever o que um adversário consegue observar em um sistema federado e quem pode atacar.
- Classificar as principais famílias de ataques: reconstrução, inferência de pertencimento, inferência de propriedades, memorização e ataques ativos.
- Explicar por que o gradiente de um único exemplo pode revelar a entrada.
- Executar uma inversão de gradientes e uma inferência de pertencimento em modelos simples.
- Relacionar o risco de vazamento ao sobreajuste, ao tamanho dos dados e ao tamanho do lote.
- Distinguir mitigação de garantia de privacidade.

7.1 O segredo que escapa

Voltemos às amigas dos cadernos de receitas. Elas combinaram enviar à coordenadora **só os ajustes**: "menos sal", "mais forno", "dobrar o açúcar".

Parece inofensivo. Mas imagine que Marta envie o ajuste "tirar toda a pimenta". A coordenadora conclui, sem ver o caderno, que a receita de Marta tinha **muita pimenta**.

Atualizações de modelo são assim: **não são os dados, mas carregam pistas sobre eles**. E, em certos casos, as pistas bastam para **reconstruir os dados** quase por completo.

> **No seu dia a dia**
>
> Alguém que vê só a lista de compras do seu mercado (leite sem lactose, fraldas, ração) consegue adivinhar muito sobre a sua casa, sem nunca ter entrado nela. Atualizações de modelo são a "lista de compras" dos dados de um cliente.

Neste capítulo, vamos **atacar de verdade** modelos simples, em laboratório, para entender o risco. O objetivo é **defensivo**: só quem conhece o ataque consegue se proteger dele.

> **Atenção**
>
> Os experimentos deste capítulo usam dados públicos (dígitos manuscritos) em modelos pequenos, para fins educacionais. Nunca teste ataques de privacidade em dados reais de pessoas sem autorização formal e aprovação ética.

7.2 Quem ataca e o que enxerga

Para falar de segurança, definimos um **modelo de ameaça**: quem é o adversário, o que ele sabe e o que ele pode fazer (Kairouz et al., 2021).

7.2.1 Os adversários

Adversário	O que é	O que enxerga
Servidor honesto-mas-curioso	Segue o protocolo, mas tenta aprender o que puder	Atualização individual de cada cliente
Servidor malicioso	Quebra o protocolo para extrair informação	Tudo acima, mais a possibilidade de enviar modelos manipulados
Cliente malicioso	Participante que ataca os outros	Modelos globais de cada rodada
Observador da rede	Espiona a comunicação	Mensagens, se não houver criptografia
Usuário do modelo final	Quem consulta o modelo pronto	Previsões e confianças (caixa-preta) ou os pesos (caixa-branca)

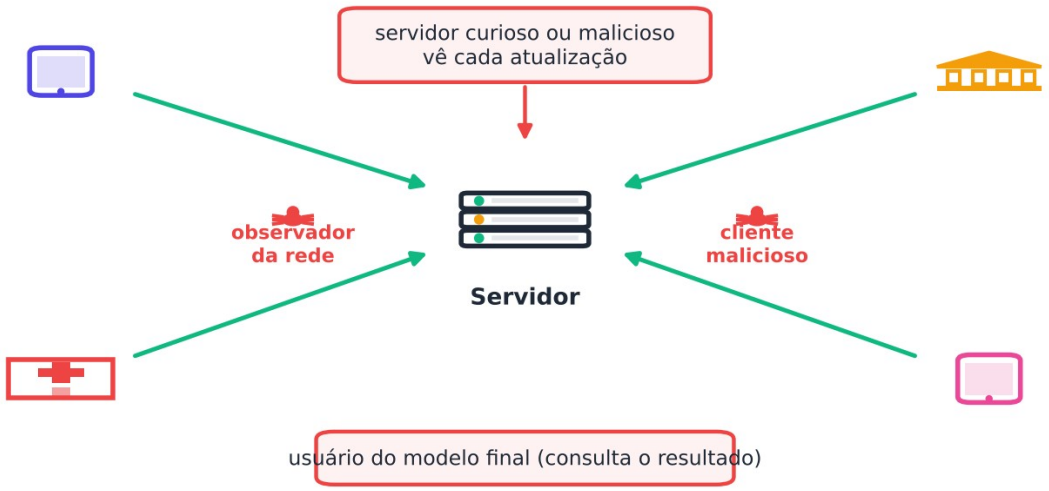


Figura 7.1 — Onde cada adversário observa o sistema. O servidor honesto-mas-curioso é o mais estudado: ele vê cada atualização individual.

7.2.2 O que trafega

Em cada rodada, o servidor recebe, de cada cliente, uma **atualização individual**. Também todos recebem o **modelo global**. Ao final, o **modelo treinado** pode ser publicado. Cada um desses três itens é uma fonte de vazamento.

7.3 Mapa dos ataques

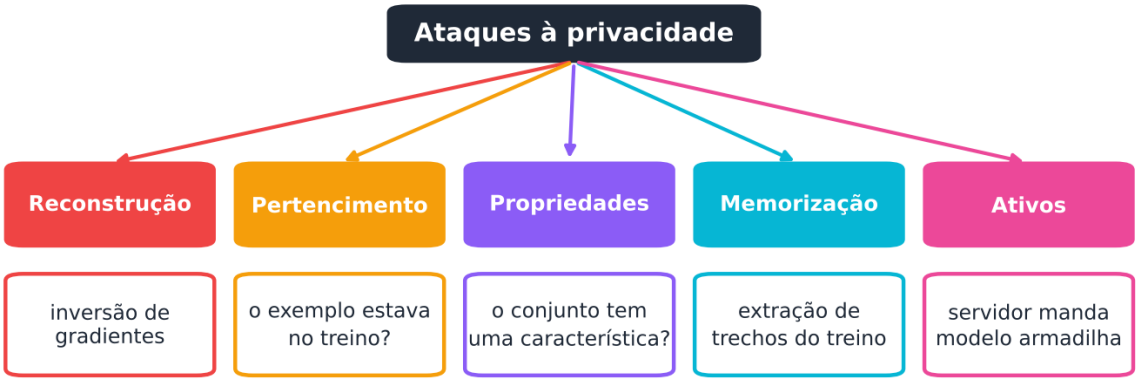


Figura 7.2 — Principais famílias de ataques à privacidade em aprendizado federado.

Ataque	Pergunta do adversário	Quem costuma atacar
Reconstrução (inversão de gradientes)	"Quais eram os dados de entrada?"	Servidor
Inferência de pertencimento	"Este exemplo foi usado no treino?"	Servidor, cliente, usuário do modelo
Inferência de propriedades	"Os dados do cliente têm esta característica?"	Servidor, cliente
Memorização e extração	"Que trechos do treino o modelo decorou?"	Usuário do modelo
Ataques ativos	"Posso manipular o modelo para facilitar o vazamento?"	Servidor malicioso

7.4 Inversão de gradientes: o gradiente entrega a entrada

7.4.1 A ideia

Zhu et al. (2019) mostraram que, em redes neurais, o gradiente de um lote pode ser usado para **recuperar as imagens de treino**. O atacante cria uma imagem falsa, calcula o gradiente dela e ajusta a imagem até que seu gradiente fique igual ao observado. Geiping et al. (2020) refinaram o ataque, e Huang et al. (2021) avaliaram sistematicamente ataques e defesas.

Em modelos simples, não precisamos de otimização: **a matemática entrega o resultado**.

7.4.2 Um caso em que a conta é exata

Considere a regressão logística multiclasse da `fedlib`. Para **um único exemplo** x com rótulo y, o gradiente da perda em relação aos pesos é:

$$\nabla W = x \otimes (p - y)$$

Em palavras: é o **produto externo** entre a entrada x e o vetor de erros $(p - y)$, em que p são as probabilidades previstas e y é o rótulo em formato one-hot (1 na classe certa, 0 nas demais). Cada coluna c do gradiente é x multiplicado pelo número $(p_c - y_c)$.

E o gradiente do **viés** é justamente *(p − y)*. Logo, dividindo uma coluna do gradiente de pesos pelo gradiente do viés correspondente, obtemos:

$$x = \nabla W[:, c] \,/\, \nabla b[c]$$

A entrada é recuperada **exatamente**, sem nenhum ruído.

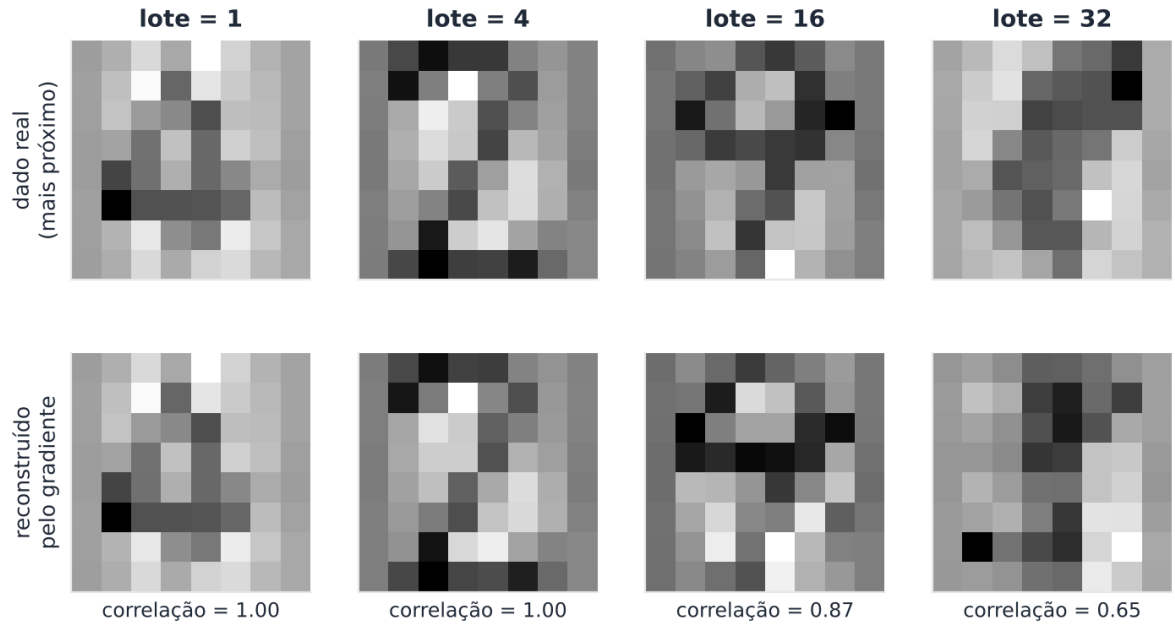


Figura 7.3 — Imagens de dígitos reconstruídas a partir do gradiente de um lote: com 1 exemplo, a reconstrução é perfeita; com lotes maiores, ela se mistura.

7.4.3 Pondo à prova

```python
import numpy as np
from fedlib import *

Xtr, Xte, ytr, yte = carregar_digits()
rng = np.random.default_rng(0)

# o atacante conhece o modelo global W, que foi enviado ao cliente
W, _ = fedavg(particao_iid(ytr, 10), Xtr, ytr, Xte, yte, rodadas=10, epocas=2, eta=0.1)

def gradiente_do_lote(idx):
    """O que o cliente envia: a soma dos gradientes dos exemplos do lote."""
    return gradiente(W, Xtr[idx], ytr[idx]) * len(idx)

def reconstruir(G):
    gb = G[-1]                          # última linha: gradiente do viés = soma de (p − y)
    c = int(np.argmax(np.abs(gb)))      # escolhe a classe com maior sinal
    return G[:-1, c] / gb[c]            # x = coluna de pesos / gradiente do viés

# caso exato: lote de 1 exemplo
i = int(rng.integers(len(ytr)))
x_rec = reconstruir(gradiente_do_lote([i]))
print("lote de 1: erro máximo de reconstrução =", float(np.abs(x_rec - Xtr[i, :-1]).max()))

# lotes maiores: quanto a melhor correlação com algum exemplo do lote?
print(f"{'tamanho do lote':>16} | correlação média (melhor exemplo)")
```

```
for B in (1, 2, 4, 8, 16, 32):
    cors = []
    for _ in range(30):
        idx = rng.choice(len(ytr), B, replace=False)
        xr = reconstruir(gradiente_do_lote(idx))
        cors.append(max(np.corrcoef(xr, Xtr[j, :-1])[0, 1] for j in idx))
    print(f"{B:>16} | {np.mean(cors):.3f}")
```

O que vemos:

- Com **um** exemplo, a reconstrução é **exata** (erro na casa do arredondamento).
- Com lotes maiores, o gradiente é uma **soma** de vários exemplos, e a reconstrução vira uma **mistura** parecida com algum deles.
- Mesmo com 32 exemplos, a imagem recuperada ainda se **parece bastante** com algum dado real.

> **Atenção**
>
> Este é o **melhor caso para o atacante**: modelo linear, um único passo de gradiente e acesso ao gradiente puro. Em redes profundas, com vários passos locais e muitos exemplos, o ataque é mais difícil, mas **não impossível**: os trabalhos citados reconstroem imagens reais de redes convolucionais.

7.5 Inferência de pertencimento

7.5.1 A pergunta

"O exemplo *z* participou do treino?" Parece uma pergunta pequena, mas é grave: se o modelo foi treinado com prontuários de uma clínica de tratamento de uma doença, saber que *z* foi usado revela que a pessoa é paciente.

7.5.2 O ataque mais simples: o limiar de perda

Modelos tendem a ter **perda menor** nos exemplos de treino do que em exemplos novos. O ataque do **limiar de perda** diz "membro" se a perda do exemplo for baixa (Yeom et al., 2018). Ataques mais sofisticados treinam "modelos-sombra" (Shokri et al., 2017) ou calibram o limiar por exemplo (Carlini et al., 2022). Nasr et al. (2019) estudaram esses ataques em cenários federados.

Mede-se o sucesso pela **AUC** (área sob a curva ROC): 0,5 é "chute"; 1,0 é ataque perfeito.

7.5.3 Pondo à prova

PYTHON

```
import numpy as np
from sklearn.metrics import roc_auc_score
from fedlib import *

Xtr, Xte, ytr, yte = carregar_digits()

def perdas(W, X, y):
    P = softmax(X @ W)
    return -np.log(P[np.arange(len(y)), y] + 1e-12)

print(f"{'dados de treino':>16}{'rodadas':>9}{'acurácia treino':>17}{'acurácia teste':>16}{'AUC do
ataque':>15}")
for n_membros, rodadas, epocas in [(150, 10, 2), (150, 50, 20), (600, 50, 20), (1347, 50, 20)]:
```

```
    aucs, a_tr, a_te = [], [], []
    for s in range(5):
        idx = np.random.default_rng(s).permutation(len(ytr))[:n_membros]       # membros
        partes = [idx[p] for p in particao_iid(ytr[idx], 6, s)]                  # 6 clientes
        W, _ = fedavg(partes, Xtr, ytr, Xte, yte, rodadas=rodadas, epocas=epocas, eta=0.1, seed=s)
        lm, ln = perdas(W, Xtr[idx], ytr[idx]), perdas(W, Xte, yte)             # membros x não membros
        rotulo = np.r_[np.ones(len(lm)), np.zeros(len(ln))]
        aucs.append(roc_auc_score(rotulo, -np.r_[lm, ln]))                       # perda baixa =>
membro
        a_tr.append(acuracia(W, Xtr[idx], ytr[idx])); a_te.append(acuracia(W, Xte, yte))
    print(f"{n_membros:>16}{rodadas:>9}{np.mean(a_tr):>17.3f}{np.mean(a_te):>16.3f}
{np.mean(aucs):>15.3f}")
```

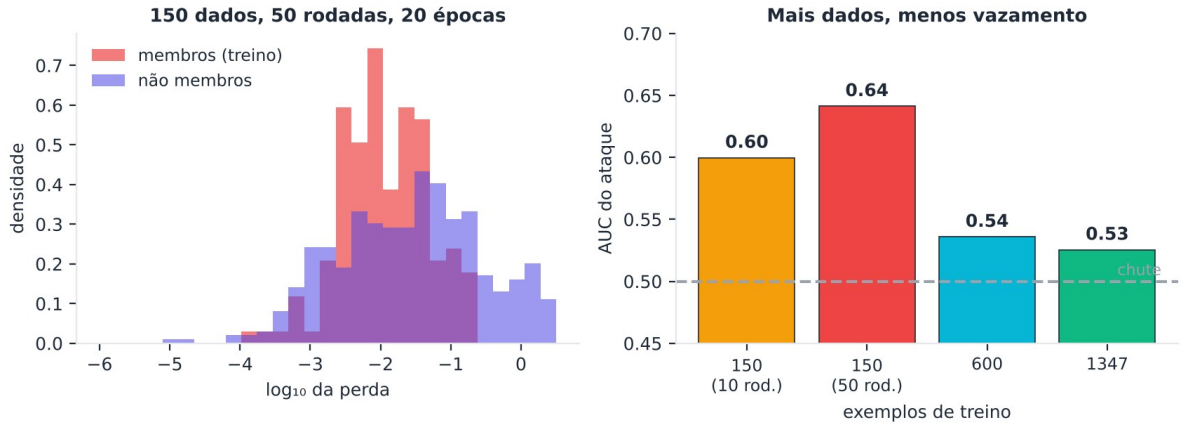


Figura 7.4 — À esquerda: perdas de membros e não membros (ataque com poucos dados e muitas rodadas). À direita: a AUC do ataque cai quando há mais dados de treino.

Resultados:

- Com **poucos dados e muito treino**, a AUC passa de **0,6**: o ataque acerta mais do que um chute.
- Com **mais dados**, a AUC se aproxima de **0,5**: o modelo "decora" menos cada exemplo.
- A lição: **sobreajuste e privacidade andam juntos**. Quanto mais o modelo decora, mais vaza.

> **Memorize**
>
> Risco de pertencimento ↑ quando: poucos dados, muitas épocas, modelo grande, sobreajuste forte. Risco ↓ quando: mais dados, regularização, parada antecipada.

7.6 Inferência de propriedades

Em vez de perguntar sobre **um exemplo**, o atacante pergunta sobre uma **propriedade do conjunto**: "os dados deste cliente incluem pessoas de óculos?". Melis et al. (2019) mostraram que um participante pode inferir propriedades que **não têm relação com a tarefa** do modelo, usando as atualizações do modelo compartilhado.

7.7 Memorização e extração

Modelos grandes, em especial de linguagem, **memorizam trechos** do treino. Carlini et al. (2021) extraíram, de um modelo de linguagem, sequências literais do treino, incluindo informações pessoais. Em modelos de fundação federados, esse risco continua presente (Capítulo 11).

7.8 Ataques ativos de um servidor malicioso

Se o servidor trapaceia, ele pode **enviar modelos construídos para vazar**. Fowl et al. (2022) mostraram que um servidor pode inserir, no modelo enviado, uma camada que "captura" os dados de cada exemplo, mesmo com lotes grandes. Boenisch et al. (2023) estenderam o ataque. Isso reforça o recado: **não basta confiar no servidor**.

7.9 Por que isso acontece?

- **Gradientes carregam informação.** Eles são funções diretas dos dados.
- **Modelos memorizam.** Principalmente com poucos dados, muitos parâmetros e muito treino.
- **O servidor controla o protocolo.** Ele escolhe o modelo enviado e vê as atualizações.
- **Clientes contribuem de forma desigual.** Quem tem poucos dados é mais exposto.

7.10 O que ajuda, e o que não ajuda

Algumas práticas **diminuem** o risco, mas **não são garantias**:

Prática	Efeito
Lotes maiores e mais passos locais	Mistura mais exemplos, dificultando a reconstrução
Regularização e parada antecipada	Menos memorização
Compressão e esparsificação	Menos informação enviada
Agregação segura	O servidor só vê a **soma** das atualizações (Capítulo 8)
Privacidade diferencial	Garantia formal por meio de ruído (Capítulo 8)
Criptografia homomórfica	Cálculos sobre dados cifrados (Capítulo 8)

> **Atenção**
>
> Esconder o servidor da atualização individual **não protege** contra o modelo final. A agregação segura, sozinha, não impede inferência de pertencimento contra o modelo publicado. Defesas se **combinam**.

7.11 Resumo do capítulo

> MEMORIZE **O essencial do Capítulo 7**
>
> - Atualizações **não são** os dados, mas **carregam pistas**.
> - Adversários: servidor curioso ou malicioso, clientes maliciosos, observadores e usuários do modelo.
> - Ataques: **reconstrução**, **pertencimento**, **propriedades**, **memorização** e **ativos**.
> - Na softmax, o gradiente de **um exemplo** é $x \otimes (p - y)$: a entrada é recuperada **exatamente**.
> - A inferência de pertencimento explora a **perda menor** nos exemplos de treino. Aumenta com sobreajuste e poucos dados.
> - Mitigações reduzem o risco; **garantias** exigem privacidade diferencial e agregação segura.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Qual a ideia central da analogia dos "ajustes das receitas" no início do capítulo?

a) Atualizações são sempre inofensivas.
b) O servidor nunca vê nada.
c) Atualizações de modelo não são os dados, mas carregam pistas sobre eles.
d) Receitas não podem ser alteradas.

Questão 2. O que é um modelo de ameaça?

a) Um método de compressão.
b) Um tipo de rede neural.
c) A descrição de quem é o adversário, o que ele sabe e o que pode fazer.
d) A lista de todos os algoritmos de aprendizado.

Questão 3. Um servidor honesto-mas-curioso:

a) não vê nada.
b) só envia dados falsos.
c) quebra o protocolo de propósito.
d) segue o protocolo, mas tenta aprender o que puder das atualizações.

Questão 4. Um servidor malicioso difere do honesto-mas-curioso porque:

a) não tem acesso às atualizações.
b) é sempre um cliente.
c) pode quebrar o protocolo, por exemplo, enviando modelos manipulados.
d) só atua depois do treino.

Questão 5. Quais itens trafegam e podem vazar informação em aprendizado federado?

a) Atualizações individuais, modelo global a cada rodada e modelo final.
b) Apenas o modelo final.
c) Apenas o código do servidor.
d) Nenhum.

Questão 6. A inversão de gradientes tem como objetivo:

a) reconstruir os dados de treino a partir dos gradientes ou atualizações.
b) acelerar o treino.
c) reduzir o modelo.
d) criptografar os dados.

Questão 7. Quem mostrou que gradientes de redes neurais podem recuperar imagens de treino (DLG)?

a) McMahan et al. (2017).
b) Zhu et al. (2019).
c) Pedregosa et al. (2011).
d) Reddi et al. (2021).

Questão 8. Para um único exemplo na softmax, o gradiente em relação aos pesos é:

a) o produto externo $x \otimes (p - y)$.
b) apenas p.
c) o rótulo y.
d) a soma dos pesos.

Questão 9. Na softmax com viés, o gradiente do viés é:

a) zero.
b) x.
c) (p − y).
d) a média de x.

Questão 10. Como se recupera x exatamente a partir do gradiente de um único exemplo?

a) Não é possível.
b) Somando todas as colunas.
c) Dividindo uma coluna do gradiente dos pesos pelo gradiente do viés correspondente.
d) Multiplicando por zero.

Questão 11. No experimento, com um lote de 1 exemplo, o erro de reconstrução foi:

a) 10%.
b) 50%.
c) da ordem de 1e-16 (arredondamento).
d) 100%.

Questão 12. O que acontece com a qualidade da reconstrução quando o lote cresce?

a) Ela fica perfeita.
b) Ela piora, pois o gradiente é uma soma de vários exemplos, mas ainda se parece com algum deles.
c) Ela melhora sempre.
d) Ela desaparece completamente.

Questão 13. Por que este experimento é considerado o melhor caso para o atacante?

a) Modelo linear, gradiente puro e um único passo.
b) Porque o servidor é honesto.
c) Porque há ruído.
d) Porque o atacante tem muitos dados.

Questão 14. Qual o objetivo da inferência de pertencimento?

a) Reconstruir todas as imagens.
b) Escolher hiperparâmetros.
c) Medir a acurácia.
d) Descobrir se um exemplo específico foi usado no treino.

Questão 15. Por que a inferência de pertencimento é grave?

a) Porque não envolve dados.
b) Saber que alguém está em um conjunto pode revelar informação sensível, como estar em um tratamento.
c) Porque aumenta a acurácia.
d) Porque reduz o tamanho do modelo.

Questão 16. No ataque do limiar de perda, um exemplo é classificado como "membro" quando:

a) seu tamanho é grande.
b) sua perda é baixa.
c) sua perda é alta.
d) seu rótulo é zero.

Questão 17. Uma AUC de 0,5 para o ataque de pertencimento significa:

a) o modelo é perfeito.
b) o ataque equivale a um chute.
c) o ataque é perfeito.
d) o ataque é pior que o acaso.

Questão 18. Uma AUC de 1,0 significa:

a) não há membros.
b) o ataque falha sempre.
c) o ataque separa perfeitamente membros de não membros.
d) o modelo está bem treinado.

Questão 19. No experimento de pertencimento, com poucos dados e muito treino a AUC foi de cerca de:

a) 0,99.
b) 0,64.
c) 0,50 exatamente.
d) 0,10.

Questão 20. Com 1.347 exemplos de treino, a AUC do ataque foi de cerca de:

a) 0,10.
b) 0,70.
c) 0,90.
d) 0,53.

Questão 21. Qual a relação entre sobreajuste e vazamento de pertencimento?

a) Quanto mais o modelo decora os exemplos, maior o risco.
b) Não há relação.
c) Só há relação em modelos lineares.
d) Quanto mais sobreajuste, menor o risco.

Questão 22. A inferência de propriedades (Melis et al., 2019) pergunta:

a) qual o rótulo de um exemplo.
b) qual a taxa de aprendizado.
c) se o conjunto de dados de um participante tem uma característica, mesmo sem relação com a tarefa.
d) quantos clientes existem.

Questão 23. Carlini et al. (2021) mostraram que modelos de linguagem podem:

a) ficar mais privados com mais parâmetros.
b) esquecer dados após cada época.
c) nunca memorizar.
d) memorizar e permitir a extração de sequências literais do treino.

Questão 24. Em um ataque ativo, um servidor malicioso pode:

a) apagar clientes.
b) corrigir rótulos.
c) enviar modelos construídos para facilitar o vazamento.
d) apenas observar.

Questão 25. Qual das práticas abaixo oferece garantia formal de privacidade?

a) Parar o treino mais cedo.
b) Privacidade diferencial.
c) Esparsificar atualizações.
d) Aumentar o lote.

Questão 26. A agregação segura, sozinha, protege contra:

a) nenhum ataque.
b) apenas observadores externos.
c) o servidor ver cada atualização individual, mas não contra inferência sobre o modelo final.
d) todos os ataques.

Questão 27. Qual o efeito de lotes maiores sobre a inversão de gradientes?

a) Facilitam a reconstrução.
b) Dificultam a reconstrução, pois o gradiente mistura mais exemplos.
c) Tornam a reconstrução exata.
d) Nenhum.

Questão 28. Qual dos clientes tende a ser mais exposto a ataques de pertencimento?

a) O que não participa.
b) O que tem muitos dados.
c) Nenhum.
d) O que tem poucos dados e muito treino.

Questão 29. Por que os experimentos do capítulo devem ser feitos só com dados públicos?

a) Porque dados públicos são melhores.
b) Porque é mais rápido.
c) Porque dados privados não funcionam.
d) Para evitar violar a privacidade de pessoas; ataques exigem autorização e aprovação ética.

Questão 30. Por que a parada antecipada (early stopping) pode ser considerada uma defesa barata contra inferência de pertencimento?

a) Porque aumenta o número de clientes.
b) Porque reduz a memorização do treino, diminuindo a diferença de perda entre membros e não membros.
c) Porque elimina o servidor.
d) Porque criptografa os dados.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique, com a analogia das receitas, por que enviar só as atualizações não garante privacidade.
2. Descreva três tipos de adversário em aprendizado federado e o que cada um observa.
3. Diferencie servidor honesto-mas-curioso de servidor malicioso.
4. Liste as cinco famílias de ataques à privacidade apresentadas e a pergunta que cada uma busca responder.
5. Demonstre por que o gradiente de um exemplo na softmax revela a entrada.
6. Explique por que o ataque do experimento é um "melhor caso" para o atacante e o que muda em redes profundas.
7. Interprete a tabela de correlação por tamanho de lote (1 a 32).
8. Explique o ataque do limiar de perda e como medir seu sucesso.
9. Por que mais dados de treino reduzem o risco de inferência de pertencimento?
10. Relacione sobreajuste, tamanho do modelo e privacidade.
11. O que é inferência de propriedades e dê um exemplo.
12. Explique por que modelos de linguagem representam um risco de memorização.
13. Descreva em linhas gerais um ataque ativo de servidor malicioso.
14. Diferencie mitigação e garantia de privacidade, com exemplos.
15. Por que a agregação segura não basta para proteger contra inferência de pertencimento sobre o modelo final?
16. Como o tamanho do lote e o número de passos locais afetam o risco de reconstrução?
17. Por que clientes com poucos dados são mais expostos?
18. Quais cuidados éticos devem existir ao testar ataques de privacidade?
19. Cite duas práticas de projeto que diminuem o risco de vazamento e uma limitação de cada uma.
20. Resuma em cinco linhas o que você aprendeu sobre ameaças à privacidade, indicando o que faria na prática.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Reconstruindo cinco dígitos, um por vez

NÍVEL: BÁSICO

Treine um modelo global com FedAvg (10 clientes IID, 10 rodadas, `epocas=2`). Para cinco exemplos de treino escolhidos ao acaso, calcule o gradiente de **um** exemplo e reconstrua a entrada com a fórmula `x = ∇W[:, c] / ∇b[c]`. Mostre o erro máximo de cada reconstrução e a classe prevista (índice `c` usado).

Pergunta: o que acontece com o erro se o modelo for inicializado com zeros (sem nenhum treino)?

Exercício 2 — Atualização com vários passos locais

NÍVEL: BÁSICO

Simule um cliente com `n` exemplos (n ∈ {1, 8, 32}) que treina `E` épocas (E ∈ {1, 5, 20}) com lote igual a `n` (um passo por época), `eta=0.1`. O atacante observa a atualização `(W − W_local)/eta` e tenta reconstruir a entrada como se fosse um gradiente. Para cada combinação, reporte a melhor correlação com algum exemplo do cliente (média de 20 repetições).

Pergunta: o que ocorre quando o cliente tem apenas 1 exemplo, mesmo com muitas épocas?

Exercício 3 — Quanto tempo de treino é demais?

NÍVEL: INTERMEDIÁRIO

Com 150 exemplos de treino, 6 clientes IID e `epocas=5`, treine por 5, 20 e 80 rodadas. Para cada duração, meça a acurácia no treino e no teste e a AUC do ataque do limiar de perda (membros contra o `Xte`), em 5 sementes.

Pergunta: o risco de pertencimento aumenta com o treino? E a acurácia no teste?

Exercício 4 — Escolhendo o limiar do atacante

NÍVEL: INTERMEDIÁRIO

Use o modelo federado do Exercício 3 (80 rodadas, 5 épocas, semente 0). Calcule, para o limiar de perda igual à **mediana das perdas dos membros**, a taxa de verdadeiros positivos (membros marcados como membros) e a taxa de falsos positivos (não membros marcados como membros). Use também `roc_curve` para encontrar o limiar que maximiza `TPR − FPR`.

Pergunta: o ataque erra muito? Ele serve para acusar alguém individualmente?

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** *Adult* e *Heart Disease*, para auditar modelos com dados tabulares sensíveis.
> - **Kaggle:** *Heart Failure Prediction* e *Credit Card Fraud Detection*, para estudar vazamento em dados desbalanceados.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Auditoria de privacidade: qual configuração aprovar?

Contexto. Uma clínica (fictícia) quer treinar um modelo federado e precisa de um **parecer de risco** antes de liberar o modelo. O comitê de ética aceita o modelo se a **AUC do ataque de pertencimento** for menor que **0,58** e se a acurácia de teste for a maior possível dentro desse limite.

Tarefas.

1. Considere as configurações: número de exemplos de treino $\in \{150, 600\}$; rodadas $\in \{10, 50\}$; épocas locais $\in \{2, 20\}$. Use 6 clientes IID.
2. Para cada configuração, em 5 sementes, calcule a acurácia de treino e de teste e a **média e o desvio da AUC** do ataque do limiar de perda.
3. Marque como **aprovada** a configuração com AUC média $< 0{,}58$.
4. Entre as aprovadas, indique a de maior acurácia de teste.
5. Escreva o parecer de risco em até cinco linhas, citando duas limitações da auditoria (por exemplo: o ataque usado é o mais simples; defesas formais não foram avaliadas).

Entregáveis. Código, tabela com AUC, acurácia e status e o parecer.

A solução de referência está no Apêndice E.



Capítulo 8 — Protegendo os Dados: Privacidade Diferencial, Agregação Segura e Criptografia

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Explicar a ideia da privacidade diferencial e interpretar os parâmetros ε e δ.
- Aplicar corte (*clipping*) e ruído gaussiano a atualizações federadas e medir a troca entre privacidade e acurácia.
- Calcular um limite de ε com contabilidade de Rényi.
- Explicar e simular a agregação segura por máscaras que se cancelam.
- Descrever a criptografia homomórfica aditiva e a computação multipartidária.
- Combinar defesas e reconhecer o que cada uma protege e o que não protege.

8.1 Camadas de defesa

No Capítulo 7, vimos que atualizações vazam informação. Agora, as defesas.

Pense em proteger uma casa. Você usa **várias camadas**: portão, fechadura, alarme, câmera. Nenhuma sozinha basta, e cada uma protege contra um tipo de invasor.

Em aprendizado federado, temos três grandes camadas:

- **Privacidade diferencial:** adiciona **ruído** para esconder a contribuição de cada indivíduo. É a única com **garantia matemática** sobre o que o modelo final revela.
- **Agregação segura:** faz o servidor ver só a **soma** das atualizações, e nunca cada uma isolada.
- **Criptografia homomórfica e computação multipartidária:** permitem **calcular sobre dados cifrados** ou repartidos.

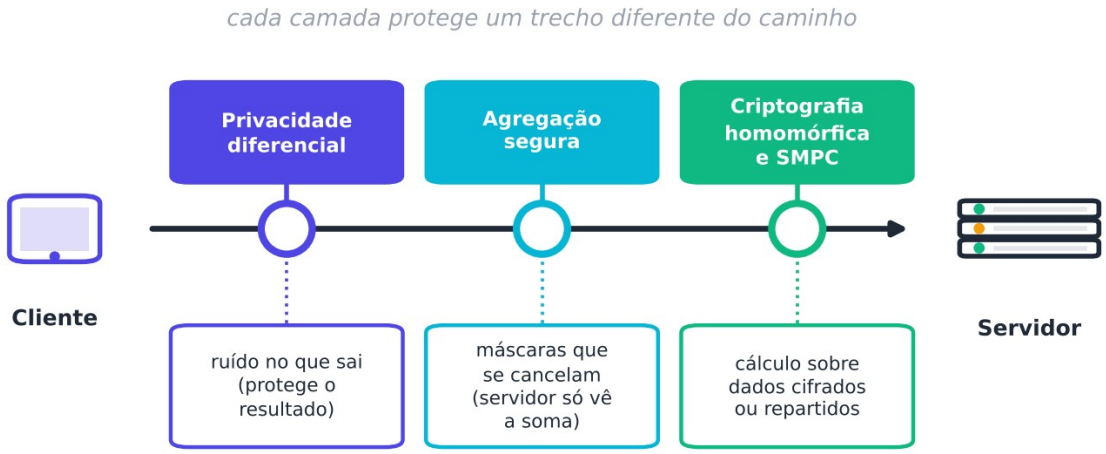


Figura 8.1 — Camadas de defesa: cada uma protege contra um adversário diferente. Combinadas, cobrem mais riscos.

> **No seu dia a dia**
>
> Pesquisas sobre temas delicados (por exemplo, "você já colou em uma prova?") sofrem com respostas falsas. Um truque antigo é o das **moedas**: antes de responder, cada pessoa joga uma moeda em segredo. É a mesma ideia da privacidade diferencial, e vamos testá-la já.

8.2 Privacidade diferencial

8.2.1 A intuição: a pesquisa da moeda

Queremos saber que **fração de pessoas** fez algo constrangedor. Cada pessoa joga uma moeda em segredo:

- **Cara:** responde a verdade.
- **Coroa:** joga outra moeda e responde "sim" (cara) ou "não" (coroa), ao acaso.

Quem vê uma resposta "sim" **não sabe** se é verdade ou sorteio. Cada indivíduo tem **negação plausível**. Mas o pesquisador, olhando para **muitas** respostas, consegue estimar a fração verdadeira, porque conhece o sorteio. Este é o método da **resposta aleatorizada** (Warner, 1965).

```python
import numpy as np

rng = np.random.default_rng(0)
n, taxa_verdadeira = 10000, 0.30
verdade = rng.random(n) < taxa_verdadeira              # quem "fez" de fato (segredo)

moeda1 = rng.random(n) < 0.5                           # cara: responde a verdade
moeda2 = rng.random(n) < 0.5                           # coroa: resposta sorteada
resposta = np.where(moeda1, verdade, moeda2)           # o que o pesquisador recebe

observado = resposta.mean()
estimado = 2 * observado - 0.5                         # desfaz o efeito do sorteio
print(f"taxa verdadeira  : {taxa_verdadeira:.3f}")
print(f"taxa observada   : {observado:.3f}   (enviesada pelo ruído)")
print(f"taxa estimada    : {estimado:.3f}   (corrigida)")
```

```
# o que um indivíduo "sim" revela? probabilidades de resposta
p_sim_dado_sim = 0.5 + 0.5 * 0.5
p_sim_dado_nao = 0.5 * 0.5
print(f"P(responder sim | fez)     = {p_sim_dado_sim:.2f}")
print(f"P(responder sim | não fez) = {p_sim_dado_nao:.2f}")
print(f"razão = {p_sim_dado_sim / p_sim_dado_nao:.1f}  ->  ε = ln(razão) = {np.log(p_sim_dado_sim /
p_sim_dado_nao):.3f}")
```

A **razão** entre as probabilidades de responder "sim" para quem fez e para quem não fez é 3. O logaritmo dessa razão, **ε ≈ 1,1**, mede o **vazamento** do mecanismo. É o parâmetro central da privacidade diferencial.

8.2.2 A definição

A **privacidade diferencial** (*differential privacy*, DP) foi proposta por Dwork et al. (2006) e é detalhada em Dwork e Roth (2014). Um mecanismo aleatório *M* é **(ε, δ)-diferencialmente privado** se, para quaisquer dois conjuntos de dados *D* e *D'* que diferem em **um único indivíduo** e para qualquer conjunto de saídas *S*:

$$P[\, M(D) \in S \,] \leq e^{\varepsilon} \cdot P[\, M(D') \in S \,] + \delta$$

Em palavras: **a presença ou ausência de qualquer pessoa muda muito pouco a distribuição das saídas**. Se a saída quase não depende de você estar nos dados, ela quase nada revela sobre você.

- **ε** (épsilon): o "orçamento de privacidade". **Menor é mais privado.** Valores de 0,1 a 1 são considerados fortes; acima de 10, a garantia é fraca.
- **δ** (delta): uma pequena probabilidade de falha da garantia. Costuma ser menor que 1/n, onde n é o número de pessoas.

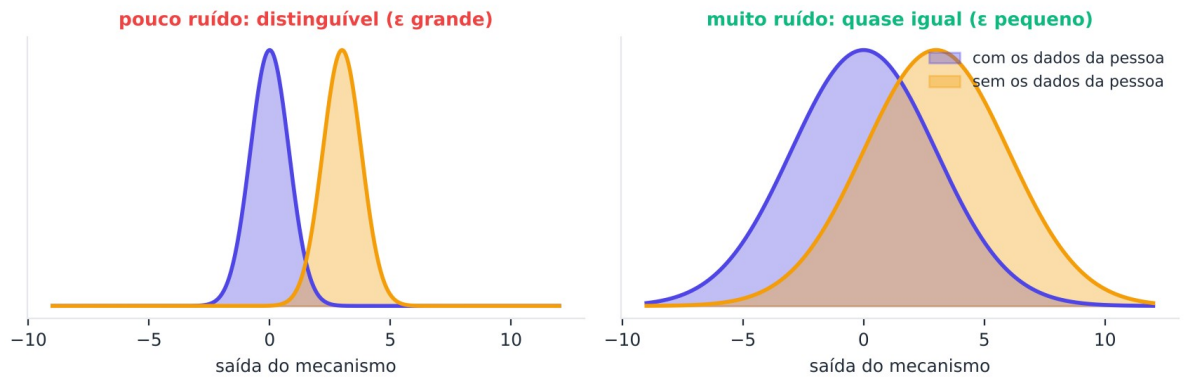


Figura 8.2 — Duas distribuições de saída: com os dados de uma pessoa (azul) e sem eles (laranja). Com pouco ruído, elas se distinguem; com muito ruído, quase se sobrepõem, e é isso que a DP exige.

> **Saiba mais**
>
> Um ponto sutil: ε não diz "quanto dos dados vaza", e sim **quanto a saída pode mudar** por causa de uma pessoa. A garantia vale **para qualquer adversário**, com qualquer informação auxiliar, o que a torna muito forte.

8.2.3 Como se obtém DP: sensibilidade e ruído

Para responder a uma consulta numérica *f* (por exemplo, a soma de atualizações), calculamos a **sensibilidade**: o **máximo** que *f* pode mudar quando uma pessoa entra ou sai.

$$\Delta = \max \| f(D) - f(D') \|$$

O **mecanismo gaussiano** soma ruído normal com desvio padrão proporcional à sensibilidade:

$$M(D) = f(D) + N(0, \sigma^2), \quad \sigma = z \cdot \Delta$$

Aqui, *z* é o **multiplicador de ruído**. Quanto maior *z*, maior a privacidade e menor a utilidade. Para uma única aplicação do mecanismo, uma escolha clássica é $\sigma = \Delta \cdot \sqrt{2\ln(1{,}25/\delta)} / \varepsilon$, válida para $\varepsilon < 1$ (Dwork e Roth, 2014).

8.2.4 Local, central e distribuída

Modelo	Quem adiciona ruído	Confiança necessária	Utilidade
Local	Cada cliente, antes de enviar	Nenhuma no servidor	Menor (muito ruído)
Central	O servidor, após agregar	Servidor confiável	Maior
Distribuída	Cada cliente, com ruído parcial + agregação segura	Servidor só vê a soma	Próxima da central

Na terceira linha, cada cliente soma uma **parte** do ruído, e a **agregação segura** faz o servidor ver só a soma já ruidosa. A soma de ruídos gaussianos independentes é gaussiana, e o total tem a variância necessária (Kairouz; Liu; Steinke, 2021).

8.2.5 DP em aprendizado federado

Há dois níveis de proteção.

- **DP no nível do exemplo:** esconde um exemplo individual (a ideia do DP-SGD, de Abadi et al., 2016).
- **DP no nível do cliente (ou usuário):** esconde **a participação inteira** de um cliente, com todos os seus dados. Esta é a mais natural em aprendizado federado (McMahan et al., 2018).

O algoritmo do **DP-FedAvg** tem dois ingredientes:

1. **Corte (*clipping*):** cada atualização é encolhida para ter norma no máximo *S*. Isso **limita a sensibilidade**: nenhum cliente consegue mudar a soma por mais que *S*.
2. **Ruído:** soma-se ao total das atualizações cortadas um ruído gaussiano com desvio $z \cdot S$. Dividindo por *m* (número de clientes), obtém-se a média ruidosa.

$$W \leftarrow W + \left(\Sigma_k \, \text{corte}_S(\Delta_k) + N(0, (z \cdot S)^2) \right) / m$$

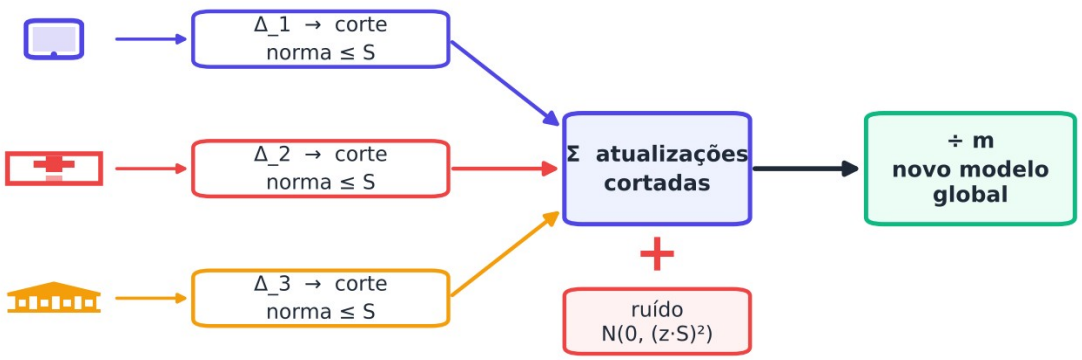


Figura 8.3 — DP-FedAvg: cada cliente corta sua atualização à norma S; o servidor soma, adiciona ruído gaussiano e divide pelo número de clientes.

> **Dica**
>
> O corte é um **compromisso**: se *S* for pequeno demais, você apaga informação útil; se for grande demais, precisa de mais ruído para proteger. Um bom ponto de partida é um valor próximo à **mediana** das normas das atualizações.

8.2.6 Contabilidade de privacidade

Cada rodada consome parte do orçamento. Com muitas rodadas, **os ε se acumulam**. Calcular o ε total é o papel da **contabilidade de privacidade** (*privacy accounting*).

Um método prático é a **contabilidade de Rényi** (Mironov, 2017). Para o mecanismo gaussiano com multiplicador *z*, aplicado *T* vezes, cada ordem α fornece o limite

$$\varepsilon(\alpha) = T \cdot \alpha / (2\, z^2) + \ln(1/\delta) / (\alpha - 1)$$

O ε final é o **menor** valor sobre todos os α > 1. A função abaixo, que também está em `privacidade.py`, calcula isso.

```python
import numpy as np

def epsilon_rdp(z, rodadas, delta=1e-5):
    alfas = np.arange(1.01, 400, 0.01)
    eps = rodadas * alfas / (2 * z ** 2) + np.log(1 / delta) / (alfas - 1)
    return float(eps.min())

print(f"{'z (ruído)':>10}{'ε após 30 rodadas':>20}{'ε após 300 rodadas':>21}")
for z in (0.5, 1, 2, 3, 5, 10):
    print(f"{z:>10}{epsilon_rdp(z, 30):>20.1f}{epsilon_rdp(z, 300):>21.1f}")
```

> **Atenção**
>
> Esse cálculo assume **participação total** dos clientes em cada rodada e **ignora** o ganho de privacidade que o sorteio de clientes traz (a "amplificação por subamostragem"). Por isso, é um **limite superior** (conservador). Bibliotecas como Opacus e TensorFlow Privacy fazem contas mais justas. Aqui, o objetivo é o raciocínio.

8.2.7 A biblioteca privacidade.py

Salve o código abaixo como **`privacidade.py`**, junto da `fedlib.py`.

```python
import numpy as np
from fedlib import *


def epsilon_rdp(z, rodadas, delta=1e-5):
    """Limite superior de epsilon para `rodadas` aplicações do mecanismo gaussiano com multiplicador de ruído z,
    com participação total (sem amplificação por sorteio). Usa a contabilidade de Rényi (Mironov, 2017)."""
    if z <= 0:
        return float("inf")
    alfas = np.arange(1.01, 400, 0.01)
    eps = rodadas * alfas / (2 * z ** 2) + np.log(1 / delta) / (alfas - 1)
```

```
    return float(eps.min())


def fedavg_dp(partes, X, y, Xte, yte, z=1.0, S=1.0, rodadas=30, epocas=5, eta=0.1, lote=32, seed=0,
classes=10):
    """FedAvg com privacidade diferencial no nível do cliente: corta cada atualização à norma S e soma
ruído gaussiano."""
    rng = np.random.default_rng(seed)
    W = np.zeros((X.shape[1], classes))
    m = sum(1 for p in partes if len(p) > 0)
    historico = []
    for r in range(rodadas):
        soma = np.zeros_like(W)
        for c, p in enumerate(partes):
            if len(p) == 0:
                continue
            d = treino_local(W, X[p], y[p], epocas, eta, lote, 0.0, seed * 1000 + r * 50 + c) - W
            norma = np.linalg.norm(d)
            soma += d * min(1.0, S / (norma + 1e-12))                    # corte (clipping)
        ruido = rng.normal(0.0, z * S, W.shape)                           # ruído calibrado pela
sensibilidade S
        W = W + (soma + ruido) / m
        historico.append(acuracia(W, Xte, yte))
    return W, historico
```

8.2.8 O experimento: privacidade contra utilidade

Treinamos o DP-FedAvg com **20 clientes** e com **100 clientes**, variando o multiplicador de ruído *z* e usando *S* = 0,5 (as normas típicas das atualizações são cerca de 1,5, então o corte age). Resultados com 3 sementes e 30 rodadas.

```
PYTHON
import numpy as np
from privacidade import *

Xtr, Xte, ytr, yte = carregar_digits()
print(f"{'clientes':>9}{'z':>6}{'ε (30 rodadas)':>16}{'acurácia':>10}")
for K in (20, 100):
    for z in (0, 0.3, 1, 3):
        acc = []
        for s in range(3):
            partes = particao_iid(ytr, K, s)
            _, h = fedavg_dp(partes, Xtr, ytr, Xte, yte, z=z, S=0.5, seed=s)
            acc.append(h[-1])
        eps = "∞" if z == 0 else f"{epsilon_rdp(z, 30):.1f}"
        print(f"{K:>9}{z:>6}{eps:>16}{np.mean(acc):>10.3f}")
```

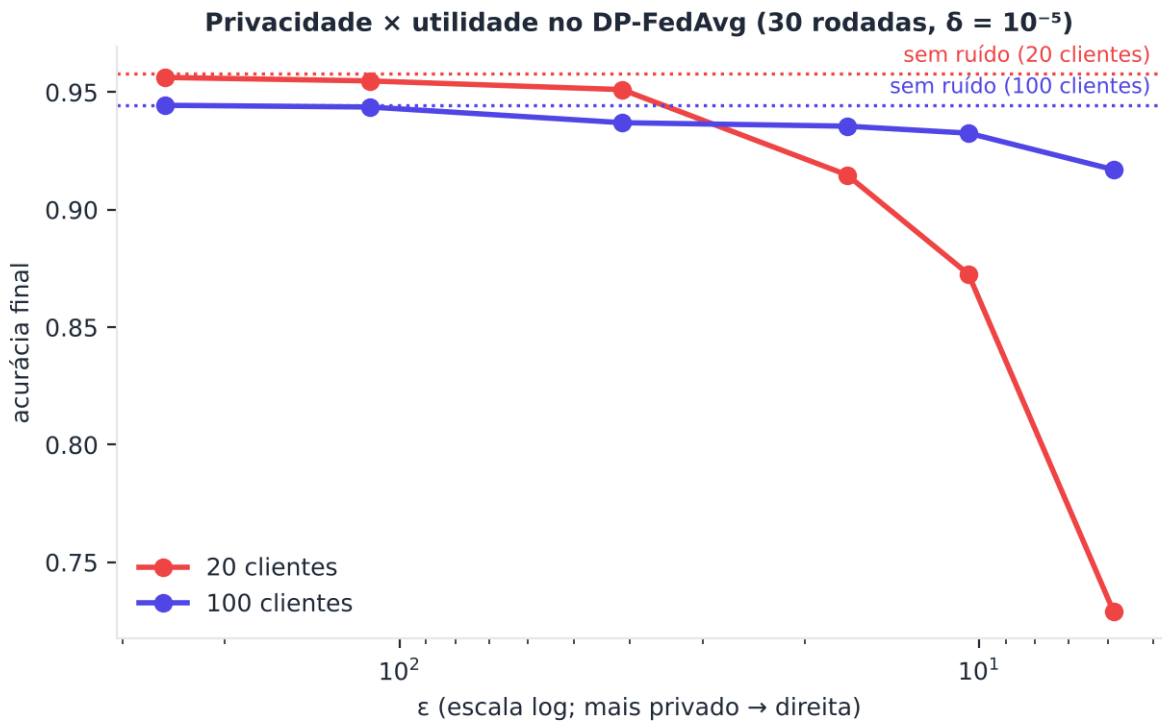


Figura 8.4 — Acurácia contra ε para 20 e 100 clientes. Com mais clientes, o mesmo ruído é "diluído" na média, e a acurácia sofre menos.

O que aprendemos:

- Com **20 clientes** e $z = 3$ ($\varepsilon \approx 10$), a acurácia cai bastante. O ruído, dividido por poucos clientes, pesa muito.
- Com **100 clientes**, o mesmo ruído quase não atrapalha. O ruído da média diminui com m.
- Mesmo com $z = 3$, o limite obtido é $\varepsilon \approx 10$: uma garantia **fraca**. Para $\varepsilon < 1$, seria preciso um ruído bem maior e, portanto, **muito mais clientes**.
- Privacidade forte ($\varepsilon < 1$) **exige muitos clientes**. Em aplicações reais de celulares, são milhares ou milhões.
- Privacidade tem **preço** em utilidade; a decisão de quanto pagar é uma escolha **social e jurídica**, e não só técnica.

> **Memorize**
>
> ε menor = mais privacidade = mais ruído = menor acurácia. Mais clientes e mais dados tornam a privacidade "mais barata". Corte e ruído limitam a influência de cada cliente.

8.2.9 DP contra os ataques do Capítulo 7

Vamos refazer a **inversão de gradientes** do Capítulo 7, agora com defesa. O cliente corta o gradiente de um exemplo à norma C e soma ruído gaussiano de desvio σ. O atacante usa a mesma fórmula de reconstrução.

```python
import numpy as np
from fedlib import *

Xtr, Xte, ytr, yte = carregar_digits()
rng = np.random.default_rng(0)
W, _ = fedavg(particao_iid(ytr, 10), Xtr, ytr, Xte, yte, rodadas=10, epocas=2, eta=0.1)

def reconstruir(G):
    gb = G[-1]; c = int(np.argmax(np.abs(gb)))
```

```
    return G[:-1, c] / gb[c]

C = 1.0
print(f"{'σ do ruído':>11} | correlação média da reconstrução (30 exemplos)")
for sigma in (0, 0.01, 0.1, 0.5, 1.0, 2.0):
    cors = []
    for i in rng.choice(len(ytr), 30, replace=False):
        G = gradiente(W, Xtr[i:i + 1], ytr[i:i + 1])
        G = G * min(1.0, C / np.linalg.norm(G))            # corte
        G = G + rng.normal(0, sigma * C, G.shape)          # ruído
        cors.append(np.corrcoef(reconstruir(G), Xtr[i, :-1])[0, 1])
    print(f"{sigma:>11} | {np.mean(cors):.3f}")
```

Sem ruído, a reconstrução é perfeita (correlação 1,0). Com ruído crescente, ela se degrada, e para valores grandes de σ, vira **ruído puro**. É a DP em ação: ela transforma a certeza do atacante em incerteza.

8.3 Agregação segura

8.3.1 A ideia: máscaras que se cancelam

A DP protege contra quem vê o **resultado**. Mas, durante o treino, o servidor vê as atualizações **individuais**. A **agregação segura** (*secure aggregation*, Bonawitz et al., 2017) resolve isso de modo engenhoso.

Imagine que cada par de clientes combina, em segredo, **um número aleatório** s. O cliente de menor índice **soma** s à sua atualização, e o de maior índice **subtrai** s. Para o servidor, cada atualização está **mascarada** e parece ruído puro. Mas, ao **somar tudo**, as máscaras se **cancelam**:

$$(\,u_1 + s_{12} + s_{13}\,) + (\,u_2 - s_{12} + s_{23}\,) + (\,u_3 - s_{13} - s_{23}\,) = u_1 + u_2 + u_3$$

O servidor obtém **apenas a soma**.

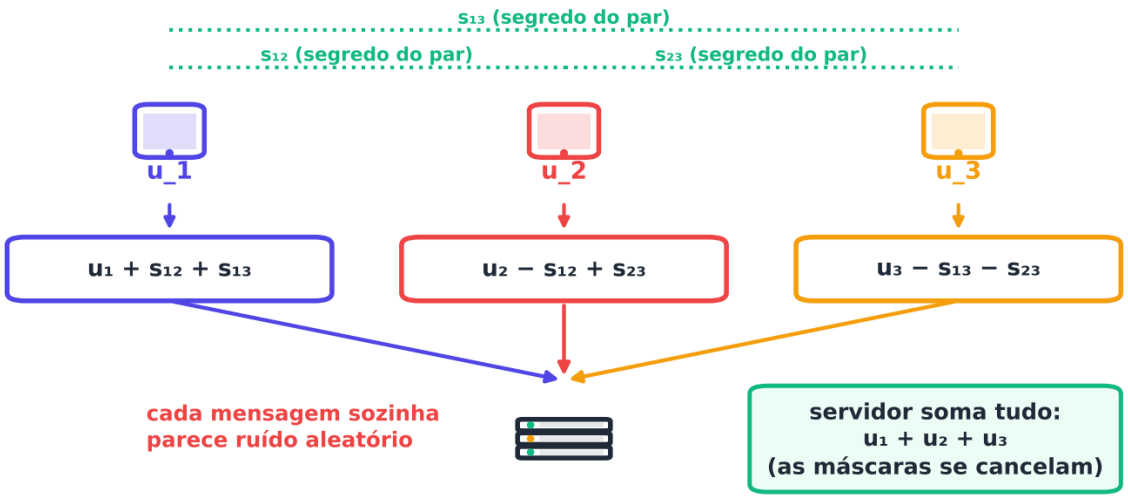


Figura 8.5 — Agregação segura com três clientes: cada par compartilha um segredo; o cliente de índice menor soma e o de índice maior subtrai. As máscaras desaparecem na soma.

8.3.2 Simulação

```
PYTHON
import numpy as np
```

```
rng = np.random.default_rng(0)
K, d = 5, 6
atualizacoes = [rng.normal(0, 1, d) for _ in range(K)]

# segredo compartilhado por cada par (i, j), com i < j; na prática, vem de um acordo de chaves
segredo = {(i, j): int(rng.integers(1 << 31)) for i in range(K) for j in range(i + 1, K)}

def mascara(i):
    m = np.zeros(d)
    for j in range(K):
        if j == i:
            continue
        v = np.random.default_rng(segredo[(min(i, j), max(i, j))]).normal(0, 100, d)   # máscara grande
        m += v if i < j else -v
    return m

enviado = [u + mascara(i) for i, u in enumerate(atualizacoes)]       # o que o servidor recebe

print("cliente 0, original :", np.round(atualizacoes[0], 2))
print("cliente 0, mascarado:", np.round(enviado[0], 2))
print("soma verdadeira     :", np.round(sum(atualizacoes), 4))
print("soma no servidor    :", np.round(sum(enviado), 4))
print("somas iguais?", np.allclose(sum(enviado), sum(atualizacoes)))
```

8.3.3 O que falta no protocolo real

O protocolo de Bonawitz et al. (2017) resolve problemas que a nossa simulação ignora:

- **Acordo de chaves:** os pares geram seus segredos com o protocolo de Diffie-Hellman, sem confiar no servidor.
- **Clientes que somem:** se um cliente sai no meio, suas máscaras não se cancelam. O protocolo usa **compartilhamento de segredos** (Shamir, 1979) para que os outros reconstruam as máscaras do que sumiu.
- **Servidor malicioso:** uma **dupla máscara** evita que o servidor engane os clientes para revelar atualizações.
- **Custo:** cresce com o quadrado do número de clientes (cada par troca segredos). Versões posteriores reduzem isso.

> **Atenção**
>
> A agregação segura esconde cada atualização **individual**, mas **não impede** que o **resultado** (a soma ou o modelo final) vaze informação. Para essa proteção, é preciso privacidade diferencial. Por isso, as duas se combinam.

8.4 Criptografia homomórfica e computação multipartidária

8.4.1 Calcular sem abrir o cofre

A **criptografia homomórfica** permite operar sobre dados **cifrados**, obtendo um resultado cifrado que, ao ser decifrado, equivale à operação feita sobre os dados originais. É como trabalhar com luvas dentro de um cofre trancado.

Há três níveis. A criptografia **parcialmente homomórfica** suporta uma única operação, como a **soma** (por exemplo, o esquema de Paillier, 1999). A **totalmente homomórfica** suporta somas e multiplicações arbitrárias (Gentry, 2009), mas é muito mais custosa.

Para o aprendizado federado, a **soma** costuma bastar: os clientes cifram suas atualizações, o servidor soma os textos cifrados, e só quem tem a chave decifra a soma.

8.4.2 Um Paillier de brinquedo

O código abaixo implementa o esquema de Paillier com chaves **minúsculas e inseguras**, só para mostrar a propriedade aditiva. **Nunca use este código em produção.**

```python
import math, random

def eh_primo(n, k=20):
    if n < 4:
        return n in (2, 3)
    if n % 2 == 0:
        return False
    d, r = n - 1, 0
    while d % 2 == 0:
        d //= 2; r += 1
    for _ in range(k):
        x = pow(random.randrange(2, n - 1), d, n)
        if x in (1, n - 1):
            continue
        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False
    return True

def primo(bits):
    while True:
        p = random.getrandbits(bits) | (1 << (bits - 1)) | 1
        if eh_primo(p):
            return p

random.seed(1)
p, q = primo(32), primo(32)                  # chaves minúsculas: INSEGURO, só didático
n = p * q; n2 = n * n; g = n + 1
lam = math.lcm(p - 1, q - 1)
mu = pow((pow(g, lam, n2) - 1) // n, -1, n)

def cifrar(m):
    r = random.randrange(2, n - 1)
    while math.gcd(r, n) != 1:
        r = random.randrange(2, n - 1)
    return (pow(g, m, n2) * pow(r, n, n2)) % n2

def decifrar(c):
    return ((pow(c, lam, n2) - 1) // n * mu) % n

# três clientes com valores inteiros (na prática, atualizações em ponto fixo)
valores = [120, 345, 77]
```

```
cifrados = [cifrar(v) for v in valores]
print("textos cifrados (parecem aleatórios):", [str(c)[:12] + "..." for c in cifrados])

produto = 1
for c in cifrados:
    produto = (produto * c) % n2            # multiplicar cifrados = somar os valores originais
print("soma decifrada:", decifrar(produto), "| soma verdadeira:", sum(valores))
```

8.4.3 Computação multipartidária

A **computação multipartidária segura** (*secure multiparty computation*, SMPC) permite que várias partes calculem juntas uma função dos seus dados privados, **sem revelar** as entradas. Técnicas comuns usam **compartilhamento de segredos**: cada valor é dividido em partes aleatórias, e nenhuma parte sozinha revela nada. A agregação segura é, na verdade, um caso especial de SMPC.

8.4.4 Ambientes de execução confiáveis

Outra alternativa é rodar a agregação em um **ambiente de execução confiável** (*trusted execution environment*, TEE): uma área protegida do processador em que nem o dono da máquina enxerga o que está sendo calculado. Funciona bem na prática, mas **exige confiar no fabricante** do hardware.

8.5 Comparando as defesas

Defesa	Protege contra	Não protege contra	Custo
Privacidade diferencial	Inferência sobre o modelo final e sobre as atualizações	— (mas custa utilidade)	Perda de acurácia
Agregação segura	Servidor ver atualizações individuais	Vazamento pelo modelo final	Comunicação e complexidade
Homomórfica	Servidor ver atualizações individuais	Vazamento pelo resultado	Computação alta
SMPC	Revelação de entradas	Vazamento pelo resultado	Comunicação alta
TEE	Operador da máquina	Falhas do hardware, ataques de canal lateral	Confiança no fabricante

Memorize

- **DP** protege o **resultado** e dá **garantia formal**.
- **Agregação segura, homomórfica e SMPC** protegem o **caminho** (as atualizações individuais).
- A melhor prática combina as duas famílias: **DP distribuída + agregação segura**.

8.6 Resumo do capítulo

MEMORIZE **O essencial do Capítulo 8**

- **DP:** a saída quase não muda se uma pessoa entra ou sai. ε menor = mais privacidade.
- **DP-FedAvg:** corte da atualização à norma S + ruído gaussiano z·S.
- Contabilidade de Rényi: ε cresce com o número de rodadas e diminui com z.
- Privacidade forte exige **muitos clientes**; há um preço em acurácia.
- **Agregação segura:** máscaras de pares que se cancelam na soma.
- **Homomórfica:** somar cifrados = cifrar a soma. **SMPC:** compartilhamento de segredos.
- Defesas se combinam: nenhuma protege tudo.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Qual defesa oferece garantia matemática formal sobre o que o modelo revela de cada indivíduo?

a) Privacidade diferencial.
b) Agregação segura.
c) Compressão das atualizações.
d) Aumento do lote.

Questão 2. Na pesquisa da moeda (resposta aleatorizada), por que cada pessoa tem "negação plausível"?

a) Porque a resposta "sim" pode ter vindo do sorteio, e não da verdade.
b) Porque as respostas são criptografadas.
c) Porque ninguém responde.
d) Porque o pesquisador ignora as respostas.

Questão 3. No experimento da moeda, a taxa verdadeira era 0,30 e a observada 0,41. A estimativa corrigida é obtida por:

a) observada − 0,3.
b) observada + 0,5.
c) observada × 0,5.
d) 2 × observada − 0,5.

Questão 4. Na resposta aleatorizada com duas moedas, P(sim | fez) = 0,75 e P(sim | não fez) = 0,25. Qual o valor de ε?

a) ln 3 ≈ 1,1.
b) 0.
c) 0,25.
d) 3.

Questão 5. Na definição de (ε, δ)-DP, o que significa um ε pequeno?

a) Mais acurácia garantida.
b) Menos privacidade.
c) Mais clientes.
d) Mais privacidade: a saída quase não muda com a presença de um indivíduo.

Questão 6. O parâmetro δ na (ε, δ)-DP representa:

a) o número de clientes.
b) uma pequena probabilidade de falha da garantia.
c) a taxa de aprendizado.
d) o tamanho do lote.

Questão 7. Em geral, valores de ε considerados fracos em termos de garantia são:

a) exatamente 0.
b) acima de 10.
c) entre 0,1 e 1.
d) menores que 0,01.

Questão 8. O que é sensibilidade (Δ) de uma consulta f?

a) O número de parâmetros.
b) A média de f.
c) O máximo que f pode mudar quando uma pessoa entra ou sai do conjunto.
d) A taxa de erro.

Questão 9. No mecanismo gaussiano, o desvio padrão do ruído é:

a) $\sigma = \Delta / z$.
b) $\sigma = z \cdot \Delta$.
c) $\sigma = 0$.
d) $\sigma = z + \Delta$.

Questão 10. Aumentar o multiplicador de ruído z produz:

a) nenhuma mudança.
b) mais privacidade e menos utilidade (acurácia).
c) menos clientes.
d) menos privacidade e mais utilidade.

Questão 11. Na DP local, o ruído é adicionado:

a) por cada cliente, antes de enviar.
b) apenas no modelo final.
c) pelo servidor, após agregar.
d) pelo observador de rede.

Questão 12. Na DP central, o ruído é adicionado:

a) pelo usuário final.
b) por um terceiro anônimo.
c) por cada cliente.
d) pelo servidor, após agregar, exigindo confiança nele.

Questão 13. Na DP distribuída:

a) cada cliente soma uma parte do ruído e a agregação segura faz o servidor ver só a soma ruidosa.
b) o ruído é removido na soma.
c) apenas o servidor adiciona ruído.
d) ninguém adiciona ruído.

Questão 14. DP no nível do cliente protege:

a) a participação inteira de um cliente, com todos os seus dados.
b) apenas um exemplo.
c) apenas rótulos.
d) o servidor.

Questão 15. Qual o papel do corte (clipping) no DP-FedAvg?

a) Aumentar a acurácia.
b) Limitar a sensibilidade: nenhuma atualização muda a soma por mais que S.
c) Comprimir o modelo.
d) Criptografar a atualização.

Questão 16. No DP-FedAvg, o ruído gaussiano tem desvio padrão:

a) z.
b) $z \cdot S$.
c) S^2.
d) S / z.

Questão 17. O que acontece se o valor de corte S for pequeno demais?

a) O modelo fica perfeito.
b) O servidor falha.
c) O ruído deixa de ser necessário.
d) Apaga-se parte da informação útil das atualizações.

Questão 18. Um bom ponto de partida para escolher S é:

a) o número de clientes.
b) zero.
c) um valor próximo à mediana das normas das atualizações.
d) infinito.

Questão 19. A contabilidade de privacidade serve para:

a) medir a taxa de aprendizado.
b) contar clientes.
c) calcular a acurácia.
d) calcular o ε acumulado ao longo de muitas rodadas.

Questão 20. Na contabilidade de Rényi para o mecanismo gaussiano, ε(α) é:

a) $T\cdot\alpha/(2z^2) + \ln(1/\delta)/(\alpha - 1)$, e usa-se o menor valor sobre α.
b) $\alpha \cdot \delta$.
c) T / z.
d) $z \cdot T$.

Questão 21. Para z = 3 e 30 rodadas ($\delta = 10^{-5}$), o limite de ε obtido foi de cerca de:

a) 0,1.
b) 254.
c) 1.000.
d) 10,4.

Questão 22. Por que o cálculo de ε do livro é um limite superior?

a) Porque ignora a amplificação de privacidade pelo sorteio de clientes.
b) Porque usa criptografia.
c) Porque ε é sempre maior.
d) Porque usa poucos clientes.

Questão 23. No experimento, com 20 clientes e z = 3, a acurácia caiu para cerca de:

a) 0,10.
b) 0,96.
c) 0,87.
d) 0,50.

Questão 24. Com 100 clientes e z = 3, a acurácia foi cerca de:

a) 0,10.
b) 0,70.
c) 0,50.
d) 0,93.

Questão 25. Qual a conclusão sobre o número de clientes em DP?

a) Mais clientes pioram a privacidade.
b) Privacidade forte exige muitos clientes.
c) Poucos clientes bastam.
d) O número de clientes não importa.

Questão 26. Na agregação segura por máscaras, o que acontece ao somar as mensagens de todos os clientes?

a) As máscaras se cancelam e o servidor obtém a soma verdadeira.
b) O servidor obtém apenas ruído.
c) As atualizações são destruídas.
d) O servidor obtém cada atualização individual.

Questão 27. Qual das opções NÃO é um problema que o protocolo real de agregação segura resolve em relação à simulação do capítulo?

a) Acordo de chaves entre pares.
b) Servidor malicioso.
c) Clientes que desistem no meio.
d) Esconder o modelo final do usuário.

Questão 28. Por que a agregação segura isolada não impede a inferência de pertencimento sobre o modelo final?

a) Porque é lenta.
b) Porque não envolve servidor.
c) Porque usa pouca memória.
d) Porque protege atualizações individuais, mas o resultado agregado ainda pode vazar informação.

Questão 29. A criptografia parcialmente homomórfica, como o Paillier, permite:

a) somar valores cifrados sem decifrá-los.
b) comprimir dados.
c) multiplicar e somar quaisquer valores.
d) decifrar sem chave.

Questão 30. Por que o ruído da média no DP-FedAvg diminui quando há mais clientes?

a) Porque o ruído é removido pelo servidor.
b) Porque o corte desaparece.
c) Porque cada cliente envia menos dados.
d) Porque o ruído é somado à soma das atualizações e depois dividido por m.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique, com a pesquisa da moeda, a ideia de privacidade diferencial.
2. Defina (ε, δ)-privacidade diferencial em palavras e interprete ε.
3. Explique o que é sensibilidade e como o mecanismo gaussiano a utiliza.
4. Compare DP local, central e distribuída quanto a confiança e utilidade.
5. Descreva os dois ingredientes do DP-FedAvg e o papel de cada um.
6. Por que privacidade forte exige muitos clientes? Use o experimento como argumento.
7. Calcule o desvio padrão do ruído somado à soma de atualizações com S = 0,5 e z = 4, e o desvio do ruído na média de m = 50 clientes.
8. Explique por que os ε se acumulam ao longo das rodadas.
9. Interprete a tabela de ε para z = 0,5 até 10 e 30 contra 300 rodadas.
10. Por que o ε calculado no livro é conservador?
11. Explique como a agregação segura por máscaras funciona com três clientes.
12. Quais problemas o protocolo real de agregação segura resolve além da simulação simples?
13. Explique por que agregação segura e DP se complementam.
14. Explique a propriedade homomórfica aditiva do esquema de Paillier no exemplo do capítulo.
15. O que é computação multipartidária segura e como se relaciona com a agregação segura?
16. Qual a diferença entre criptografia parcialmente e totalmente homomórfica?
17. Descreva um ambiente de execução confiável (TEE) e sua principal limitação.
18. Interprete o experimento de defesa contra inversão de gradientes.
19. Dê um exemplo de decisão de projeto em que privacidade e acurácia entram em conflito e como você a resolveria.
20. Resuma em cinco linhas as defesas estudadas e o que cada uma protege.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Resposta aleatorizada: privacidade contra precisão

NÍVEL: BÁSICO

Generalize a pesquisa da moeda: com probabilidade `p` a pessoa responde a verdade; com probabilidade `1 − p`, responde "sim" ou "não" por sorteio (50% cada). Para p ∈ {0,25; 0,5; 0,75}, calcule o ε do mecanismo e o erro quadrático médio (RMSE) da estimativa da taxa verdadeira (0,30) com n = 2.000 respostas, repetindo 300 vezes.

Pergunta: o que ocorre com o erro quando a privacidade aumenta (ε menor)?

Exercício 2 — Qual ruído preciso para ε = 2?

NÍVEL: BÁSICO

Usando `epsilon_rdp` (de `privacidade.py`), encontre por bisseção o menor multiplicador de ruído `z` que garante ε ≤ 2 com $\delta = 10^{-5}$ para 10, 30, 100 e 300 rodadas.

Pergunta: como o ruído necessário cresce com o número de rodadas?

Exercício 3 — Escolhendo a norma de corte S

NÍVEL: INTERMEDIÁRIO

Com 50 clientes IID, `z = 1`, 30 rodadas e 3 sementes, rode `fedavg_dp` para S ∈ {0,1; 0,5; 2; 5}. Calcule a acurácia final média. Mostre também a norma média das atualizações sem corte (mediana na primeira rodada).

Pergunta: por que nem um S muito pequeno nem um muito grande é o melhor?

Exercício 4 — Agregação segura e clientes que desistem

NÍVEL: INTERMEDIÁRIO

Simule 6 clientes com a agregação segura por máscaras. O cliente 3 envia a mensagem mascarada, mas **desiste** antes de a rodada terminar (o servidor não recebe a dele). (a) Mostre que a soma recebida **não** é a soma das atualizações dos sobreviventes. (b) Implemente a **correção**: o servidor, ao receber (por compartilhamento de segredos) as sementes do cliente que desistiu, subtrai as máscaras residuais. Verifique que a soma corrigida é exata.

Pergunta: por que as máscaras não se cancelam quando alguém some?

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** *Adult* e *Statlog (German Credit Data)*, para estudar DP em dados financeiros e censitários.
> - **Kaggle:** *Heart Failure Prediction* e *Credit Card Fraud Detection*.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Orçamento de privacidade da clínica

Contexto. Uma clínica (fictícia) treinará um modelo federado com os dados de poucos pacientes por cliente. O comitê de ética quer entender o **preço** de proteger o modelo: o quanto cada nível de privacidade diminui a acurácia e o quanto reduz o risco de inferência de pertencimento (Capítulo 7).

Tarefas.

1. Considere dois cenários: (A) 150 pacientes divididos em 30 clientes; (B) 300 pacientes em 50 clientes.
2. Para $z \in \{0; 1; 3; 5{,}8\}$, rode `fedavg_dp` ($S = 0{,}5$, 50 rodadas, `epocas=10`) com 5 sementes e relate: acurácia no teste, **AUC** do ataque do limiar de perda (membros contra `Xte`) e o ε (use `epsilon_rdp(z, 50)`).
3. Monte uma tabela com os dois cenários.
4. Escreva um parecer de cinco linhas: que nível de privacidade a clínica consegue pagar em cada cenário? O que seria necessário para ter ε baixo sem perder acurácia?

Entregáveis. Código, tabela e parecer.

A solução de referência está no Apêndice E.



Capítulo 9 — Quando Alguém Trapaceia: Robustez e Ataques de Envenenamento

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Distinguir ataques de envenenamento de dados e de modelo, e ataques não direcionados e direcionados (*backdoor*).
- Explicar por que a média é frágil diante de um único participante malicioso.
- Descrever e implementar agregadores robustos: mediana, média aparada, Krum e corte de norma.
- Executar um ataque de *backdoor* simples e medir seu sucesso.
- Reconhecer o custo da robustez quando os dados são não-IID.
- Entender a tensão entre agregação segura e detecção de clientes maliciosos.

9.1 Quando alguém trapaceia

Uma turma de 20 alunos decide a nota de um trabalho pela **média** das notas sugeridas por cada um. Um aluno malicioso sugere 1.000. A média da turma passa de 7 para 56. Um único participante, escolhendo um valor absurdo, **sequestrou** a decisão.

O aprendizado federado tem o mesmo ponto fraco. O servidor combina atualizações de clientes que ele **não controla**. Alguns podem estar com defeito, ser invadidos ou agir de má-fé.

> **No seu dia a dia**
>
> Aplicativos que dependem de avaliações de usuários (restaurantes, lojas) sofrem com avaliações falsas. Um bom sistema desconfia de notas muito fora do padrão. É a ideia dos agregadores robustos.

9.2 Quem trapaceia e o que quer

9.2.1 O adversário bizantino

Na computação distribuída, chama-se **bizantino** um participante que pode se comportar de forma **arbitrária**: falhar, mentir, combinar-se com outros (Lamport et al., 1982). Em aprendizado federado, supomos que uma fração f dos clientes é maliciosa e envia o que quiser.

9.2.2 Envenenamento de dados e de modelo

- **Envenenamento de dados** (*data poisoning*): o atacante só controla os **seus dados** e treina normalmente, mas com dados manipulados (por exemplo, rótulos trocados).

- **Envenenamento de modelo** (*model poisoning*): o atacante controla **a atualização enviada** e pode mandar qualquer vetor, como o inverso da atualização honesta, amplificada (Bhagoji et al., 2019; Fang et al., 2020).

9.2.3 Qual o objetivo?

- **Não direcionado** (*untargeted*): destruir a acurácia do modelo em geral.
- **Direcionado** (*targeted*), ou **backdoor**: o modelo continua bom em tudo, **exceto** quando vê um **gatilho** secreto, caso em que dá a resposta escolhida pelo atacante (Bagdasaryan et al., 2020; Wang et al., 2020a).

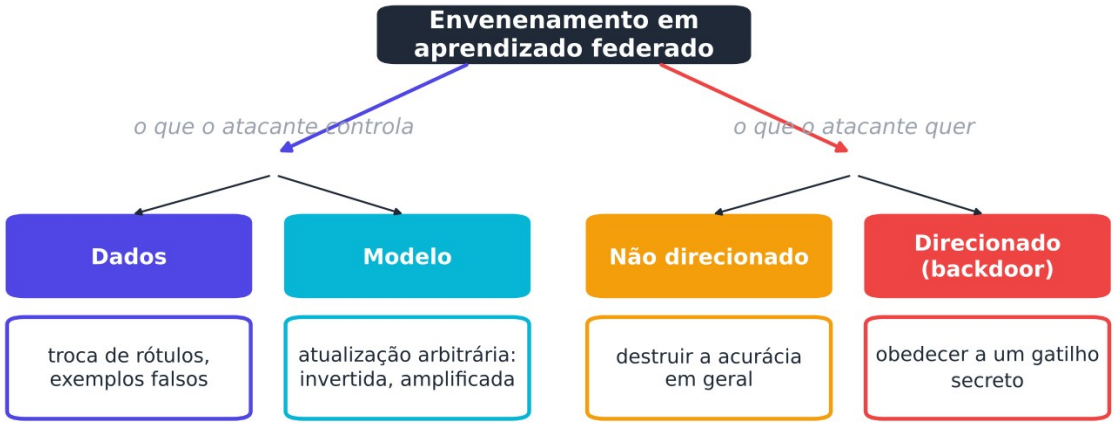


Figura 9.1 — Famílias de ataques de envenenamento em aprendizado federado.

> **Atenção**
>
> O *backdoor* é o ataque mais perigoso, porque **não aparece** na acurácia normal. Um modelo de reconhecimento facial com backdoor pode funcionar bem para todos, e liberar o acesso a quem usar um certo óculos.

9.3 Por que a média é frágil

A média tem **ponto de ruptura zero**: **um** valor suficientemente grande pode levá-la a qualquer lugar. Já a **mediana** só se move quando mais da metade dos valores muda.

```python
import numpy as np

notas = np.array([7.0, 6.5, 7.5, 8.0, 7.0, 6.0, 7.2, 7.8, 6.8, 7.1])
print("sem atacante  : média = %.2f | mediana = %.2f" % (notas.mean(), np.median(notas)))

for valor in (50, 1000, 1e6):
    com = np.append(notas, valor)
    print("atacante manda %-9g: média = %-12.2f | mediana = %.2f" % (valor, com.mean(),
np.median(com)))
```

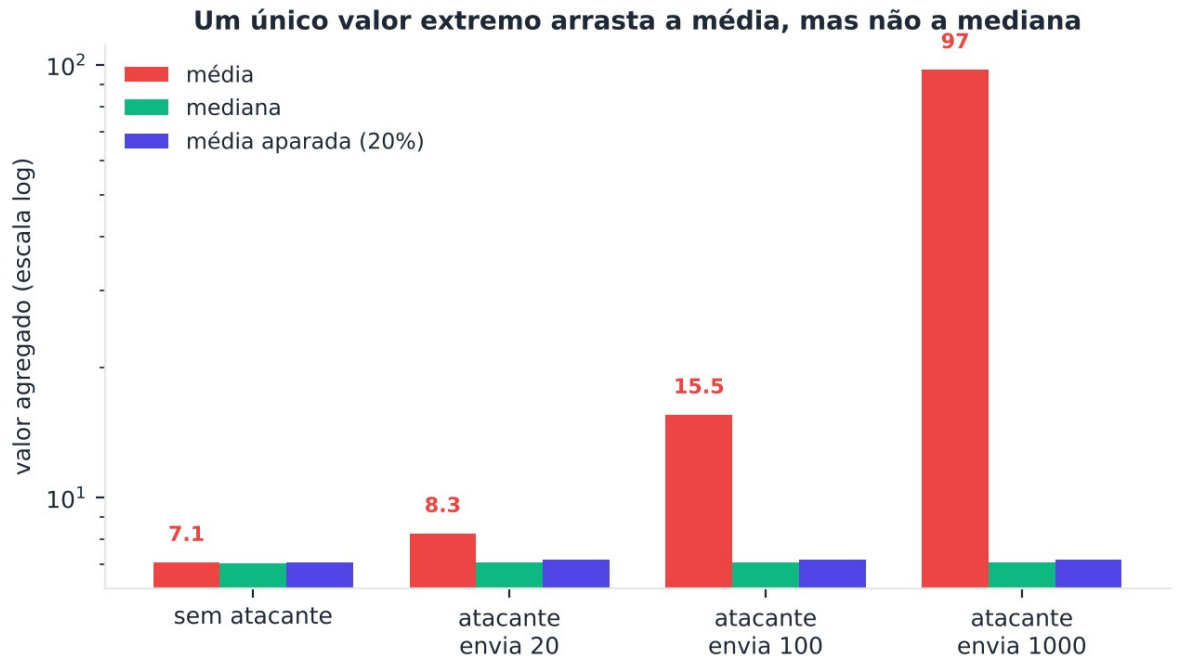


Figura 9.2 — Um único valor extremo arrasta a média, mas quase não move a mediana nem a média aparada.

Em aprendizado federado, o raciocínio é aplicado a **cada coordenada** do vetor de atualização.

9.4 Agregadores robustos

Seja um conjunto de *m* atualizações. Os agregadores robustos mais usados são:

9.4.1 Mediana coordenada a coordenada

Para cada parâmetro, toma-se a **mediana** dos valores enviados pelos clientes (Yin et al., 2018). Resiste a até quase metade de clientes maliciosos, mas descarta informação.

9.4.2 Média aparada

Para cada parâmetro, **ordenam-se** os valores, **descartam-se** os *k* menores e os *k* maiores, e calcula-se a média do resto (Yin et al., 2018). Se há até *k* maliciosos de cada lado, eles são eliminados.

9.4.3 Krum e Multi-Krum

O **Krum** (Blanchard et al., 2017) escolhe **uma única atualização**, a mais "central". Para cada cliente *i*, calcula-se uma pontuação: a soma das distâncias ao quadrado até seus $m - f - 2$ vizinhos mais próximos. Escolhe-se a de **menor** pontuação. Em palavras: é a atualização que tem muitas outras parecidas. O **Multi-Krum** média as *q* melhores.

$$\text{pontuação}(i) = \Sigma_{j \in \text{vizinhos de } i} \| \Delta_i - \Delta_j \|^2$$

9.4.4 Corte de norma

Corta a norma de cada atualização a um limite *S* antes da média. Limita o dano de atualizações gigantes, mas **não** impede um ataque sutil, de norma pequena.

9.4.5 Outros

O **Bulyan** combina Krum e média aparada (El Mhamdi et al., 2018). A **mediana geométrica** é uma versão multidimensional da mediana (Pillutla et al., 2022).

Agregador	Ideia	Pontos fortes	Pontos fracos
Média	Soma ponderada	Simples, ótimo sem ataque	Ruptura com 1 atacante
Mediana	Valor central por coordenada	Muito robusta	Perde eficiência
Média aparada	Descarta extremos	Boa robustez e eficiência	Precisa saber k
Krum	Escolhe a mais central	Defesa contra vetores arbitrários	Descarta quase tudo; sensível a não-IID
Corte de norma	Limita o tamanho	Barato	Não pega ataques sutis

9.5 A biblioteca robustez.py

Salve o código como `robustez.py`, junto da `fedlib.py`. Ele traz os agregadores, os ataques (troca de rótulos, sinal invertido, ruído e *backdoor*) e uma versão do FedAvg com clientes maliciosos.

```python
import numpy as np
from fedlib import *


# ---------- agregadores (recebem um array de atualizações com forma (m, d, classes)) ----------
def agg_media(D):
    return D.mean(axis=0)


def agg_mediana(D):
    return np.median(D, axis=0)


def agg_media_aparada(D, beta=0.2):
    m = D.shape[0]
    k = int(np.floor(beta * m))
    S = np.sort(D, axis=0)
    return S[k:m - k].mean(axis=0)


def agg_krum(D, f=2, multi=1):
    """Krum (Blanchard et al., 2017): escolhe a(s) atualização(ões) mais 'central(is)'."""
    m = D.shape[0]
    X = D.reshape(m, -1)
    dist = ((X[:, None, :] - X[None, :, :]) ** 2).sum(-1)
    k = max(1, m - f - 2)
    scores = np.array([np.sort(np.delete(dist[i], i))[:k].sum() for i in range(m)])
    escolhidos = np.argsort(scores)[:multi]
    return D[escolhidos].mean(axis=0)


def agg_corte_norma(D, S=1.0):
    """Corta a norma de cada atualização a S e tira a média."""
    n = np.linalg.norm(D.reshape(len(D), -1), axis=1)
    fator = np.minimum(1.0, S / (n + 1e-12))
```

```
    return (D * fator[:, None, None]).mean(axis=0)


# ---------- ataques ----------
def com_gatilho(X, tamanho=2):
    """Gatilho (backdoor): acende o canto inferior direito da imagem 8x8 (nas coordenadas
padronizadas)."""
    Xg = X.copy()
    img = Xg[:, :-1].reshape(-1, 8, 8)
    img[:, 8 - tamanho:, 8 - tamanho:] = 6.0
    Xg[:, :-1] = img.reshape(len(Xg), -1)
    return Xg


def fedavg_robusto(partes, X, y, Xte, yte, agregador=agg_media, maliciosos=(), ataque="nenhum",
gama=3.0,
                   rodadas=30, epocas=5, eta=0.1, lote=32, seed=0, classes=10, alvo=0):
    """FedAvg com clientes maliciosos e agregador à escolha. Devolve o modelo e os históricos de
acurácia e de sucesso do backdoor."""
    rng = np.random.default_rng(seed)
    W = np.zeros((X.shape[1], classes))
    maliciosos = set(maliciosos)
    Xte_gat = com_gatilho(Xte[yte != alvo])
    hist_acc, hist_asr = [], []
    for r in range(rodadas):
        deltas = []
        for c, p in enumerate(partes):
            if len(p) == 0:
                continue
            Xc, yc = X[p], y[p]
            if c in maliciosos and ataque == "rotulos":
                yc = (yc + 1) % classes                                  # troca de rótulos
            if c in maliciosos and ataque == "backdoor":
                n_g = len(p) // 2
                Xc = np.vstack([Xc, com_gatilho(Xc[:n_g])]); yc = np.r_[yc, np.full(n_g, alvo)]
            d = treino_local(W, Xc, yc, epocas, eta, lote, 0.0, seed * 1000 + r * 50 + c) - W
            if c in maliciosos and ataque == "sinal":
                d = -gama * d                                            # inverte e amplifica
            if c in maliciosos and ataque == "ruido":
                d = rng.normal(0, gama, d.shape)
            if c in maliciosos and ataque == "backdoor":
                d = gama * d                                             # amplifica (substituição de
modelo)
            deltas.append(d)
        W = W + agregador(np.array(deltas))
        hist_acc.append(acuracia(W, Xte, yte))
        hist_asr.append(float(np.mean(np.argmax(Xte_gat @ W, axis=1) == alvo)))
    return W, hist_acc, hist_asr
```

9.6 Ataques não direcionados

Com 20 clientes IID, simulamos três ataques. **Sinal invertido:** o malicioso envia o oposto da sua atualização, multiplicado por 3. **Troca de rótulos:** treina com rótulos deslocados. **Ruído:** envia vetores gaussianos. Variamos a fração de maliciosos e comparamos os agregadores. Médias de 3 sementes.

PYTHON

```
import numpy as np
from robustez import *

Xtr, Xte, ytr, yte = carregar_digits()
K = 20
agregadores = {
    "média":         agg_media,
    "mediana":       agg_mediana,
    "média aparada": lambda D: agg_media_aparada(D, 0.25),
    "Krum":          lambda D: agg_krum(D, f=6, multi=1),
    "Multi-Krum":    lambda D: agg_krum(D, f=6, multi=8),
}
print(f"{'ataque':<10}{'f':>5} | " + "".join(f"{n:>15}" for n in agregadores))
for ataque in ("sinal", "rotulos", "ruido"):
    for f in (0.0, 0.1, 0.3):
        linha = []
        for nome, ag in agregadores.items():
            accs = []
            for s in range(3):
                partes = particao_iid(ytr, K, s)
                _, h, _ = fedavg_robusto(partes, Xtr, ytr, Xte, yte, agregador=ag,
maliciosos=range(int(f * K)),
                                         ataque=ataque if f > 0 else "nenhum", seed=s)
                accs.append(h[-1])
            linha.append(np.mean(accs))
        print(f"{ataque:<10}{f:>5} | " + "".join(f"{a:>15.3f}" for a in linha))
```

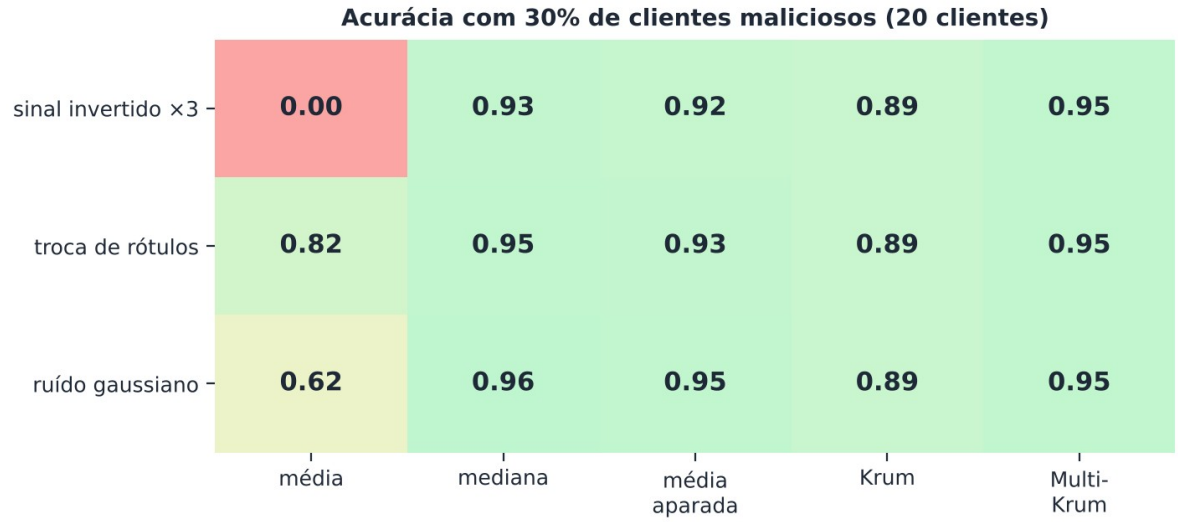


Figura 9.3 — Acurácia com 30% de clientes maliciosos, por ataque e por agregador. A média sucumbe; mediana e média aparada resistem.

O que vemos:

- Com **sinal invertido** e 30% de maliciosos, a média **colapsa** (acurácia perto de zero).
- Com **ruído**, até 10% de maliciosos já derruba a média para cerca de 0,75.
- **Mediana**, **média aparada** e **Multi-Krum** seguem perto de 0,92 a 0,96.
- O **Krum** puro, que escolhe um único cliente, perde cerca de 7 pontos mesmo **sem ataque**: descartar informação tem custo.

9.7 Backdoor

9.7.1 O ataque

O atacante quer que o modelo erre de propósito **só** quando vê um **gatilho**. No nosso experimento, o gatilho é um pequeno bloco de pixels acesos no canto inferior direito da imagem. O alvo é a classe **0**: toda imagem com gatilho deve ser classificada como "0".

O cliente malicioso treina com **dados normais mais dados com gatilho rotulados como 0**. Pode também **amplificar** sua atualização por um fator γ para que ela se sobreponha às demais, uma técnica chamada de **substituição de modelo** (Bagdasaryan et al., 2020).

Medimos:

- a **acurácia normal** (o modelo continua bom?);
- a **taxa de sucesso do ataque** (ASR, *attack success rate*): fração das imagens com gatilho (que não são zeros) classificadas como 0.

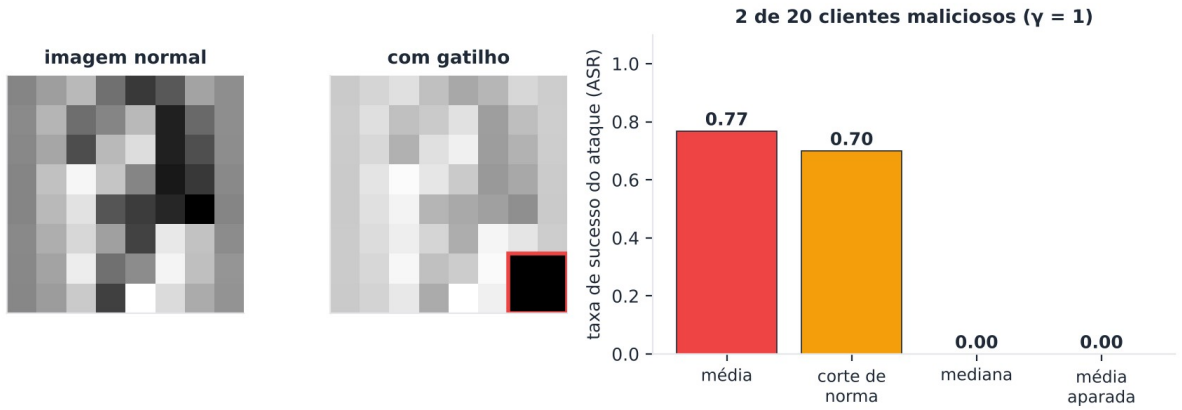


Figura 9.4 — O gatilho (bloco brilhante no canto) e o resultado: com a média, o gatilho funciona quase sempre; com mediana ou média aparada, quase nunca.

```python
import numpy as np
from robustez import *

Xtr, Xte, ytr, yte = carregar_digits()
K = 20
agregadores = {
    "média":                  agg_media,
    "corte de norma (S=0,5)":  lambda D: agg_corte_norma(D, 0.5),
    "mediana":                agg_mediana,
    "média aparada":          lambda D: agg_media_aparada(D, 0.25),
}
print(f"{'maliciosos':>10}{'γ':>4} | {'agregador':<24}{'acurácia':>9}{'ASR':>7}")
for f, gama in ((0.1, 1), (0.1, 5), (0.2, 1)):
    for nome, ag in agregadores.items():
        accs, asrs = [], []
        for s in range(3):
            partes = particao_iid(ytr, K, s)
            _, h, r = fedavg_robusto(partes, Xtr, ytr, Xte, yte, agregador=ag, maliciosos=range(int(f *
K)),
                                     ataque="backdoor", gama=gama, seed=s)
            accs.append(h[-1]); asrs.append(r[-1])
```

```
        print(f"{int(f * 100):>9}%{gama:>4} | {nome:<24}{np.mean(accs):>9.3f}{np.mean(asrs):>7.2f}")
```

Resultados:

- Com a **média**, 10% de maliciosos já dão **ASR de 0,77**; com amplificação, **0,97**; com 20%, **0,92.** A acurácia normal **mal muda**: o ataque é invisível.
- O **corte de norma não** resolve: o ASR continua em torno de 0,7 a 0,9.
- **Mediana** e **média aparada** reduzem o ASR a **praticamente zero** neste cenário IID.

> **Atenção**
>
> A mediana vence aqui porque os clientes honestos são **parecidos** entre si e os maliciosos são minoria. Em dados não-IID, a história muda, como veremos a seguir. Ataques de *backdoor* mais sofisticados, como os de "cauda" (Wang et al., 2020a), foram desenhados para escapar das defesas.

9.8 O custo da robustez com dados não-IID

Agregadores robustos supõem que **os honestos são parecidos**. Mas, com dados não-IID, os clientes honestos **também são diferentes** entre si. Um agregador robusto pode confundir um cliente honesto "diferente" com um atacante e descartá-lo.

Testamos **sem nenhum atacante**, com 20 clientes Dirichlet de heterogeneidade crescente.

```python
import numpy as np
from robustez import *

Xtr, Xte, ytr, yte = carregar_digits()
agregadores = {
    "média":         agg_media,
    "mediana":       agg_mediana,
    "média aparada": lambda D: agg_media_aparada(D, 0.25),
    "Krum":          lambda D: agg_krum(D, f=6, multi=1),
    "Multi-Krum":    lambda D: agg_krum(D, f=6, multi=8),
}
print(f"{'α':>6} | " + "".join(f"{n:>15}" for n in agregadores))
for alfa in (100, 0.3, 0.1):
    linha = []
    for nome, ag in agregadores.items():
        accs = []
        for s in range(3):
            partes = particao_dirichlet(ytr, 20, alfa, seed=s)
            _, h, _ = fedavg_robusto(partes, Xtr, ytr, Xte, yte, agregador=ag, seed=s)
            accs.append(h[-1])
        linha.append(np.mean(accs))
    print(f"{alfa:>6} | " + "".join(f"{a:>15.3f}" for a in linha))
```

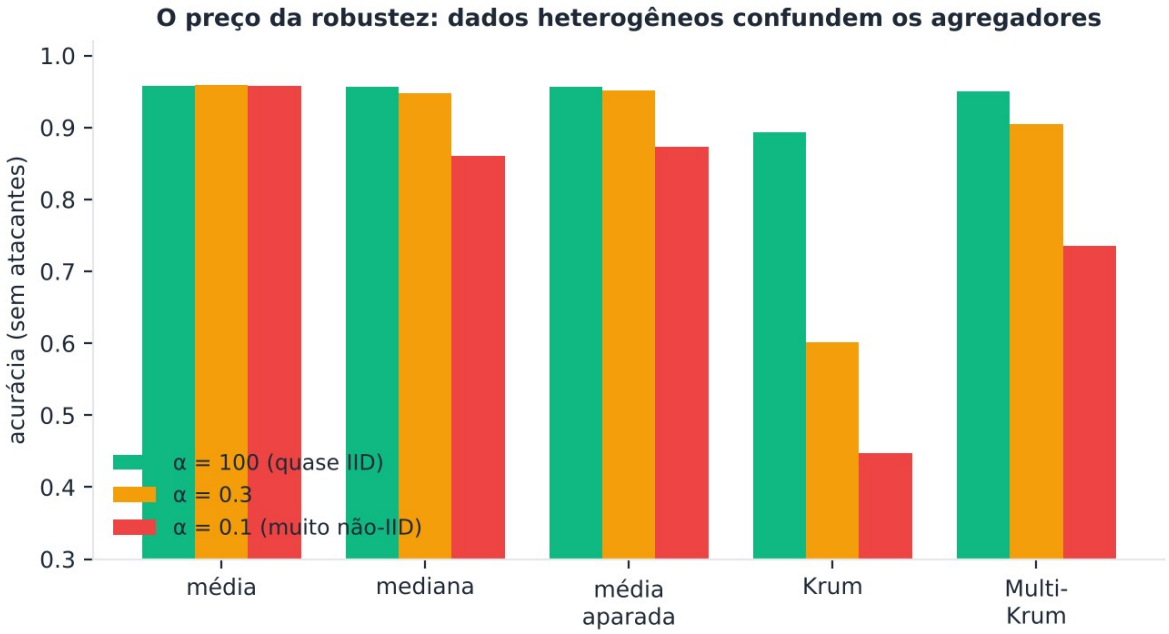


Figura 9.5 — Sem ataque, a robustez custa acurácia quando os dados são heterogêneos. O Krum sofre mais.

Com heterogeneidade alta (α = 0,1), a mediana cai de 0,96 para 0,86, a média aparada para 0,87, e o Krum para menos de 0,5, enquanto a média simples continua em 0,96. **Defender-se de atacantes custa utilidade para os honestos.**

> **Memorize**
>
> Robustez e heterogeneidade estão em **tensão**. Quanto mais diferentes os honestos, mais difícil separar o "diferente legítimo" do "diferente malicioso".

9.9 Outras defesas e limites

- **Detecção e reputação:** acompanhar clientes ao longo do tempo e reduzir o peso dos suspeitos.
- **Validação com dados de teste no servidor:** conferir se o modelo agregado não piora.
- **Privacidade diferencial:** o corte e o ruído limitam a influência de cada cliente, o que também dificulta alguns ataques.
- **Verificação de provas:** provas de conhecimento zero de que a atualização respeita limites (usadas em protocolos recentes).
- **Conluio:** vários atacantes combinados podem se esconder entre os honestos; "um pouco é suficiente" (Baruch et al., 2019).

9.9.1 A tensão com a agregação segura

Aqui há um dilema importante. Para **detectar** atualizações suspeitas, o servidor precisa **vê-las**. Mas a agregação segura (Capítulo 8) existe justamente para que o servidor **não veja** atualizações individuais. Ou seja, **privacidade e robustez colidem**. Pesquisas recentes tentam reconciliá-las, por exemplo, verificando limites de norma sobre dados cifrados, mas não há solução simples (Kairouz et al., 2021).

Saiba mais

Os ataques de envenenamento e as defesas formam uma **corrida armamentista**: cada defesa nova inspira ataques adaptativos que a contornam (Fang et al., 2020; Wang et al., 2020a). Avalie defesas **contra o atacante mais forte que você consegue imaginar**, e não apenas contra os ataques simples.

9.10 Resumo do capítulo

MEMORIZE **O essencial do Capítulo 9**

- Clientes podem ser **bizantinos**: falhos ou maliciosos, enviando o que quiserem.
- Ataques: **dados** ou **modelo**; **não direcionados** ou **backdoor**.
- A **média** quebra com 1 atacante. **Mediana**, **média aparada**, **Krum** e **corte de norma** são alternativas.
- Backdoor: o modelo parece normal, mas obedece a um **gatilho**.
- Com dados **não-IID**, defesas robustas **custam acurácia**.
- **Privacidade (agregação segura) e robustez** estão em tensão.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. O que é um participante bizantino em aprendizado federado?

a) Um cliente que usa criptografia.
b) Um cliente que pode se comportar de forma arbitrária: falhar, mentir ou combinar-se com outros.
c) Um cliente com poucos dados.
d) Um servidor lento.

Questão 2. No envenenamento de dados, o atacante:

a) controla o código dos outros clientes.
b) controla só os próprios dados, treinando normalmente com dados manipulados.
c) controla o canal de rede.
d) controla o servidor.

Questão 3. No envenenamento de modelo, o atacante:

a) só consulta o modelo final.
b) controla a atualização enviada e pode mandar qualquer vetor.
c) só altera os rótulos.
d) só observa.

Questão 4. Um ataque não direcionado (untargeted) busca:

a) acelerar o treino.
b) destruir a acurácia do modelo em geral.
c) roubar dados.
d) fazer o modelo errar só com um gatilho.

Questão 5. Um ataque de backdoor busca:

a) reduzir o número de clientes.
b) tornar o modelo mais privado.
c) fazer o modelo funcionar bem, exceto quando vê um gatilho, caso em que dá a resposta escolhida pelo atacante.
d) quebrar a criptografia.

Questão 6. Por que o backdoor é perigoso?

a) Porque só ocorre em dados IID.
b) Porque não aparece na acurácia normal do modelo.
c) Porque destrói toda a acurácia.
d) Porque é fácil de detectar.

Questão 7. Qual é o ponto de ruptura da média?

a) Zero: um único valor suficientemente grande pode levá-la a qualquer lugar.
b) 50%.
c) 30%.
d) 100%.

Questão 8. Com o atacante enviando 1.000 em um conjunto de 10 valores próximos de 7, a mediana:

a) vai para zero.
b) quase não muda.
c) vai para 1.000.
d) vai para 100.

Questão 9. Em 20 alunos, um envia a nota 1.000 e os outros 19 enviam 7. A média é aproximadamente:

a) 56,65.
b) 100.
c) 1.007.
d) 7.

Questão 10. A mediana coordenada a coordenada:

a) toma, para cada parâmetro, a mediana dos valores enviados.
b) soma tudo.
c) elimina o corte.
d) escolhe um cliente.

Questão 11. A média aparada (trimmed mean) com $\beta = 0{,}2$ e $m = 20$ clientes descarta:

a) nenhum valor.
b) os 4 menores e os 4 maiores valores de cada coordenada.
c) 20 valores.
d) apenas o maior valor.

Questão 12. O Krum escolhe:

a) a atualização maior.
b) a atualização com menor soma de distâncias aos seus vizinhos mais próximos.
c) uma atualização aleatória.
d) a média de todas.

Questão 13. No Krum, quantos vizinhos mais próximos entram na pontuação?

a) $m - f - 2$.
b) 2.
c) m.
d) f.

Questão 14. Qual a diferença entre Krum e Multi-Krum?

a) O Multi-Krum faz a média das q melhores atualizações em vez de escolher uma só.
b) O Multi-Krum usa criptografia.
c) Não há diferença.
d) O Multi-Krum escolhe a pior.

Questão 15. O corte de norma consiste em:

a) trocar o servidor.
b) apagar atualizações grandes.
c) limitar a norma de cada atualização a S antes da média.
d) aumentar o ruído.

Questão 16. Qual a limitação do corte de norma?

a) Só funciona com um cliente.
b) Não impede ataques sutis, de norma pequena, como backdoors.
c) Destrói a acurácia.
d) Exige criptografia.

Questão 17. No experimento, com 30% de clientes com sinal invertido, a média teve acurácia aproximada de:

a) 0,50.
b) 0,95.
c) 0,80.
d) 0,00.

Questão 18. No ataque de ruído com 10% de maliciosos, a média caiu para cerca de:

a) 0,75.
b) 0,10.
c) 0,60.
d) 0,95.

Questão 19. No experimento sem ataque, o Krum puro teve acurácia cerca de 0,88 enquanto a média teve 0,96. Qual a explicação?

a) O Krum descarta quase toda a informação, ficando com uma única atualização.
b) O Krum é sempre pior.
c) A média usa dados de teste.
d) O Krum usa menos clientes.

Questão 20. No backdoor do experimento, qual era o gatilho?

a) Uma senha numérica.
b) Uma palavra secreta.
c) Um cliente específico.
d) Um pequeno bloco de pixels acesos no canto inferior direito da imagem.

Questão 21. O que mede o ASR (taxa de sucesso do ataque)?

a) O tamanho do modelo.
b) A fração de imagens com gatilho (não zeros) classificadas como a classe alvo.
c) O número de clientes.
d) A acurácia normal.

Questão 22. Com a média e 10% de maliciosos ($\gamma = 1$), o ASR foi cerca de:

a) 0,77.
b) 0,30.
c) 0,00.
d) 0,10.

Questão 23. Com a mediana e 10% de maliciosos, o ASR foi:

a) 0,90.
b) 0,50.
c) 1,00.
d) praticamente zero.

Questão 24. Por que a mediana funcionou bem no backdoor do experimento?

a) Porque não havia ruído.
b) Porque os clientes honestos eram parecidos entre si e os maliciosos eram minoria.
c) Porque a mediana é criptografada.
d) Porque os dados eram não-IID.

Questão 25. O que acontece com agregadores robustos quando os dados são não-IID, mesmo sem ataque?

a) Ganham acurácia.
b) Perdem acurácia, pois clientes honestos diferentes podem ser tratados como atacantes.
c) Nada muda.
d) Ficam mais rápidos.

Questão 26. No experimento não-IID ($\alpha = 0{,}1$), qual agregador teve o pior desempenho?

a) Nenhum perdeu acurácia.
b) A média.
c) Krum, com menos de 0,5.
d) A mediana, com 0,96.

Questão 27. Qual é a tensão entre agregação segura e robustez?

a) A agregação segura aumenta o número de clientes.
b) Elas são idênticas.
c) Para detectar atualizações suspeitas, o servidor precisa vê-las; a agregação segura esconde as individuais.
d) A robustez exige criptografia.

Questão 28. O que significa "um pouco é suficiente" (Baruch et al., 2019)?

a) Que poucos clientes bastam para treinar.
b) Que atacantes podem se esconder entre os honestos, com mudanças pequenas, e ainda derrubar defesas.
c) Que pouco ruído é suficiente para privacidade.
d) Que poucas rodadas bastam.

Questão 29. Qual agregador resiste a até quase 50% de clientes maliciosos (em atualizações semelhantes entre honestos)?

a) O corte de norma isolado.
b) O FedAvg ponderado.
c) A mediana coordenada a coordenada.
d) A média simples.

Questão 30. Qual o risco de escolher um β pequeno demais na média aparada?

a) O servidor travar.
b) A média ficar perfeita.
c) Os clientes honestos serem sempre descartados.
d) Restarem atacantes entre os valores mantidos, que continuam arrastando a média.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique com a analogia da nota da turma por que a média é frágil.
2. Diferencie envenenamento de dados e de modelo, com um exemplo de cada.
3. Diferencie ataque não direcionado de backdoor e diga qual é mais difícil de detectar.
4. Descreva o funcionamento da média aparada e o parâmetro β.
5. Explique o Krum em palavras e uma limitação.
6. Explique por que o corte de norma não impede um backdoor.
7. Descreva o backdoor do experimento: gatilho, alvo, cliente malicioso e métricas.
8. Interprete os resultados do backdoor com média, corte de norma e mediana.
9. Por que os agregadores robustos pioram com dados não-IID?
10. Compare o desempenho do Krum e da média sem atacantes e explique.
11. Explique a tensão entre agregação segura e robustez, e uma possível direção de solução.
12. O que é a substituição de modelo e por que amplificar a atualização ajuda o atacante?
13. Dê duas defesas, além de agregadores robustos, contra envenenamento.
14. Por que as defesas devem ser avaliadas contra ataques adaptativos?
15. Calcule a média aparada (β = 0,2) para os valores 1, 2, 3, 4, 100 e interprete.
16. Qual a diferença entre falha bizantina e ataque malicioso coordenado?
17. Como você escolheria o parâmetro β da média aparada em um sistema real?
18. Em que situação você recomendaria a mediana em vez da média e em qual evitaria?
19. Resuma em cinco linhas o que você aprendeu sobre robustez em aprendizado federado.
20. Por que nenhuma defesa por agregação resiste a mais da metade de clientes maliciosos?

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Ponto de ruptura na prática

NÍVEL: BÁSICO

Simule 20 clientes cujas atualizações (vetores de 50 dimensões) são honestas: `N(1, 0,1)` em cada coordenada. Faça `m` deles maliciosos, enviando o vetor constante 100. Para `m` de 0 a 9, calcule o erro (norma da diferença até o vetor honesto médio) da **média**, da **mediana**, da **média aparada (β = 0,25)** e do **Krum** (use `agg_krum` de `robustez.py` com `f=m`).

Pergunta: a partir de quantos maliciosos cada agregador deixa de funcionar?

Exercício 2 — Troca de rótulos: até onde vai a média?

NÍVEL: BÁSICO

Com 20 clientes IID e o ataque `rotulos`, varie a fração de maliciosos f ∈ {0; 0,1; 0,2; 0,3; 0,4}. Compare a acurácia final (3 sementes) da média e da mediana.

Pergunta: como a degradação da média cresce com f? A mediana resiste a 40%?

Exercício 3 — Quanto aparar?

NÍVEL: INTERMEDIÁRIO

Com o ataque `sinal` e 30% de maliciosos (20 clientes), varie o parâmetro β da média aparada ∈ {0; 0,1; 0,2; 0,3; 0,4}. Para cada β, relate a acurácia com ataque e **sem** ataque (3 sementes).

Pergunta: qual β é seguro? Que preço se paga por um β grande?

Exercício 4 — Quando a mediana também cai

NÍVEL: INTERMEDIÁRIO

Com o ataque de backdoor (`gama=1`), varie a fração de maliciosos f ∈ {0,3; 0,4; 0,5; 0,6} para a **mediana** e a **média aparada (β = 0,25)**, com 20 clientes IID. Relate a acurácia normal e o ASR (3 sementes).

Pergunta: a partir de que fração a mediana deixa de proteger?

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** *Adult*, *Statlog (German Credit Data)* e *Spambase* (bom para estudar envenenamento por troca de rótulos).
> - **Kaggle:** *Credit Card Fraud Detection* e *SMS Spam Collection*.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Parecer de segurança: qual agregador adotar?

Contexto. Uma rede de clínicas (fictícia) vai treinar um modelo federado com 20 unidades. O time de segurança teme que **até 20%** das unidades possam ser comprometidas, mas os dados das clínicas são **moderadamente heterogêneos**. Você deve recomendar o agregador.

Tarefas.

1. Compare cinco agregadores: média, mediana, média aparada (β = 0,25), Multi-Krum (f = 6, multi = 8) e **corte de norma** (S = 0,5).
2. Meça, em 3 sementes, três cenários: (a) **sem ataque, dados Dirichlet α = 0,3**; (b) **ataque de sinal invertido com 20% de maliciosos, dados IID**; (c) **backdoor com 10% de maliciosos (γ = 1), dados IID**, medindo a acurácia e o ASR.
3. Monte uma tabela com os resultados.
4. Defina o critério: aprovado se a acurácia em (a) e em (b) for ≥ 0,90 e o ASR em (c) for ≤ 0,10. Entre os aprovados, indique o de maior média das acurácias (a, b).
5. Escreva o parecer de cinco linhas, com duas limitações.

Entregáveis. Código, tabela e parecer.

A solução de referência está no Apêndice E.

PARTE IV

Parte IV — Arquiteturas, Aplicações e Fronteiras

Outros desenhos de aprendizado federado (vertical, descentralizado, split learning), aprendizado não supervisionado federado, modelos de fundação, ferramentas, aplicações reais, regulação e ética.

Nesta parte:

- Cap. 10 — Outros Desenhos: Federado Vertical, Split Learning, Descentralizado e por Transferência
- Cap. 11 — Fronteiras e Prática: Não Supervisionado, Modelos de Fundação, Ferramentas, Aplicações e Ética



Capítulo 10 — Outros Desenhos: Federado Vertical, Split Learning, Descentralizado e por Transferência

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Distinguir aprendizado federado horizontal, vertical e por transferência, conforme a partição dos dados.
- Explicar e implementar o treino de uma regressão logística vertical, com troca de resíduos.
- Reconhecer o vazamento de rótulos no aprendizado federado vertical.
- Descrever o *split learning* e verificar sua equivalência com o treino convencional.
- Explicar o aprendizado descentralizado e como a topologia afeta o consenso.
- Escolher a arquitetura adequada a um cenário.

10.1 Os dados podem ser repartidos de maneiras diferentes

Até aqui, em todos os capítulos, os clientes tinham **o mesmo tipo de informação** (os mesmos atributos) sobre **pessoas diferentes**. Imagine uma tabela. Cada cliente tinha **algumas linhas**.

Mas há outros casos. Considere um **banco** e uma **loja virtual** de uma mesma cidade. Muitos clientes são os mesmos, mas cada empresa guarda **informações diferentes** sobre eles: o banco sabe renda e dívidas; a loja sabe o que a pessoa compra. Nenhuma das duas pode entregar seus dados à outra. Aqui, cada participante tem **algumas colunas** da tabela.

Yang et al. (2019) organizam as situações em três categorias.

Categoria	Como os dados são repartidos	Exemplo
Horizontal (HFL)	Mesmos atributos, **pessoas diferentes**	Dois hospitais com o mesmo tipo de exame
Vertical (VFL)	Mesmas pessoas, **atributos diferentes**	Banco e loja com clientes em comum
Por transferência (FTL)	Poucas pessoas e poucos atributos em comum	Dois países com poucos clientes em comum e dados distintos

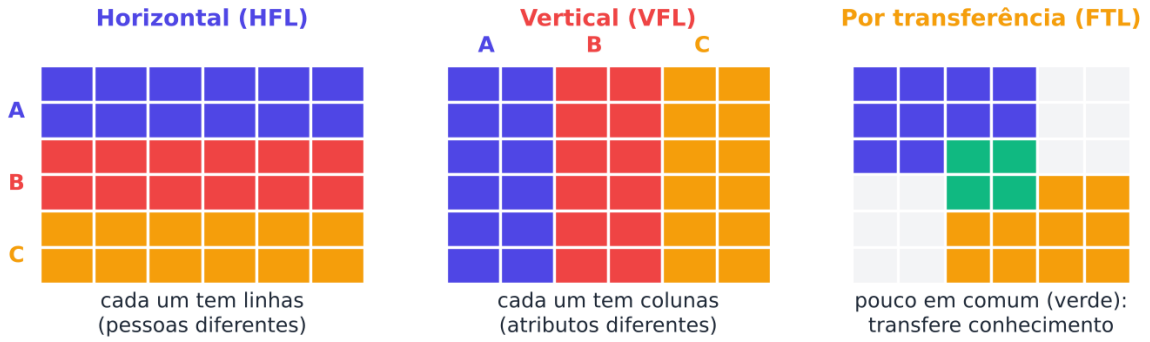


Figura 10.1 — As três maneiras de repartir uma tabela de dados entre participantes. No horizontal, cada um tem linhas; no vertical, colunas; na transferência, só um pedaço comum.

> **No seu dia a dia**
>
> Pense na sua saúde. O **plano de saúde** sabe suas consultas, o **relógio inteligente** sabe seus batimentos, e a **farmácia** sabe seus remédios. Juntos, formariam um retrato muito melhor, mas nenhum quer entregar os dados aos outros. É o cenário vertical.

10.2 Aprendizado federado vertical

10.2.1 Os desafios

No vertical, aparecem dificuldades novas.

1. **Alinhamento de entidades:** como descobrir **quais pessoas** os participantes têm em comum, sem revelar quem são as demais? Usam-se técnicas de **interseção privada de conjuntos** (*private set intersection*).
2. **Treino conjunto:** cada participante só tem **algumas colunas**, e o modelo precisa combiná-las.
3. **Quem tem o rótulo?** Normalmente, um participante (por exemplo, o banco, que sabe quem deu calote) guarda o rótulo.

10.2.2 Como treinar uma regressão logística vertical

A regressão logística calcula uma nota z somando as contribuições de cada atributo:

$$z = X_1 \cdot w_1 + X_2 \cdot w_2 + \ldots + X_K \cdot w_K + b$$

Se cada participante k guarda suas colunas X_k e seus pesos w_k, cada um consegue calcular **a sua parcela** $z_k = X_k \cdot w_k$ sem ver as colunas dos outros. O treino segue em quatro passos.

1. Cada participante calcula z_k e **envia a parcela** à parte que tem o rótulo.
2. Essa parte soma as parcelas, calcula a probabilidade p e o **resíduo** $(p - y)$.
3. Ela **devolve o resíduo** a todos.
4. Cada participante atualiza só seus pesos: $w_k \leftarrow w_k - \eta \cdot X_k^{\top} \cdot (p - y) / n$.

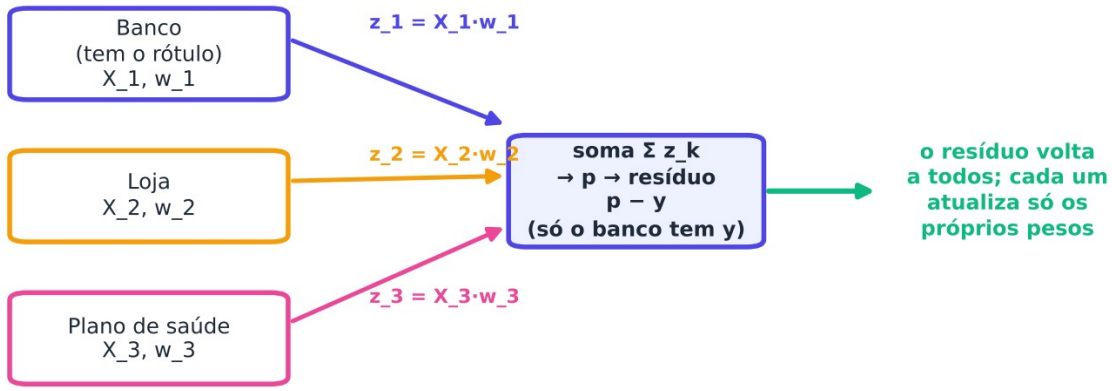


Figura 10.2 — Aprendizado federado vertical: cada parte envia sua contribuição parcial; a parte com o rótulo devolve o resíduo (p − y).

10.2.3 A biblioteca arquiteturas.py

Salve o código abaixo como **`arquiteturas.py`**. Ele reúne o aprendizado vertical, o *split learning* e o consenso descentralizado deste capítulo.

```python
import numpy as np


def sigmoide(z):
    return 1 / (1 + np.exp(-np.clip(z, -50, 50)))


# ---------------------------------------------------------------- Aprendizado federado vertical
def vfl_treinar(X_partes, y, epocas=300, eta=0.5):
    """Regressão logística vertical: cada parte tem só as suas colunas; apenas a parte 0 conhece o
rótulo y.
    Cada parte envia z_k = X_k w_k; a parte 0 soma, calcula o resíduo (p - y) e o devolve."""
    n = len(y)
    w = [np.zeros(X.shape[1]) for X in X_partes]
    b = 0.0
    historico = []
    for _ in range(epocas):
        z = sum(X @ wk for X, wk in zip(X_partes, w)) + b          # soma das contribuições parciais
        p = sigmoide(z)
        residuo = p - y                                            # calculado pela parte com o rótulo
        for k, X in enumerate(X_partes):
            w[k] -= eta * X.T @ residuo / n                        # cada parte atualiza só os seus
pesos
        b -= eta * residuo.mean()
        historico.append(-np.mean(y * np.log(p + 1e-12) + (1 - y) * np.log(1 - p + 1e-12)))
    return w, b, historico


def vfl_prever(X_partes, w, b):
    return (sum(X @ wk for X, wk in zip(X_partes, w)) + b > 0).astype(int)


# ---------------------------------------------------------------- Split learning
```

```
def treinar_junto(X, y, W1, W2, eta=0.1, passos=50):
    """Rede de 2 camadas (tanh + softmax) treinada de forma convencional."""
    W1, W2 = W1.copy(), W2.copy()
    Y = np.eye(W2.shape[1])[y]
    for _ in range(passos):
        H = np.tanh(X @ W1)
        P = np.exp(H @ W2); P /= P.sum(1, keepdims=True)
        dZ = (P - Y) / len(y)
        dW2 = H.T @ dZ
        dH = dZ @ W2.T * (1 - H ** 2)
        W2 -= eta * dW2
        W1 -= eta * X.T @ dH
    return W1, W2


def treinar_dividido(X, y, W1, W2, eta=0.1, passos=50):
    """Mesma rede, mas dividida: o cliente tem W1 e os dados; o servidor tem W2 e os rótulos.
    Só trafegam as ativações H ('smashed data') e o gradiente em H."""
    W1, W2 = W1.copy(), W2.copy()
    Y = np.eye(W2.shape[1])[y]
    for _ in range(passos):
        H = np.tanh(X @ W1)                                      # cliente: passagem direta até a
camada de corte
        # --- H sobe ao servidor ---
        P = np.exp(H @ W2); P /= P.sum(1, keepdims=True)         # servidor: conclui a passagem
direta
        dZ = (P - Y) / len(y)
        dH = dZ @ W2.T                                           # servidor: gradiente na camada de
corte
        W2 -= eta * H.T @ dZ
        # --- dH desce ao cliente ---
        W1 -= eta * X.T @ (dH * (1 - H ** 2))                    # cliente: conclui a
retropropagação
    return W1, W2


# ---------------------------------------------------------------- Aprendizado descentralizado
def matriz_mistura(topologia, n):
    """Matriz duplamente estocástica de mistura para uma topologia de comunicação."""
    A = np.zeros((n, n))
    if topologia == "anel":
        for i in range(n):
            A[i, i] = 1 / 3; A[i, (i + 1) % n] = 1 / 3; A[i, (i - 1) % n] = 1 / 3
    elif topologia == "completa":
        A[:] = 1 / n
    elif topologia == "estrela":
        for i in range(1, n):
            A[0, i] = A[i, 0] = 1 / n
            A[i, i] = 1 - 1 / n
        A[0, 0] = 1 - (n - 1) / n
    return A


def consenso(valores, A, rodadas):
    """Cada nó repete: novo valor = média ponderada dos valores dos vizinhos. Devolve o erro até a
média global."""
    x = valores.copy()
    alvo = x.mean(axis=0)
```

```
    erros = [np.abs(x - alvo).max()]
    for _ in range(rodadas):
        x = A @ x
        erros.append(np.abs(x - alvo).max())
    return erros
```

10.2.4 Experimento: três bancos de dados, uma pessoa

Usamos o conjunto *Breast Cancer Wisconsin*, da UCI, com 30 atributos. Repartimos as **colunas** ao acaso entre 3 participantes (10 cada). Só o participante 0 conhece o diagnóstico (rótulo).

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from arquiteturas import *

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
esc = StandardScaler().fit(Xtr)
Xtr, Xte = esc.transform(Xtr), esc.transform(Xte)

rng = np.random.default_rng(0)
colunas = np.array_split(rng.permutation(30), 3)           # 10 atributos por participante
Xp = [Xtr[:, c] for c in colunas]
Xtp = [Xte[:, c] for c in colunas]

for k, c in enumerate(colunas):
    m = LogisticRegression(max_iter=2000).fit(Xtr[:, c], ytr)
    print(f"participante {k} sozinho (10 atributos): acurácia = {m.score(Xte[:, c], yte):.3f}")

w, b, perdas = vfl_treinar(Xp, ytr)
print(f"VFL com os 3 participantes (sem juntar colunas): acurácia = {(vfl_prever(Xtp, w, b) ==
yte).mean():.3f}")
print(f"centralizado (todas as colunas juntas)          : acurácia =
{LogisticRegression(max_iter=2000).fit(Xtr, ytr).score(Xte, yte):.3f}")
print(f"perda: início = {perdas[0]:.3f} -> fim = {perdas[-1]:.3f}")
```

Cada participante sozinho fica **abaixo** do modelo conjunto. O treino vertical, **sem que nenhuma coluna saísse de seu dono**, chega ao nível do centralizado.

10.2.5 O perigo: o resíduo entrega o rótulo

Atenção a um detalhe do passo 3. Os participantes **sem rótulo** recebem o resíduo $(p - y)$. Como p está entre 0 e 1:

- se $y = 1$, o resíduo é **negativo**;
- se $y = 0$, o resíduo é **positivo**.

Ou seja, o **sinal** do resíduo **revela o rótulo** de cada pessoa. Vamos conferir.

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
```

```
from sklearn.preprocessing import StandardScaler
from arquiteturas import *

X, y = load_breast_cancer(return_X_y=True)
X = StandardScaler().fit_transform(X)
colunas = np.array_split(np.random.default_rng(0).permutation(30), 3)
Xp = [X[:, c] for c in colunas]

# a primeira iteração do treino: o que o participante 1 (sem rótulo) recebe?
w = [np.zeros(Xk.shape[1]) for Xk in Xp]
p = sigmoide(sum(Xk @ wk for Xk, wk in zip(Xp, w)))
residuo = p - y                                  # é isto que a parte com o rótulo envia

rotulo_inferido = (residuo < 0).astype(int)      # sinal negativo => rótulo 1
print("fração de rótulos recuperados pelo sinal do resíduo:", float((rotulo_inferido == y).mean()))
```

> **Atenção**
>
> Este vazamento é um **defeito conhecido** do aprendizado vertical simples. Na prática, protege-se o resíduo com **criptografia homomórfica**, **privacidade diferencial** (ruído no resíduo) ou **agregação segura**. O desafio é manter a utilidade enquanto se reduz o vazamento.

10.3 Split learning

10.3.1 A ideia: partir a rede em duas

No **split learning** (Gupta e Raskar, 2018), a rede neural é **cortada** em uma camada. As primeiras camadas ficam com o **cliente**, as últimas com o **servidor**.

- O cliente processa os dados até a **camada de corte** e envia as **ativações** (*smashed data*, "dados amassados").
- O servidor completa o cálculo, computa a perda e a retropropagação até a camada de corte.
- O servidor devolve o **gradiente** nessa camada, e o cliente conclui a retropropagação.

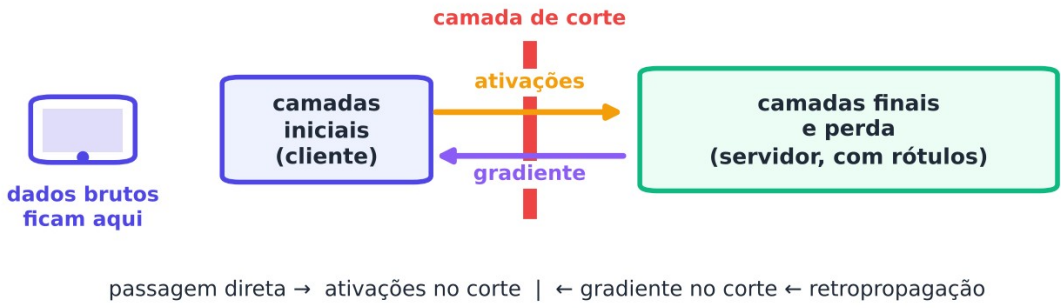


Figura 10.3 — Split learning: o cliente só envia ativações na camada de corte e recebe o gradiente de volta; os dados brutos nunca saem.

Vantagens: o cliente faz **pouco cálculo** (só as primeiras camadas), e os dados brutos ficam com ele.
Desvantagens: o treino é **sequencial** por padrão, e as ativações podem **vazar informação** sobre a entrada.

10.3.2 A matemática é a mesma

O ponto surpreendente: o split learning **não muda o algoritmo**. A retropropagação, pela regra da cadeia, passa de uma camada à outra de modo que o resultado é **idêntico** ao do treino convencional. Vamos verificar.

```python
import numpy as np
from sklearn.datasets import load_digits
from sklearn.preprocessing import StandardScaler
from arquiteturas import *

X, y = load_digits(return_X_y=True)
X = StandardScaler().fit_transform(X)
rng = np.random.default_rng(0)
W1 = rng.normal(0, 0.1, (64, 16))      # camada do cliente
W2 = rng.normal(0, 0.1, (16, 10))      # camada do servidor

a1, a2 = treinar_junto(X, y, W1, W2, eta=0.5, passos=100)
b1, b2 = treinar_dividido(X, y, W1, W2, eta=0.5, passos=100)

print("pesos do cliente  iguais?", np.allclose(a1, b1), "| diferença máxima:", float(np.abs(a1 -
b1).max()))
print("pesos do servidor iguais?", np.allclose(a2, b2), "| diferença máxima:", float(np.abs(a2 -
b2).max()))
```

10.4 Aprendizado descentralizado: sem servidor

10.4.1 E se não houver servidor?

O servidor central é um **ponto único de falha** (se cair, tudo para) e de **confiança** (quem o controla?). No **aprendizado descentralizado**, os clientes conversam **diretamente** entre si, em uma **rede de vizinhos**.

A ideia é a do **boato** (*gossip*): a cada rodada, cada cliente **tira a média** do seu modelo com os dos vizinhos. Com o tempo, todos convergem para um valor comum, o **consenso** (Boyd et al., 2006). Lian et al. (2017) mostraram que o treino descentralizado pode igualar o centralizado, e Koloskova et al. (2020) deram uma teoria unificada.

10.4.2 A matriz de mistura e a topologia

Se há *n* clientes, a regra de uma rodada é

$$x \leftarrow A \cdot x$$

em que *A* é a **matriz de mistura**: a linha *i* diz com que peso o cliente *i* combina os valores dos vizinhos. Cada linha e cada coluna somam 1.

A velocidade do consenso depende da **topologia**, isto é, de quem conversa com quem.

- **Completa:** todos com todos. O consenso é imediato, mas exige muita comunicação.
- **Anel:** cada um fala só com dois vizinhos. Barato, mas **lento**.
- **Estrela:** todos falam só com o nó central (um quase-servidor). Com esses pesos de mistura, converge até mais devagar que o anel, pois todo o fluxo passa por um único nó.

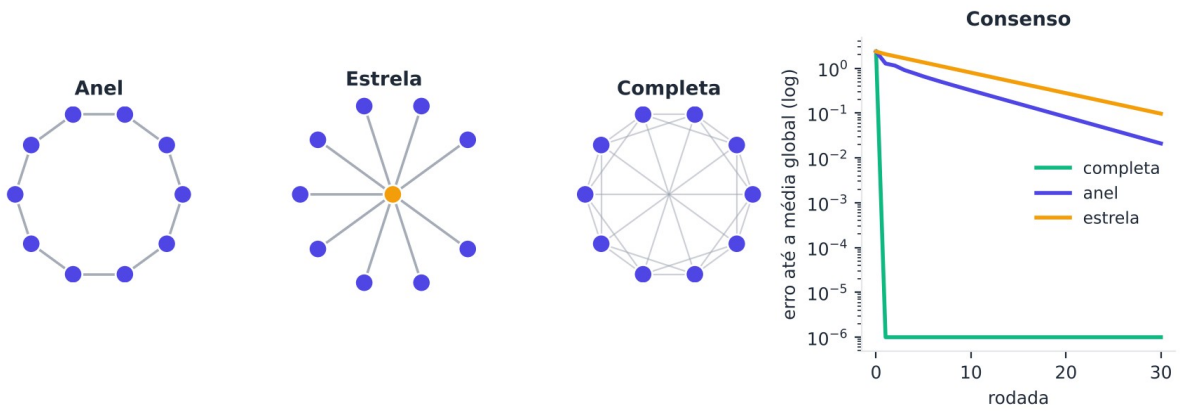


Figura 10.4 — Topologias de comunicação (anel, estrela e completa) e o erro de consenso ao longo das rodadas.

```python
import numpy as np
from arquiteturas import *

rng = np.random.default_rng(0)
n = 10
valores = rng.normal(0, 1, (n, 5))          # um "modelo" de 5 números em cada cliente

print(f"{'topologia':>10} | erro máximo até a média global, após 0, 10, 20 e 30 rodadas")
for topo in ("completa", "estrela", "anel"):
    A = matriz_mistura(topo, n)
    erros = consenso(valores, A, 30)
    print(f"{topo:>10} | {erros[0]:.3f}  {erros[10]:.3f}  {erros[20]:.3f}  {erros[30]:.4f}")
```

10.4.3 Vantagens e desafios

- **Vantagem:** nenhum ponto único de falha; ausência de um servidor que veja tudo.
- **Desafio:** convergência mais lenta com topologias esparsas; mais difícil de proteger com agregação segura; sincronização mais complexa.
- **Uso típico:** redes de dispositivos que não confiam em um coordenador central e redes de borda.

10.5 Aprendizado federado por transferência

Quando **quase não há** pessoas nem atributos em comum, o aprendizado vertical e o horizontal não se aplicam. A **transferência federada** (Liu et al., 2020) usa um **espaço comum de representação**: cada participante aprende, com seus dados, uma representação em um espaço compartilhado; um pequeno conjunto de pessoas em comum (ou de pares de exemplos) ajuda a **alinhar** os espaços, e o conhecimento é transferido de quem tem rótulos para quem não tem.

Esta é a categoria mais difícil e menos madura. O livro apresenta a ideia, e não uma implementação.

10.6 Comparação das arquiteturas

Arquitetura	Partição	Quem fala com quem	Principal vantagem	Principal desafio
HFL (FedAvg)	Linhas	Clientes ↔ servidor	Simples, escalável	Dados não-IID
VFL	Colunas	Participantes ↔ parte com rótulo	Combina fontes distintas	Alinhamento; vazamento de rótulos

Arquitetura	Partição	Quem fala com quem	Principal vantagem	Principal desafio
Split learning	Camadas do modelo	Cliente ↔ servidor	Pouco cálculo no cliente	Sequencial; vazamento de ativações
Descentralizado	Linhas	Cliente ↔ vizinhos	Sem ponto único de falha	Convergência e proteção
FTL	Poucas sobreposições	Varia	Transfere conhecimento	Maturidade

Memorize

- **Horizontal:** mesmas colunas, pessoas diferentes. **Vertical:** mesmas pessoas, colunas diferentes.
- No vertical, o **resíduo** (p − y) pode **vazar o rótulo**.
- **Split learning:** a rede é cortada em duas; a matemática é a mesma do treino convencional.
- **Descentralizado:** sem servidor; a topologia define a velocidade do consenso.

10.7 Resumo do capítulo

MEMORIZE **O essencial do Capítulo 10**

- A partição dos dados define o tipo: **horizontal, vertical ou por transferência**.
- **VFL:** cada parte envia z_k; o resíduo volta; cada uma atualiza seus pesos. Requer alinhar entidades.
- **Split learning:** cliente e servidor dividem as camadas e trocam ativações e gradientes.
- **Descentralizado:** gossip e consenso; anel é lento, completa é rápida.
- **FTL:** transferência de conhecimento por um espaço comum.
- Nenhuma arquitetura elimina o risco de vazamento: cada uma tem o seu.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. No aprendizado federado horizontal, os participantes têm:

a) os mesmos atributos, mas pessoas diferentes.
b) apenas os rótulos.
c) poucas pessoas e atributos em comum.
d) as mesmas pessoas, mas atributos diferentes.

Questão 2. No aprendizado federado vertical, os participantes têm:

a) apenas o modelo.
b) nenhum dado.
c) as mesmas pessoas, mas atributos diferentes.
d) os mesmos atributos e pessoas diferentes.

Questão 3. Um banco e uma loja virtual com clientes em comum, mas dados diferentes sobre eles, ilustram:

a) split learning.
b) aprendizado federado horizontal.
c) aprendizado descentralizado.
d) aprendizado federado vertical.

Questão 4. Dois hospitais com o mesmo tipo de exame, mas pacientes distintos, ilustram:

a) aprendizado descentralizado.
b) aprendizado federado horizontal.
c) aprendizado federado vertical.
d) aprendizado por transferência.

Questão 5. Quando o aprendizado federado por transferência (FTL) é mais indicado?

a) Quando não há rótulos em nenhum lugar.
b) Quando há um único participante.
c) Quando todos têm a mesma tabela.
d) Quando há poucas pessoas e poucos atributos em comum entre os participantes.

Questão 6. Qual é o primeiro desafio específico do aprendizado vertical?

a) Quantizar atualizações.
b) Cortar a norma.
c) Alinhar as entidades (descobrir quais pessoas são comuns) sem revelar as demais.
d) Escolher o número de clientes.

Questão 7. O que é interseção privada de conjuntos (PSI)?

a) Uma função de perda.
b) Um tipo de ruído.
c) Um método de compressão.
d) Um protocolo que descobre os elementos em comum de dois conjuntos sem revelar os demais.

Questão 8. Na regressão logística vertical, o que cada participante k envia à parte que tem o rótulo?

a) Suas colunas X_k.
b) Sua contribuição parcial $z_k = X_k \cdot w_k$.
c) Seus pesos w_k.
d) Seus rótulos.

Questão 9. Quem calcula o resíduo (p − y) no aprendizado vertical simples?

a) Todos os participantes.
b) A parte que possui o rótulo.
c) O servidor, sem rótulos.
d) Ninguém.

Questão 10. Como cada participante atualiza seus pesos no aprendizado vertical?

a) $w_k \leftarrow w_k - \eta \cdot X_k^T \cdot (p - y) / n$, usando só as próprias colunas.
b) Usando os rótulos diretamente.
c) Não atualiza.
d) Copiando os pesos dos outros.

Questão 11. No experimento com três participantes de 10 atributos cada, a acurácia do VFL (≈ 0,965) comparada à de cada participante sozinho (≈ 0,92 a 0,96):

a) foi pior que todos.
b) foi zero.
c) não pôde ser medida.
d) foi igual ou maior do que a de qualquer um sozinho.

Questão 12. Qual é o vazamento do aprendizado vertical simples?

a) O sinal do resíduo (p − y) revela o rótulo de cada pessoa aos participantes passivos.
b) As colunas são enviadas ao servidor.
c) Os pesos são públicos.
d) O servidor vê os dados brutos.

Questão 13. Por que o resíduo é negativo quando y = 1?

a) Porque p está entre 0 e 1, e $p - 1 < 0$.
b) Porque p é sempre negativo.
c) Porque o resíduo é aleatório.
d) Porque y é negativo.

Questão 14. No experimento do capítulo, qual fração dos rótulos foi recuperada pelo sinal do resíduo na primeira iteração?

a) 50%.
b) 0%.
c) 10%.
d) 100%.

Questão 15. Como proteger o resíduo no aprendizado vertical?

a) Usando mais rodadas.
b) Aumentando o lote.
c) Removendo clientes.
d) Com criptografia homomórfica, privacidade diferencial (ruído) ou agregação segura.

Questão 16. No split learning, o que o cliente envia ao servidor?

a) O modelo inteiro.
b) As ativações na camada de corte.
c) Os dados brutos.
d) Os rótulos.

Questão 17. No split learning, o que o servidor devolve ao cliente?

a) Os pesos do servidor.
b) Os rótulos.
c) O gradiente na camada de corte.
d) Nada.

Questão 18. O experimento do capítulo mostrou que o split learning, comparado ao treino convencional:

a) produz pesos aleatórios.
b) produz pesos muito piores.
c) produz exatamente os mesmos pesos.
d) não treina.

Questão 19. Qual a principal vantagem do split learning para o cliente?

a) Não precisa de dados.
b) Dispensa servidor.
c) Elimina a comunicação.
d) Faz pouco cálculo (só as primeiras camadas) e mantém os dados brutos locais.

Questão 20. Qual a principal desvantagem do split learning simples?

a) Treino sequencial e possível vazamento de informação pelas ativações.
b) É sempre menos preciso.
c) Não funciona com redes neurais.
d) Exige muitos clientes.

Questão 21. O aprendizado descentralizado é aquele em que:

a) apenas um cliente treina.
b) os clientes conversam diretamente entre si, sem servidor central.
c) os dados são juntados.
d) o servidor faz tudo.

Questão 22. A técnica de gossip consiste em:

a) criptografar os dados.
b) espalhar rumores falsos.
c) cada nó tirar, a cada rodada, a média do seu modelo com os dos vizinhos.
d) enviar tudo ao servidor.

Questão 23. Uma matriz de mistura A válida para o consenso deve ter:

a) apenas zeros.
b) todas as entradas negativas.
c) linhas e colunas somando 1 (duplamente estocástica).
d) a diagonal zerada.

Questão 24. No experimento de topologias, qual conseguiu consenso imediato?

a) A completa (todos com todos).
b) O anel.
c) Nenhuma.
d) A estrela.

Questão 25. Por que o anel é "barato, mas lento"?

a) Cada nó fala só com dois vizinhos, e a informação leva muitas rodadas para se espalhar.
b) Porque exige muita banda.
c) Porque usa criptografia.
d) Porque não há vizinhos.

Questão 26. Quais são vantagens do aprendizado descentralizado?

a) Dispensa dados.
b) Convergência sempre mais rápida.
c) Não há ponto único de falha nem servidor que veja tudo.
d) Segurança garantida.

Questão 27. O que é a transferência federada (FTL)?

a) Usar um espaço comum de representação para transferir conhecimento entre participantes com pouca sobreposição.
b) Um tipo de ruído.
c) Quantizar atualizações.
d) Enviar dados ao servidor.

Questão 28. Qual arquitetura reparte o modelo entre cliente e servidor por camadas?

a) FTL.
b) Gossip.
c) HFL.
d) Split learning.

Questão 29. Qual dos pares abaixo associa corretamente arquitetura e principal desafio?

a) HFL: alinhar entidades.
b) Split learning: topologias.
c) Descentralizado: interseção privada.
d) VFL: alinhamento e vazamento de rótulos.

Questão 30. Quando o aprendizado vertical deixa de compensar, segundo o projeto do capítulo?

a) Quando poucas pessoas são comuns aos participantes (pouco alinhamento).
b) Quando os dados são IID.
c) Quando o modelo é linear.
d) Quando há muitos atributos.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique as três categorias de partição de dados (horizontal, vertical e por transferência) com um exemplo do cotidiano de cada.
2. Descreva os quatro passos do treino de uma regressão logística vertical.
3. Explique por que o resíduo vaza o rótulo e como a defesa funciona.
4. Como o ruído gaussiano no resíduo afeta a recuperação de rótulos e a acurácia no experimento do projeto?
5. Explique o problema do alinhamento de entidades no aprendizado vertical.
6. Descreva o split learning e explique por que o resultado é idêntico ao treino convencional.
7. Compare a comunicação do split learning com a do FedAvg para um cliente com muitos dados.
8. Quais riscos de privacidade o split learning apresenta?
9. Explique o conceito de consenso em aprendizado descentralizado e a função da matriz de mistura.
10. Interprete a tabela de erros de consenso por topologia (completa, estrela e anel).
11. Dê vantagens e desvantagens do aprendizado descentralizado.
12. Qual a diferença entre cliente em HFL e participante em VFL quanto aos dados que possuem?
13. Explique por que, no VFL, a parte com o rótulo tem papel especial.
14. Descreva o experimento de alinhamento parcial e sua conclusão.
15. Em que cenário você preferiria o aprendizado descentralizado ao FedAvg?
16. Explique o que é a camada de corte e como escolhê-la.
17. Cite duas semelhanças e duas diferenças entre agregação segura e aprendizado vertical protegido por homomórfica.
18. Resuma em cinco linhas qual arquitetura você escolheria para três cenários distintos e por quê.
19. Por que o treino vertical com K participantes é, em essência, equivalente à regressão logística centralizada?
20. Calcule o tráfego do split learning por época para 5.000 exemplos locais, 32 neurônios na camada de corte e 32 bits, e compare com um modelo de 10.000 parâmetros no FedAvg (por rodada).

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Quantos participantes?

NÍVEL: BÁSICO

No conjunto *Breast Cancer Wisconsin*, reparta as 30 colunas ao acaso (`np.random.default_rng(0).permutation`) entre K ∈ {1, 2, 3, 5, 10} participantes (partes quase iguais). Treine o VFL (`vfl_treinar`) e compare a acurácia no teste com a da regressão logística centralizada.

Pergunta: o número de participantes altera a qualidade do modelo vertical?

Exercício 2 — Ruído no resíduo

NÍVEL: BÁSICO

Modifique o treino vertical para que as partes **passivas** (todas, exceto a parte 0) recebam o resíduo com ruído gaussiano de desvio σ. Para σ ∈ {0; 0,1; 0,5; 1; 2} (5 sementes), reporte a acurácia no teste e a fração de rótulos que um participante passivo recupera pelo sinal do resíduo ruidoso na primeira iteração (p = 0,5).

Pergunta: qual σ reduz o vazamento sem estragar o modelo?

Exercício 3 — Comunicação: split learning contra FedAvg

NÍVEL: INTERMEDIÁRIO

Para os dígitos (1.347 exemplos de treino, 64 atributos) e uma rede de 16 neurônios ocultos e 10 classes, calcule: (a) bytes trocados por época no split learning (ativações para cima e gradientes para baixo, 32 bits); (b) bytes trocados por rodada no FedAvg com 1 cliente com todos os dados (modelo para baixo e para cima). Varie a quantidade de dados locais (100, 1.347, 10.000 e 100.000 exemplos).

Pergunta: quando o split learning é mais caro do que o FedAvg?

Exercício 4 — Velocidade do consenso no anel

NÍVEL: INTERMEDIÁRIO

Para um anel com n ∈ {10, 20, 50} nós, calcule o segundo maior autovalor em módulo da matriz de mistura (`matriz_mistura("anel", n)`) e estime o número de rodadas para o erro cair a 1% do inicial, usando `consenso`. Compare com a previsão `ln(0,01)/ln(λ₂)`.

Pergunta: como o número de rodadas cresce com n?

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** *Adult* (separe colunas demográficas e financeiras), *Bank Marketing* e *Statlog (German Credit Data)* (separe atributos do cliente e do crédito).
> - **Kaggle:** *Telco Customer Churn* (separe colunas de contrato e de uso) e *Heart Failure Prediction*.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Banco e loja: quando o aprendizado vertical compensa?

Contexto. Um banco (colunas A) e uma loja (colunas B), fictícios, querem prever o risco de um cliente. O banco possui o rótulo. Os dois têm **apenas parte dos clientes em comum**. O time quer saber: *quanto de alinhamento é necessário para que o treino vertical valha a pena, e o que acontece se protegermos o resíduo com ruído?*

Tarefas.

1. No conjunto *Breast Cancer Wisconsin*, divida as colunas ao acaso entre A e B (15 cada, `np.random.default_rng(1)`).
2. Para a fração de clientes alinhados $\rho \in \{1; 0{,}5; 0{,}2; 0{,}1\}$ (5 sementes), treine: (a) só A (todos os dados de treino); (b) só B (idem); (c) VFL usando **apenas os clientes alinhados**.
3. Reporte a acurácia no teste e indique a partir de que ρ o VFL deixa de superar o melhor participante isolado.
4. Para $\rho = 1$, repita o VFL com ruído $\sigma = 1$ no resíduo enviado às partes passivas, relatando a acurácia e a fração de rótulos recuperada.
5. Escreva uma recomendação em cinco linhas.

Entregáveis. Código, tabela e recomendação.

A solução de referência está no Apêndice E.



Capítulo 11 — Fronteiras e Prática: Não Supervisionado, Modelos de Fundação, Ferramentas, Aplicações e Ética

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Executar K-means e PCA de forma federada e verificar que reproduzem o resultado centralizado.
- Explicar por que modelos de fundação pedem ajuste eficiente em parâmetros e como adaptadores de baixo posto reduzem a comunicação.
- Comparar frameworks de aprendizado federado e escolher um conforme o cenário.
- Descrever aplicações reais em saúde, dispositivos móveis, IoT e finanças.
- Relacionar o aprendizado federado à LGPD, ao GDPR e a princípios de justiça e responsabilidade.
- Montar uma lista de verificação para um projeto federado real.

11.1 Além da classificação

Quase tudo que fizemos até aqui foi **classificar** (aprendizado supervisionado). Mas o aprendizado federado vai muito além:

- **Aprendizado não supervisionado:** agrupar, reduzir dimensões e descobrir padrões **sem rótulos**.
- **Análises federadas:** calcular estatísticas (médias, histogramas, contagens) sobre dados distribuídos sem juntá-los.
- **Modelos de fundação:** especializar grandes modelos de linguagem e de imagens com dados privados.

Vamos conhecer cada um, começando pelo mais simples.

No seu dia a dia

Uma rede de lojas quer saber **quais perfis de cliente existem**, sem que cada loja entregue seu cadastro. Ninguém rotulou os clientes: o objetivo é **descobrir grupos**. É um problema não supervisionado, e dá para resolvê-lo de forma federada.

11.2 K-means federado

O **K-means** divide os dados em *k* grupos. Em cada iteração (algoritmo de Lloyd):

1. cada ponto é atribuído ao **centroide mais próximo**;
2. cada centroide vira a **média** dos seus pontos.

A beleza: o passo 2 só precisa, para cada grupo, da **soma dos pontos** e da **contagem**. Essas são estatísticas que **se combinam por soma**, como a média e a variância do Capítulo 2.

K-means federado:

1. o servidor envia os *k* centroides;
2. cada cliente atribui seus pontos e devolve, por grupo, **(soma, contagem)**;
3. o servidor soma tudo e calcula os novos centroides.

O resultado é **idêntico** ao do K-means centralizado, com a mesma inicialização.

11.3 PCA federado

A **análise de componentes principais** (PCA) acha as direções de maior variação dos dados. Ela depende da **matriz de covariância**, que se calcula com somas:

$$\text{cov} = \left(\Sigma\, x\, x^{T} - n \cdot \mu\, \mu^{T} \right) / \left(n - 1 \right)$$

Cada cliente envia n, Σx e $\Sigma\, x\, x^{T}$. O servidor soma, calcula a covariância e seus autovetores. De novo, o resultado é **exatamente o do PCA centralizado**.

11.4 A biblioteca fronteiras.py

Salve o código abaixo como `fronteiras.py`, ao lado da `fedlib.py`. Ele reúne o K-means e o PCA federados e o ajuste de adaptadores de baixo posto da seção 11.6.

```python
import numpy as np
from fedlib import *


# ---------------------------------------------------------------- K-means federado
def kmeans_federado(partes, X, centroides, iteracoes=30):
    """Algoritmo de Lloyd distribuído: cada cliente devolve, por grupo, a soma dos pontos e a
contagem."""
    C = centroides.copy()
    for _ in range(iteracoes):
        S, N = np.zeros_like(C), np.zeros(len(C))
        for p in partes:                                  # (em um sistema real, isto roda em cada
cliente)
            Xk = X[p]
            a = ((Xk[:, None, :] - C[None]) ** 2).sum(-1).argmin(1)
            for j in range(len(C)):
                S[j] += Xk[a == j].sum(0)
                N[j] += (a == j).sum()
        C = np.where(N[:, None] > 0, S / np.maximum(N, 1)[:, None], C)   # servidor: nova média de cada
grupo
    return C


# ---------------------------------------------------------------- PCA federado
def pca_federado(partes, X, n_componentes):
    """PCA exato a partir de somas locais: cada cliente envia n, Σx e Σxxᵀ."""
    n, s, ss = 0, 0.0, 0.0
    for p in partes:
```

```
        Xk = X[p]
        n += len(Xk); s = s + Xk.sum(0); ss = ss + Xk.T @ Xk
    mu = s / n
    cov = (ss - n * np.outer(mu, mu)) / (n - 1)
    valores, vetores = np.linalg.eigh(cov)
    ordem = np.argsort(valores)[::-1][:n_componentes]
    return mu, valores[ordem], vetores[:, ordem]


# ------------------------------------------------------------------ Adaptadores de baixo posto (estilo LoRA)
def fedavg_adaptador(partes, X, y, Xte, yte, W0, posto, rodadas=20, epocas=3, eta=0.1, lote=32, seed=0):
    """Mantém W0 congelado e treina só ΔW = A·B, com A (d×r) e B (r×K). Só A e B trafegam.
    Com posto=None, treina o modelo inteiro (referência)."""
    rng = np.random.default_rng(seed)
    d, K = W0.shape
    A = rng.normal(0, 0.01, (d, posto)) if posto else None
    B = np.zeros((posto, K)) if posto else None
    W = W0.copy()
    bytes_por_rodada = 2 * 4 * (d * K if posto is None else posto * (d + K))
    historico = []
    for r in range(rodadas):
        locais, tam = [], []
        for c, p in enumerate(partes):
            if len(p) == 0:
                continue
            rk = np.random.default_rng(seed * 1000 + r * 50 + c)
            if posto is None:
                w = W.copy()
                for _ in range(epocas):
                    o = rk.permutation(len(p))
                    for i in range(0, len(p), lote):
                        b = p[o[i:i + lote]]; w -= eta * gradiente(w, X[b], y[b])
                locais.append(w)
            else:
                a, bb = A.copy(), B.copy()
                for _ in range(epocas):
                    o = rk.permutation(len(p))
                    for i in range(0, len(p), lote):
                        b = p[o[i:i + lote]]
                        G = gradiente(W0 + a @ bb, X[b], y[b])          # gradiente em relação a ΔW
                        a, bb = a - eta * G @ bb.T, bb - eta * a.T @ G   # regra da cadeia para A e B
                locais.append((a, bb))
            tam.append(len(p))
        pesos = np.array(tam) / sum(tam)
        if posto is None:
            W = sum(pi * wi for pi, wi in zip(pesos, locais)); acc = acuracia(W, Xte, yte)
        else:
            A = sum(pi * l[0] for pi, l in zip(pesos, locais)); B = sum(pi * l[1] for pi, l in zip(pesos, locais))
            acc = acuracia(W0 + A @ B, Xte, yte)
        historico.append(acc)
    return historico, bytes_por_rodada
```

11.5 Experimento: agrupar e comprimir sem juntar os dados

Usamos o conjunto *Wine*, da UCI, em **5 clientes muito desiguais** (distribuição de Dirichlet com α = 0,3). Comparamos com o `KMeans` e o `PCA` do scikit-learn aplicados aos dados juntos.

```python
import numpy as np
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import adjusted_rand_score
from fronteiras import *

X, y = load_wine(return_X_y=True)
X = StandardScaler().fit_transform(X)          # (didático; ver aviso do Capítulo 2 sobre padronização federada)
partes = particao_dirichlet(y, 5, 0.3, seed=0)
print("clientes (tamanhos):", [len(p) for p in partes])

# ---- K-means ----
rng = np.random.default_rng(0)
C0 = X[rng.choice(len(X), 3, replace=False)]                    # mesma inicialização para os dois
Cf = kmeans_federado(partes, X, C0, iteracoes=50)
km = KMeans(3, init=C0, n_init=1, max_iter=300, tol=0, algorithm="lloyd").fit(X)
rotulos_f = ((X[:, None, :] - Cf[None]) ** 2).sum(-1).argmin(1)
print("K-means: diferença máxima entre centroides:", float(np.abs(Cf - km.cluster_centers_).max()))
print("K-means: ARI entre federado e centralizado:", adjusted_rand_score(rotulos_f, km.labels_))
print("K-means: ARI do federado contra as classes reais:", round(adjusted_rand_score(y, rotulos_f), 3))

# ---- PCA ----
mu, valores, vetores = pca_federado(partes, X, 3)
pca = PCA(3).fit(X)
print("PCA: variâncias federadas   :", np.round(valores, 4))
print("PCA: variâncias centralizadas:", np.round(pca.explained_variance_, 4))
print("PCA: |cos| entre componentes :", np.round([abs(vetores[:, i] @ pca.components_[i]) for i in range(3)], 6))
```

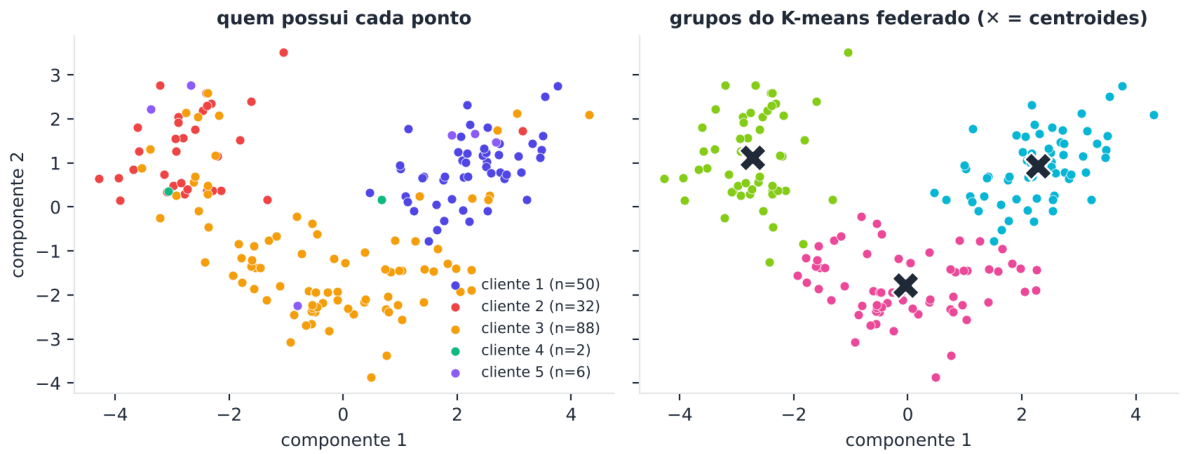


Figura 11.3 — Projeção dos vinhos nas duas primeiras componentes principais. À esquerda, cada ponto tem a cor do cliente que o possui (muito desiguais); à direita, o agrupamento federado, idêntico ao centralizado.

Resultados: os centroides coincidem até a casa dos 10^{-16}, o agrupamento federado é **idêntico** ao centralizado (ARI = 1), e as variâncias e direções do PCA são as mesmas.

> **Memorize**
>
> Sempre que um algoritmo depende só de **somas de estatísticas locais** (médias, contagens, covariâncias), a versão federada é **exata**. Isso vale para K-means, PCA, regressão linear por mínimos quadrados e Naive Bayes.

> **Atenção**
>
> Somas e contagens por grupo **também vazam informação**, sobretudo em grupos pequenos (por exemplo, um grupo com um único cliente revela sua média exata). Combine com **agregação segura** e **privacidade diferencial** (Capítulo 8).

11.6 Modelos de fundação e modelos de linguagem

11.6.1 O cenário

Modelos de fundação (*foundation models*) são modelos enormes pré-treinados com dados gerais e depois **especializados**. Muitos dos dados mais valiosos para especializá-los (prontuários, contratos, conversas de atendimento) são **privados** e não podem ser juntados. Daí o interesse em **ajustar modelos de linguagem de forma federada** (Ye et al., 2024; Kuang et al., 2024).

Os obstáculos são claros:

- **Tamanho:** bilhões de parâmetros. Enviar o modelo inteiro a cada rodada é inviável.
- **Memória e cálculo:** muitos clientes não conseguem sequer treinar o modelo completo.
- **Memorização:** modelos grandes decoram trechos do treino (Carlini et al., 2021).

11.6.2 A saída: ajuste eficiente em parâmetros

Em vez de treinar tudo, **congela-se** o modelo pré-treinado e treinam-se **poucos parâmetros extras**:

- **Adaptadores** (*adapters*): pequenas camadas inseridas no modelo (Houlsby et al., 2019).
- **LoRA** (*low-rank adaptation*): aprende uma correção de **baixo posto** nos pesos (Hu et al., 2022).

A ideia do LoRA: em vez de aprender uma correção ΔW com $d \times K$ números, aprendem-se duas matrizes pequenas, A ($d \times r$) e B ($r \times K$), com r pequeno, e usa-se

$$W = W_0 + A \cdot B$$

O número de parâmetros treináveis cai de $d \cdot K$ para $r \cdot (d + K)$. Só A e B trafegam entre cliente e servidor. A economia de comunicação é enorme.

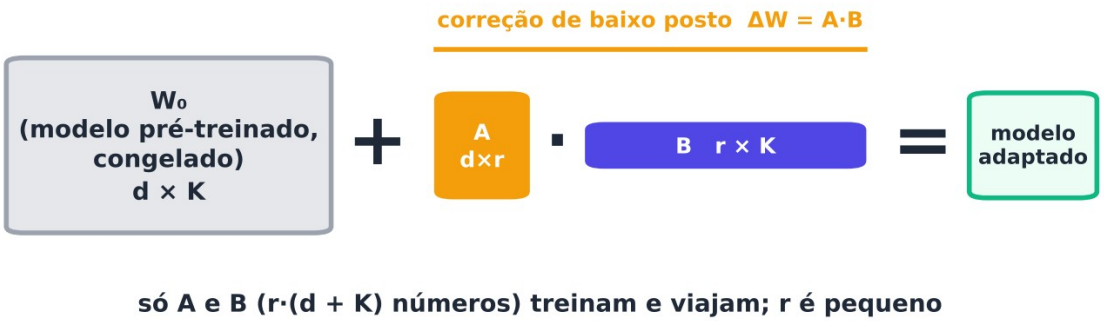


Figura 11.1 — O modelo pré-treinado W_0 fica congelado. Cada cliente treina só as matrizes A e B, de baixo posto. Só elas trafegam.

11.6.3 Um experimento em miniatura

Imitamos o cenário com um modelo pequeno. O "modelo de fundação" é uma softmax treinada com os dígitos (acurácia de 0,97). Aparece então uma **tarefa nova**: o mundo trocou os nomes de duas duplas de classes (3 ↔ 5 e 4 ↔ 9). O modelo antigo erra muito (acurácia 0,58). Vinte clientes (Dirichlet α = 0,5) ajustam o modelo, ou por completo, ou só com adaptadores de posto *r*.

```python
import numpy as np
from fronteiras import *

Xtr, Xte, ytr, yte = carregar_digits()

# "modelo de fundação": softmax treinada com os rótulos originais
W0 = np.zeros((65, 10))
for _ in range(300):
    W0 -= 0.5 * gradiente(W0, Xtr, ytr)
print(f"modelo pré-treinado, tarefa original : {acuracia(W0, Xte, yte):.3f}")

# tarefa nova: troca dos nomes 3<->5 e 4<->9
perm = np.arange(10); perm[3], perm[5] = 5, 3; perm[4], perm[9] = 9, 4
ytr2, yte2 = perm[ytr], perm[yte]
print(f"modelo pré-treinado, tarefa nova     : {acuracia(W0, Xte, yte2):.3f}")

print(f"\n{'método':<14}{'KB por rodada':>15}{'acc rodada 5':>14}{'acc rodada 20':>15}")
for nome, posto in [("completo", None), ("LoRA r=1", 1), ("LoRA r=2", 2), ("LoRA r=4", 4), ("LoRA r=8",
8)]:
    hs = []
    for s in range(3):
        partes = particao_dirichlet(ytr, 20, 0.5, seed=s)
        h, bytes_rodada = fedavg_adaptador(partes, Xtr, ytr2, Xte, yte2, W0, posto, seed=s)
        hs.append(h)
    h = np.mean(hs, axis=0)
    print(f"{nome:<14}{bytes_rodada / 1024:>15.1f}{h[4]:>14.3f}{h[-1]:>15.3f}")
```

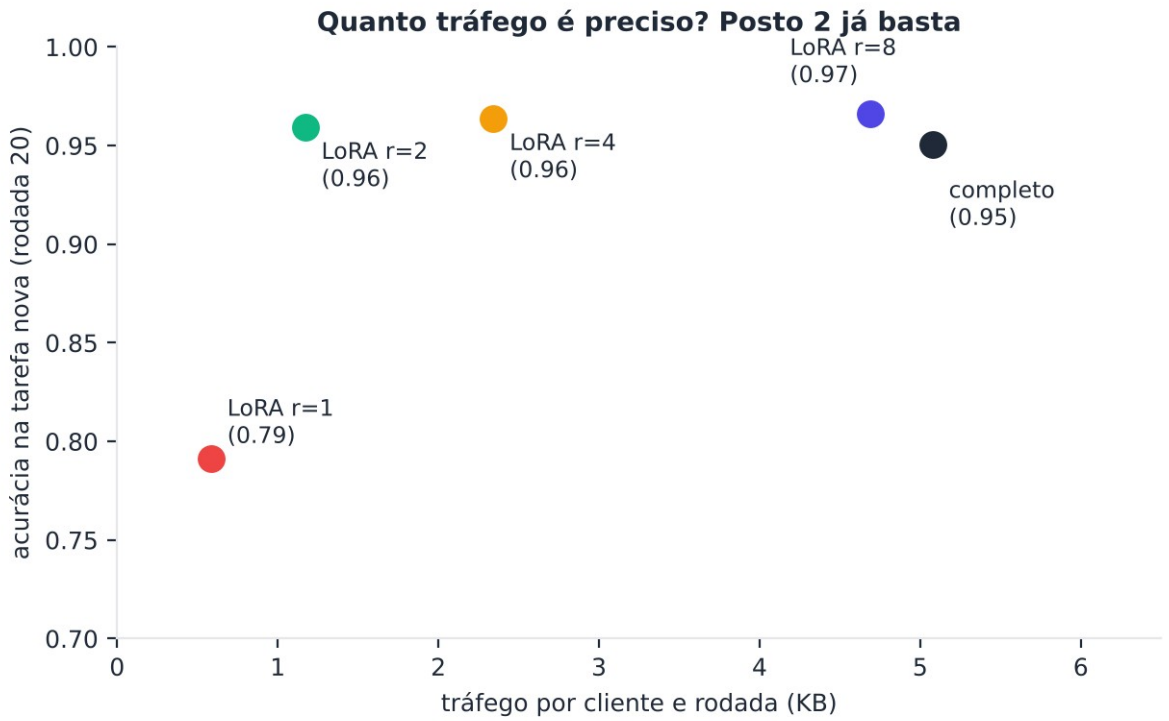


Figura 11.2 — Acurácia na tarefa nova contra o tráfego por cliente e rodada: o posto 2 já recupera o desempenho do ajuste completo com cerca de um quarto do tráfego; o posto 1 não basta.

O que aprendemos:

- O **ajuste completo** chega a cerca de 0,95, trafegando **5,1 KB** por cliente e rodada.
- O **LoRA com posto 2** atinge **0,96**, com **1,2 KB** (4 vezes menos).
- No começo, os adaptadores são **mais lentos** (rodada 5: 0,66 contra 0,79 do ajuste completo), pois precisam de mais rodadas para aprender a correção.
- O **posto 1 não basta** (≈ 0,79): trocar duas duplas de classes exige uma correção de posto **pelo menos 2**. A capacidade do adaptador precisa casar com a tarefa.
- Em redes grandes, a diferença de tamanho é gigantesca (milhões contra bilhões de parâmetros).

> **Atenção**
>
> Uma sutileza: a média de A e a média de B **não** são a média dos produtos $A \cdot B$. A média das matrizes separadas introduz um erro de agregação. Métodos recentes tratam disso (por exemplo, agregando os produtos ou empilhando os adaptadores). No nosso experimento, o erro é pequeno, mas em modelos grandes merece atenção.

> **Saiba mais**
>
> Outras ideias para modelos de fundação federados: **prompts** aprendidos (em vez de pesos), **destilação** (os clientes enviam previsões, e não pesos), **modelos pequenos locais com grandes remotos**, e o uso de **dados sintéticos** gerados pelo modelo para reduzir a exposição de dados reais. A área está em rápida evolução: consulte as publicações recentes.

11.7 Ferramentas e frameworks

Para sair da simulação e construir sistemas reais, usam-se frameworks. Os principais são software livre.

Ferramenta	Foco principal	Observações
Flower	Simulação e implantação, independente de biblioteca de ML	Suporta PyTorch, TensorFlow, scikit-learn
TensorFlow Federated (TFF)	Pesquisa e simulação com TensorFlow	Integra privacidade diferencial
PySyft	Privacidade e computação remota	Foco em ciência de dados com dados privados
FATE	Cross-silo industrial, incluindo vertical	Voltado a finanças e empresas
FedML	Pesquisa, simulação e implantação	Dispositivos móveis e nuvem
NVIDIA FLARE	Cross-silo, saúde e indústria	Foco em implantação
OpenFL	Cross-silo, saúde	Originado em colaborações médicas
LEAF e FedScale	Conjuntos de referência (*benchmarks*)	Para comparar algoritmos

Para escolher, pergunte:

1. É **simulação** (pesquisa) ou **implantação** (produção)?
2. É **cross-device** ou **cross-silo**? É horizontal ou vertical?
3. Qual **biblioteca de ML** a equipe usa?
4. Que **garantias de segurança** são necessárias (agregação segura, DP, TEE)?
5. A ferramenta é **mantida ativamente** e tem comunidade?

> **Atenção**
>
> Ferramentas mudam rápido. Antes de decidir, consulte a **documentação atual** e a licença de cada projeto. Este livro não valida versões específicas.

11.8 Aplicações reais

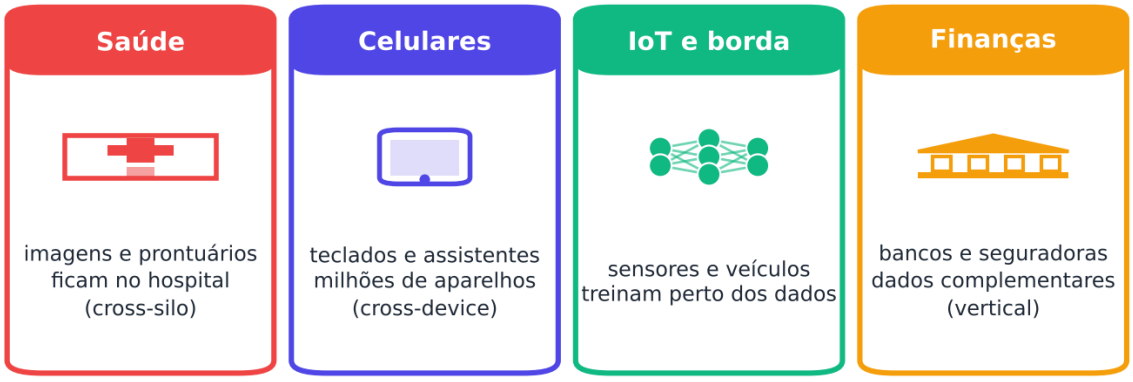


Figura 11.4 — Quatro áreas de aplicação e o que cada uma protege ou combina.

11.8.1 Saúde

É a área de maior destaque. Colaborações entre hospitais usaram aprendizado federado em imagens de tumores cerebrais (Sheller et al., 2020) e em previsão de necessidades de oxigênio na COVID-19 com dados de vinte instituições (Dayan et al., 2021). Rieke et al. (2020) e Xu et al. (2021) discutem oportunidades e obstáculos: dados muito heterogêneos entre hospitais, protocolos de imagem diferentes, governança e confiança.

11.8.2 Dispositivos móveis

Teclados, assistentes de voz e recomendações em celulares. O sistema de produção do Google, descrito por Bonawitz et al. (2019), treinou com **milhões** de dispositivos, selecionando clientes apenas quando carregando, conectados a redes sem franquia de dados e ociosos.

11.8.3 Internet das coisas e redes

Sensores industriais, veículos conectados e redes elétricas inteligentes geram dados na borda. Nguyen et al. (2021) revisam as aplicações em IoT: detecção de intrusão, manutenção preditiva, veículos e cidades inteligentes.

11.8.4 Finanças e seguros

Bancos e seguradoras têm obrigações de sigilo e dados complementares: o cenário clássico do aprendizado **vertical** (Yang et al., 2019), por exemplo, para avaliação de crédito e detecção de fraude entre instituições.

11.9 Regulação, ética e responsabilidade

11.9.1 Aprendizado federado não é anonimização

Um erro perigoso é achar que "os dados não saem, logo não há problema". Como vimos nos Capítulos 7 a 9, atualizações vazam informação, e modelos memorizam. Do ponto de vista jurídico, um modelo que memoriza dados pessoais **pode ser tratado como dado pessoal**, a depender da jurisdição e do caso.

11.9.2 Princípios de proteção de dados

A **LGPD** (Lei nº 13.709/2018), no Brasil, e o **GDPR**, na União Europeia, estabelecem princípios que se aplicam a projetos federados:

- **Finalidade:** o uso dos dados precisa ter um propósito claro e legítimo.
- **Minimização:** coletar e tratar só o necessário.
- **Transparência:** informar claramente o titular.
- **Segurança:** adotar medidas técnicas e administrativas.
- **Responsabilização:** poder demonstrar a conformidade.
- **Base legal** para o tratamento, como consentimento ou legítimo interesse.

Projetos de alto risco costumam exigir um **relatório de impacto à proteção de dados** (RIPD, no Brasil; DPIA, na Europa). Regulamentos de inteligência artificial, como o da União Europeia aprovado em 2024, acrescentam exigências conforme o risco do sistema.

> **Atenção**
>
> Este livro **não é aconselhamento jurídico**. Antes de usar dados pessoais, consulte o encarregado de dados (DPO) e o jurídico da organização.

11.9.3 Justiça (fairness)

Um modelo global pode servir **bem à maioria** e **mal a minorias**. Em aprendizado federado, pode favorecer clientes grandes ou com dados típicos. Duas propostas conhecidas:

- **Aprendizado federado agnóstico** (Mohri et al., 2019): otimiza o **pior caso** sobre misturas dos clientes, em vez da média.

- **q-FFL** (Li et al., 2020a): dá mais peso aos clientes com **maior perda**; um parâmetro q regula o quanto a justiça pesa sobre a média.

Meça sempre por cliente (como no Capítulo 6): média, pior cliente e percentil 10.

11.9.4 Incentivos e governança

- **Por que participar?** Se o modelo beneficia a todos, há tentação de "pegar carona". Mecanismos de incentivo e de **atribuição de valor** à contribuição de cada cliente são um problema aberto (Kairouz et al., 2021).
- **Governança:** em cross-silo, definir por contrato quem controla o modelo, quem pode usá-lo e o que acontece quando um participante sai.
- **Transparência e auditoria:** documentar dados, protocolos e riscos conhecidos (por exemplo, com fichas técnicas de modelo e de conjuntos de dados).
- **Impacto ambiental:** muitos clientes treinando consome energia. Compare com o custo do treino centralizado.

11.10 Lista de verificação de um projeto federado

> MEMORIZE **Antes de começar um projeto federado**
>
> 1. **Necessidade:** os dados realmente não podem ser centralizados? Há alternativa mais simples? 2. **Cenário:** cross-device ou cross-silo? Horizontal ou vertical? 3. **Dados:** quão heterogêneos? Há desbalanceamento entre clientes? 4. **Modelo:** cabe nos clientes? Vale usar adaptadores ou modelos menores? 5. **Comunicação:** qual orçamento por rodada? Compressão necessária? 6. **Privacidade:** modelo de ameaça definido? DP, agregação segura? 7. **Robustez:** risco de clientes maliciosos? Agregador robusto? 8. **Avaliação:** testes locais por cliente, com média, pior cliente e percentil 10. 9. **Legal e ético:** base legal, RIPD, governança, justiça, auditoria. 10. **Operação:** monitoramento, versão do modelo, atualização e retirada de participantes.

11.11 Problemas em aberto

Kairouz et al. (2021) reúnem dezenas de problemas em aberto. Alguns dos mais relevantes:

- garantias de privacidade úteis **com poucos clientes**;
- **robustez** sem abrir mão da agregação segura;
- **heterogeneidade** extrema e **personalização** em escala;
- **modelos de fundação** federados: eficiência, memorização e avaliação;
- **justiça** e **incentivos** em sistemas reais;
- **benchmarks** realistas, com dados e sistemas heterogêneos.

11.12 Resumo do capítulo

MEMORIZE **O essencial do Capítulo 11**

- Algoritmos baseados em **somas de estatísticas** (K-means, PCA) têm versões federadas **exatas**.
- Modelos de fundação pedem **ajuste eficiente** (adaptadores, LoRA): só A e B trafegam.
- A capacidade do adaptador (posto) deve casar com a tarefa.
- Frameworks: Flower, TFF, PySyft, FATE, FedML, FLARE, OpenFL. Escolha conforme o cenário.
- Aplicações: saúde, celulares, IoT e finanças.
- Federado **não é anonimização**: respeite LGPD, GDPR, finalidade e minimização.
- Justiça, incentivos e governança são parte do projeto, e não um detalhe.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Por que o K-means federado produz o mesmo resultado do centralizado (com a mesma inicialização)?

a) Porque a atualização dos centroides depende só de somas e contagens por grupo, que se combinam exatamente.
b) Porque os dados são copiados.
c) Porque usa ruído.
d) Porque ignora os clientes pequenos.

Questão 2. No K-means federado, o que cada cliente devolve ao servidor em cada iteração?

a) Todos os seus pontos.
b) Apenas os rótulos.
c) Por grupo, a soma dos pontos e a contagem.
d) Uma cópia do modelo.

Questão 3. O PCA federado calcula a covariância global a partir de:

a) todos os dados brutos.
b) apenas os rótulos.
c) apenas as médias.
d) n, Σx e $\Sigma\, x\, x^{T}$ de cada cliente.

Questão 4. No experimento com o conjunto Wine, a diferença máxima entre os centroides federados e centralizados foi da ordem de:

a) 1.
b) 10^{-16}.
c) 10^{-1}.
d) 10.

Questão 5. O que significa ARI = 1 entre o agrupamento federado e o centralizado?

a) Um dos agrupamentos falhou.
b) Os agrupamentos são opostos.
c) Os agrupamentos são idênticos (a menos de nomes dos grupos).
d) Os agrupamentos são aleatórios.

Questão 6. Para quais algoritmos a versão federada exata é possível, segundo o capítulo?

a) Os que dependem só de somas de estatísticas locais (K-means, PCA, mínimos quadrados).
b) Nenhum.
c) Apenas redes profundas.
d) Qualquer algoritmo.

Questão 7. Por que somas e contagens por grupo ainda podem vazar informação?

a) Porque são sempre criptografadas.
b) Em grupos pequenos, revelam valores individuais ou médias exatas de um cliente.
c) Porque não envolvem dados.
d) Porque são enormes.

Questão 8. O que são modelos de fundação (foundation models)?

a) Modelos sem parâmetros.
b) Grandes modelos pré-treinados com dados gerais, depois especializados.
c) Modelos lineares simples.
d) Modelos treinados só localmente.

Questão 9. Por que federar a especialização de modelos de linguagem é interessante?

a) Porque elimina o pré-treino.
b) Porque reduz o tamanho do modelo.
c) Muitos dados valiosos (prontuários, contratos) são privados e não podem ser juntados.
d) Porque dados privados são inúteis.

Questão 10. Qual o principal obstáculo de federar modelos com bilhões de parâmetros?

a) Falta de dados.
b) A matemática não funciona.
c) Enviar o modelo inteiro a cada rodada é inviável em comunicação, memória e cálculo.
d) Excesso de privacidade.

Questão 11. A ideia do LoRA é:

a) aprender a correção dos pesos como o produto de duas matrizes de baixo posto $A \cdot B$.
b) quantizar o modelo para 1 bit.
c) treinar do zero.
d) apagar o modelo.

Questão 12. Em um LoRA com $d = 1.000$, $K = 1.000$ e $r = 8$, quantos parâmetros treináveis há?

a) 16.000.
b) 2.000.
c) 8.000.
d) 1.000.000.

Questão 13. No mesmo caso, qual a fração aproximada em relação ao ajuste completo ($d \cdot K = 1.000.000$)?

a) 1,6%.
b) 16%.
c) 100%.
d) 50%.

Questão 14. Quais parâmetros trafegam entre cliente e servidor em um LoRA federado?

a) O modelo W_0 inteiro.
b) Os rótulos.
c) Apenas A e B.
d) Os dados.

Questão 15. No experimento, o LoRA de posto 1 foi insuficiente porque:

a) trocar duas duplas de classes exige uma correção de posto pelo menos 2.
b) os clientes eram poucos.
c) o posto 1 usa mais bytes.
d) a taxa de aprendizado era zero.

Questão 16. No experimento, o LoRA de posto 2 obteve acurácia final de cerca de:

a) 0,10.
b) 0,96.
c) 0,79.
d) 0,58.

Questão 17. No experimento, quanto tráfego (KB por cliente e rodada) usou o ajuste completo e o LoRA de posto 2?

a) Cerca de 5,1 KB e 1,2 KB.
b) 1 KB e 5 KB.
c) 10 KB e 10 KB.
d) 0,1 KB e 0,1 KB.

Questão 18. Qual a sutileza na agregação de adaptadores A e B?

a) Os produtos são sempre iguais.
b) A e B não podem ser somados.
c) Não há sutileza.
d) A média de A e a média de B não é a média dos produtos A·B.

Questão 19. Qual das ferramentas abaixo é um framework de aprendizado federado independente de biblioteca de ML?

a) PCA.
b) LEAF.
c) Flower.
d) K-means.

Questão 20. Qual framework é voltado a cenários industriais cross-silo, incluindo o vertical?

a) FATE.
b) Nenhum.
c) PySyft apenas.
d) LEAF.

Questão 21. LEAF e FedScale são principalmente:

a) bibliotecas de criptografia.
b) conjuntos de referência (benchmarks) para comparar algoritmos.
c) servidores de produção.
d) algoritmos de compressão.

Questão 22. Qual pergunta ajuda a escolher um framework?

a) Quantos funcionários a empresa tem?
b) É simulação ou implantação, cross-device ou cross-silo, e qual biblioteca de ML a equipe usa?
c) Qual a cor do logotipo?
d) Quantas estrelas o repositório tem, apenas?

Questão 23. Em saúde, qual aplicação real com aprendizado federado é citada no capítulo?

a) Segmentação de tumores cerebrais e previsão de oxigênio na COVID-19.
b) Tradução automática.
c) Reconhecimento de placas.
d) Jogos.

Questão 24. No sistema de produção descrito por Bonawitz et al. (2019), os dispositivos participavam quando:

a) carregando, em rede sem franquia de dados e ociosos.
b) desligados.
c) sempre, a qualquer momento.
d) em uso intenso.

Questão 25. Por que "os dados não saem" não equivale a anonimização?

a) Atualizações vazam informação e modelos memorizam, podendo ser tratados como dados pessoais.
b) Porque os dados são públicos.
c) Porque o servidor é lento.
d) Porque não há risco.

Questão 26. Qual dos itens abaixo é um princípio de proteção de dados citado no capítulo?

a) Retenção ilimitada.
b) Sigilo sobre o uso.
c) Minimização: coletar e tratar só o necessário.
d) Maximização do volume de dados.

Questão 27. O relatório de impacto à proteção de dados (RIPD / DPIA) serve para:

a) criptografar os dados.
b) acelerar o treino.
c) escolher o modelo.
d) avaliar e documentar riscos de um tratamento de dados de alto risco.

Questão 28. O aprendizado federado agnóstico (Mohri et al., 2019) busca:

a) eliminar clientes.
b) otimizar o pior caso sobre misturas dos clientes.
c) usar menos dados.
d) minimizar a média do erro apenas.

Questão 29. O q-FFL (Li et al., 2020) dá mais peso:

a) aos clientes com mais dados.
b) aos clientes mais rápidos.
c) a nenhum cliente.
d) aos clientes com maior perda.

Questão 30. Qual dos itens abaixo é um problema em aberto citado por Kairouz et al. (2021)?

a) Instalar o Python.
b) Reduzir o número de rodadas a zero.
c) Escolher o nome do projeto.
d) Incentivos e atribuição de valor à contribuição dos clientes.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique como o K-means federado funciona e por que é exato.
2. Explique como se calcula o PCA de forma federada.
3. Dê três outros algoritmos que podem ser federados de modo exato por somas de estatísticas e justifique um deles.
4. Por que as somas do K-means federado podem violar a privacidade e como mitigar?
5. Explique por que modelos de fundação pedem ajuste eficiente em parâmetros em cenários federados.
6. Descreva o LoRA e calcule o número de parâmetros treináveis para d = 4.096, K = 4.096 e r = 8.
7. Interprete os resultados do experimento de LoRA (posto 1, 2, 4, 8 e ajuste completo).
8. Qual a sutileza de agregar A e B separadamente e como a literatura trata?
9. Cite duas outras ideias de modelos de fundação federados além do LoRA.
10. Compare Flower, TFF e FATE quanto ao foco.
11. Liste cinco perguntas para escolher um framework de aprendizado federado.
12. Descreva duas aplicações reais de aprendizado federado e o que fica local em cada uma.
13. Explique por que aprendizado federado não é anonimização e as implicações jurídicas.
14. Cite quatro princípios da LGPD/GDPR relevantes ao aprendizado federado.
15. O que é justiça em aprendizado federado e que propostas existem?
16. Quais questões de governança surgem em um projeto cross-silo?
17. Por que incentivos são um problema em aprendizado federado?
18. Aplique a lista de verificação do capítulo a um cenário seu, destacando três itens críticos.
19. Cite três problemas em aberto no aprendizado federado e explique um.
20. Resuma em cinco linhas o que você aprendeu e como aplicaria o aprendizado federado no seu contexto.

Exercícios práticos em Python (scikit-learn)

Resolva os exercícios no seu computador (ou no Google Colab). Eles usam conjuntos de dados da UCI que já vêm no scikit-learn ou dados simulados. Tente resolver **antes** de consultar a solução, no Apêndice D. As saídas apresentadas lá foram obtidas com as versões de bibliotecas listadas na Apresentação.

Exercício 1 — Escolhendo k no K-means federado

NÍVEL: BÁSICO

No conjunto *Wine* (padronizado), em 5 clientes Dirichlet (α = 0,3), rode `kmeans_federado` para k = 1 a 6, com inicialização por `np.random.default_rng(0)`. Calcule a **soma dos quadrados dentro dos grupos** (SSE) de forma federada (cada cliente envia seu SSE local, e o servidor soma). Compare com a `inertia_` do `KMeans` do scikit-learn com a mesma inicialização.

Pergunta: o "cotovelo" aparece em k = 3? Os valores federados coincidem com os centralizados?

Exercício 2 — PCA federado nos dígitos

NÍVEL: BÁSICO

Nos dígitos (`load_digits`, 64 atributos, padronizados), em 10 clientes Dirichlet (α = 0,5), calcule o PCA federado e a **variância explicada acumulada** com as primeiras 5, 10, 20 e 30 componentes. Compare com `PCA` do scikit-learn e determine quantas componentes explicam 90% da variância.

Pergunta: o PCA federado coincide com o centralizado? Quanto se reduz a dimensão?

Exercício 3 — Regressão linear federada exata

NÍVEL: INTERMEDIÁRIO

No conjunto de diabetes (`load_diabetes`, 10 atributos, alvo numérico), reparta os dados em 6 clientes de tamanhos diferentes. Cada cliente envia X^TX e X^Ty (com uma coluna de 1 para o intercepto). O servidor soma e resolve as equações normais. Compare os coeficientes com `LinearRegression`. Depois repita com regularização de Ridge (λ = 1) e compare com `Ridge(alpha=1)`.

Pergunta: por que o resultado é exatamente igual ao centralizado?

Exercício 4 — Quanto posto o adaptador precisa?

NÍVEL: INTERMEDIÁRIO

Reproduza o experimento de LoRA do capítulo com **três tarefas novas**, cada uma trocando o nome de mais classes: (a) 3 ↔ 5; (b) 3 ↔ 5 e 4 ↔ 9; (c) 3 ↔ 5, 4 ↔ 9 e 1 ↔ 7. Para os postos 1, 2, 4 e 8, relate a acurácia na rodada 20 (média de 2 sementes, 20 clientes Dirichlet α = 0,5).

Pergunta: como o posto necessário cresce com o número de trocas?

> SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**
>
> - **UCI:** *Wholesale Customers* (perfis de clientes), *Online Retail* e *Iris*, para praticar K-means e PCA.
> - **Kaggle:** *Mall Customer Segmentation Data* e *Telco Customer Churn*.
>
> Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador do capítulo

Perfil de clientes sem juntar os cadastros

Contexto. Uma rede de 5 lojas (fictícia) quer entender **perfis de clientes** sem juntar os cadastros. Como não há dados reais, use o conjunto *Wine* da UCI como substituto: cada vinho é um "cliente" descrito por 13 atributos, e as 3 classes são perfis "ocultos". As lojas têm perfis muito desiguais (Dirichlet $\alpha = 0{,}3$).

Tarefas.

1. Faça a **padronização federada** (n, Σx, Σx^2 por loja).
2. Faça o **PCA federado** e reduza a 2 componentes.
3. Faça o **K-means federado** ($k = 3$) nos dados projetados.
4. Compare com o pipeline centralizado equivalente (padronização, PCA e K-means do scikit-learn, com os mesmos exemplos como centroides iniciais): proximidade das componentes principais e ARI entre os agrupamentos.
5. Meça o ARI do agrupamento federado contra as classes ocultas.
6. **Efeito da privacidade:** repita o K-means federado somando ruído gaussiano de desvio σ às somas agregadas por grupo (e à contagem) em cada iteração, para $\sigma \in \{0; 1; 5; 20\}$. Reporte o ARI contra o agrupamento sem ruído.
7. Escreva uma recomendação em cinco linhas.

Entregáveis. Código, tabela e recomendação.

A solução de referência está no Apêndice E.

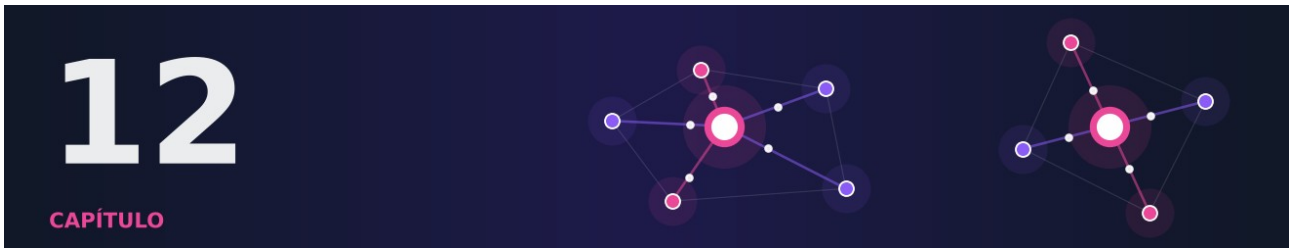
PARTE V

Parte V — Projeto Final

Um estudo completo que integra simulação federada, dados não-IID, comunicação, personalização, privacidade e robustez, da pergunta de negócio ao relatório.

Nesta parte:

- Cap. 12 — Projeto Integrador Final: do Problema ao Relatório



Capítulo 12 — Projeto Integrador Final: do Problema ao Relatório

OBJETIVOS DE APRENDIZAGEM

Ao final deste capítulo você será capaz de:

- Conduzir um estudo federado completo, da pergunta ao relatório.
- Combinar agregação robusta, compressão, privacidade e personalização em um mesmo motor de simulação.
- Avaliar o sistema por cliente e sob ataque, e não apenas pela acurácia global.
- Justificar escolhas técnicas com números e reconhecer limitações.
- Redigir um parecer técnico claro para quem decide.

12.1 O desafio

Você chegou ao final da jornada. Agora, em vez de estudar uma técnica por vez, vai **juntar todas**.

Imagine a **Federa Saúde** (fictícia): uma rede de **20 clínicas** que quer treinar um modelo de apoio ao diagnóstico **sem centralizar os dados** dos pacientes. A diretoria tem cinco preocupações:

1. **Qualidade:** o modelo precisa funcionar bem **em cada clínica**, e não só na média.
2. **Dados diferentes:** as clínicas atendem públicos muito distintos (dados não-IID).
3. **Segurança: duas** clínicas podem estar comprometidas e tentarão implantar um **backdoor**.
4. **Custo:** a rede entre as clínicas é lenta. Cada byte conta.
5. **Privacidade:** o comitê de ética pergunta qual é o preço de proteger os pacientes.



Figura 12.1 — As nove etapas do projeto, do diagnóstico dos dados ao relatório.

> **No seu dia a dia**
>
> Um projeto real nunca é "rodar um algoritmo". É uma sequência de decisões com compromissos: qualidade contra custo, privacidade contra utilidade, robustez contra eficiência. O seu trabalho é **medir esses compromissos** e **explicá-los** a quem decide.

12.2 Os dados e o cenário

Como não temos dados clínicos reais, usamos o conjunto de **dígitos manuscritos** (UCI, via scikit-learn) como substituto didático. Cada "clínica" é um cliente. O cenário fixo é:

- **20 clientes**, com dados **muito heterogêneos nos rótulos** (Dirichlet, $\alpha = 0{,}3$).
- Cada cliente separa **70% para treino e 30% para teste local**.
- **2 clientes maliciosos** (os de índice 0 e 1) executam um **ataque de *backdoor***: treinam também com imagens com gatilho rotuladas como "0".
- Um **teste global** (`Xte`) mede a acurácia geral e o **ASR**, a taxa de sucesso do backdoor.

12.3 O motor de simulação projeto.py

Para facilitar, o código abaixo reúne, em uma só função, as peças dos capítulos anteriores: **agregador** (média, mediana ou média aparada), **quantização** das atualizações, **privacidade diferencial no nível do cliente** (corte e ruído), **clientes maliciosos** e **contagem de bytes**. Salve-o como **`projeto.py`**. Ele usa também `compressao.py`, `robustez.py` e `personalizacao.py`.

```python
import numpy as np
from fedlib import *
from compressao import quantizar
from robustez import agg_media, agg_mediana, agg_media_aparada, com_gatilho
from personalizacao import dividir_local, acuracias_locais

AGREGADORES = {"media": agg_media, "mediana": agg_mediana, "aparada": lambda D: agg_media_aparada(D,
0.25)}
```

```
def fedavg_integrado(treino, X, y, Xte, yte, agregador="media", bits=None, dp_z=None, S=0.5,
                     maliciosos=(), gama=1.0, alvo=0, rodadas=30, epocas=5, eta=0.1, lote=32, seed=0,
classes=10):
    """Motor do projeto final: junta agregador robusto, compressão por quantização, DP no nível do
cliente
    (corte + ruído) e clientes maliciosos com backdoor. Devolve o modelo, a acurácia, o ASR e os bytes
de subida."""
    rng = np.random.default_rng(seed)
    W = np.zeros((X.shape[1], classes))
    n_par = W.size
    maliciosos = set(maliciosos)
    Xte_gat = com_gatilho(Xte[yte != alvo])
    bytes_subida = 0
    for r in range(rodadas):
        deltas = []
        for c, p in enumerate(treino):
            if len(p) == 0:
                continue
            Xc, yc = X[p], y[p]
            if c in maliciosos:                                          # backdoor: metade extra com
gatilho
                n_g = len(p) // 2
                Xc = np.vstack([Xc, com_gatilho(Xc[:n_g])]); yc = np.r_[yc, np.full(n_g, alvo)]
            d = treino_local(W, Xc, yc, epocas, eta, lote, 0.0, seed * 1000 + r * 50 + c) - W
            if c in maliciosos:
                d = gama * d
            if dp_z is not None:                                         # corte à norma S
                d = d * min(1.0, S / (np.linalg.norm(d) + 1e-12))
            if bits is not None:
                d = quantizar(d, bits, rng); bytes_subida += int(np.ceil(n_par * bits / 8)) + 8
            else:
                bytes_subida += 4 * n_par
            deltas.append(d)
        D = np.array(deltas)
        if dp_z is not None:
            media = D.mean(axis=0) + rng.normal(0, dp_z * S / len(D), W.shape)    # DP: ruído na média
            W = W + media
        else:
            W = W + AGREGADORES[agregador](D)
    acc = acuracia(W, Xte, yte)
    asr = float(np.mean(np.argmax(Xte_gat @ W, axis=1) == alvo))
    return W, acc, asr, bytes_subida
```

Observe duas decisões de projeto.

- A **DP** (`dp_z`) usa a **média com corte e ruído**, e não a mediana: a garantia formal do Capítulo 8 depende da sensibilidade controlada por corte e média.
- A **quantização** é aplicada **depois** do corte e **antes** da agregação, como faria um cliente real.

12.4 As nove etapas

Siga a ordem abaixo. Cada etapa tem uma pergunta e um número a registrar.

Etapa	Pergunta	O que registrar
1. Diagnóstico	Quão heterogêneos são os clientes?	Tamanhos, distância de variação total média (Cap. 4)
2. Linha de base	Como se sai o FedAvg sem proteção?	Acurácia global, por cliente (média e percentil 10)
3. Ameaça	O backdoor funciona?	ASR com a média
4. Robustez	Qual agregador resolve?	ASR e acurácia com mediana e média aparada
5. Comunicação	Quantos bytes podemos economizar?	Acurácia e KB com 8 e 4 bits
6. Privacidade	Qual o preço da DP?	Acurácia, ASR e ε (Cap. 8) para z escolhido
7. Personalização	O ajuste fino local ajuda?	Média e percentil 10 por cliente, antes e depois
8. Síntese	Qual configuração recomendar?	Tabela com todas as métricas
9. Relatório	Como explicar a decisão?	Parecer de uma página

12.5 Modelo de relatório

Um bom relatório técnico tem **uma página** e segue esta estrutura.

1. **Resumo executivo** (3 linhas): o que foi feito e a recomendação.
2. **Cenário e método**: dados, clientes, ameaças, métricas.
3. **Resultados**: uma tabela (e, se quiser, um gráfico) com as configurações.
4. **Riscos e limitações**: o que o estudo **não** garante.
5. **Recomendação e próximos passos**.

> **Dica**
>
> Escreva para **quem não é técnico**. Em vez de "o ASR caiu para 0,00", diga "com a mediana, o ataque deixou de funcionar". Em vez de "ε = 14", diga "a proteção formal é fraca com tão poucos clientes".

12.6 Critérios de avaliação

- **Correção técnica** (30%): experimentos corretos, com várias sementes.
- **Cobertura** (20%): todas as nove etapas atendidas.
- **Análise** (25%): interpretação honesta dos compromissos, com números.
- **Comunicação** (15%): relatório claro e acessível.
- **Rigor e ética** (10%): limitações reconhecidas; sem alegar privacidade formal sem garantia.

12.6.1 Para comparar depois

Quando terminar, compare o seu resultado com o de referência da Figura 12.2. As configurações seguem a ordem do enunciado: (1) base com média; (2) mediana; (3) mediana com 8 bits; (4) mediana com 4 bits; (5) DP com $z = 1$; (6) mediana com 4 bits e ajuste fino local.

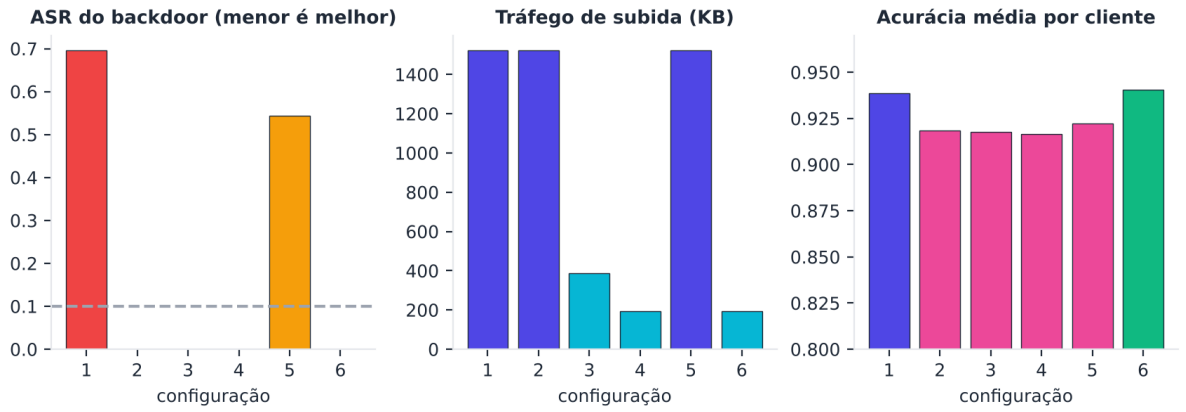


Figura 12.2 — Resultado de referência: ASR do backdoor, tráfego de subida e acurácia média por cliente, para as seis configurações. Média de 3 sementes.

12.7 O que você aprendeu

Percorra mentalmente o livro.

- **Capítulos 1 a 3:** por que federar, as bases do treino e o FedAvg.
- **Capítulos 4 a 6:** dados não-IID, comunicação e personalização.
- **Capítulos 7 a 9:** ataques, defesas e robustez.
- **Capítulos 10 e 11:** outras arquiteturas, aprendizado não supervisionado, modelos de fundação, ferramentas, aplicações e ética.

E mais importante: aprendeu que **todo método tem um preço**, e que decidir bem é medir esse preço e comunicá-lo com honestidade.

MEMORIZE **Para levar**

- Meça **por cliente**, **sob ataque** e **por byte**.
- Nenhuma técnica resolve tudo; as defesas **se combinam**.
- Privacidade e robustez estão em tensão; heterogeneidade e robustez também.
- Documente **limitações**. Um resultado honesto vale mais que um resultado bonito.

12.8 Para continuar estudando

- Releia o artigo de revisão de Kairouz et al. (2021): ele organiza os problemas em aberto.
- Teste um framework (Flower, TFF ou outro) e leve seus experimentos de simulação a um ambiente com processos separados.
- Aplique a lista de verificação do Capítulo 11 a um problema do seu trabalho.
- Contribua com a comunidade: reproduza resultados, relate falhas e compartilhe conjuntos de dados abertos.

Exercícios de múltipla escolha

Escolha **uma** alternativa em cada questão. O gabarito está no Apêndice A.

Questão 1. Qual é o objetivo central do aprendizado federado?

a) Substituir a privacidade diferencial.
b) Treinar um modelo com vários donos de dados sem juntar os dados brutos.
c) Reduzir sempre o tamanho do modelo.
d) Eliminar a necessidade de dados.

Questão 2. Na simulação do projeto final, o que o ASR mede?

a) A acurácia global.
b) A taxa de sucesso do backdoor: imagens com gatilho classificadas como a classe alvo.
c) A taxa de aprendizado.
d) O tráfego total.

Questão 3. Por que a acurácia global sozinha é insuficiente no projeto final?

a) Porque não existe.
b) Porque depende do servidor.
c) Porque não revela o backdoor nem a qualidade em cada cliente.
d) Porque é sempre igual a 1.

Questão 4. Na base do projeto (média sem proteção), o ASR foi de cerca de:

a) 1,00 em todas as sementes.
b) 0,10.
c) 0,00.
d) 0,70.

Questão 5. O que a mediana fez com o backdoor no projeto?

a) Impediu o treino.
b) Não alterou o ASR.
c) Reduziu o ASR a praticamente zero, com pequeno custo de acurácia.
d) Aumentou o ASR.

Questão 6. Por que a quantização a 8 e 4 bits foi vantajosa no projeto?

a) Eliminou o ruído.
b) Aumentou o tráfego.
c) Reduziu o tráfego em 4 e 8 vezes sem piorar os resultados.
d) Substituiu o agregador.

Questão 7. Por que a DP (z = 1) não impediu o backdoor no projeto?

a) Corte e ruído limitam a influência de cada cliente, mas não distinguem o malicioso do honesto.
b) Porque a DP exige mais clientes honestos.
c) Porque a DP é sempre inútil.
d) Porque a DP aumenta o ASR.

Questão 8. O ajuste fino local, aplicado após o treino federado, no projeto:

a) eliminou o backdoor.
b) triplicou o tráfego.
c) piorou a média.
d) elevou a média e o percentil 10 por cliente, sem custo de comunicação.

Questão 9. Qual configuração foi recomendada no projeto de referência?

a) A média sem proteção.
b) DP com z = 1 apenas.
c) Nenhuma foi aprovada.
d) Mediana com 4 bits e ajuste fino local.

Questão 10. Qual a fórmula do objetivo global do aprendizado federado?

a) $f(w) = \Sigma F_k(w)^2$.
b) $f(w) = 0$.
c) $f(w) = \max F_k(w)$.
d) $f(w) = \Sigma (n_k/n) \cdot F_k(w)$.

Questão 11. Qual a diferença entre FedSGD e FedAvg?

a) FedAvg não usa servidor.
b) FedSGD dá um passo de gradiente por rodada; FedAvg faz vários passos locais.
c) FedSGD não usa gradientes.
d) Não há diferença.

Questão 12. A deriva dos clientes ocorre porque:

a) o servidor falha.
b) o modelo é linear.
c) com muitos passos locais e dados diferentes, cada cliente se afasta do mínimo global.
d) os dados são IID.

Questão 13. Qual das opções ajuda a combater a heterogeneidade dos dados?

a) SCAFFOLD, com variáveis de controle.
b) Aumentar o ruído.
c) Quantizar para 1 bit.
d) Reduzir o número de clientes a um.

Questão 14. A memória de erro (error feedback) é essencial para:

a) aumentar o lote.
b) eliminar o servidor.
c) esparsificação agressiva, pois guarda o que foi cortado e o envia depois.
d) aumentar o ASR.

Questão 15. No Ditto, o parâmetro λ controla:

a) a coleira que prende o modelo pessoal ao global.
b) a quantização.
c) a taxa de aprendizado do servidor.
d) o número de clientes.

Questão 16. O IFCA é um método de personalização baseado em:

a) split learning.
b) agrupamento de clientes: cada um escolhe o modelo de menor perda.
c) criptografia homomórfica.
d) compressão.

Questão 17. No Capítulo 7, o gradiente de um exemplo na softmax revela a entrada porque:

a) é aleatório.
b) é criptografado.
c) é o produto externo $x \otimes (p - y)$, e o gradiente do viés é $(p - y)$.
d) é sempre zero.

Questão 18. A inferência de pertencimento é mais bem-sucedida quando:

a) há poucos dados, muito treino e sobreajuste.
b) o modelo não treina.
c) o lote é enorme.
d) há muitos dados e regularização.

Questão 19. O que exprime o parâmetro ε na privacidade diferencial?

a) A acurácia.
b) O vazamento: menor ε, mais privacidade.
c) O número de clientes.
d) A taxa de aprendizado.

Questão 20. Qual a função do corte (clipping) no DP-FedAvg?

a) Eliminar o ruído.
b) Limitar a sensibilidade de cada cliente.
c) Aumentar o tráfego.
d) Comprimir o modelo.

Questão 21. A agregação segura por máscaras garante que:

a) o servidor veja só a soma das atualizações, e não cada uma.
b) o modelo final não vaze nada.
c) não haja atacantes.
d) o ruído seja removido.

Questão 22. Qual agregador é frágil a um único cliente malicioso?

a) O Multi-Krum.
b) A média aparada.
c) A média simples.
d) A mediana.

Questão 23. Por que os agregadores robustos pioram com dados não-IID?

a) Clientes honestos diferentes passam a parecer outliers.
b) Porque são mais lentos.
c) Porque exigem mais memória.
d) Porque usam criptografia.

Questão 24. Por que a agregação segura e a detecção de clientes maliciosos entram em tensão?

a) Porque ambas usam ruído.
b) Para detectar atualizações suspeitas, o servidor precisa vê-las, e a agregação segura as esconde.
c) Porque ambas são lentas.
d) Porque não há tensão.

Questão 25. No aprendizado vertical, o que pode vazar o rótulo?

a) O sinal do resíduo (p − y) enviado às partes passivas.
b) A quantização.
c) O corte de norma.
d) A matriz de mistura.

Questão 26. Qual arquitetura divide a rede neural em duas partes, trocando ativações e gradientes na camada de corte?

a) FedAvg.
b) Split learning.
c) Gossip.
d) HFL.

Questão 27. Por que o K-means federado reproduz o centralizado?

a) Porque usa mediana.
b) Porque depende só de somas e contagens por grupo, que se combinam exatamente.
c) Porque usa ruído.
d) Porque ignora clientes pequenos.

Questão 28. Por que ajustar modelos de fundação de forma federada pede métodos como LoRA?

a) Porque LoRA aumenta o modelo.
b) Enviar bilhões de parâmetros por rodada é inviável; só A e B trafegam.
c) Porque elimina o pré-treino.
d) Porque LoRA dá garantia de privacidade.

Questão 29. O aprendizado federado é uma forma de anonimização?

a) Sim, se usar quantização.
b) Sim, apenas em cross-silo.
c) Sim, sempre.
d) Não: atualizações vazam e modelos memorizam, podendo ser tratados como dados pessoais.

Questão 30. Qual das proposições resume bem a lição final do livro?

a) A acurácia global basta.
b) Um bom algoritmo resolve tudo.
c) Privacidade e utilidade nunca conflitam.
d) Todo método tem um preço; decidir bem é medir esse preço e comunicá-lo com honestidade.

Exercícios discursivos

Responda com suas palavras, em poucas linhas. Compare depois com a resposta-modelo do Apêndice A.

1. Explique em suas palavras o que é aprendizado federado, usando uma analogia do cotidiano, e cite dois motivos para adotá-lo.
2. Descreva uma rodada do FedAvg e cite quatro hiperparâmetros.
3. Explique por que dados não-IID prejudicam o treino e cite duas soluções.
4. Compare quantização e esparsificação top-k, indicando o papel da memória de erro.
5. Diferencie modelo global, ajuste fino local, Ditto e IFCA.
6. Explique o ataque de inversão de gradientes na softmax e por que lotes maiores o dificultam.
7. Explique a inferência de pertencimento e como ela se relaciona com o sobreajuste.
8. Explique a privacidade diferencial, o papel de ε e do corte e do ruído no DP-FedAvg.
9. Por que a DP exige muitos clientes para ter ε baixo sem destruir a acurácia?
10. Explique como a agregação segura funciona e suas limitações.
11. Explique por que a média é frágil e cite dois agregadores robustos, com uma limitação.
12. Descreva o backdoor do experimento e por que a acurácia global não o revela.
13. Explique a tensão entre privacidade e robustez em aprendizado federado.
14. Diferencie aprendizado federado horizontal, vertical e por transferência, com exemplos.
15. Explique o vazamento do resíduo no aprendizado vertical e uma defesa.
16. Explique por que K-means e PCA têm versões federadas exatas e dê um aviso de privacidade.
17. Explique o papel do LoRA em modelos de fundação federados e a importância do posto.
18. Descreva três princípios da LGPD relevantes e uma proposta de justiça em aprendizado federado.
19. Redija um parágrafo de recomendação para a Federa Saúde, justificando a configuração escolhida com os números do projeto.
20. Se a Federa Saúde dobrasse o número de clientes maliciosos (de 2 para 4 em 20), quais configurações do projeto você reavaliaria e por quê?

SAIBA MAIS **Para ir além: bases públicas da UCI e do Kaggle**

- **UCI:** *Optical Recognition of Handwritten Digits*, *Letter Recognition* e *Human Activity Recognition Using Smartphones* (cada voluntário como um cliente).
- **Kaggle:** *Digit Recognizer*, *Fashion-MNIST* e *Heart Failure Prediction*, para repetir o estudo com dados diferentes.

Estes conjuntos não foram usados nem testados nos códigos do livro. Confira, nos sites, a descrição, a licença e as colunas antes de usá-los.

Projeto integrador final

Federa Saúde: a decisão final

Contexto. A rede de 20 clínicas da seção 12.1 precisa de uma **recomendação técnica** sobre como treinar o modelo federado. O cenário está descrito na seção 12.2: dígitos manuscritos como substituto de dados clínicos, 20 clientes com Dirichlet $\alpha = 0{,}3$, 70% de treino e 30% de teste local em cada cliente, e **2 clientes maliciosos** (índices 0 e 1) com ataque de *backdoor* (alvo: classe 0).

Tarefas. Use o motor `fedavg_integrado` (arquivo `projeto.py`) e as bibliotecas dos capítulos anteriores.

1. **Diagnóstico:** reporte os tamanhos dos clientes e a distância de variação total média entre as distribuições de rótulos e a global.
2. **Configurações:** avalie, com **3 sementes**, estas seis configurações:

1. base: agregador `media`, sem proteção; 2. `mediana`; 3. `mediana` com 8 bits; 4. `mediana` com 4 bits; 5. DP no nível do cliente (`dp_z = 1`, `S = 0,5`); 6. `mediana` com 4 bits **e ajuste fino local** (3 épocas) após o treino federado.

1. **Métricas:** para cada configuração, acurácia global (teste `Xte`), **ASR** do backdoor, **KB de subida** totais, e acurácia **por cliente** (média e percentil 10) no teste local.
2. **Critério de aprovação:** ASR ≤ 0,10 **e** acurácia global ≥ 0,90 **e** percentil 10 por cliente ≥ 0,84. Entre as aprovadas, escolha a de **menor tráfego** (em caso de empate, a de maior acurácia média por cliente).
3. **Relatório:** redija o parecer de uma página (resumo executivo, cenário e método, tabela de resultados, riscos e limitações, recomendação e próximos passos), em linguagem acessível.

Extensão opcional. Meça a AUC do ataque de pertencimento (Capítulo 7) para as configurações 1 e 5 e discuta o efeito da DP. Repita a análise com o $\alpha = 0{,}1$ e relate se a recomendação muda.

Entregáveis. Código, tabela e o relatório.

A solução de referência está no Apêndice E.

Apêndice A — Gabarito dos exercícios

Esta seção traz a resposta de todas as questões. Nas questões de múltipla escolha, aparecem a alternativa correta e uma breve justificativa. Nas discursivas, aparece uma **resposta-modelo**: a sua resposta pode ser redigida de outro modo, desde que contemple as mesmas ideias.

Capítulo 1 — Bem-vindo ao Aprendizado Federado

Múltipla escolha

Questão	Resposta	Justificativa
1	a)	No federado, o treino acontece onde os dados estão, e só atualizações de modelo trafegam.
2	b)	Os ajustes representam as atualizações de modelo; os cadernos (dados) não saem de casa.
3	b)	Cliente é o participante dono de dados, como um celular ou um hospital.
4	b)	O servidor envia o modelo, recebe atualizações e as agrega.
5	c)	Uma rodada reúne envio, treino local, devolução e agregação.
6	c)	Silos são repositórios isolados que seus donos não podem ou não querem abrir.
7	d)	Reduzir a precisão não é objetivo; os motivos são privacidade, leis, custo e confiança.
8	b)	LGPD significa Lei Geral de Proteção de Dados.
9	a)	Minimização é coletar e usar só o necessário para a finalidade declarada.
10	c)	No centralizado, os dados são juntados em um repositório único.
11	b)	Cross-device envolve muitos clientes pequenos e instáveis.
12	d)	Cross-silo envolve poucas organizações, geralmente confiáveis e sempre disponíveis.
13	c)	Só uma fração disponível, e sorteada, participa de cada rodada.
14	d)	Clientes com mais exemplos têm mais peso na média.
15	d)	F_k mede o erro local do cliente k.
16	b)	McMahan et al. (AISTATS, 2017) propuseram o FedAvg.
17	b)	FedAvg é Federated Averaging, a média federada de modelos.
18	b)	O federado respeita a localização dos dados, que em geral não são parecidos entre clientes.
19	b)	Não-IID é a heterogeneidade das distribuições entre clientes.
20	d)	Ataques podem inferir informações a partir das atualizações; são necessárias defesas.
21	d)	A privacidade diferencial adiciona ruído calibrado.
22	c)	Na agregação segura, o servidor só obtém a soma das atualizações.
23	a)	Com dados não-IID e problemas mais difíceis, a média pode ser pior.
24	c)	O capítulo usa o conjunto de câncer de mama, que vem no scikit-learn.
25	d)	Dayan et al. (2021) mostraram esse estudo multicêntrico.
26	d)	Atualização é a diferença de parâmetros resultante do treino local.
27	b)	Agregação combina contribuições, por exemplo, por média ponderada.
28	b)	A média e o desvio globais exigiriam acesso a todos os dados; será resolvido de forma federada no Capítulo 2.
29	b)	Confiança e concorrência impedem o compartilhamento direto de dados.
30	b)	Evita transportar grandes volumes de dados brutos até um servidor distante.

Discursivas (respostas-modelo)

1. É treinar um modelo com a colaboração de vários donos de dados sem que os dados brutos saiam de cada um. Analogia: as amigas com cadernos de receitas secretos que só trocam ajustes da receita-base, nunca os cadernos; o modelo vai até os dados.

2. 1) O servidor envia o modelo atual aos clientes selecionados. 2) Cada cliente treina o modelo com seus dados locais. 3) Cada cliente devolve a atualização (não os dados). 4) O servidor agrega as atualizações em um novo modelo global e inicia nova rodada.
3. Privacidade e risco de vazamento; leis e regulamentos de proteção de dados; custo e lentidão de mover grandes volumes; falta de confiança entre organizações, sobretudo concorrentes.
4. Cross-device: milhões de clientes pequenos (celulares), instáveis, com poucos dados cada, e só uma fração participa por rodada. Cross-silo: poucos clientes grandes (hospitais, bancos), quase sempre disponíveis e com muitos dados.
5. O erro global do modelo w é a média ponderada dos erros locais F_k(w), onde cada cliente pesa proporcionalmente ao seu número de exemplos n_k. Assim, o objetivo equivale a minimizar o erro sobre todos os dados, mesmo sem reuni-los.
6. No data center os dados são do treinador, ficam em máquinas controladas e são embaralhados para ficarem parecidos; o objetivo é velocidade. No federado, os dados ficam com seus donos, tendem a ser diferentes entre clientes, a comunicação é lenta e os participantes variam; o objetivo é respeitar a localização e a privacidade.
7. Significa que os clientes têm distribuições diferentes. Exemplo: um hospital de referência em câncer vê muito mais casos malignos do que uma clínica de check-up, e cada pessoa escreve de modo distinto no celular.
8. Porque as atualizações enviadas podem vazar informações sobre os dados: há ataques que reconstroem exemplos ou inferem participação no treino. Para garantias é preciso combinar com técnicas como privacidade diferencial e agregação segura.
9. O federado evita copiar dados brutos para um local central, coletando e circulando apenas o necessário (atualizações de modelo). Isso se alinha à minimização, mas não dispensa a análise jurídica nem outras medidas de segurança.
10. Três hospitais com 50 pacientes cada treinaram regressão logística sozinhos e tiveram acurácia um pouco menor. A média dos três modelos, sem troca de dados, igualou o modelo centralizado. O resultado é otimista (dados IID, problema simples, uma só rodada).
11. Teclado móvel: o texto digitado fica no celular, que só envia atualizações do modelo. Saúde: prontuários e imagens ficam no hospital, e só parâmetros do modelo trafegam entre instituições.
12. Porque o objetivo global é o erro médio sobre todos os exemplos; um cliente com mais exemplos contribui com mais termos nessa média, e seu modelo local é estimado com mais evidência.
13. Ele calculou média e desvio padrão usando os dados de treino de todos os hospitais juntos, o que exigiria centralizá-los. Em um sistema real, essas estatísticas precisariam ser calculadas de forma federada, combinando somas locais.
14. Vantagens: não move dados brutos, respeita leis e silos, economiza banda e permite colaboração. Limitações: comunicação cara, dados não-IID, dispositivos instáveis, maior complexidade e riscos de privacidade e de participantes maliciosos.
15. É um servidor que segue o protocolo, mas tenta aprender o que puder das atualizações recebidas. Importa porque mostra que enviar só atualizações pode não bastar; defesas como agregação segura e ruído protegem contra essa curiosidade.
16. Em vez de copiar dados para um local central, o modelo é enviado aos locais onde estão os dados e treinado ali. Vantagem: dados sensíveis não saem do dono, e se evita transportar grandes volumes.
17. Conexões de clientes como celulares são lentas e instáveis, e modelos grandes exigem muitos bytes por rodada; além disso, são necessárias muitas rodadas e o envio de subida costuma ser mais lento que o de descida.
18. Resposta pessoal. Exemplo: clínicas de uma rede que querem um modelo de triagem; cada clínica é cliente, uma entidade central é o servidor e os prontuários permanecem em cada clínica.
19. Porque, com dados IID, cada cliente enxerga a mesma distribuição e os modelos locais apontam para direções parecidas; a média resulta em um bom modelo. Com dados não-IID, as direções divergem e a média pode piorar.

20. Resposta pessoal, coerente com o resumo do capítulo: definição de aprendizado federado; ciclo de quatro passos; motivos (privacidade, leis, custo, confiança); cross-device versus cross-silo; e a ressalva de que federado não garante privacidade sozinho.

Capítulo 2 — Preparando o Terreno

Múltipla escolha

Questão	Resposta	Justificativa
1	d)	Há X (atributos) e y (rótulos) para cada exemplo.
2	d)	Atributos são colunas; exemplos são linhas.
3	c)	Parâmetros são aprendidos; hiperparâmetros são escolhidos por nós.
4	d)	A perda quantifica o quanto a previsão está errada.
5	a)	$-\log(0{,}9) \approx 0{,}105$: confiança no acerto gera perda pequena.
6	b)	$-\log(0{,}1) \approx 2{,}30$: pouca confiança no acerto gera perda grande.
7	d)	$F(w) = (1/n)\ \Sigma$ perda$(w; x_i, y_i)$.
8	b)	O gradiente indica a subida mais íngreme; andamos no sentido contrário.
9	d)	η controla o tamanho do passo na direção oposta ao gradiente.
10	c)	Passos muito grandes ultrapassam o vale e podem se afastar dele.
11	b)	Passos minúsculos exigem muitos passos para chegar ao vale.
12	c)	O gradiente é estimado em mini-lotes, o que é barato e, na média, correto.
13	d)	1.000 / 50 = 20 passos por época.
14	c)	Época é uma passada completa; passo é uma atualização.
15	d)	O modelo decora o treino e não generaliza.
16	c)	A validação orienta escolhas sem usar o teste.
17	c)	O teste deve simular dados novos; usá-lo para escolhas o contamina.
18	a)	O termo $\lambda\|w\|^2$ desencoraja pesos muito grandes.
19	b)	Alta acurácia, mas revocação zero para a classe de interesse.
20	a)	Revocação = VP / (VP + FN).
21	b)	Precisão = VP / (VP + FP).
22	a)	F1 só é alto se precisão e revocação forem ambas altas.
23	d)	Convexidade facilita a otimização e a análise.
24	c)	$64 \times 10 + 10 = 650$.
25	a)	A agregação opera sobre os parâmetros, que são listas de números.
26	d)	partial_fit retoma o treino de onde parou, útil para treino local em rodadas.
27	a)	O treino é retomado do estado anterior em vez de reiniciado.
28	c)	Com n, Σx e Σx^2 o servidor obtém média e variância globais.
29	a)	$\sigma^2 = E[x^2] - (E[x])^2$, obtida a partir das somas.
30	c)	Só resumos (contagens e somas) são enviados.

Discursivas (respostas-modelo)

1. Você só enxerga o chão sob seus pés; sente para onde o terreno desce mais e dá um passo nessa direção, repetindo. O gradiente aponta a subida mais íngreme; andamos no sentido oposto para reduzir o erro, com passo de tamanho η.
2. A perda dá uma nota ao erro de uma previsão em um exemplo. O risco empírico é a média dessas perdas sobre os exemplos de treino. Treinar é minimizar o risco empírico.

3. Com 0,5: −log(0,5) ≈ 0,69. Com 0,99: −log(0,99) ≈ 0,01. Quanto mais confiante e correto o modelo, menor a perda; indecisão gera perda intermediária.
4. Define o tamanho do passo. Pequena demais: convergência lentíssima. Grande demais: o passo ultrapassa o vale, a perda oscila ou diverge.
5. O gradiente completo usa todos os exemplos a cada passo: caminho suave, mas caro. O SGD usa mini-lotes sorteados: gradiente ruidoso, porém barato, e na média correto. Vantagem: muito mais passos por unidade de tempo.
6. Época é uma passada completa pelos dados; passo é uma atualização com um mini-lote. 2.400/32 = 75 passos por época.
7. É quando o modelo decora o treino e generaliza mal. Detecta-se quando o erro de treino é muito menor que o de validação, e a validação para de melhorar ou piora.
8. Treino ajusta os parâmetros; validação escolhe hiperparâmetros; teste dá a nota final, uma única vez. A separação evita resultados otimistas por vazamento.
9. Soma-se à perda um termo $\lambda\|w\|^2$ que penaliza pesos grandes, produzindo modelos mais simples e menos sobreajustados. λ controla a força: λ grande restringe demais, λ pequeno quase não regulariza.
10. Em classes desbalanceadas, prever sempre a classe majoritária gera acurácia alta sem utilidade (ex.: 99% de transações legítimas). Use revocação, precisão e F1 da classe de interesse.
11. Precisão: das transações marcadas como fraude, quantas eram fraude. Revocação: das fraudes existentes, quantas foram detectadas. O F1 combina ambas.
12. VP = 8, FN = 2, FP = 4, VN = 86. Precisão = 8/12 ≈ 0,67. Revocação = 8/10 = 0,80. F1 = 2·0,67·0,80/(0,67+0,80) ≈ 0,73.
13. Porque os parâmetros formam listas numéricas; o servidor pode enviá-las, os clientes devolver versões atualizadas e o servidor combinar por média ou soma, sem tocar nos dados.
14. A softmax para dígitos tem 650 parâmetros e otimização convexa. Uma MLP com 32 neurônios tem 2.410, é mais flexível, mas não convexa, o que torna a otimização mais delicada.
15. Em cada rodada o cliente recebe os parâmetros globais e continua o treino a partir deles. O scikit-learn oferece partial_fit e warm_start para retomar o treino.
16. Cada cliente envia n_k e a soma local $S1_k$. O servidor calcula $\mu = \Sigma S1_k / \Sigma n_k$. Nenhum valor individual é enviado.
17. Cada cliente envia também a soma dos quadrados $S2_k$. O servidor calcula $\sigma^2 = \Sigma S2_k/n - \mu^2$ e $\sigma = \sqrt{\sigma^2}$. Resultado idêntico ao de olhar todos os dados.
18. Em grupos muito pequenos, médias e somas podem revelar valores individuais; um servidor curioso pode inferir informações. Por isso, mais adiante, usam-se agregação segura e privacidade diferencial.
19. Cobrar pênaltis: chuta, vê onde a bola foi, corrige a mira. No treino, o modelo prevê, mede a perda e ajusta os parâmetros na direção que reduz o erro.
20. Resposta pessoal coerente com o resumo: perda e risco empírico; gradiente descendente e SGD; treino/validação/teste e F1; modelo como vetor de números; partial_fit e estatísticas por somas locais.

Capítulo 3 — Anatomia de um Sistema Federado

Múltipla escolha

Questão	Resposta	Justificativa
1	c)	O FedAvg combina treino local em mini-lotes com média ponderada dos modelos no servidor.
2	a)	A rodada começa com seleção de clientes e envio do modelo atual.
3	d)	Todos devem partir do mesmo modelo global da rodada.
4	b)	Devolve-se o modelo local atualizado ou a atualização (delta).
5	d)	$w_{t+1} = \Sigma (n_k / n_S) \cdot w_k$.
6	c)	$0{,}1 \cdot 0{,}80 + 0{,}3 \cdot 0{,}60 + 0{,}6 \cdot 0{,}50 = 0{,}56$.
7	c)	$(0{,}80 + 0{,}60 + 0{,}50)/3 \approx 0{,}63$.
8	c)	$(50 \cdot 1{,}0 + 150 \cdot 0{,}0)/200 = 0{,}25$.
9	c)	Um passo de gradiente completo por rodada é o FedSGD.
10	a)	O custo de comunicação é alto: um passo por rodada.
11	d)	Troca comunicação por computação local.
12	b)	C controla a participação por rodada.
13	c)	$0{,}1 \times 100 = 10$ clientes.
14	d)	$0{,}3 \times 50 = 15$.
15	c)	E é quanto cada cliente treina antes de devolver o modelo.
16	a)	B é o tamanho do mini-lote do SGD local.
17	a)	$u = E \cdot n_k / B = 5 \cdot 200 / 20 = 50$.
18	a)	$u = 2 \cdot 600 / 30 = 40$.
19	a)	Com muitos passos locais e dados diferentes, os modelos divergem do ótimo global.
20	c)	McMahan et al. observaram que a média só funciona bem com inicialização compartilhada.
21	d)	A média ficou ao redor de 0,89, enquanto cada cliente tinha cerca de 0,96.
22	b)	É uma biblioteca didática de simulação.
23	d)	O modelo simples foca o protocolo e não o modelo.
24	d)	$65 \times 10 = 650$.
25	d)	A cópia evita que um cliente modifique o modelo compartilhado.
26	c)	A mesma semente gera os mesmos sorteios e resultados.
27	a)	$2 \cdot 4 \cdot 1.000 \cdot 8 = 64.000$ bytes.
28	a)	Reddi et al. (2021) propuseram FedAdam, FedYogi e FedAdagrad.
29	b)	Isso mantém os clientes na mesma vizinhança do espaço de parâmetros.
30	d)	O número de rodadas é o que custa comunicação.

Discursivas (respostas-modelo)

1. 1) O servidor sorteia S_t e envia w_t. 2) Cada cliente k treina localmente por E épocas, com lote B e passo η, obtendo w_{t+1}^k. 3) Cada cliente devolve w_{t+1}^k. 4) O servidor agrega: $w_{t+1} = \Sigma (n_k/n_S)\, w_{t+1}^k$.
2. Porque o objetivo global é a média dos erros sobre todos os exemplos; clientes com mais exemplos devem pesar mais. A média simples superestimaria clientes pequenos e enviesaria o modelo.
3. $n_S = 100$. $w = 0{,}2 \cdot 2{,}0 + 0{,}3 \cdot 4{,}0 + 0{,}5 \cdot 1{,}0 = 0{,}4 + 1{,}2 + 0{,}5 = 2{,}1$.

4. No FedSGD, cada cliente dá um passo de gradiente por rodada; no FedAvg, vários passos locais (E épocas, lote B). Como cada rodada aproveita muito mais cálculo local, o FedAvg precisa de bem menos rodadas.
5. Porque a rede costuma ser lenta, instável e cara, enquanto o dispositivo tem capacidade de sobra. Fazer mais trabalho localmente reduz o número de rodadas e o tráfego.
6. C: fração de clientes por rodada; E: épocas locais; B: tamanho do lote; η: passo. Aumentar E acelera o progresso por rodada, mas, com dados diferentes, aumenta a deriva dos clientes.
7. u = E · n_k / B = 4 · 450 / 30 = 60 passos.
8. É o afastamento do modelo local em direção ao mínimo do próprio cliente. Quanto mais diferentes os dados, mais diferentes os mínimos locais, e a média dos modelos fica longe do mínimo global.
9. Redes podem aprender os mesmos conceitos em neurônios permutados; a média mistura pesos que não correspondem e destrói a função aprendida. Com mesma inicialização, as redes permanecem alinhadas.
10. Duas redes pequenas treinaram em metades dos dígitos. Com inicializações diferentes, a média ficou em torno de 0,89, abaixo dos clientes (cerca de 0,96); com mesma inicialização, ficou em torno de 0,97. Ensina a enviar o mesmo modelo a todos.
11. 2 · 4 · 500.000 · 20 = 80.000.000 bytes, ou cerca de 80 MB por rodada (subida e descida somadas).
12. Cada parâmetro ocuparia 2 bytes; o tráfego cairia pela metade, para cerca de 40 MB por rodada.
13. Controla os sorteios (partição dos dados, seleção de clientes, ordem dos mini-lotes). Com a mesma semente, resultados são reproduzíveis; variando a semente, avalia-se a variabilidade.
14. Para focar no protocolo, ter modelo pequeno (650 parâmetros), rápido e convexo. Limitações: menos expressividade e, por ser convexo, efeitos como a falha da média com inicializações diferentes não aparecem.
15. Porque a rodada envolve comunicação, que é o principal custo; dois métodos com o mesmo número de épocas totais podem exigir números muito diferentes de rodadas.
16. Cada rodada usa uma amostra menor, o que pode atrasar um pouco o início do treino; como qualquer subconjunto representa bem o todo, ao final a diferença costuma ser pequena.
17. f é o erro médio sobre todos os exemplos. O FedAvg aproxima a minimização fazendo cada cliente reduzir seu F_k localmente e combinando os modelos com pesos n_k/n.
18. Na simulação tudo roda em uma máquina, sem rede, falhas, autenticação ou criptografia; no real há latência, clientes que somem, segurança e heterogeneidade de hardware.
19. Aumentar E (menos rodadas), reduzir o tamanho do modelo, quantizar ou esparsificar as atualizações, e sortear menos clientes por rodada. O Capítulo 5 aprofunda o tema.
20. Resposta pessoal coerente com o resumo: sortear clientes, enviar o mesmo modelo, treino local com E, B e η, devolução, média ponderada por n_k. Limitações: deriva dos clientes com dados diferentes, custo de comunicação e ausência de garantias de privacidade.

Capítulo 4 — Quando os Dados Não São Iguais

Múltipla escolha

Questão	Resposta	Justificativa
1	d)	IID: independentes e identicamente distribuídos entre clientes.
2	b)	P_k(x, y) difere entre clientes.
3	b)	Receitas locais divergentes, quando promediadas, podem não servir a nenhum local.
4	c)	O que muda é P(y), a distribuição das classes.
5	d)	P(x) muda: a aparência dos dados difere entre clientes.
6	d)	Clientes têm volumes de dados muito diferentes.
7	c)	Para um cliente "bom" e para outro "regular" o mesmo dado.
8	b)	P(y) é a distribuição marginal dos rótulos.
9	a)	Como os dados foram ordenados por classe, cada pedaço contém praticamente uma classe.
10	b)	α grande gera proporções próximas de iguais.
11	c)	α pequeno gera repartições concentradas.
12	c)	Com α = 100, os clientes têm todas as classes em proporções parecidas (≈ 1/10).
13	b)	Com α = 0,1, mais da metade dos exemplos de um cliente é, em média, da classe mais comum.
14	a)	Cada cliente caminha para seu próprio mínimo, afastando-se do global.
15	b)	Mais passos e mais heterogeneidade ampliam a divergência.
16	a)	Com passos locais diferentes e dados heterogêneos, o ponto de convergência muda.
17	c)	Cada cliente vai ao seu mínimo; a média desses mínimos não é o mínimo global.
18	a)	Curvas mais lentas e oscilantes, e clientes grandes dominando a média.
19	a)	Em regressão logística o impacto é mais visível na velocidade e na estabilidade.
20	d)	0 indica distribuições idênticas; 1, totalmente diferentes.
21	b)	A "coleira" mantém o modelo local perto do global.
22	d)	Sem penalização, o treino local é o de sempre.
23	c)	A coleira muito curta limita o aprendizado local.
24	a)	Corrige cada passo local por g(w) − c_k + c.
25	b)	Variáveis de controle também trafegam e precisam ser guardadas.
26	d)	Corrige a inconsistência do objetivo causada por números diferentes de passos.
27	a)	O SCAFFOLD chegou a cerca de 0,96, contra 0,93 do FedAvg.
28	b)	As curvas ficaram quase sobrepostas; modelo simples e convexo se beneficia pouco da coleira.
29	b)	Uma única semente pode enganar; médias e desvios dão evidência mais sólida.
30	b)	FedAdam, FedYogi e FedAdagrad são otimizadores adaptativos no servidor (Reddi et al., 2021).

Discursivas (respostas-modelo)

1. IID: todos os exemplos vêm da mesma fonte, como bolinhas sorteadas de uma única urna. Não-IID: cada cliente tem a sua urna, como pizzarias com públicos diferentes. Uma receita média pode não servir a ninguém.
2. Rótulos: hospital oncológico com mais casos de câncer. Atributos: câmeras ou caligrafias diferentes. Quantidade: usuários muito ativos versus pouco ativos. Conceito: o mesmo dado recebe rótulos diferentes por cliente.

3. Para cada classe, sorteia-se uma proporção de repartição entre os clientes. α grande dá repartições quase uniformes (quase IID); α pequeno concentra cada classe em poucos clientes (muito não-IID).
4. Ordenam-se os exemplos por classe, cortam-se em pedaços (shards) e entregam-se poucos pedaços a cada cliente. Com um pedaço por cliente, cada um vê uma única classe.
5. É o afastamento dos modelos locais em direção a seus mínimos locais. Ao tirar a média, o modelo global pode ficar longe do mínimo global, convergindo mais lentamente ou para um ponto pior.
6. Se cada cliente treinar até convergir, a média dos modelos é a média dos mínimos locais, que não é o mínimo da função global. O algoritmo, portanto, estaciona em um ponto que não minimiza o objetivo desejado.
7. Convergência mais lenta; maior oscilação da acurácia entre rodadas; viés em favor de clientes grandes; maior dificuldade de ajustar hiperparâmetros.
8. TV = ½ (|0,7−0,4| + |0,2−0,3| + |0,1−0,3|) = ½ (0,3 + 0,1 + 0,2) = 0,3.
9. p_k = (1, 0, 0). TV = ½ (|1−0,5| + |0−0,3| + |0−0,2|) = ½ (0,5 + 0,3 + 0,2) = 0,5.
10. Adiciona ao erro local um termo $(\mu/2)\|w - w_t\|^2$ que penaliza o afastamento do modelo global, reduzindo a deriva. μ controla a força: μ = 0 equivale ao FedAvg; μ grande quase impede o aprendizado local.
11. Mantém c no servidor e c_k em cada cliente, estimando a diferença entre direções locais e global. O passo local usa $g(w) - c_k + c$, corrigindo a deriva. O custo é o dobro da comunicação (envio dos controles) e estado extra nos clientes.
12. Corrige a inconsistência do objetivo causada por clientes com números diferentes de passos locais, normalizando cada atualização pelo número de passos antes de agregar.
13. Em problemas convexos a deriva é mais branda; a coleira pouco altera a trajetória, e μ grande apenas reduz o progresso. Os ganhos reportados costumam ocorrer em cenários mais heterogêneos em dispositivos e redes neurais.
14. Com 10% de participação, E = 20 e η = 0,3, o SCAFFOLD foi mais rápido e estável (≈ 0,96 contra ≈ 0,93). O FedProx (μ = 0,1) pouco mudou o resultado. Nenhum algoritmo é melhor em tudo; é preciso testar.
15. A agregação pondera por n_k; clientes grandes têm mais peso. Clientes pequenos ou diferentes ficam mal atendidos pelo modelo global, o que pede personalização ou métodos de justiça.
16. Comparando distribuições de classes por cliente (por exemplo, com TV), tamanhos n_k e estatísticas dos atributos; treinando modelos locais e avaliando-os nos dados dos outros clientes.
17. Em rótulos, as classes têm frequências diferentes, mas a relação entre x e y é a mesma. Em conceito, a própria relação P(y|x) muda: o mesmo x tem rótulos diferentes para clientes diferentes.
18. Mais passos locais aumentam a distância percorrida em direção ao mínimo local de cada cliente; com dados heterogêneos, isso amplifica a deriva e pode piorar a média.
19. Há muitas fontes de aleatoriedade (particionamento, sorteio de clientes, ordem dos lotes). Uma única execução pode superestimar ou subestimar um método; média e desvio mostram a variabilidade.
20. Resposta pessoal coerente com o resumo: tipos de não-IID, simulação por shards e Dirichlet, deriva dos clientes, FedProx/SCAFFOLD/FedNova, e a lição de que cada método depende do problema.

Capítulo 5 — Falando Menos

Múltipla escolha

Questão	Resposta	Justificativa
1	a)	A transmissão de atualizações de modelos grandes por redes lentas e instáveis limita o treino.
2	b)	A assimetria torna a subida da atualização o trecho mais caro.
3	d)	Cada rodada soma treino local, subida e descida.
4	c)	20 × 8 = 160 Mbit; 160 / 5 = 32 s.
5	a)	160 Mbit / 20 Mbit/s = 8 s.
6	a)	30 + 8 + 32 = 70 s.
7	a)	500 × 70 s = 35.000 s ≈ 9,7 h.
8	c)	30 + 8 + 4 = 42 s.
9	c)	Os valores são aproximados por um número pequeno de níveis.
10	c)	2 bits dão 4 níveis; 4 bits, 16 níveis.
11	a)	Em média o erro se cancela, evitando acúmulo de viés.
12	c)	A probabilidade de subir é a parte fracionária (0,3).
13	d)	650 × 4 / 8 = 325 bytes (333 com o cabeçalho de mínimo e máximo).
14	b)	650 × 4 bytes = 2.600 bytes.
15	d)	Mantém os valores mais importantes e descarta os demais.
16	d)	Sem os índices, os valores não teriam posição.
17	d)	Por valor enviado: 4 + 2 = 6 bytes.
18	a)	10% de 650 = 65.
19	b)	O viés sistemático impede o aprendizado dessas componentes.
20	c)	O resíduo da compressão é adiado, não perdido.
21	c)	Top-10% + EF chegou a 0,959, igual ao float32, com 114 KB contra 762 KB.
22	a)	Descartar quase tudo, sem compensação, prejudicou o aprendizado.
23	b)	A memória de erro permite que os valores adiados sejam enviados depois.
24	d)	A 1 bit, a acurácia caiu para cerca de 0,93.
25	b)	762 KB contra 50 KB.
26	a)	Enviar o sinal reduz cada valor a 1 bit.
27	d)	Com participação rara, o resíduo guardado perde relevância.
28	a)	Um método só é bom se mantiver a acurácia com custo de comunicação menor.
29	d)	Mais rodadas, com o mesmo tamanho, aumentam o tráfego.
30	c)	8 bits ocupam 1/4 do espaço de 32 bits; no experimento a acurácia caiu menos de 0,5 ponto.

Discursivas (respostas-modelo)

1. Porque a conexão costuma ter upload mais lento que download e o cliente precisa enviar toda a atualização. Exemplo: 20 MB a 5 Mbit/s levam 32 s para subir, contra 8 s para descer a 20 Mbit/s.
2. $T \approx R \cdot (t_{comp} + t_{sub} + t_{desc})$. Se a subida cai 8 vezes, t_{sub} diminui e T diminui proporcionalmente à parcela da subida. Ex.: 70 s por rodada vira 42 s.

3. Divide-se o intervalo [mínimo, máximo] em 2^b − 1 degraus e cada valor é aproximado por um degrau. Menos bits significam menos níveis e menos bytes, mas aproximação mais grosseira.
4. Arredonda-se para cima com probabilidade igual à parte fracionária, de modo que a esperança do valor quantizado é igual ao original. Os erros são aleatórios e se cancelam ao agregar muitos clientes.
5. 32 bits: 4.000 bytes; 8 bits: 1.000 bytes; 2 bits: 250 bytes.
6. Envia-se apenas os k maiores valores em módulo. Como o servidor precisa saber a posição, cada valor vem acompanhado do índice, que ocupa bytes adicionais (por exemplo, 2 bytes por índice).
7. k = 500; custo = 500 × 6 = 3.000 bytes, contra 40.000 bytes sem compressão (cerca de 13 vezes menos).
8. Cada cliente guarda o resíduo do que não enviou e o soma à atualização seguinte. Sem ela, valores pequenos nunca seriam enviados e o modelo deixaria de aprender; com ela, são apenas adiados.
9. Sem memória de erro, a acurácia caiu a cerca de 0,87; com ela, subiu a cerca de 0,94, usando os mesmos bytes. A memória de erro recupera grande parte do que a esparsificação perde.
10. 2 bits usou cerca de 50 KB e atingiu 0,955; top-10% + EF usou cerca de 114 KB e atingiu 0,959. Ambos mantêm a acurácia, e 2 bits é mais econômico, neste caso.
11. Para que o servidor reconstrua os valores: ele precisa saber o intervalo que os níveis representam. São apenas 8 bytes por atualização.
12. Os clientes passam a treinar a partir de um modelo aproximado, o que pode introduzir erro sistemático e prejudicar a convergência; é preciso cuidado e, às vezes, compensações.
13. Envia apenas o sinal de cada valor da atualização (1 bit). Limitações: perde a magnitude, pode ser enviesado e costuma exigir correções, como memória de erro.
14. Porque cada rodada passa a fazer mais progresso, reduzindo o número de rodadas necessárias e, portanto, o tráfego total. Há, porém, o risco de deriva dos clientes com dados não-IID.
15. Redes móveis são lentas e instáveis, com restrições de bateria e participação intermitente; em cross-silo, os clientes têm enlaces rápidos e estáveis.
16. Enviar menos informação pode dificultar alguns ataques, mas não oferece garantia formal. Para garantias, é preciso privacidade diferencial e agregação segura.
17. Sem perdas (como zip) recupera exatamente o original, mas comprime pouco dados numéricos densos. Com perdas (quantização, top-k) aceita erro em troca de grande economia, e é a usada nas atualizações.
18. Estimaria o tamanho do modelo e testaria quantização com poucos bits e top-k com memória de erro, comparando acurácia por bytes em validação; escolheria o que mantém a acurácia dentro do orçamento.
19. Modelo pequeno e convexo, dados dos dígitos (simples), 20 clientes simulados, poucas sementes e contagem simplificada de bytes (sem protocolo de rede). Resultados podem mudar com redes profundas e dados reais.
20. Resposta pessoal coerente com o resumo: quantização (poucos bits, arredondamento estocástico), top-k com memória de erro (compressão forte), mais épocas locais (menos rodadas), modelos menores/adaptadores e comparação por acurácia × bytes.

Capítulo 6 — Cada Um do Seu Jeito

Múltipla escolha

Questão	Resposta	Justificativa
1	c)	Um modelo único é bom na média, mas pode ser ruim para clientes específicos.
2	a)	A personalização seria o alfaiate, que ajusta ao corpo de cada um.
3	d)	A média não revela o pior cliente nem a cauda inferior.
4	a)	90% dos clientes têm acurácia pelo menos igual a esse valor.
5	d)	O modelo será usado nos dados locais; o teste deve refleti-los.
6	c)	Se P(y|x) muda entre grupos, nenhum modelo único acerta para todos.
7	d)	O cliente adapta o modelo recebido aos seus dados.
8	a)	A adaptação acontece localmente após receber o modelo.
9	d)	Passos demais afastam o modelo do global e podem decorar poucos dados.
10	d)	$F_k(v_k) + (\lambda/2)\|v_k - w\|^2$.
11	d)	Sem penalização, não há ligação com o global.
12	d)	A coleira forte prende o modelo pessoal ao global.
13	a)	Ambos usam o termo proximal, mas com propósitos diferentes.
14	a)	O corpo é compartilhado; a cabeça é pessoal.
15	a)	Só o corpo trafega.
16	c)	O agrupamento emerge da escolha do modelo de menor perda.
17	c)	K, o número de modelos, é um hiperparâmetro.
18	c)	Um único modelo não serve a três permutações diferentes de rótulos.
19	d)	Recuperou os três grupos e treinou dentro de cada um.
20	d)	O global misturava conceitos incompatíveis; pouco treino local não desfaz isso.
21	d)	Clientes com poucos dados não aprendem sozinhos; colaborar os protege.
22	c)	A média subiu cerca de 2 pontos percentuais.
23	a)	É uma abordagem de meta-aprendizado: aprender a se adaptar rápido.
24	d)	Sem dados locais não há como personalizar.
25	c)	Cada cliente precisa armazenar um modelo próprio.
26	a)	Modelos ajustados a poucos dados tendem a memorizá-los mais.
27	b)	MOCHA usa aprendizado multitarefa federado.
28	d)	Um modelo por grupo captura as diferenças entre grupos.
29	a)	Escolher entre K modelos exige poucos dados; o treino pesado já foi feito pelo grupo.
30	a)	Se o global mistura conceitos incompatíveis, poucas épocas não corrigem.

Discursivas (respostas-modelo)

1. A camisa de prateleira serve bem à maioria, mas mal a quem tem medidas fora do padrão; da mesma forma, o modelo global é bom na média, mas pode ser ruim para clientes com dados diferentes. A personalização ajusta ao cliente.
2. Média: desempenho geral. Pior cliente: quem é menos bem servido. Percentil 10: valor que 90% dos clientes atingem ou superam, mostrando a cauda inferior sem depender de um único extremo.

3. Mudança de rótulos: as frequências das classes variam, mas P(y|x) é igual. Mudança de conceito: o mesmo x recebe rótulos diferentes entre grupos. Na mudança de conceito, nenhum modelo único acerta para todos, e o global falha mais.
4. O cliente continua o treino do modelo global com dados locais. Vantagem: simples e sem custo de comunicação. Risco: passos demais afastam o modelo do global e podem sobreajustar poucos dados.
5. λ controla a coleira que prende o modelo pessoal ao global. λ = 0: modelo só local. λ muito grande: igual ao global. Valores intermediários equilibram personalização e colaboração.
6. No FedProx, o termo proximal estabiliza o treino do modelo global. No Ditto, serve para treinar um modelo pessoal próximo ao global, que é treinado normalmente em paralelo.
7. As camadas iniciais (corpo) aprendem características gerais e são treinadas em conjunto; a camada final (cabeça) é pessoal e não é enviada. Todos compartilham conhecimento geral, e há economia de comunicação.
8. 1) O servidor envia K modelos. 2) Cada cliente escolhe o de menor perda nos seus dados. 3) Treina esse modelo e devolve a atualização. 4) O servidor faz a média dentro de cada grupo. Repete.
9. Exige escolher K, é sensível à inicialização, pode juntar grupos diferentes no mesmo modelo, e custa mais comunicação por enviar K modelos.
10. Global (≈ 0,38) falha por misturar conceitos incompatíveis. Só local (≈ 0,87) aprende cada conceito, mas só com dados próprios. Ajuste fino com 5 épocas (≈ 0,79) não desfaz o global. IFCA (≈ 0,93) recupera os grupos e compartilha dentro deles.
11. O modelo só local teve média razoável, mas um pior cliente de cerca de 0,17; os métodos que partem do global protegem quem tem pouco dado, com pior cliente em torno de 0,82 a 0,84.
12. Porque clientes com poucos exemplos não conseguem aprender sozinhos; a média esconde casos extremos, e a avaliação por cliente revela o pior cliente.
13. Busca um modelo global que se adapte bem a cada cliente com poucos passos de ajuste (meta-aprendizado). É mais custoso porque a formulação envolve derivadas de segunda ordem ou aproximações.
14. Um cliente novo não tem dados para personalizar. Mitiga-se começando com o modelo global, atribuindo-o a um grupo semelhante ou usando poucos passos de ajuste conforme chegam dados.
15. Memória extra para modelos pessoais; maior risco de memorização de dados do cliente; comunicação extra em métodos com vários modelos (como IFCA).
16. Quando há grupos distintos de clientes com conceitos ou comportamentos diferentes, de modo que o modelo global é ruim como ponto de partida, como no experimento B.
17. Usando um teste local com dados do próprio cliente, medindo o desempenho antes e depois da personalização, e comparando-o com o global e com o modelo só local.
18. Porque as partições, os tamanhos e os sorteios variam; um cliente pequeno pode distorcer resultados de uma única execução. Várias sementes dão média e variabilidade.
19. Resposta pessoal coerente com o resumo: modelo global, medidas por cliente, ajuste fino, Ditto, partição, IFCA e Per-FedAvg; personalizar é indispensável em mudança de conceito.
20. Sem a penalização, o modelo pessoal minimiza apenas o erro local a partir do global, o que é exatamente o ajuste fino. A coleira só diferencia os métodos quando λ > 0.

Capítulo 7 — O Que Pode Vazar

Múltipla escolha

Questão	Resposta	Justificativa
1	c)	Mesmo enviando só ajustes, é possível deduzir características dos dados.
2	c)	Define adversário, conhecimento e capacidades.
3	d)	Ele cumpre o protocolo, mas analisa o que recebe.
4	c)	Pode desviar-se do protocolo para extrair mais informação.
5	a)	Cada um desses itens é uma fonte potencial de vazamento.
6	a)	O atacante recupera as entradas a partir do gradiente observado.
7	b)	Zhu, Liu e Han (NeurIPS 2019), "Deep leakage from gradients".
8	a)	Cada coluna é x multiplicado por (p_c − y_c).
9	c)	O viés atua como atributo constante 1, e seu gradiente é o próprio erro.
10	c)	x = ∇W[:, c] / ∇b[c].
11	c)	A reconstrução foi exata, a menos de erros numéricos.
12	b)	A correlação cai de 1,0 (lote 1) para cerca de 0,82 (lote 32).
13	a)	Em redes profundas e com vários passos, o ataque é mais difícil.
14	d)	Pergunta se o exemplo z fez parte dos dados de treino.
15	b)	A pertinência a um conjunto específico pode ser, em si, um dado sensível.
16	b)	Modelos tendem a ter perda menor em exemplos de treino.
17	b)	AUC 0,5 indica ausência de capacidade de distinguir.
18	c)	É o ataque ideal.
19	b)	Com 150 exemplos e 50 rodadas, o ataque atingiu cerca de 0,64.
20	d)	Mais dados reduzem a memorização individual e o vazamento.
21	a)	Perda muito menor nos membros facilita o ataque.
22	c)	Infere propriedades do conjunto a partir das atualizações.
23	d)	Foi demonstrada a extração de dados de treino de modelos de linguagem.
24	c)	Fowl et al. (2022) inseriram camadas para capturar dados dos clientes.
25	b)	As demais apenas mitigam o risco; DP oferece garantia matemática.
26	c)	Ela esconde atualizações individuais; o modelo final ainda pode vazar.
27	b)	Mais exemplos somados reduzem a fidelidade da reconstrução de cada um.
28	d)	Poucos dados e muitas épocas favorecem a memorização.
29	d)	Experimentos de privacidade em dados pessoais sem autorização são antiéticos e ilegais.
30	b)	Treinar menos reduz o sobreajuste; o ataque do limiar de perda perde força.

Discursivas (respostas-modelo)

1. Mesmo sem ver o caderno, quem recebe o ajuste "tirar toda a pimenta" deduz que a receita tinha muita pimenta. Atualizações de modelo carregam pistas sobre os dados que as geraram.
2. Servidor curioso: vê cada atualização individual. Cliente malicioso: vê os modelos globais de cada rodada. Observador de rede: vê mensagens, se não houver criptografia. Usuário do modelo final: vê previsões ou pesos.

3. O curioso segue o protocolo, mas analisa o que recebe. O malicioso se desvia do protocolo, por exemplo, enviando modelos manipulados para facilitar o vazamento.
4. Reconstrução: quais eram os dados? Pertencimento: o exemplo estava no treino? Propriedades: o conjunto tem tal característica? Memorização: que trechos foram decorados? Ativos: posso manipular o modelo para vazar mais?
5. O gradiente dos pesos é $x \otimes (p - y)$ e o do viés é $(p - y)$. Dividindo uma coluna do primeiro pelo componente correspondente do segundo, obtém-se x exatamente.
6. Modelo linear, um passo e gradiente puro permitem solução fechada. Em redes profundas, há não linearidades, vários passos e lotes maiores; é preciso otimizar uma entrada falsa, o que é mais difícil, mas ainda possível.
7. A correlação cai de 1,0 (lote 1) a cerca de 0,82 (lote 32): lotes maiores misturam exemplos e dificultam a reconstrução, mas ainda restam semelhanças com exemplos reais.
8. Marca como membro quem tem perda baixa, pois modelos ajustam melhor os exemplos de treino. Mede-se pela AUC: 0,5 equivale a chute; 1,0 é perfeito.
9. Cada exemplo tem menor influência individual e o modelo decora menos; a diferença de perda entre membros e não membros diminui.
10. Modelos grandes, treinados muito e com poucos dados tendem a memorizar, tornando exemplos de treino distinguíveis e aumentando o vazamento. Regularização e mais dados reduzem o risco.
11. É deduzir uma característica do conjunto de dados de um participante, mesmo irrelevante para a tarefa, por exemplo, se há pessoas de óculos nas fotos de um cliente.
12. Têm muitos parâmetros e treinam com grandes volumes de texto, podendo memorizar sequências literais, inclusive informações pessoais, que podem ser extraídas por consultas apropriadas.
13. O servidor envia um modelo modificado, por exemplo com uma camada que captura entradas, de modo que a atualização devolvida revele diretamente os dados dos clientes, mesmo com lotes grandes.
14. Mitigações (lotes grandes, regularização, compressão) diminuem o risco, sem prova formal. Garantia (privacidade diferencial) oferece limite matemático ao vazamento.
15. Ela esconde apenas as atualizações individuais durante o treino. O modelo agregado ainda pode ser consultado ou inspecionado e pode ter memorizado exemplos.
16. Lotes maiores misturam mais exemplos e dificultam a reconstrução; mais passos locais adicionam não linearidade e mistura, embora em modelos lineares com um único exemplo a direção da atualização continue proporcional a x.
17. Cada exemplo tem peso maior na atualização, o modelo local memoriza mais e o gradiente mistura menos exemplos, facilitando reconstrução e inferência de pertencimento.
18. Usar dados públicos ou sintéticos, ter autorização formal e aprovação ética, não reidentificar pessoas e relatar vulnerabilidades de modo responsável.
19. Lotes maiores: reduzem a fidelidade da reconstrução, mas não dão garantia. Regularização/parada antecipada: reduzem memorização, mas não impedem vazamento por gradientes.
20. Resposta pessoal coerente com o resumo: atualizações carregam pistas; adversários; famílias de ataques; inversão exata em softmax; pertencimento e sobreajuste; mitigação versus garantia.

Capítulo 8 — Protegendo os Dados

Múltipla escolha

Questão	Resposta	Justificativa
1	a)	A DP limita, por definição, a influência de qualquer pessoa na saída.
2	a)	O sorteio embaralha a resposta individual, mas permite estimar a taxa agregada.
3	d)	Como observada = 0,25 + 0,5 · verdadeira, a verdadeira é 2·observada − 0,5.
4	a)	ε = ln da razão das probabilidades = ln(0,75/0,25) = ln 3.
5	d)	Quanto menor ε, mais próximas são as distribuições de saída com e sem o indivíduo.
6	b)	δ costuma ser menor que o inverso do número de pessoas.
7	b)	ε muito grande dá uma garantia pouco significativa.
8	c)	Δ = max ‖f(D) − f(D')‖ para conjuntos vizinhos.
9	b)	O ruído é proporcional à sensibilidade, com multiplicador z.
10	b)	Mais ruído esconde melhor o indivíduo, mas prejudica a acurácia.
11	a)	Não exige confiar no servidor, mas costuma ter menor utilidade.
12	d)	Maior utilidade, mas o servidor precisa ser confiável.
13	a)	Soma de ruídos gaussianos independentes é gaussiana; a utilidade se aproxima da DP central.
14	a)	Esconde se o cliente participou do treino.
15	b)	Encolhe cada atualização para norma no máximo S.
16	b)	A sensibilidade é S (norma de corte), e o multiplicador é z.
17	d)	Atualizações são encolhidas, reduzindo o progresso do treino.
18	c)	Equilibra corte e quantidade de ruído.
19	d)	Os ε se acumulam a cada rodada.
20	a)	Fórmula usada em `epsilon_rdp` do livro.
21	d)	O livro obtém ε ≈ 10,4 com participação total.
22	a)	Com subamostragem, o ε real é menor; bibliotecas especializadas calculam isso.
23	c)	O ruído dividido por poucos clientes pesa muito.
24	d)	Mais clientes diluem o ruído na média.
25	b)	O ruído da média diminui com o número de clientes.
26	a)	O cliente de menor índice soma s, o de maior subtrai; a soma é exata.
27	d)	O protocolo não esconde o modelo final; isso exige DP.
28	d)	O resultado (soma ou modelo) continua revelando a influência dos dados.
29	a)	O produto de textos cifrados corresponde à soma dos valores originais.
30	d)	O ruído de desvio z·S é dividido pelo número de clientes m na média.

Discursivas (respostas-modelo)

1. Cada pessoa responde a verdade ou um sorteio, conforme a moeda; ninguém sabe se um "sim" é verdadeiro, mas o pesquisador, conhecendo o sorteio, estima a taxa verdadeira. O ruído esconde o indivíduo e preserva a estatística coletiva.

2. Para quaisquer conjuntos vizinhos, a probabilidade de qualquer saída muda no máximo por um fator e^ε (mais δ) quando uma pessoa entra ou sai. ε pequeno significa que a saída quase não depende de uma pessoa: mais privacidade.
3. Sensibilidade é o máximo que a consulta pode mudar com uma pessoa a mais ou a menos. O mecanismo gaussiano soma ruído normal com desvio σ = z·Δ, tornando a saída insensível à presença de um indivíduo.
4. Local: sem confiança no servidor, mas menor utilidade. Central: maior utilidade, exige servidor confiável. Distribuída: ruído parcial por cliente e agregação segura; confiança mínima e utilidade próxima da central.
5. Corte: limita a norma de cada atualização a S, limitando a sensibilidade. Ruído: soma-se ao total das atualizações cortadas um ruído gaussiano com desvio z·S; depois divide-se por m.
6. O ruído é adicionado à soma e dividido por m; com mais clientes, seu efeito na média diminui. Com z = 3, 20 clientes caíram a 0,87, enquanto 100 clientes ficaram em 0,93.
7. σ da soma = z·S = 2,0. Na média, σ/m = 2,0/50 = 0,04.
8. Cada rodada é uma nova consulta aos dados que revela alguma informação; pela composição, os custos de privacidade somam-se (ou crescem, em contabilidades mais finas), de modo que mais rodadas exigem mais ruído para manter o mesmo ε.
9. ε diminui conforme z aumenta (mais ruído) e aumenta com o número de rodadas. Para z = 10, ε ≈ 2,8 em 30 rodadas e ≈ 9,8 em 300. Mesmo ruído alto acumula ε com muitas rodadas.
10. Assume participação total e ignora a amplificação de privacidade que o sorteio de clientes traz; bibliotecas como Opacus fazem contas mais justas, com ε menores.
11. Cada par combina um segredo s. O cliente de menor índice soma s, o de maior subtrai. Cada mensagem parece ruído, mas ao somar tudo as máscaras se cancelam e o servidor obtém $u_1 + u_2 + u_3$.
12. Acordo de chaves sem confiar no servidor (Diffie-Hellman), clientes que desistem (compartilhamento de segredos de Shamir), servidor malicioso (dupla máscara) e custo de comunicação.
13. A agregação segura esconde atualizações individuais durante o treino, mas o resultado ainda pode vazar. A DP limita o que o resultado revela. Juntas, protegem o caminho e o produto.
14. Multiplicar os textos cifrados de 120, 345 e 77 e decifrar o produto resulta em 542, a soma dos valores. O servidor soma sem ver os valores individuais.
15. Permite que várias partes calculem juntas uma função dos seus dados privados sem revelar as entradas, tipicamente por compartilhamento de segredos. A agregação segura é um caso particular, em que a função é a soma.
16. A parcial suporta uma operação (por exemplo, soma) sobre cifrados; a totalmente suporta somas e multiplicações arbitrárias, mas é muito mais custosa computacionalmente.
17. Área protegida do processador em que nem o dono da máquina vê o cálculo. Exige confiar no fabricante do hardware e pode sofrer ataques de canal lateral.
18. Sem ruído, a correlação da reconstrução é 1,0. Com σ = 0,1, cai para cerca de 0,23, e com σ ≥ 0,5, fica próxima de zero: a combinação de corte e ruído destrói a reconstrução.
19. Escolher z: valores altos dão mais privacidade, mas pioram a acurácia em poucos clientes. Resolveria aumentando o número de clientes, ajustando S, e definindo com juristas e a organização um ε aceitável.
20. Resposta pessoal coerente com o resumo: DP (resultado, garantia formal); agregação segura (atualizações individuais); homomórfica e SMPC (cálculo cifrado); TEE (operador); combinação em camadas.

Capítulo 9 — Quando Alguém Trapaceia

Múltipla escolha

Questão	Resposta	Justificativa
1	b)	O termo vem do problema dos generais bizantinos (Lamport et al., 1982).
2	b)	Exemplo: trocar rótulos nos dados locais.
3	b)	Pode, por exemplo, enviar o oposto da atualização honesta, amplificada.
4	b)	O objetivo é degradar o desempenho geral.
5	c)	O modelo parece normal, mas obedece ao gatilho secreto.
6	b)	O modelo se comporta bem em dados normais, escondendo o ataque.
7	a)	A média não resiste nem a um valor arbitrário.
8	b)	A mediana só se move se mais da metade dos valores mudar.
9	a)	$(19 \cdot 7 + 1.000)/20 = 56{,}65$.
10	a)	Aplica a mediana a cada coordenada do vetor.
11	b)	$k = \text{floor}(0{,}2 \cdot 20) = 4$ de cada lado.
12	b)	É a mais "central", com muitas outras parecidas.
13	a)	Soma-se a distância aos $m - f - 2$ vizinhos mais próximos.
14	a)	Isso reduz a perda de informação do Krum.
15	c)	Limita o dano de atualizações gigantes.
16	b)	No experimento, o ASR continuou em torno de 0,7 a 0,9.
17	d)	A média colapsou.
18	a)	Mesmo uma minoria já prejudica muito a média.
19	a)	Escolher uma só atualização desperdiça as demais.
20	d)	As imagens com gatilho devem ser classificadas como a classe alvo (0).
21	b)	Mede o quanto o backdoor funciona.
22	a)	Mesmo uma minoria implanta o backdoor com a média.
23	d)	Em dados IID, a mediana eliminou o backdoor.
24	b)	Em IID, as atualizações honestas se agrupam, e as maliciosas ficam nas pontas.
25	b)	Com $\alpha = 0{,}1$, a mediana caiu de 0,96 para 0,86.
26	c)	O Krum depende de haver muitas atualizações parecidas, o que não ocorre em não-IID.
27	c)	Privacidade e robustez colidem.
28	b)	Ataques sutis, ajustados à variabilidade honesta, contornam defesas.
29	c)	A mediana só é movida quando mais da metade dos valores muda.
30	d)	Se $\beta \cdot m$ é menor que o número de atacantes, sobram atacantes na média.

Discursivas (respostas-modelo)

1. Com 20 alunos, se um sugere 1.000 e os outros 7, a média vira cerca de 57: um único participante arrasta a decisão. A mediana continuaria perto de 7.
2. Dados: o atacante só controla os próprios dados (ex.: trocar rótulos). Modelo: controla a atualização enviada (ex.: enviar o oposto da atualização honesta, amplificada).

3. O não direcionado destrói a acurácia geral e é facilmente notado. O backdoor mantém a acurácia normal e só falha com um gatilho, sendo muito mais difícil de detectar.
4. Para cada coordenada, ordenam-se os valores, descartam-se os β·m menores e maiores e faz-se a média do restante. β deve ser pelo menos a fração de maliciosos.
5. Pontua cada atualização pela soma das distâncias aos m − f − 2 vizinhos mais próximos e escolhe a de menor pontuação (a mais central). Limitação: usa uma única atualização, perdendo informação, e é sensível a dados não-IID.
6. O backdoor pode ser implantado com atualizações de norma pequena, que o corte não altera; o corte só limita atualizações gigantes, e no experimento o ASR seguiu alto.
7. Gatilho: bloco de pixels acesos no canto. Alvo: classe 0. O malicioso treina com dados normais e com gatilho rotulados como 0, amplificando a atualização. Métricas: acurácia normal e ASR.
8. Com a média, ASR de 0,77 a 0,97 e acurácia normal intacta; corte de norma não ajudou (0,7 a 0,9); mediana e média aparada zeraram o ASR em dados IID.
9. Eles supõem honestos parecidos entre si. Com dados heterogêneos, clientes honestos diferem muito, e o agregador pode descartá-los como se fossem atacantes, perdendo informação útil.
10. Sem ataque, a média tem 0,96 e o Krum 0,88 (IID), pois o Krum usa uma só atualização. Em não-IID o Krum cai ainda mais (0,45 com α = 0,1).
11. O servidor precisa ver as atualizações para filtrá-las, mas a agregação segura as esconde. Direções: verificar limites de norma sobre dados cifrados e usar provas de conhecimento zero de que a atualização é válida.
12. O atacante multiplica sua atualização por um fator γ para que ela domine a média e substitua o modelo global por um modelo com backdoor. Amplificar aumenta a influência na agregação.
13. Detecção e reputação de clientes ao longo do tempo; validação do modelo agregado com dados no servidor; privacidade diferencial (corte e ruído); provas de que a atualização respeita limites.
14. Atacantes que conhecem a defesa podem ajustar o ataque para contorná-la (como em "um pouco é suficiente" e "cauda"). Defesas avaliadas só contra ataques simples dão falsa sensação de segurança.
15. m = 5; k = floor(0,2·5) = 1; descartam-se 1 e 100; média de 2, 3, 4 = 3. A média comum seria 22. O valor extremo é neutralizado.
16. Falha bizantina pode ser aleatória ou acidental (bug, hardware). Ataque coordenado envolve vários clientes combinando-se para maximizar o dano, o que é mais difícil de defender.
17. Estimando a fração máxima plausível de clientes maliciosos e usando β um pouco acima; testando perda de acurácia sem ataque para equilibrar robustez e eficiência.
18. Recomendaria quando há risco de clientes maliciosos e os dados são aproximadamente IID. Evitaria com forte heterogeneidade e sem atacantes, em que a média é mais eficiente.
19. Resposta pessoal coerente com o resumo: bizantinos, tipos de ataque, fragilidade da média, agregadores robustos, backdoor, custo em não-IID e tensão com a agregação segura.
20. Porque, se os maliciosos são maioria, eles definem o "centro" dos valores (mediana, vizinhança do Krum), e os honestos passam a parecer os outliers. Defesas baseadas em maioria ou centralidade falham.

Capítulo 10 — Outros Desenhos

Múltipla escolha

Questão	Resposta	Justificativa
1	a)	Cada cliente possui linhas (exemplos) diferentes da mesma tabela.
2	c)	Cada participante guarda colunas diferentes sobre as mesmas pessoas.
3	d)	Mesmas pessoas e atributos distintos: partição vertical.
4	b)	Mesmos atributos, pessoas diferentes.
5	d)	FTL é usado quando quase não há sobreposição de pessoas ou atributos.
6	c)	Usa-se, por exemplo, interseção privada de conjuntos.
7	d)	Permite alinhar registros comuns preservando o restante.
8	b)	Apenas as parcelas, sem expor as colunas.
9	b)	Só quem conhece y consegue calcular p − y.
10	a)	O resíduo recebido e suas próprias colunas bastam.
11	d)	O treino conjunto aproveitou todas as colunas, sem juntá-las.
12	a)	Se y = 1, o resíduo é negativo; se y = 0, é positivo.
13	a)	$p \in (0, 1)$, então $p - 1 < 0$ e $p - 0 > 0$.
14	d)	O sinal do resíduo identifica y exatamente.
15	d)	Essas técnicas reduzem o vazamento do rótulo.
16	b)	São os "dados amassados" (smashed data).
17	c)	Permite ao cliente concluir a retropropagação.
18	c)	A regra da cadeia garante equivalência matemática.
19	d)	O resto do modelo roda no servidor.
20	a)	Ativações podem permitir reconstruir a entrada.
21	b)	Não há ponto central; a comunicação é entre vizinhos.
22	c)	O consenso emerge de médias locais repetidas.
23	c)	Isso preserva a média global.
24	a)	Com pesos iguais a 1/n, todos recebem a média global em uma rodada.
25	a)	Pouca comunicação por rodada, mas convergência mais lenta.
26	c)	A principal motivação é remover o coordenador central.
27	a)	Alinha representações com poucos exemplos em comum.
28	d)	A rede é cortada em uma camada.
29	d)	VFL exige alinhar entidades e protege mal o resíduo.
30	a)	Com poucos registros alinhados, o treino conjunto é limitado.

Discursivas (respostas-modelo)

1. Horizontal: mesmas colunas, pessoas diferentes (dois hospitais). Vertical: mesmas pessoas, colunas diferentes (banco e loja). Transferência: pouco em comum, transfere-se conhecimento (países com poucos clientes comuns).

2. 1) Cada participante calcula $z_k = X_k \cdot w_k$ e envia à parte com o rótulo. 2) Essa parte soma, calcula p e o resíduo (p − y). 3) Devolve o resíduo. 4) Cada participante atualiza w_k com $X_k^T(p - y)/n$.

3. Como $p \in (0,1)$, o sinal de $p - y$ indica y (negativo se 1, positivo se 0). Defesas: criptografar o resíduo (homomórfica), adicionar ruído (DP) ou usar agregação segura, reduzindo a informação sobre y.
4. Com ruído maior, a fração de rótulos recuperada pelo sinal cai (de 100% a cerca de 60% com $\sigma = 2$), enquanto a acurácia de teste cai pouco (de 0,965 a 0,951): o ruído protege com custo moderado.
5. Para treinar, é preciso saber quais registros de cada participante pertencem à mesma pessoa, sem revelar os registros que não são comuns. Usa-se interseção privada de conjuntos.
6. A rede é cortada: cliente computa até a camada de corte e envia ativações; o servidor completa, calcula a perda e devolve o gradiente no corte. Pela regra da cadeia, as atualizações são as mesmas do treino conjunto.
7. No split learning, trafegam ativações e gradientes proporcionais ao número de exemplos por época; no FedAvg, trafega o modelo uma vez por rodada. Com muitos dados locais, o split pode custar bem mais comunicação.
8. As ativações na camada de corte podem permitir reconstruir ou inferir a entrada; o servidor conhece os rótulos e as ativações; o treino sequencial pode expor ordem e participação.
9. Cada nó repete a média ponderada com os vizinhos até todos terem o mesmo valor (a média global). A matriz de mistura A define os pesos; duplamente estocástica, preserva a média e permite convergência.
10. A completa converge em uma rodada; o anel reduz o erro gradualmente (cerca de 0,02 em 30 rodadas); a estrela, com esses pesos, é mais lenta (cerca de 0,10), por causa do gargalo no nó central.
11. Vantagens: sem ponto único de falha, sem coordenador que veja tudo. Desvantagens: convergência mais lenta em topologias esparsas, sincronização complexa, mais difícil aplicar agregação segura.
12. Em HFL, cada cliente tem linhas completas (todos os atributos) de algumas pessoas; em VFL, cada participante tem algumas colunas de todas (ou muitas) as mesmas pessoas.
13. Só ela calcula p e o resíduo $(p - y)$, que coordena o treino; ao devolver o resíduo, ela expõe informação sobre os rótulos às demais, o que exige proteção.
14. Com alinhamento total, o VFL supera cada participante sozinho; com 50%, ainda é um pouco melhor; com 20% ou menos, deixa de superar um participante sozinho. Poucos registros alinhados limitam o benefício do VFL.
15. Quando não há um coordenador confiável ou se quer evitar ponto único de falha, como em redes de dispositivos de borda cooperantes; aceitando convergência mais lenta e maior complexidade.
16. É a camada onde a rede é dividida. Escolha-se equilibrando o cálculo no cliente (quanto mais camadas, mais cálculo), o tamanho das ativações (comunicação) e o vazamento de informação pelas ativações.
17. Semelhanças: ambos protegem contribuições individuais e usam criptografia. Diferenças: a agregação segura soma atualizações de modelo entre clientes horizontais; no vertical, combinam-se parcelas de colunas diferentes e protege-se o resíduo de rótulos.
18. Resposta pessoal coerente com a tabela do capítulo: HFL para hospitais com mesmas colunas, VFL para banco e loja com clientes em comum, split learning para dispositivos fracos com redes grandes, descentralizado sem coordenador confiável.
19. Porque a nota z é a soma das parcelas $X_k \cdot w_k$, e o gradiente de cada bloco de pesos depende só do resíduo e das próprias colunas. Reparti-las entre participantes não muda as contas, apenas onde elas são feitas.
20. Split: $2 \cdot 4 \cdot 5.000 \cdot 32 = 1.280.000$ bytes (cerca de 1,25 MB). FedAvg: $2 \cdot 4 \cdot 10.000 = 80.000$ bytes (cerca de 78 KB). O split custa cerca de 16 vezes mais.

Capítulo 11 — Fronteiras e Prática

Múltipla escolha

Questão	Resposta	Justificativa
1	a)	Soma de somas e soma de contagens reproduzem a média global de cada grupo.
2	c)	O servidor soma as estatísticas e recalcula os centroides.
3	d)	$\text{cov} = (\Sigma\, x\, x^T - n\, \mu\, \mu^T)/(n-1)$.
4	b)	Praticamente zero, diferença de arredondamento.
5	c)	ARI é 1 quando as partições coincidem.
6	a)	Se os resultados dependem de somas, o servidor reproduz o centralizado.
7	b)	A estatística de um grupo de poucos exemplos expõe seus valores.
8	b)	Ex.: grandes modelos de linguagem.
9	c)	Federar permite usar dados que não podem ser centralizados.
10	c)	Por isso se usam ajustes eficientes em parâmetros.
11	a)	$W = W_0 + A{\cdot}B$, com W_0 congelado.
12	a)	$r{\cdot}(d + K) = 8 \cdot 2.000 = 16.000$.
13	a)	16.000 / 1.000.000 = 1,6%.
14	c)	W_0 é congelado e já está em todos.
15	a)	A capacidade do adaptador deve casar com a complexidade da tarefa.
16	b)	Chegou ao nível do ajuste completo com cerca de um quarto do tráfego.
17	a)	O LoRA r = 2 usou cerca de 4 vezes menos tráfego.
18	d)	Isso gera erro de agregação, tratado por métodos recentes.
19	c)	Flower suporta PyTorch, TensorFlow e scikit-learn, entre outros.
20	a)	FATE é voltado a finanças e empresas, com suporte vertical.
21	b)	Servem para avaliar métodos de modo comparável.
22	b)	Essas questões definem os requisitos técnicos do projeto.
23	a)	Sheller et al. (2020) e Dayan et al. (2021).
24	a)	Isso evita impacto no usuário.
25	a)	Defesas técnicas e análise jurídica continuam necessárias.
26	c)	A minimização é princípio da LGPD e do GDPR.
27	d)	É uma exigência típica de projetos de maior risco.
28	b)	Visa justiça ao evitar que um grupo seja sacrificado.
29	d)	O parâmetro q regula o quanto a justiça pesa na média.
30	d)	Mecanismos de incentivo e valoração são questões abertas.

Discursivas (respostas-modelo)

1. O servidor envia os centroides; cada cliente atribui seus pontos e devolve, por grupo, a soma e a contagem; o servidor soma e divide. Como média = soma/contagem e ambas são somáveis, o resultado é idêntico ao centralizado com a mesma inicialização.
2. Cada cliente envia n, Σx e $\Sigma\, x\, x^T$. O servidor calcula μ e a covariância, $\text{cov} = (\Sigma\, x\, x^T - n\mu\mu^T)/(n-1)$, e extrai os autovetores principais, obtendo o PCA centralizado exato.

3. Mínimos quadrados (somas de X^TX e X^Ty), Naive Bayes (contagens por classe) e médias/variâncias. Em mínimos quadrados, X^TX e X^Ty são somas sobre exemplos e, portanto, somáveis entre clientes.
4. Em grupos com poucos exemplos, a soma e a contagem revelam valores individuais. Mitiga-se com agregação segura (servidor vê só a soma total) e privacidade diferencial (ruído).
5. Enviar bilhões de parâmetros por rodada é inviável em comunicação, memória e cálculo. Ajustar poucos parâmetros (adaptadores, LoRA) reduz tráfego e custo local.
6. $W = W_0 + A \cdot B$, com A (d × r) e B (r × K). Parâmetros: $r(d + K) = 8 \cdot 8.192 = 65.536$, contra $d \cdot K \approx 16,8$ milhões (cerca de 0,4%).
7. O posto 1 não basta (≈ 0,79). Posto 2 atinge ≈ 0,96 com cerca de 1/4 do tráfego do ajuste completo (≈ 0,95). Postos 4 e 8 melhoram um pouco, mas gastam mais bytes. O posto precisa casar com a tarefa.
8. A média de A e a de B não é a média de A·B, o que causa erro de agregação. Métodos agregam os produtos $\Delta W = A \cdot B$ ou empilham adaptadores para reduzir o erro.
9. Prompts aprendidos, destilação (enviar previsões em vez de pesos), modelos pequenos locais com apoio de grandes remotos, e dados sintéticos.
10. Flower: independente de biblioteca, simulação e implantação. TFF: pesquisa com TensorFlow. FATE: cross-silo industrial com suporte vertical, voltado a empresas e finanças.
11. Simulação ou implantação? Cross-device ou cross-silo (horizontal ou vertical)? Qual biblioteca de ML? Quais garantias de segurança são necessárias? A ferramenta é mantida e tem comunidade?
12. Saúde: imagens e prontuários permanecem nos hospitais, e só parâmetros trafegam. Teclados móveis: o texto digitado permanece no celular, e só atualizações do modelo são enviadas.
13. Atualizações podem vazar e modelos podem memorizar dados pessoais; por isso o modelo pode ser tratado como dado pessoal. É preciso base legal, minimização, segurança e análise de risco.
14. Finalidade, minimização, transparência, segurança, responsabilização e base legal para o tratamento.
15. Evitar que o modelo global sirva bem à maioria e mal a minorias. Propostas: aprendizado federado agnóstico (otimiza o pior caso) e q-FFL (mais peso a clientes com maior perda).
16. Quem controla e pode usar o modelo, o que ocorre quando um participante sai, como compartilhar custos e benefícios, auditoria, e responsabilidades legais.
17. Participantes podem "pegar carona" no modelo sem contribuir ou contribuir com dados de baixa qualidade; avaliar e recompensar a contribuição de cada um é difícil e é um problema em aberto.
18. Resposta pessoal coerente com a lista: necessidade, cenário, heterogeneidade, comunicação, privacidade, robustez, avaliação por cliente, base legal e operação, escolhendo os mais críticos para o caso.
19. Privacidade útil com poucos clientes, robustez sem abrir mão da agregação segura, modelos de fundação federados, justiça e incentivos. Exemplo: com poucos clientes, o ruído necessário para ε baixo destrói a acurácia.
20. Resposta pessoal coerente com o resumo: algoritmos de somas são exatos, LoRA reduz comunicação, escolha de framework, aplicações, LGPD e justiça.

Capítulo 12 — Projeto Integrador Final

Múltipla escolha

Questão	Resposta	Justificativa
1	b)	Leva o modelo até os dados em vez de levar os dados até o modelo.
2	b)	ASR = attack success rate.
3	c)	É preciso medir por cliente e sob ataque.
4	d)	Com 2 de 20 clientes maliciosos, o backdoor funciona com a média.
5	c)	Em dados com heterogeneidade moderada, a mediana resistiu ao backdoor.
6	c)	386 e 195 KB contra 1.523 KB, com métricas equivalentes.
7	a)	A DP protege contra inferência de pertencimento, não contra envenenamento direcionado.
8	d)	A média por cliente subiu de 0,92 para 0,94.
9	d)	Foi aprovada e teve o menor tráfego, com melhor média por cliente no desempate.
10	d)	Média ponderada dos erros locais pelo tamanho dos dados.
11	b)	O FedAvg troca comunicação por computação local.
12	c)	Cada cliente caminha para o seu próprio mínimo local.
13	a)	Corrige a deriva ao custo de mais comunicação.
14	c)	Evita que valores pequenos nunca sejam enviados.
15	a)	$\lambda = 0$ equivale ao ajuste fino; λ grande iguala o global.
16	b)	O servidor mantém K modelos e agrupa os clientes iterativamente.
17	c)	Dividir uma coluna pelo gradiente do viés devolve x.
18	a)	A perda nos membros fica muito menor que nos não membros.
19	b)	Mede quanto a saída pode mudar pela presença de uma pessoa.
20	b)	Nenhum cliente muda a soma por mais que S.
21	a)	As máscaras de pares se cancelam na soma.
22	c)	Ponto de ruptura zero.
23	a)	Eles supõem honestos parecidos entre si.
24	b)	Privacidade e robustez colidem.
25	a)	Se y = 1, o resíduo é negativo; se y = 0, é positivo.
26	b)	O cliente computa as primeiras camadas e o servidor, as últimas.
27	b)	Soma de estatísticas suficientes = estatística global.
28	b)	O ajuste de baixo posto reduz comunicação e cálculo local.
29	d)	É preciso base legal, minimização e defesas técnicas.
30	d)	Compromissos entre qualidade, custo, privacidade e robustez estão em toda decisão.

Discursivas (respostas-modelo)

1. É treinar um modelo com a colaboração de vários donos de dados sem juntar os dados, como amigas que trocam só os ajustes de uma receita. Motivos: privacidade e leis; custo de mover dados; falta de confiança entre organizações.
2. O servidor sorteia clientes e envia o modelo; cada um treina localmente e devolve a atualização; o servidor agrega por média ponderada. Hiperparâmetros: C, E, B e η.

3. Cada cliente vai ao seu mínimo local (deriva), tornando a média pior e o treino lento e instável. Soluções: SCAFFOLD, FedProx, FedNova, otimizadores no servidor, personalização.
4. Quantização reduz bits por valor; top-k envia só os maiores valores e seus índices. A memória de erro guarda o que foi cortado e o soma depois, sendo crucial na esparsificação forte.
5. Global: um modelo para todos. Ajuste fino: continua o treino local a partir do global. Ditto: modelo pessoal com coleira λ ao global. IFCA: agrupa clientes e treina um modelo por grupo.
6. O gradiente de um exemplo é x $\otimes$ (p − y), e o do viés é (p − y), logo x = coluna / gradiente do viés. Com lotes maiores, o gradiente é uma soma de vários exemplos e a reconstrução fica misturada.
7. Pergunta se um exemplo estava no treino, explorando a perda menor nos membros. Quanto mais o modelo decora (poucos dados, muito treino), maior a AUC do ataque.
8. A saída quase não muda com a presença de um indivíduo; ε menor, mais privacidade. O corte limita a sensibilidade e o ruído gaussiano z·S esconde a contribuição; ambos custam acurácia.
9. O ruído é dividido pelo número de clientes na média; com poucos clientes, o mesmo ruído pesa muito. Com mais clientes, a acurácia sofre menos.
10. Pares de clientes compartilham segredos que se somam e subtraem, mascarando cada atualização; na soma, as máscaras se cancelam. Limitações: não protege o resultado, exige tratar desistências e tem custo de comunicação.
11. Um único valor extremo a arrasta. Mediana e média aparada resistem; limitação: perdem acurácia com dados não-IID e não resistem a mais da metade de maliciosos.
12. O atacante treina com imagens com gatilho rotuladas como a classe alvo e amplifica a atualização. O modelo funciona bem em imagens normais, e só erra com o gatilho; por isso a acurácia global fica intacta.
13. Detectar atualizações maliciosas exige vê-las, mas a agregação segura as esconde. Defesas que inspecionam atualizações enfraquecem a privacidade, e vice-versa.
14. Horizontal: mesmas colunas, pessoas diferentes (hospitais). Vertical: mesmas pessoas, colunas diferentes (banco e loja). Transferência: pouca sobreposição, transfere-se conhecimento.
15. O sinal de p − y revela o rótulo aos participantes passivos. Defesas: criptografia homomórfica, ruído (DP) ou agregação segura.
16. Ambos dependem de somas de estatísticas locais (somas, contagens, covariâncias). Aviso: somas por grupo pequeno revelam informação; combine com agregação segura e DP.
17. Congela W_0 e treina só A e B, reduzindo o que trafega. O posto precisa casar com a tarefa: no experimento, o posto 1 não bastou para duas duplas trocadas, e o posto 2 sim.
18. Finalidade, minimização, segurança (também transparência e responsabilização). Justiça: aprendizado federado agnóstico ou q-FFL, que dá mais peso a clientes com maior perda.
19. Recomenda-se mediana com quantização de 4 bits e ajuste fino local: o ASR cai de cerca de 0,70 para zero, o tráfego cai de 1.523 para 195 KB, a acurácia global fica em cerca de 0,94 e a média por cliente sobe a 0,94. Limites: ataque simples, modelo pequeno, dados substitutos e ausência de garantia formal de privacidade.
20. Reavaliaria a mediana e a média aparada, que perdem robustez à medida que a fração de maliciosos aproxima-se de 30 a 50%, e a DP, que não distingue maliciosos. Mediria o ASR em cada uma e consideraria Multi-Krum ou detecção adicional.

Apêndice B — Glossário

Termos técnicos usados no livro, explicados de forma simples.

Acurácia — fração de previsões corretas de um modelo.

Adaptador — pequena camada extra inserida em um modelo pré-treinado, treinada no lugar do modelo inteiro.

Agregação segura — protocolo criptográfico em que o servidor obtém apenas a soma das atualizações, e não cada uma delas.

Agregação — combinação das atualizações dos clientes em um único modelo global.

Agregador robusto — regra de agregação (mediana, média aparada, Krum) que limita a influência de clientes maliciosos.

Alinhamento de entidades — etapa do aprendizado vertical que descobre quais registros de diferentes participantes pertencem à mesma pessoa.

Aprendizado federado agnóstico — método que otimiza o pior caso sobre misturas dos clientes.

Arredondamento estocástico — arredondamento em que se sobe com probabilidade igual à parte fracionária, evitando viés.

ASR (taxa de sucesso do ataque) — fração de exemplos com gatilho que o modelo classifica como a classe escolhida pelo atacante.

Ataque ativo — ataque em que o adversário se desvia do protocolo, como um servidor que envia modelos manipulados.

Ataque de envenenamento — ataque em que clientes maliciosos manipulam dados ou atualizações para prejudicar o modelo.

AUC — área sob a curva ROC; mede a capacidade de um classificador de separar duas classes; 0,5 é chute e 1,0 é perfeito.

Backdoor — ataque em que o modelo funciona bem, exceto quando recebe um gatilho secreto, caso em que dá a resposta escolhida pelo atacante.

Base legal — fundamento jurídico que autoriza o tratamento de dados pessoais, como o consentimento.

Bizantino — participante que pode se comportar de forma arbitrária, inclusive mentindo ou falhando.

Centroide — ponto que representa o centro de um grupo, em geral a média de seus pontos.

Cliente — participante que possui dados e treina localmente.

Clipping (corte) — limitação da norma de um vetor, como a atualização de um cliente, a um valor máximo.

Cold start — situação em que um cliente novo não tem dados para personalizar o modelo.

Compartilhamento de segredos — divisão de um valor em partes de modo que nenhuma parte sozinha o revele.

Comunicação (rodada de) — ciclo em que o servidor envia o modelo, os clientes treinam e devolvem atualizações, e o servidor agrega.

Concept shift (mudança de conceito) — situação em que o mesmo atributo recebe rótulos diferentes em clientes ou grupos diferentes.

Consenso — estado em que todos os nós de uma rede descentralizada têm o mesmo valor, em geral a média global.

Contabilidade de privacidade — cálculo do ε acumulado ao longo de várias aplicações de um mecanismo diferencialmente privado.

Convexo — diz-se do problema cuja superfície de perda tem um único vale, o que facilita a otimização.

Corte de norma — técnica que limita a norma de cada atualização antes de agregá-las.

Criptografia homomórfica — criptografia que permite operar sobre dados cifrados.

Cross-device — cenário com milhões de clientes pequenos e instáveis, como celulares.

Cross-silo — cenário com poucos clientes grandes e confiáveis, como hospitais e bancos.

Dados de teste — parte dos dados usada uma única vez para avaliar o modelo final.

Dados de treino — parte dos dados usada para ajustar os parâmetros do modelo.

Dados de validação — parte dos dados usada para escolher hiperparâmetros.

Dados não-IID — dados cujas distribuições diferem entre os clientes.

Delta (δ) — probabilidade pequena de falha da garantia de privacidade diferencial.

Deriva dos clientes (client drift) — afastamento do modelo local em direção ao mínimo do próprio cliente durante o treino local.

Descida do gradiente — método de otimização que repete passos na direção oposta ao gradiente da perda.

Diffie-Hellman — protocolo para que duas partes combinem um segredo comum por um canal aberto.

Dirichlet — distribuição de probabilidade usada para sortear proporções que somam 1, útil para simular dados não-IID.

DP (privacidade diferencial) — propriedade segundo a qual a saída de um algoritmo quase não muda com a presença ou ausência de qualquer indivíduo.

DP-FedAvg — FedAvg com privacidade diferencial no nível do cliente, por meio de corte e ruído gaussiano.

Entropia cruzada — função de perda usada em classificação; vale −log da probabilidade atribuída à classe correta.

Época — uma passada completa pelos exemplos de treino.

Épsilon (ε) — parâmetro da privacidade diferencial; quanto menor, mais privacidade.

Esparsificação — envio apenas dos valores mais importantes de uma atualização, zerando os demais.

Estatística suficiente — resumo dos dados, como soma e contagem, que se combina entre clientes e basta para um cálculo.

F1 — média harmônica entre precisão e revocação.

FedAdam — variante do FedAvg com otimizador adaptativo (Adam) aplicado no servidor.

FedAvg — algoritmo de médias federadas: clientes treinam localmente e o servidor faz a média ponderada dos modelos.

Federated analytics (análises federadas) — cálculo de estatísticas sobre dados distribuídos sem juntá-los.

FedNova — algoritmo que normaliza cada atualização pelo número de passos locais antes de agregá-la.

FedProx — algoritmo que adiciona ao erro local uma penalização por se afastar do modelo global.

FedSGD — versão do FedAvg com um único passo de gradiente por rodada.

Fine-tuning (ajuste fino) — continuação do treino de um modelo com dados específicos.

Foundation model (modelo de fundação) — grande modelo pré-treinado com dados gerais, posteriormente especializado.

Gatilho — padrão que ativa o comportamento malicioso de um modelo com backdoor.

GDPR — regulamento de proteção de dados da União Europeia, em vigor desde 2018.

Gossip — método de comunicação em que cada nó troca informação com poucos vizinhos para que a rede alcance consenso.

Gradiente — vetor que aponta a direção em que a perda mais cresce.

Heterogeneidade — diferença entre clientes, de dados (não-IID), de recursos ou de comportamento.

HFL (aprendizado federado horizontal) — partição em que os participantes têm os mesmos atributos e pessoas diferentes.

Hiperparâmetro — ajuste do algoritmo escolhido pelo programador, e não aprendido pelo modelo.

IFCA — algoritmo de agrupamento iterativo de clientes, em que cada cliente escolhe o modelo de menor perda.

IID — independentes e identicamente distribuídos; dados sorteados de uma mesma distribuição.

Inferência de pertencimento — ataque que tenta descobrir se um exemplo específico foi usado no treino.

Inferência de propriedades — ataque que tenta descobrir uma característica do conjunto de dados de um participante.

Interseção privada de conjuntos (PSI) — protocolo que revela apenas os elementos em comum entre dois conjuntos.

Inversão de gradientes — ataque que reconstrói dados de treino a partir de gradientes ou atualizações.

K-means — algoritmo de agrupamento que alterna atribuir pontos ao centroide mais próximo e recalcular os centroides.

Krum — agregador robusto que escolhe a atualização mais central entre as recebidas.

Label skew — heterogeneidade em que a frequência das classes varia entre clientes.

LGPD — Lei Geral de Proteção de Dados (Lei nº 13.709/2018), do Brasil.

LoRA — método de ajuste eficiente que aprende uma correção de baixo posto A·B sobre os pesos congelados.

Lote (mini-batch) — pequeno grupo de exemplos usado para calcular uma estimativa do gradiente.

Matriz de confusão — tabela que conta acertos e erros de um classificador em cada classe.

Matriz de mistura — matriz duplamente estocástica que define como os nós combinam os valores dos vizinhos.

Mecanismo gaussiano — mecanismo de privacidade que soma ruído gaussiano proporcional à sensibilidade.

Média aparada — média calculada após descartar os valores extremos de cada coordenada.

Mediana coordenada a coordenada — agregador robusto que toma a mediana de cada parâmetro.

Memória de erro (error feedback) — técnica em que o cliente guarda o que não enviou e soma à atualização seguinte.

Minimização de dados — princípio de coletar e tratar apenas os dados necessários.

Modelo de ameaça — descrição de quem é o adversário, o que sabe e o que pode fazer.

Modelo — função com parâmetros ajustáveis que faz previsões.

Multi-Krum — variante do Krum que faz a média de várias atualizações centrais.

Ótimo global — valor dos parâmetros que minimiza a perda sobre todos os dados.

Overfitting (sobreajuste) — situação em que o modelo vai bem nos dados de treino e mal em dados novos.

Padronização — transformação dos atributos para média 0 e desvio padrão 1.

Paillier — esquema de criptografia parcialmente homomórfica que permite somar valores cifrados.

Parâmetros (pesos) — números que o modelo ajusta durante o aprendizado.

PCA — análise de componentes principais; encontra as direções de maior variação dos dados.

Per-FedAvg — abordagem de meta-aprendizado que busca um modelo global fácil de adaptar a cada cliente.

Percentil 10 — valor abaixo do qual estão os 10% piores resultados.

Perda (loss) — número que mede o erro de uma previsão.

Personalização — adaptação do modelo ao cliente, por ajuste fino, regularização, partição ou agrupamento.

Precisão — dos exemplos que o modelo chamou de positivos, fração que realmente era positiva.

Privacidade diferencial local — variante em que cada cliente adiciona ruído antes de enviar seus dados ou atualização.

q-FFL — método de justiça que dá mais peso aos clientes com maior perda.

Quantização — representação dos valores com menos bits.

Regularização — técnica que penaliza modelos complexos para reduzir o sobreajuste.

Resposta aleatorizada — técnica de pesquisa em que respostas são sorteadas para proteger cada indivíduo, mas permitem estimar taxas coletivas.

Revocação (recall) — dos exemplos realmente positivos, fração que o modelo encontrou.

Risco empírico — média da perda sobre os exemplos de treino.

Rodada — ver Comunicação (rodada de).

SCAFFOLD — algoritmo que usa variáveis de controle para corrigir a deriva dos clientes.

Sensibilidade — máximo que uma consulta pode mudar quando uma pessoa entra ou sai do conjunto de dados.

Servidor — coordenador que organiza o treino e agrega as contribuições dos clientes.

SGD (gradiente descendente estocástico) — descida do gradiente com mini-lotes sorteados.

Shards — pedaços de dados ordenados por classe, usados para simular dados muito heterogêneos.

Silo de dados — repositório de dados fechado, controlado por um dono.

Smashed data — ativações enviadas pelo cliente na camada de corte do split learning.

SMPC — computação multipartidária segura; várias partes calculam juntas sem revelar suas entradas.

Softmax — função que transforma notas em probabilidades que somam 1; usada na regressão logística multiclasse.

Split learning — aprendizado em que a rede neural é dividida entre cliente e servidor em uma camada de corte.

Taxa de aprendizado — tamanho do passo na descida do gradiente.

TEE — ambiente de execução confiável; área protegida do processador.

Topologia — estrutura de quem se comunica com quem em uma rede.

Transferência federada (FTL) — aprendizado federado com pouca sobreposição de pessoas e atributos, por meio de um espaço comum de representação.

Variável de controle — vetor mantido por servidor e clientes no SCAFFOLD para estimar e corrigir a deriva.

VFL (aprendizado federado vertical) — partição em que os participantes têm as mesmas pessoas e atributos diferentes.

Viés (bias) do modelo — parâmetro de deslocamento somado à nota do modelo.

Viés estatístico — erro sistemático que não se cancela ao fazer médias.

Apêndice C — Guia rápido de algoritmos e códigos

C.1 Mapa de decisão

Se você quer...	Considere	Capítulo
Treinar sem juntar dados horizontais	FedAvg	3
Lidar com dados muito diferentes entre clientes	SCAFFOLD, FedProx, FedNova, otimizadores no servidor	4
Reduzir o tráfego de subida	Quantização, top-k com memória de erro, mais épocas locais	5
Atender bem a cada cliente	Ajuste fino, Ditto, partição corpo e cabeça	6
Atender grupos com conceitos diferentes	IFCA, agrupamento de clientes	6
Entender o que pode vazar	Inversão de gradientes, inferência de pertencimento	7
Garantia formal de privacidade	Privacidade diferencial (corte e ruído)	8
Esconder atualizações individuais	Agregação segura, criptografia homomórfica, SMPC	8
Resistir a clientes maliciosos	Mediana, média aparada, Multi-Krum, corte de norma	9
Combinar colunas de diferentes donos	Aprendizado federado vertical	10
Poucos recursos no cliente, rede neural	Split learning	10
Não ter servidor	Aprendizado descentralizado (gossip)	10
Agrupar ou reduzir dimensões sem rótulos	K-means e PCA federados	11
Especializar modelos grandes	Adaptadores de baixo posto (LoRA)	11

C.2 Arquivos de código do livro

Arquivo	Capítulo	O que contém
`fedlib.py`	3	Dados dos dígitos, partições (IID, shards, Dirichlet), treino local, FedAvg
`scaffold.py`	4	Algoritmo SCAFFOLD
`compressao.py`	5	Quantização, top-k, memória de erro e FedAvg com compressão
`personalizacao.py`	6	Teste local, Ditto, mudança de conceito e IFCA
`privacidade.py`	8	Contabilidade de Rényi e DP-FedAvg
`robustez.py`	9	Agregadores robustos, ataques e FedAvg com maliciosos
`arquiteturas.py`	10	VFL, split learning e consenso descentralizado
`fronteiras.py`	11	K-means e PCA federados e adaptadores de baixo posto
`projeto.py`	12	Motor do projeto final

Os códigos completos de cada arquivo aparecem nos respectivos capítulos. A seguir, a base de todos eles.

C.3 Código completo da biblioteca fedlib.py

PYTHON

```
"""fedlib.py - biblioteca didática do livro 'Aprendizado Federado'.
Simula clientes e servidor em um único computador, usando NumPy e scikit-learn."""
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler


def carregar_digits(seed=0):
    """Dígitos manuscritos da UCI (via scikit-learn), padronizados, com coluna de viés."""
    X, y = load_digits(return_X_y=True)
    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=seed, stratify=y)
    esc = StandardScaler().fit(Xtr)
    mais_um = lambda A: np.hstack([esc.transform(A), np.ones((len(A), 1))])
    return mais_um(Xtr), mais_um(Xte), ytr, yte


def particao_iid(y, k, seed=0):
    """Embaralha tudo e reparte em k clientes de tamanho parecido."""
    idx = np.random.default_rng(seed).permutation(len(y))
    return [np.array(p) for p in np.array_split(idx, k)]


def particao_shards(y, k, por_cliente=2, seed=0):
    """Ordena por classe, corta em k*por_cliente pedaços e sorteia pedaços para cada cliente."""
    rng = np.random.default_rng(seed)
    pedacos = np.array_split(np.argsort(y, kind="stable"), k * por_cliente)
    ordem = rng.permutation(len(pedacos))
    return [np.concatenate([pedacos[ordem[i * por_cliente + j]] for j in range(por_cliente)]) for i in
range(k)]


def particao_dirichlet(y, k, alfa=0.5, seed=0):
    """Cada classe é repartida entre os clientes com proporções sorteadas de uma Dirichlet(alfa)."""
    rng = np.random.default_rng(seed)
    partes = [[] for _ in range(k)]
    for c in np.unique(y):
        ic = rng.permutation(np.where(y == c)[0])
        p = rng.dirichlet([alfa] * k)
        cortes = (np.cumsum(p) * len(ic)).astype(int)[:-1]
        for i, pedaco in enumerate(np.split(ic, cortes)):
            partes[i] += pedaco.tolist()
    return [np.array(p, dtype=int) for p in partes]


def softmax(Z):
    Z = Z - Z.max(axis=1, keepdims=True)
    E = np.exp(Z)
    return E / E.sum(axis=1, keepdims=True)


def gradiente(W, X, y, mu=0.0, W_ref=None):
    """Gradiente da entropia cruzada (regressão logística multiclasse) + termo proximal opcional."""
    P = softmax(X @ W)
    P[np.arange(len(y)), y] -= 1.0
    g = X.T @ P / len(y)
    if mu > 0 and W_ref is not None:
        g = g + mu * (W - W_ref)
```

```
    return g


def treino_local(W, X, y, epocas=1, eta=0.1, lote=32, mu=0.0, seed=0):
    """Treino local do cliente: mini-batch SGD a partir do modelo global W."""
    rng = np.random.default_rng(seed)
    W0, W = W, W.copy()
    for _ in range(epocas):
        ordem = rng.permutation(len(y))
        for i in range(0, len(y), lote):
            b = ordem[i:i + lote]
            W -= eta * gradiente(W, X[b], y[b], mu, W0)
    return W


def acuracia(W, X, y):
    return float(np.mean(np.argmax(X @ W, axis=1) == y))


def perda(W, X, y):
    P = softmax(X @ W)
    return float(-np.mean(np.log(P[np.arange(len(y)), y] + 1e-12)))


def fedavg(partes, X, y, Xte, yte, rodadas=30, epocas=1, eta=0.1, lote=32,
           frac=1.0, mu=0.0, seed=0, W0=None, classes=10):
    """FedAvg: em cada rodada, clientes sorteados treinam localmente e o servidor faz a média
ponderada."""
    rng = np.random.default_rng(seed)
    W = np.zeros((X.shape[1], classes)) if W0 is None else W0.copy()
    historico = []
    for r in range(rodadas):
        k = max(1, int(round(frac * len(partes))))
        escolhidos = rng.choice(len(partes), k, replace=False)
        locais, tamanhos = [], []
        for c in escolhidos:
            if len(partes[c]) == 0:
                continue
            locais.append(treino_local(W, X[partes[c]], y[partes[c]], epocas, eta, lote, mu, seed *
1000 + r * 50 + int(c)))
            tamanhos.append(len(partes[c]))
        p = np.array(tamanhos) / sum(tamanhos)
        W = sum(pi * Wi for pi, Wi in zip(p, locais))
        historico.append(acuracia(W, Xte, yte))
    return W, historico
```

C.4 Modelo de código para um experimento federado

```
PYTHON
import numpy as np
from fedlib import *

# 1) Dados e partição entre clientes
Xtr, Xte, ytr, yte = carregar_digits()
partes = particao_dirichlet(ytr, k=20, alfa=0.3, seed=0)      # clientes heterogêneos
```

```
# 2) Treino federado, com várias sementes
resultados = []
for semente in range(3):
    partes = particao_dirichlet(ytr, 20, 0.3, seed=semente)
    W, hist = fedavg(partes, Xtr, ytr, Xte, yte, rodadas=30, epocas=5, eta=0.1, frac=0.5, seed=semente)
    resultados.append(hist[-1])

# 3) Relato honesto: média e desvio, e não uma única execução
print(f"acurácia final: {np.mean(resultados):.3f} ± {np.std(resultados):.3f}")

# 4) Custo de comunicação
n_param = Xtr.shape[1] * 10
print("bytes por rodada:", 2 * 4 * n_param * int(0.5 * 20))
```

C.5 Lista de verificação final

MEMORIZE **Antes de apresentar seus resultados**

- Os dados foram divididos entre clientes de forma realista (IID e não-IID)?
- Os resultados usam **várias sementes**, com média e desvio?
- A avaliação inclui **por cliente** (média, pior e percentil 10)?
- O **custo de comunicação** foi medido?
- O **modelo de ameaça** está explícito? Há ataque e defesa testados?
- As **limitações** estão escritas (dados simulados, modelo simples, ataque simples)?
- Nenhuma afirmação de privacidade sem **garantia formal** (ε, δ) ou sem ressalvas?
- A parte jurídica e ética foi consultada (LGPD, GDPR, RIPD)?

Apêndice D — Soluções dos exercícios práticos

Cada solução traz o código, a **saída obtida** na elaboração do livro e um comentário sobre o que observar. Há várias maneiras corretas de resolver um exercício: o importante é que o seu resultado seja coerente e que você consiga explicá-lo. Pequenas diferenças numéricas podem ocorrer com outras versões das bibliotecas. Os gráficos aparecem quando você executa o código.

Capítulo 1 — Bem-vindo ao Aprendizado Federado

Exercício 1 — Conhecendo os dados dos "hospitais"

PYTHON

```python
import numpy as np
from sklearn.datasets import load_breast_cancer

dados = load_breast_cancer()
X, y = dados.data, dados.target
print("Formato de X:", X.shape)
print("Classes:", list(map(str, dados.target_names)), "->", np.bincount(y))

rng = np.random.default_rng(1)
idx = rng.permutation(len(y))
hospitais = np.array_split(idx[:180], 3)
for i, h in enumerate(hospitais, 1):
    cont = np.bincount(y[h], minlength=2)
    print(f"Hospital {i}: {len(h)} pacientes | malignos = {cont[0]}, benignos = {cont[1]}")
```

SAÍDA

```
Formato de X: (569, 30)
Classes: ['malignant', 'benign'] -> [212 357]
Hospital 1: 60 pacientes | malignos = 23, benignos = 37
Hospital 2: 60 pacientes | malignos = 22, benignos = 38
Hospital 3: 60 pacientes | malignos = 21, benignos = 39
```

EXEMPLO **O que observar**

Como o sorteio foi aleatório, as proporções são parecidas entre os hospitais (dados IID). Em um cenário real, um hospital de referência em oncologia teria muito mais casos malignos. Essa diferença é a heterogeneidade não-IID, tema do Capítulo 4.

Exercício 2 — Quanto o tamanho do hospital importa?

PYTHON

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
esc = StandardScaler().fit(Xtr)
Xtr, Xte = esc.transform(Xtr), esc.transform(Xte)

def media_de_modelos(modelos):
    m = LogisticRegression(max_iter=1000)
    m.coef_ = np.mean([k.coef_ for k in modelos], axis=0)
    m.intercept_ = np.mean([k.intercept_ for k in modelos], axis=0)
    m.classes_ = modelos[0].classes_
```

```
    return m

rng = np.random.default_rng(0)
idx = rng.permutation(len(ytr))
print("pacientes/hospital | local (média) | colaborando (média dos modelos)")
for n in [10, 25, 50, 100]:
    partes = [idx[i * n:(i + 1) * n] for i in range(3)]
    locais = [LogisticRegression(max_iter=1000).fit(Xtr[p], ytr[p]) for p in partes]
    acc_local = np.mean([m.score(Xte, yte) for m in locais])
    acc_fed = media_de_modelos(locais).score(Xte, yte)
    print(f"{n:>18} | {acc_local:13.3f} | {acc_fed:.3f}")
```

SAÍDA

```
pacientes/hospital | local (média) | colaborando (média dos modelos)
                10 |         0.871 | 0.912
                25 |         0.914 | 0.918
                50 |         0.942 | 0.942
               100 |         0.959 | 0.947
```

EXEMPLO **O que observar**

Com 10 pacientes por hospital, a colaboração traz um ganho claro (cerca de 4 pontos percentuais). Com 25 e 50, a diferença some. Com 100, a média dos modelos até fica um pouco abaixo do local médio: a média de modelos treinados uma só vez não é igual a treinar com todos os dados. Colaborar vale mais para quem tem pouco dado, e o ganho depende do problema.

Exercício 3 — Repetindo para não se enganar com o acaso

PYTHON

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
esc = StandardScaler().fit(Xtr)
Xtr, Xte = esc.transform(Xtr), esc.transform(Xte)

def media_de_modelos(modelos):
    m = LogisticRegression(max_iter=1000)
    m.coef_ = np.mean([k.coef_ for k in modelos], axis=0)
    m.intercept_ = np.mean([k.intercept_ for k in modelos], axis=0)
    m.classes_ = modelos[0].classes_
    return m

loc, fed, cen = [], [], []
for semente in range(30):
    rng = np.random.default_rng(semente)
    idx = rng.permutation(len(ytr))[:150]
    partes = np.array_split(idx, 3)
    modelos = [LogisticRegression(max_iter=1000).fit(Xtr[p], ytr[p]) for p in partes]
    loc.append(np.mean([m.score(Xte, yte) for m in modelos]))
```

```
    fed.append(media_de_modelos(modelos).score(Xte, yte))
    cen.append(LogisticRegression(max_iter=1000).fit(Xtr[idx], ytr[idx]).score(Xte, yte))

for nome, v in [("local", loc), ("média dos modelos", fed), ("centralizado", cen)]:
    print(f"{nome:<18}: {np.mean(v):.3f} ± {np.std(v):.3f}")
```

SAÍDA

```
local             : 0.939 ± 0.009
média dos modelos : 0.950 ± 0.010
centralizado      : 0.954 ± 0.010
```

EXEMPLO **O que observar**

Em média, a média dos modelos fica acima do local e muito próxima do centralizado. O desvio padrão mostra que o resultado varia com o sorteio, e por isso a repetição é importante.

Capítulo 2 — Preparando o Terreno

Exercício 1 — Taxa de aprendizado em ação

PYTHON

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
esc = StandardScaler().fit(Xtr)
Xtr = esc.transform(Xtr)

def sigmoide(z):
    return 1 / (1 + np.exp(-np.clip(z, -50, 50)))

def treinar(eta, passos=100):
    w, b = np.zeros(Xtr.shape[1]), 0.0
    for _ in range(passos):
        p = sigmoide(Xtr @ w + b)
        erro = p - ytr
        w, b = w - eta * (Xtr.T @ erro / len(ytr)), b - eta * erro.mean()
    p = sigmoide(Xtr @ w + b)
    return -np.mean(ytr * np.log(p + 1e-12) + (1 - ytr) * np.log(1 - p + 1e-12))

for eta in [0.001, 0.1, 10, 1000]:
    print(f"η = {eta:<6} | perda final = {treinar(eta):.4f}")
```

SAÍDA

```
η = 0.001  | perda final = 0.5391
η = 0.1    | perda final = 0.0943
η = 10     | perda final = 0.0441
η = 1000   | perda final = 0.3194
```

EXEMPLO **O que observar**

Com η = 0,001 a perda quase não sai do lugar (passos minúsculos). Com 0,1 e 10, ela cai bem: dados padronizados e perda logística tolerantes permitem passos grandes. Com η = 1000, os passos ultrapassam o vale e a perda fica alta e instável. Cada problema tem sua faixa boa: teste valores em escala de potências de 10.

Exercício 2 — Gradiente completo versus mini-lotes

PYTHON

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
```

```
esc = StandardScaler().fit(Xtr)
Xtr, Xte = esc.transform(Xtr), esc.transform(Xte)

sig = lambda z: 1 / (1 + np.exp(-np.clip(z, -50, 50)))
def perda(w, b):
    p = sig(Xtr @ w + b)
    return -np.mean(ytr * np.log(p + 1e-12) + (1 - ytr) * np.log(1 - p + 1e-12))
def acc(w, b):
    return np.mean((sig(Xte @ w + b) > 0.5) == yte)

eta, d = 0.05, Xtr.shape[1]

# gradiente completo: 1 passo por época
w, b = np.zeros(d), 0.0
for _ in range(5):
    e = sig(Xtr @ w + b) - ytr
    w, b = w - eta * Xtr.T @ e / len(ytr), b - eta * e.mean()
print(f"completo (5 passos)  : perda = {perda(w, b):.4f} | acurácia = {acc(w, b):.3f}")

# SGD: muitos passos por época
rng = np.random.default_rng(0)
w, b = np.zeros(d), 0.0
for _ in range(5):
    ordem = rng.permutation(len(ytr))
    for i in range(0, len(ytr), 16):
        lote = ordem[i:i + 16]
        e = sig(Xtr[lote] @ w + b) - ytr[lote]
        w, b = w - eta * Xtr[lote].T @ e / len(lote), b - eta * e.mean()
print(f"SGD (5 épocas)       : perda = {perda(w, b):.4f} | acurácia = {acc(w, b):.3f}")
```

SAÍDA

```
completo (5 passos)  : perda = 0.4099 | acurácia = 0.912
SGD (5 épocas)       : perda = 0.1115 | acurácia = 0.942
```

EXEMPLO **O que observar**

Em 5 épocas, o SGD dá cerca de 125 atualizações (398 exemplos / 16 por lote, 5 vezes), contra 5 do gradiente completo. Por isso avança muito mais por época de leitura dos dados.

Exercício 3 — Acurácia que engana

PYTHON

```
from sklearn.datasets import make_classification
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=2000, n_features=10, n_informative=6, n_redundant=0,
                           weights=[0.97, 0.03], class_sep=2.0, flip_y=0.01, random_state=0)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)

modelos = {
    "ingênuo": DummyClassifier(strategy="most_frequent"),
    "logística": LogisticRegression(max_iter=1000),
```

```
    "balanceada": LogisticRegression(max_iter=1000, class_weight="balanced"),
}
for nome, m in modelos.items():
    pred = m.fit(Xtr, ytr).predict(Xte)
    print(f"{nome:<11} acc = {accuracy_score(yte, pred):.3f} | prec = {precision_score(yte, pred,
zero_division=0):.3f} | "
          f"rev = {recall_score(yte, pred):.3f} | F1 = {f1_score(yte, pred):.3f}")
print("Matriz de confusão (logística):")
print(confusion_matrix(yte, modelos["logística"].predict(Xte)))
```

SAÍDA

```
ingênuo     acc = 0.963 | prec = 0.000 | rev = 0.000 | F1 = 0.000
logística   acc = 0.987 | prec = 1.000 | rev = 0.636 | F1 = 0.778
balanceada  acc = 0.918 | prec = 0.286 | rev = 0.818 | F1 = 0.424
Matriz de confusão (logística):
[[578   0]
 [  8  14]]
```

EXEMPLO **O que observar**

O modelo ingênuo tem acurácia de cerca de 96%, mas revocação e F1 iguais a zero: nunca acha um caso positivo. A logística comum tem acurácia maior, mas ainda deixa passar cerca de um terço dos positivos. Com `class_weight="balanced"`, a revocação sobe, ao custo de mais falsos alarmes e acurácia menor. Não existe escolha sem custo: depende de quanto custa perder um positivo.

Exercício 4 — Padronização federada com clientes de tamanhos diferentes

PYTHON

```
import numpy as np
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X, y = load_wine(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)

rng = np.random.default_rng(0)
idx = rng.permutation(len(Xtr))
clientes = np.split(idx, [10, 30, 60, 90])          # tamanhos desiguais
print("tamanhos:", [len(c) for c in clientes])

resumos = [(len(c), Xtr[c].sum(axis=0), (Xtr[c] ** 2).sum(axis=0)) for c in clientes]
n = sum(r[0] for r in resumos)
mu = sum(r[1] for r in resumos) / n
sigma = np.sqrt(sum(r[2] for r in resumos) / n - mu ** 2)

ref = StandardScaler().fit(Xtr)
print("erro na média  (somas)         :", np.abs(mu - ref.mean_).max())
print("erro no desvio (somas)         :", np.abs(sigma - ref.scale_).max())

media_errada = np.mean([Xtr[c].mean(axis=0) for c in clientes], axis=0)
print("erro na média  (média de médias):", np.abs(media_errada - ref.mean_).max())
```

SAÍDA

```
tamanhos: [10, 20, 30, 30, 34]
erro na média  (somas)         : 5.329070518200751e-15
erro no desvio (somas)         : 4.618527782440651e-14
erro na média  (média de médias): 5.401992409867148
```

EXEMPLO O que observar

Com somas, o erro é da ordem do arredondamento. A média simples das médias dá peso igual a todos os clientes, ignorando que alguns têm muito mais dados, e resulta em erro visível. É o mesmo motivo da ponderação por n_k/n na agregação do FedAvg.

Capítulo 3 — Anatomia de um Sistema Federado

Exercício 1 — Sua primeira agregação à mão

PYTHON

```python
import numpy as np

W = np.array([[0.8, -1.0],
              [0.6,  0.0],
              [0.5,  2.0]])
n = np.array([100, 300, 600])

pesos = n / n.sum()
ponderada = (pesos[:, None] * W).sum(axis=0)
simples = W.mean(axis=0)

print("pesos          :", pesos)
print("média ponderada:", ponderada)
print("média simples  :", simples)
print("distância ao cliente 3 (ponderada):", np.linalg.norm(ponderada - W[2]).round(3))
print("distância ao cliente 3 (simples)  :", np.linalg.norm(simples - W[2]).round(3))
```

SAÍDA

```
pesos          : [0.1 0.3 0.6]
média ponderada: [0.56 1.1 ]
média simples  : [0.63333333 0.33333333]
distância ao cliente 3 (ponderada): 0.902
distância ao cliente 3 (simples)  : 1.672
```

EXEMPLO **O que observar**

O resultado ponderado fica perto do cliente 3, que tem 60% dos dados. A média simples ignora o tamanho e fica mais afastada dele.

Exercício 2 — Reproduzindo a curva do FedAvg

PYTHON

```python
import numpy as np
from fedlib import *

Xtr, Xte, ytr, yte = carregar_digits()
finais = []
for semente in (0, 1, 2):
    partes = particao_iid(ytr, 10, seed=semente)
    W, hist = fedavg(partes, Xtr, ytr, Xte, yte, rodadas=30, epocas=5, eta=0.1, lote=32, seed=semente)
    finais.append(hist[-1])
    print(f"semente {semente}: ", [round(hist[r - 1], 3) for r in (1, 5, 10, 20, 30)])
print(f"acurácia final: {np.mean(finais):.3f} ± {np.std(finais):.3f}")
```

SAÍDA

```
semente 0:  [0.909, 0.947, 0.958, 0.958, 0.962]
```

```
semente 1:  [0.896, 0.949, 0.956, 0.958, 0.962]
semente 2:  [0.909, 0.94, 0.958, 0.96, 0.967]
acurácia final: 0.964 ± 0.002
```

EXEMPLO O que observar

A variação é pequena (da ordem de meio ponto percentual): com dados IID e modelo simples, o resultado é estável. Mesmo assim, é uma boa prática relatar média e desvio sobre várias sementes.

Exercício 3 — Variando E e η

PYTHON

```python
from fedlib import *

Xtr, Xte, ytr, yte = carregar_digits()
partes = particao_iid(ytr, 10, seed=0)
epocas_lista, etas = [1, 5, 20], [0.01, 0.1, 0.5]

print("E \\ eta " + "".join(f"{e:>8}" for e in etas))
for E in epocas_lista:
    linha = []
    for eta in etas:
        _, hist = fedavg(partes, Xtr, ytr, Xte, yte, rodadas=15, epocas=E, eta=eta, lote=32)
        linha.append(hist[-1])
    print(f"{E:>7} " + "".join(f"{a:8.3f}" for a in linha))
```

SAÍDA

```
E \ eta     0.01     0.1     0.5
      1    0.869   0.931   0.958
      5    0.927   0.958   0.969
     20    0.947   0.967   0.973
```

EXEMPLO O que observar

Passos pequenos com poucas épocas ficam para trás; combinações com E e η maiores chegam rápido a valores altos. O custo computacional local cresce com E: a partir de certo ponto, mais épocas rendem pouco, e a computação extra é desperdiçada. O objetivo é equilibrar rodadas (comunicação) e computação local.

Exercício 4 — Calculando o custo de comunicação

PYTHON

```python
from fedlib import *

Xtr, Xte, ytr, yte = carregar_digits()
n_param = Xtr.shape[1] * 10          # 65 atributos (com viés) x 10 classes

def bytes_por_rodada(n_parametros, n_clientes, bits=32):
    return 2 * (bits // 8) * n_parametros * n_clientes

for bits in (32, 16):
    por_rodada = bytes_por_rodada(n_param, 10, bits)
```

```
    print(f"{bits} bits: {por_rodada:,} bytes por rodada | 30 rodadas = {30 * por_rodada / 1024:.1f}
KB")

limite = 10 * 1024 * 1024
print("rodadas em 10 MB (32 bits):", limite // bytes_por_rodada(n_param, 10, 32))
```

SAÍDA

```
32 bits: 52,000 bytes por rodada | 30 rodadas = 1523.4 KB
16 bits: 26,000 bytes por rodada | 30 rodadas = 761.7 KB
rodadas em 10 MB (32 bits): 201
```

EXEMPLO **O que observar**

Com 650 parâmetros, o tráfego é de 52.000 bytes por rodada, e 30 rodadas somam cerca de 1.523 KB. Reduzir para 16 bits corta o tráfego pela metade. Em 10 MB cabem cerca de 201 rodadas. Para modelos grandes, as contas ficam bem mais exigentes.

Capítulo 4 — Quando os Dados Não São Iguais

Exercício 1 — Medindo a heterogeneidade de rótulos

PYTHON

```python
import numpy as np
from fedlib import *

Xtr, Xte, ytr, yte = carregar_digits()
global_p = np.bincount(ytr, minlength=10) / len(ytr)

def tv_medio(partes):
    tvs = []
    for p in partes:
        if len(p) == 0:
            continue
        pk = np.bincount(ytr[p], minlength=10) / len(p)
        tvs.append(0.5 * np.abs(pk - global_p).sum())
    return np.mean(tvs)

for alfa in (100, 1.0, 0.1):
    partes = particao_dirichlet(ytr, 10, alfa, seed=0)
    print(f"α = {alfa:>5}: TV médio = {tv_medio(partes):.3f}")
print(f"shards (1 classe): TV médio = {tv_medio(particao_shards(ytr, 10, 1, seed=0)):.3f}")
print(f"IID              : TV médio = {tv_medio(particao_iid(ytr, 10, seed=0)):.3f}")
```

SAÍDA

```
α =   100: TV médio = 0.035
α =   1.0: TV médio = 0.312
α =   0.1: TV médio = 0.666
shards (1 classe): TV médio = 0.890
IID              : TV médio = 0.088
```

EXEMPLO **O que observar**

A distância cresce à medida que α diminui: cerca de 0,04 com α = 100, 0,31 com α = 1 e 0,67 com α = 0,1. Mesmo no IID o valor não é zero (cerca de 0,09), por causa da amostragem finita: com ~135 exemplos por cliente, as proporções flutuam ao acaso. Para shards de uma classe, o valor chega a cerca de 0,89, pois cada cliente tem só uma das 10 classes.

Exercício 2 — Épocas locais e heterogeneidade

PYTHON

```python
from fedlib import *

Xtr, Xte, ytr, yte = carregar_digits()
cenarios = {"IID": particao_iid(ytr, 10, seed=0),
            "shards (2 classes)": particao_shards(ytr, 10, por_cliente=2, seed=0)}
print(f"{'cenário':<20}{'E':>4}{'rodada 5':>10}{'rodada 30':>11}")
for nome, partes in cenarios.items():
    for E in (1, 5, 20):
        _, h = fedavg(partes, Xtr, ytr, Xte, yte, rodadas=30, epocas=E, eta=0.1, lote=32, frac=0.3,
```

```
seed=0)
        print(f"{nome:<20}{E:>4}{h[4]:>10.3f}{h[-1]:>11.3f}")
```

SAÍDA

```
cenário               E  rodada 5  rodada 30
IID                   1     0.918      0.951
IID                   5     0.949      0.964
IID                  20     0.953      0.971
shards (2 classes)    1     0.693      0.951
shards (2 classes)    5     0.747      0.967
shards (2 classes)   20     0.762      0.962
```

EXEMPLO **O que observar**

A heterogeneidade atrasa o início: na rodada 5, o IID está em 0,92 a 0,95 e os shards de 2 classes em 0,69 a 0,76. Ao fim de 30 rodadas, porém, os dois cenários ficam equivalentes (cerca de 0,95 a 0,97). Mais épocas locais ajudam bastante no início; no final, E = 20 não superou E = 5. Neste problema simples, a heterogeneidade moderada é bem tolerada. A deriva domina com heterogeneidade extrema e poucos clientes por rodada, como na Figura 4.4. Experimente `por_cliente=1` e `frac=0.1`.

Exercício 3 — Heterogeneidade de quantidade: ponderar ou não?

PYTHON

```python
import numpy as np
from fedlib import *

Xtr, Xte, ytr, yte = carregar_digits()

def partes_desiguais(y, k, seed):
    rng = np.random.default_rng(seed)
    tam = rng.lognormal(0, 1, k)
    cortes = (np.cumsum(tam) / tam.sum() * len(y)).astype(int)[:-1]
    return [np.array(p) for p in np.split(rng.permutation(len(y)), cortes)]

def fedavg_simples(partes, X, y, Xte, yte, rodadas=30, epocas=5, eta=0.1, lote=32, seed=0):
    W = np.zeros((X.shape[1], 10))
    for r in range(rodadas):
        locais = [treino_local(W, X[p], y[p], epocas, eta, lote, 0.0, seed * 1000 + r * 50 + i)
                  for i, p in enumerate(partes) if len(p) > 0]
        W = np.mean(locais, axis=0)              # média simples
    return acuracia(W, Xte, yte)

pond, simp = [], []
for s in range(3):
    partes = partes_desiguais(ytr, 10, s)
    if s == 0:
        print("tamanhos (semente 0):", sorted(len(p) for p in partes))
    pond.append(fedavg(partes, Xtr, ytr, Xte, yte, rodadas=30, epocas=5, eta=0.1, seed=s)[1][-1])
    simp.append(fedavg_simples(partes, Xtr, ytr, Xte, yte, seed=s))
print(f"ponderada: {np.mean(pond):.3f} ± {np.std(pond):.3f}")
print(f"simples  : {np.mean(simp):.3f} ± {np.std(simp):.3f}")
```

SAÍDA

```
tamanhos (semente 0): [27, 48, 56, 84, 107, 108, 137, 181, 247, 352]
ponderada: 0.968 ± 0.002
simples  : 0.964 ± 0.002
```

EXEMPLO O que observar

Na média simples, um cliente minúsculo tem o mesmo peso que um gigante, e seu modelo, treinado com pouquíssimos dados, é ruidoso. Isso tende a piorar o modelo global. A ponderação por n_k evita essa distorção. A diferença depende dos tamanhos sorteados.

Exercício 4 — Quanto vale a "coleira" do FedProx?

PYTHON

```python
import numpy as np
from fedlib import *

Xtr, Xte, ytr, yte = carregar_digits()
print(f"{'mu':>6}{'rodada 10':>11}{'rodada 40':>11}")
for mu in (0, 0.01, 0.1, 1, 10):
    h10, h40 = [], []
    for s in range(3):
        partes = particao_dirichlet(ytr, 20, 0.1, seed=s)
        _, h = fedavg(partes, Xtr, ytr, Xte, yte, rodadas=40, epocas=10, eta=0.1, frac=0.25, mu=mu,
seed=s)
        h10.append(h[9]); h40.append(h[-1])
    print(f"{mu:>6}{np.mean(h10):>11.3f}{np.mean(h40):>11.3f}")
```

SAÍDA

```
    mu  rodada 10  rodada 40
     0      0.924      0.959
  0.01      0.923      0.959
   0.1      0.917      0.959
     1      0.883      0.947
    10      0.759      0.877
```

EXEMPLO O que observar

Valores pequenos de μ (até 0,1) ficam praticamente iguais ao FedAvg. Com μ = 1 e, sobretudo, com μ = 10, o progresso é freado: o cliente fica preso ao modelo global e quase não aprende localmente, e a acurácia cai. Aqui não houve ganho com a coleira: o resultado depende do problema.

Capítulo 5 — Falando Menos

Exercício 1 — Viés do arredondamento: determinístico contra estocástico

PYTHON

```python
import numpy as np
from compressao import *

rng = np.random.default_rng(0)
valor, n = 0.3, 20000

# vetor com 0 e 1 fixa o intervalo [0, 1]; o primeiro elemento é o valor de interesse
est = []
for _ in range(n):
    v = np.array([valor, 0.0, 1.0])
    est.append(quantizar(v, 2, rng)[0])
est = np.array(est)

niveis = np.array([0, 1/3, 2/3, 1])
det = niveis[np.argmin(np.abs(niveis - valor))]          # mais próximo, sempre o mesmo

print(f"valor original            : {valor}")
print(f"arredondamento determinístico: {det:.4f} (erro fixo = {det - valor:+.4f})")
print(f"arredondamento estocástico   : média = {est.mean():.4f} | níveis usados = {[float(x) for x in
sorted(set(np.round(est, 3)))]}")
```

SAÍDA

```
valor original            : 0.3
arredondamento determinístico: 0.3333 (erro fixo = +0.0333)
arredondamento estocástico   : média = 0.2997 | níveis usados = [0.0, 0.333]
```

EXEMPLO **O que observar**

O determinístico devolve sempre 0,3333: erro de +0,0333 que **nunca** se cancela. O estocástico sorteia entre 0 e 1/3, e a média fica muito próxima de 0,3. Ao somar clientes cujo valor real é parecido, o erro sistemático do determinístico se acumula, enquanto o estocástico se cancela.

Exercício 2 — Calculadora de tempo por rodada

PYTHON

```python
def tempo_rodada(n_parametros, bits_subida, desc_mbps, sub_mbps, t_comp):
    t_desc = n_parametros * 32 / 1e6 / desc_mbps
    t_sub = n_parametros * bits_subida / 1e6 / sub_mbps
    return t_comp + t_desc + t_sub, t_desc, t_sub

print(f"{'bits':>5}{'subida (s)':>12}{'rodada (s)':>12}{'500 rodadas (h)':>17}")
for bits in (32, 8, 4, 2):
    total, t_desc, t_sub = tempo_rodada(5_000_000, bits, 20, 5, 30)
    print(f"{bits:>5}{t_sub:>12.1f}{total:>12.1f}{500 * total / 3600:>17.2f}")
```

SAÍDA

```
 bits  subida (s)  rodada (s)  500 rodadas (h)
   32        32.0        70.0             9.72
    8         8.0        46.0             6.39
    4         4.0        42.0             5.83
    2         2.0        40.0             5.56
```

EXEMPLO O que observar

Passar de 32 para 8 bits reduz muito o tempo (a subida cai de 32 s para 8 s). De 8 para 2 bits, o ganho absoluto é menor, pois o tempo de treino (30 s) e de descida (8 s) passam a dominar. É a lei dos retornos decrescentes: comprimir só um trecho do ciclo.

Exercício 3 — Esparsificação com e sem memória de erro

PYTHON

```python
import numpy as np
from compressao import *

Xtr, Xte, ytr, yte = carregar_digits()
print(f"{'fração':>7}{'sem EF':>9}{'com EF':>9}{'KB':>7}")
for frac in (0.5, 0.1, 0.05, 0.01):
    accs = {False: [], True: []}
    kb = 0
    for ef in (False, True):
        for s in range(3):
            partes = particao_dirichlet(ytr, 20, 1.0, seed=s)
            _, h, t = fedavg_comprimido(partes, Xtr, ytr, Xte, yte, metodo="topk", param=frac,
                                        erro_acumulado=ef, seed=s)
            accs[ef].append(h[-1]); kb = t / 1024
    print(f"{frac:>7}{np.mean(accs[False]):>9.3f}{np.mean(accs[True]):>9.3f}{kb:>7.0f}")
```

SAÍDA

```
 fração   sem EF   com EF     KB
    0.5    0.960    0.959    571
    0.1    0.952    0.959    114
   0.05    0.940    0.954     56
   0.01    0.867    0.941     11
```

EXEMPLO O que observar

Com 50%, o corte é leve e a memória de erro é irrelevante (0,960 contra 0,959). Já com 10% ela começa a ajudar (0,952 para 0,959), e com 5% e 1% a diferença é grande: com 1%, a acurácia sobe de 0,867 para 0,941 usando os mesmos 11 KB. O tráfego (KB) é o total de subida em cada fração, igual com e sem memória.

Exercício 4 — Quantização com dados muito heterogêneos

PYTHON

```python
import numpy as np
from compressao import *

Xtr, Xte, ytr, yte = carregar_digits()
```

```
configs = [("32 bits", dict()), ("8 bits", dict(metodo="quantizar", param=8)), ("4 bits",
dict(metodo="quantizar", param=4)),
           ("2 bits", dict(metodo="quantizar", param=2)), ("1 bit", dict(metodo="quantizar", param=1))]
print(f"{'método':<9}{'IID':>8}{'1 classe/cliente':>18}")
for nome, kw in configs:
    res = {}
    for cen, mk in (("iid", lambda s: particao_iid(ytr, 20, s)),
                    ("shard", lambda s: particao_shards(ytr, 20, 1, s))):
        accs = []
        for s in range(3):
            _, h, _ = fedavg_comprimido(mk(s), Xtr, ytr, Xte, yte, rodadas=40, epocas=5, frac=0.25,
seed=s, **kw)
            accs.append(h[-1])
        res[cen] = np.mean(accs)
    print(f"{nome:<9}{res['iid']:>8.3f}{res['shard']:>18.3f}")
```

SAÍDA

```
método        IID  1 classe/cliente
32 bits     0.961             0.950
8 bits      0.962             0.964
4 bits      0.962             0.960
2 bits      0.958             0.939
1 bit       0.910             0.687
```

EXEMPLO O que observar

Com 8 e 4 bits, a quantização não prejudica nem no cenário muito heterogêneo (0,964 e 0,960). Com 2 bits, a perda é pequena no IID (0,958) e maior com uma classe por cliente (0,939). Com 1 bit, o efeito é dramático: de 0,910 no IID para 0,687 com uma classe por cliente. Quanto mais heterogêneos os dados, menos tolerante o treino é à compressão agressiva, pois cada cliente puxa o modelo em uma direção diferente.

Capítulo 6 — Cada Um do Seu Jeito

Exercício 1 — Avaliando o modelo global cliente a cliente

PYTHON

```python
import numpy as np
from personalizacao import *

Xtr, Xte, ytr, yte = carregar_digits()
partes = particao_dirichlet(ytr, 10, 0.3, seed=0)
treino, teste = dividir_local(partes, 0.3, seed=0)
ok = [i for i in range(10) if len(teste[i]) >= 5]
treino = [treino[i] for i in ok]; teste = [teste[i] for i in ok]

Wg, hist = fedavg(treino, Xtr, ytr, Xte, yte, rodadas=30, epocas=5, eta=0.1, seed=0)
por_cliente = acuracias_locais([Wg] * len(treino), teste, Xtr, ytr)
print(f"acurácia no teste global: {hist[-1]:.3f}")
print("por cliente:", np.round(por_cliente, 3))
print(resumo(por_cliente))
```

SAÍDA

```
acurácia no teste global: 0.964
por cliente: [0.889 1.    1.    0.957 0.952 0.95  0.917 0.981 1.    1.   ]
média = 0.964 | pior cliente = 0.889 | percentil 10 = 0.914
```

EXEMPLO **O que observar**

A acurácia global é uma média sobre todos os exemplos, e não sobre clientes. A lista por cliente mostra que há clientes acima e abaixo dessa média. Quanto mais heterogêneos os dados, maior a dispersão entre clientes.

Exercício 2 — Quantas épocas de ajuste fino?

PYTHON

```python
import numpy as np
from personalizacao import *

Xtr, Xte, ytr, yte = carregar_digits()
R = {}
for s in range(3):
    partes = particao_dirichlet(ytr, 10, 0.1, s)
    treino, teste = dividir_local(partes, 0.3, s)
    ok = [i for i in range(10) if len(teste[i]) >= 5]
    treino = [treino[i] for i in ok]; teste = [teste[i] for i in ok]
    Wg, _ = fedavg(treino, Xtr, ytr, Xte, yte, rodadas=30, epocas=5, eta=0.1, seed=s)
    for E in (0, 1, 3, 5, 10, 30):
        modelos = [Wg] * len(treino) if E == 0 else \
                  [treino_local(Wg, Xtr[p], ytr[p], E, 0.1, 32, 0.0, s + i) for i, p in
enumerate(treino)]
        R.setdefault(E, []).append(acuracias_locais(modelos, teste, Xtr, ytr))
for E, v in R.items():
    print(f"E = {E:>2}:", resumo(np.concatenate(v)))
```

SAÍDA

```
E =  0: média = 0.954 | pior cliente = 0.818 | percentil 10 = 0.893
E =  1: média = 0.965 | pior cliente = 0.818 | percentil 10 = 0.921
E =  3: média = 0.973 | pior cliente = 0.818 | percentil 10 = 0.941
E =  5: média = 0.973 | pior cliente = 0.841 | percentil 10 = 0.941
E = 10: média = 0.980 | pior cliente = 0.886 | percentil 10 = 0.952
E = 30: média = 0.977 | pior cliente = 0.833 | percentil 10 = 0.950
```

EXEMPLO O que observar

Poucas épocas já trazem ganho grande em relação ao global puro (E = 0). O ganho se estabiliza por volta de 3 a 10 épocas. Com 30 épocas, o resultado não melhora e o pior cliente piora: sinal de sobreajuste. Mais passos nem sempre ajudam.

Exercício 3 — O botão λ do Ditto

PYTHON

```python
import numpy as np
from personalizacao import *

Xtr, Xte, ytr, yte = carregar_digits()
R = {}
for s in range(3):
    partes = particao_dirichlet(ytr, 10, 0.1, s)
    treino, teste = dividir_local(partes, 0.3, s)
    ok = [i for i in range(10) if len(teste[i]) >= 5]
    treino = [treino[i] for i in ok]; teste = [teste[i] for i in ok]
    Wg, _ = fedavg(treino, Xtr, ytr, Xte, yte, rodadas=30, epocas=5, eta=0.1, seed=s)
    R.setdefault("global", []).append(acuracias_locais([Wg] * len(treino), teste, Xtr, ytr))
    for lam in (0, 0.01, 0.1, 0.5, 2, 10):
        R.setdefault(f"λ = {lam}", []).append(acuracias_locais(ditto(treino, Xtr, ytr, Wg, lam,
seed=s), teste, Xtr, ytr))
for nome, v in R.items():
    print(f"{nome:<10}", resumo(np.concatenate(v)))
```

SAÍDA

```
global     média = 0.954 | pior cliente = 0.818 | percentil 10 = 0.893
λ = 0      média = 0.973 | pior cliente = 0.841 | percentil 10 = 0.941
λ = 0.01   média = 0.973 | pior cliente = 0.841 | percentil 10 = 0.941
λ = 0.1    média = 0.973 | pior cliente = 0.841 | percentil 10 = 0.941
λ = 0.5    média = 0.971 | pior cliente = 0.818 | percentil 10 = 0.941
λ = 2      média = 0.966 | pior cliente = 0.818 | percentil 10 = 0.930
λ = 10     média = 0.961 | pior cliente = 0.818 | percentil 10 = 0.904
```

EXEMPLO O que observar

Com λ ≤ 0,1, os resultados são idênticos ao ajuste fino (o modelo pessoal começa no global e a coleira quase não age). Com λ = 10, o modelo pessoal fica preso ao global e a média e o percentil 10 caem em direção aos do global. Os valores intermediários (0,5 a 2) equilibram, mas neste problema o melhor resultado foi com coleira fraca.

Exercício 4 — IFCA com o número errado de grupos

PYTHON

```python
import numpy as np
from personalizacao import *

Xtr, Xte, ytr0, yte0 = carregar_digits()
print(f"{'K':>3}{'acurácia média':>16}{'modelos usados (por semente)':>32}")
for K in (2, 3, 5):
    accs, usados = [], []
    for s in range(3):
        partes, y, grupo = particao_conceitos(ytr0, 12, 3, s)
        treino, teste = dividir_local(partes, 0.3, s)
        modelos, escolha = ifca(treino, Xtr, y, K=K, seed=s)
        accs.append(acuracias_locais([modelos[j] for j in escolha], teste, Xtr, y).mean())
        usados.append(len(set(escolha)))
    print(f"{K:>3}{np.mean(accs):>16.3f}{str(usados):>32}")
```

SAÍDA

```
  K  acurácia média    modelos usados (por semente)
  2           0.641                      [2, 2, 2]
  3           0.926                      [3, 3, 3]
  5           0.922                      [4, 3, 3]
```

EXEMPLO **O que observar**

Com K = 2, dois grupos verdadeiros precisam compartilhar um modelo, e a acurácia média despenca (cerca de 0,64). Com K = 3, os grupos são recuperados (cerca de 0,93). Com K = 5, sobram modelos: alguns ficam sem clientes e o resultado é próximo do de K = 3. Errar para menos é bem pior do que errar para mais.

Capítulo 7 — O Que Pode Vazar

Exercício 1 — Reconstruindo cinco dígitos, um por vez

PYTHON

```python
import numpy as np
from fedlib import *

Xtr, Xte, ytr, yte = carregar_digits()
rng = np.random.default_rng(1)

def reconstruir(G):
    gb = G[-1]
    c = int(np.argmax(np.abs(gb)))
    return G[:-1, c] / gb[c], c

Wtreinado, _ = fedavg(particao_iid(ytr, 10), Xtr, ytr, Xte, yte, rodadas=10, epocas=2, eta=0.1)
modelos = {"treinado": Wtreinado, "zeros (sem treino)": np.zeros_like(Wtreinado)}
for nome, W in modelos.items():
    print(f"modelo {nome}:")
    for i in rng.choice(len(ytr), 5, replace=False):
        G = gradiente(W, Xtr[i:i + 1], ytr[i:i + 1])
        xr, c = reconstruir(G)
        print(f"   exemplo {i:>4} (dígito {ytr[i]}) | coluna usada = {c} | erro máx. = {np.abs(xr -
Xtr[i, :-1]).max():.2e}")
```

SAÍDA

```
modelo treinado:
   exemplo  687 (dígito 5) | coluna usada = 5 | erro máx. = 2.22e-16
   exemplo   46 (dígito 5) | coluna usada = 5 | erro máx. = 1.11e-16
   exemplo 1015 (dígito 8) | coluna usada = 8 | erro máx. = 1.11e-16
   exemplo 1279 (dígito 7) | coluna usada = 7 | erro máx. = 2.22e-16
   exemplo  635 (dígito 6) | coluna usada = 6 | erro máx. = 2.22e-16
modelo zeros (sem treino):
   exemplo  418 (dígito 8) | coluna usada = 8 | erro máx. = 1.11e-16
   exemplo  569 (dígito 9) | coluna usada = 9 | erro máx. = 2.22e-16
   exemplo  367 (dígito 6) | coluna usada = 6 | erro máx. = 2.22e-16
   exemplo 1114 (dígito 0) | coluna usada = 0 | erro máx. = 1.11e-16
   exemplo 1167 (dígito 4) | coluna usada = 4 | erro máx. = 1.78e-15
```

EXEMPLO **O que observar**

Nos dois casos, a reconstrução é exata (erro da ordem de 1e-16 ou menor). Com o modelo zerado, todos os componentes de (p − y) têm módulo parecido, e qualquer coluna serve. O ataque independe de o modelo estar treinado: basta ter o gradiente de um exemplo.

Exercício 2 — Atualização com vários passos locais

PYTHON

```python
import numpy as np
from fedlib import *
```

```
Xtr, Xte, ytr, yte = carregar_digits()
rng = np.random.default_rng(0)
W, _ = fedavg(particao_iid(ytr, 10), Xtr, ytr, Xte, yte, rodadas=10, epocas=2, eta=0.1)

def reconstruir(G):
    gb = G[-1]; c = int(np.argmax(np.abs(gb)))
    return G[:-1, c] / gb[c]

print(f"{'n':>3}{'E':>4}  correlação média")
for n in (1, 8, 32):
    for E in (1, 5, 20):
        cs = []
        for _ in range(20):
            idx = rng.choice(len(ytr), n, replace=False)
            Wl = treino_local(W, Xtr[idx], ytr[idx], E, 0.1, lote=n)
            xr = reconstruir((W - Wl) / 0.1)
            cs.append(max(np.corrcoef(xr, Xtr[i, :-1])[0, 1] for i in idx))
        print(f"{n:>3}{E:>4}  {np.mean(cs):.3f}")
```

SAÍDA

```
  n   E  correlação média
  1   1  1.000
  1   5  1.000
  1  20  1.000
  8   1  0.937
  8   5  0.936
  8  20  0.906
 32   1  0.811
 32   5  0.792
 32  20  0.778
```

EXEMPLO O que observar

Com um único exemplo, a atualização continua proporcional a x, mesmo após 20 épocas (cada passo soma um múltiplo de x ⊗ erro): a correlação segue 1,00. Com 8 ou 32 exemplos, a correlação cai, e mais épocas a reduzem um pouco. Atenção: isto vale para um modelo linear. Em redes profundas, os múltiplos passos mudam a direção e a história é diferente.

Exercício 3 — Quanto tempo de treino é demais?

PYTHON

```
import numpy as np
from sklearn.metrics import roc_auc_score
from fedlib import *

Xtr, Xte, ytr, yte = carregar_digits()
def perdas(W, X, y):
    P = softmax(X @ W)
    return -np.log(P[np.arange(len(y)), y] + 1e-12)

print(f"{'rodadas':>8}{'acc treino':>12}{'acc teste':>11}{'AUC ataque':>12}")
for rodadas in (5, 20, 80):
    a_tr, a_te, aucs = [], [], []
    for s in range(5):
```

```
        idx = np.random.default_rng(s).permutation(len(ytr))[:150]
        partes = [idx[p] for p in particao_iid(ytr[idx], 6, s)]
        W, _ = fedavg(partes, Xtr, ytr, Xte, yte, rodadas=rodadas, epocas=5, eta=0.1, seed=s)
        lm, ln = perdas(W, Xtr[idx], ytr[idx]), perdas(W, Xte, yte)
        aucs.append(roc_auc_score(np.r_[np.ones(len(lm)), np.zeros(len(ln))], -np.r_[lm, ln]))
        a_tr.append(acuracia(W, Xtr[idx], ytr[idx])); a_te.append(acuracia(W, Xte, yte))
    print(f"{rodadas:>8}{np.mean(a_tr):>12.3f}{np.mean(a_te):>11.3f}{np.mean(aucs):>12.3f}")
```

SAÍDA

```
 rodadas  acc treino  acc teste  AUC ataque
       5       0.948      0.878       0.602
      20       0.988      0.908       0.620
      80       1.000      0.910       0.635
```

EXEMPLO O que observar

Mais rodadas elevam a acurácia de treino e, com isso, a AUC do ataque, enquanto a acurácia de teste estaciona. Ou seja, depois de certo ponto, treinar mais **só aumenta o vazamento**. A parada antecipada é uma defesa barata.

Exercício 4 — Escolhendo o limiar do atacante

PYTHON

```python
import numpy as np
from sklearn.metrics import roc_curve
from fedlib import *

Xtr, Xte, ytr, yte = carregar_digits()
def perdas(W, X, y):
    P = softmax(X @ W)
    return -np.log(P[np.arange(len(y)), y] + 1e-12)

idx = np.random.default_rng(0).permutation(len(ytr))[:150]
partes = [idx[p] for p in particao_iid(ytr[idx], 6, 0)]
W, _ = fedavg(partes, Xtr, ytr, Xte, yte, rodadas=80, epocas=5, eta=0.1, seed=0)
lm, ln = perdas(W, Xtr[idx], ytr[idx]), perdas(W, Xte, yte)

limiar = np.median(lm)
print(f"limiar = mediana da perda dos membros = {limiar:.4f}")
print(f"TPR (membros marcados como membros)     : {np.mean(lm <= limiar):.3f}")
print(f"FPR (não membros marcados como membros) : {np.mean(ln <= limiar):.3f}")

rotulo = np.r_[np.ones(len(lm)), np.zeros(len(ln))]
fpr, tpr, lim = roc_curve(rotulo, -np.r_[lm, ln])
k = int(np.argmax(tpr - fpr))
print(f"melhor ponto: TPR = {tpr[k]:.3f}, FPR = {fpr[k]:.3f}, limiar de perda = {-lim[k]:.4f}")
```

SAÍDA

```
limiar = mediana da perda dos membros = 0.0245
TPR (membros marcados como membros)     : 0.500
FPR (não membros marcados como membros) : 0.351
melhor ponto: TPR = 0.860, FPR = 0.600, limiar de perda = 0.1034
```

EXEMPLO **O que observar**

Com o limiar na mediana, o TPR é 0,50 por construção e o FPR é cerca de 0,35: para cada três não membros marcados, há cinco membros. No melhor ponto da curva, o TPR sobe a 0,86, mas o FPR vai a 0,60. O ataque erra bastante: dá uma **vantagem estatística**, e não uma prova sobre indivíduos. Mesmo assim, em saúde ou finanças, qualquer vantagem acima do acaso pode ser considerada risco. Um ataque calibrado por exemplo (Carlini et al., 2022) costuma ser bem mais forte.

Capítulo 8 — Protegendo os Dados

Exercício 1 — Resposta aleatorizada: privacidade contra precisão

PYTHON

```python
import numpy as np

rng = np.random.default_rng(0)
n, taxa, repeticoes = 2000, 0.30, 300

print(f"{'p (verdade)':>12}{'ε':>8}{'RMSE da estimativa':>22}")
for p in (0.25, 0.5, 0.75):
    p_sim_fez = p + (1 - p) / 2
    p_sim_nao = (1 - p) / 2
    eps = np.log(p_sim_fez / p_sim_nao)
    estimativas = []
    for _ in range(repeticoes):
        verdade = rng.random(n) < taxa
        diz_verdade = rng.random(n) < p
        resposta = np.where(diz_verdade, verdade, rng.random(n) < 0.5)
        estimativas.append((resposta.mean() - (1 - p) / 2) / p)         # desfaz o sorteio
    rmse = np.sqrt(np.mean((np.array(estimativas) - taxa) ** 2))
    print(f"{p:>12}{eps:>8.2f}{rmse:>22.4f}")
```

SAÍDA

```
 p (verdade)       ε    RMSE da estimativa
        0.25    0.51                0.0471
         0.5    1.10                0.0228
        0.75    1.95                0.0146
```

EXEMPLO **O que observar**

Quanto menor p (mais sorteio, mais privacidade), menor o ε, mas maior o erro: com p = 0,25, o ε fica em cerca de 0,51 e o erro é bem maior que com p = 0,75 (ε ≈ 1,95). É o compromisso central: privacidade custa precisão, e mais respostas (n maior) compensam parte do custo.

Exercício 2 — Qual ruído preciso para ε = 2?

PYTHON

```python
from privacidade import *

def z_minimo(eps_alvo, rodadas, delta=1e-5):
    lo, hi = 0.1, 500.0
    for _ in range(50):
        meio = (lo + hi) / 2
        if epsilon_rdp(meio, rodadas, delta) > eps_alvo:
            lo = meio
        else:
            hi = meio
    return hi

print(f"{'rodadas':>8}{'z mínimo para ε ≤ 2':>22}")
```

```
for T in (10, 30, 100, 300):
    print(f"{T:>8}{z_minimo(2, T):>22.2f}")
```

SAÍDA

```
 rodadas   z mínimo para ε ≤ 2
      10                  7.90
      30                 13.69
     100                 24.99
     300                 43.29
```

EXEMPLO O que observar

O ruído necessário cresce aproximadamente com a raiz do número de rodadas: para quadruplicar as rodadas (de 30 para 120), é preciso cerca de o dobro do ruído. Treinar menos rodadas, ou treinar de forma mais eficiente, ajuda a economizar o orçamento de privacidade.

Exercício 3 — Escolhendo a norma de corte S

PYTHON

```
import numpy as np
from privacidade import *

Xtr, Xte, ytr, yte = carregar_digits()

partes = particao_iid(ytr, 50, seed=0)
W0 = np.zeros((Xtr.shape[1], 10))
normas = [np.linalg.norm(treino_local(W0, Xtr[p], ytr[p], 5, 0.1, 32) - W0) for p in partes]
print(f"mediana das normas das atualizações (rodada 1): {np.median(normas):.2f}")

print(f"{'S':>6}{'acurácia final':>16}")
for S in (0.1, 0.5, 2, 5):
    accs = []
    for s in range(3):
        _, h = fedavg_dp(particao_iid(ytr, 50, s), Xtr, ytr, Xte, yte, z=1.0, S=S, seed=s)
        accs.append(h[-1])
    print(f"{S:>6}{np.mean(accs):>16.3f}")
```

SAÍDA

```
mediana das normas das atualizações (rodada 1): 0.78
     S  acurácia final
   0.1          0.918
   0.5          0.941
     2          0.888
     5          0.727
```

EXEMPLO O que observar

Com S muito pequeno (0,1), o corte encolhe demais as atualizações e o treino fica lento. Com S grande (5), quase nada é cortado, e o ruído (proporcional a S) fica enorme. O melhor aqui foi S = 0,5, próximo da mediana das normas (cerca de 0,78); S = 2 já perde acurácia por excesso de ruído. A mediana das normas orienta a escolha.

Exercício 4 — Agregação segura e clientes que desistem

PYTHON

```python
import numpy as np

rng = np.random.default_rng(0)
K, d = 6, 5
u = [rng.normal(0, 1, d) for _ in range(K)]
segredo = {(i, j): int(rng.integers(1 << 31)) for i in range(K) for j in range(i + 1, K)}

def termo(i, j):
    """Máscara que o cliente i soma (se i < j) ou subtrai (se i > j) por causa do par (i, j)."""
    v = np.random.default_rng(segredo[(min(i, j), max(i, j))]).normal(0, 100, d)
    return v if i < j else -v

def mascara(i):
    return sum(termo(i, j) for j in range(K) if j != i)

enviado = {i: u[i] + mascara(i) for i in range(K)}

desistente = 3
sobreviventes = [i for i in range(K) if i != desistente]
soma_recebida = sum(enviado[i] for i in sobreviventes)
soma_certa = sum(u[i] for i in sobreviventes)
print("(a) soma recebida == soma dos sobreviventes?", np.allclose(soma_recebida, soma_certa))

# (b) correção: tirar de cada sobrevivente o termo ligado ao desistente
residuo = sum(termo(i, desistente) for i in sobreviventes)
soma_corrigida = soma_recebida - residuo
print("(b) soma corrigida == soma dos sobreviventes?", np.allclose(soma_corrigida, soma_certa))
```

SAÍDA

```
(a) soma recebida == soma dos sobreviventes? False
(b) soma corrigida == soma dos sobreviventes? True
```

EXEMPLO **O que observar**

Cada máscara de um par (i, j) só se cancela se **ambos** enviarem. Quando o cliente 3 some, os termos (i, 3) dos sobreviventes ficam sem seu par e sobram na soma, a qual vira lixo. No protocolo real, as sementes do desistente são reconstruídas a partir de partes compartilhadas (Shamir) entre os sobreviventes, e o servidor remove os resíduos, como no código. O protocolo também impede que o servidor peça as sementes de clientes que estão vivos.

Capítulo 9 — Quando Alguém Trapaceia

Exercício 1 — Ponto de ruptura na prática

PYTHON

```python
import numpy as np
from robustez import *

rng = np.random.default_rng(0)
K, d = 20, 50
honestos = rng.normal(1.0, 0.1, (K, d))
referencia = honestos.mean(axis=0)

print(f"{'maliciosos':>10}{'média':>10}{'mediana':>10}{'aparada':>10}{'Krum':>10}")
for m in range(0, 10):
    D = honestos.copy()
    D[:m] = 100.0
    D3 = D.reshape(K, d, 1)
    erro = lambda v: np.linalg.norm(v.ravel() - referencia)
    print(f"{m:>10}{erro(agg_media(D3)):>10.2f}{erro(agg_mediana(D3)):>10.2f}"
          f"{erro(agg_media_aparada(D3, 0.25)):>10.2f}{erro(agg_krum(D3, f=max(m, 1))):>10.2f}")
```

SAÍDA

```
maliciosos     média   mediana   aparada      Krum
         0      0.00      0.10      0.06      0.52
         1     35.00      0.10      0.08      0.52
         2     70.00      0.12      0.11      0.52
         3    105.00      0.17      0.17      0.52
         4    140.01      0.23      0.25      0.56
         5    175.01      0.30      0.35      0.56
         6    210.02      0.41     70.33      0.56
         7    245.02      0.53    140.33      0.56
         8    280.02      0.71    210.32      0.56
         9    315.02      0.93    280.29      0.56
```

EXEMPLO **O que observar**

A média é arrastada já com **um** atacante (erro enorme). A mediana resiste enquanto os maliciosos forem menos da metade. A média aparada com β = 0,25 descarta 5 de cada lado, por isso funciona até 5 maliciosos e quebra com 6 ou mais. O Krum escolhe sempre um vetor honesto (erro constante de cerca de 0,5, pois usa um único vetor, mesmo sem atacantes), enquanto f é bem estimado e m é menor que cerca de metade dos clientes.

Exercício 2 — Troca de rótulos: até onde vai a média?

PYTHON

```python
import numpy as np
from robustez import *

Xtr, Xte, ytr, yte = carregar_digits()
K = 20
print(f"{'f':>5}{'média':>9}{'mediana':>10}")
for f in (0.0, 0.1, 0.2, 0.3, 0.4):
```

```
    linha = []
    for ag in (agg_media, agg_mediana):
        accs = []
        for s in range(3):
            partes = particao_iid(ytr, K, s)
            _, h, _ = fedavg_robusto(partes, Xtr, ytr, Xte, yte, agregador=ag, maliciosos=range(int(f *
K)),
                                     ataque="rotulos" if f > 0 else "nenhum", seed=s)
            accs.append(h[-1])
        linha.append(np.mean(accs))
    print(f"{f:>5}{linha[0]:>9.3f}{linha[1]:>10.3f}")
```

SAÍDA

```
    f    média   mediana
  0.0    0.958     0.957
  0.1    0.938     0.956
  0.2    0.904     0.956
  0.3    0.817     0.949
  0.4    0.661     0.915
```

EXEMPLO O que observar

A média cai de forma progressiva com f (de 0,96 para cerca de 0,66 com 40%). A mediana se mantém em cerca de 0,95 até 30%, mas com 40% já cede um pouco (0,92), pois se aproxima do ponto de ruptura de 50%. A troca de rótulos é um ataque mais brando do que o sinal invertido.

Exercício 3 — Quanto aparar?

PYTHON

```
import numpy as np
from robustez import *

Xtr, Xte, ytr, yte = carregar_digits()
K = 20
print(f"{'β':>5}{'com ataque 30%':>16}{'sem ataque':>12}")
for beta in (0.0, 0.1, 0.2, 0.3, 0.4):
    ag = lambda D, b=beta: agg_media_aparada(D, b)
    com, sem = [], []
    for s in range(3):
        partes = particao_iid(ytr, K, s)
        com.append(fedavg_robusto(partes, Xtr, ytr, Xte, yte, agregador=ag, maliciosos=range(6),
ataque="sinal", seed=s)[1][-1])
        sem.append(fedavg_robusto(partes, Xtr, ytr, Xte, yte, agregador=ag, seed=s)[1][-1])
    print(f"{beta:>5}{np.mean(com):>16.3f}{np.mean(sem):>12.3f}")
```

SAÍDA

```
    β  com ataque 30%  sem ataque
  0.0          0.004       0.958
  0.1          0.244       0.958
  0.2          0.880       0.959
  0.3          0.934       0.959
  0.4          0.935       0.957
```

EXEMPLO **O que observar**

Com β = 0, é a média simples e o ataque a derruba (acurácia próxima de zero). Com β = 0,1 (2 de cada lado), ainda sobram 4 atacantes na média, e o dano é grande (0,24). Com β = 0,2 (4 de cada lado), a recuperação é parcial (0,88). A partir de β = 0,3 (6 de cada lado), todos os 6 maliciosos são descartados, e a acurácia chega a cerca de 0,93. Sem ataque, β grande custa pouco em IID. Em dados não-IID, esse custo seria maior.

Exercício 4 — Quando a mediana também cai

PYTHON

```python
import numpy as np
from robustez import *

Xtr, Xte, ytr, yte = carregar_digits()
K = 20
agregadores = {"mediana": agg_mediana, "média aparada": lambda D: agg_media_aparada(D, 0.25)}
print(f"{'f':>5} | {'agregador':<15}{'acurácia':>9}{'ASR':>7}")
for f in (0.3, 0.4, 0.5, 0.6):
    for nome, ag in agregadores.items():
        accs, asrs = [], []
        for s in range(3):
            partes = particao_iid(ytr, K, s)
            _, h, r = fedavg_robusto(partes, Xtr, ytr, Xte, yte, agregador=ag, maliciosos=range(int(f *
K)),
                                     ataque="backdoor", gama=1, seed=s)
            accs.append(h[-1]); asrs.append(r[-1])
        print(f"{f:>5} | {nome:<15}{np.mean(accs):>9.3f}{np.mean(asrs):>7.2f}")
```

SAÍDA

```
    f | agregador       acurácia    ASR
  0.3 | mediana            0.959   0.08
  0.3 | média aparada      0.954   0.87
  0.4 | mediana            0.959   0.39
  0.4 | média aparada      0.952   0.98
  0.5 | mediana            0.952   0.99
  0.5 | média aparada      0.950   1.00
  0.6 | mediana            0.949   1.00
  0.6 | média aparada      0.950   1.00
```

EXEMPLO **O que observar**

Com 30% de maliciosos, a mediana ainda segura o backdoor (ASR 0,08); com 40%, já cede parcialmente (0,39); com 50% ou mais, os maliciosos viram maioria e o ASR dispara (0,99). A média aparada com β = 0,25 só descarta 25% de cada lado, e por isso cede já a partir de 30%. Nenhuma defesa por agregação resiste a mais da metade de clientes maliciosos.

Capítulo 10 — Outros Desenhos

Exercício 1 — Quantos participantes?

PYTHON

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from arquiteturas import *

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
esc = StandardScaler().fit(Xtr)
Xtr, Xte = esc.transform(Xtr), esc.transform(Xte)
ordem = np.random.default_rng(0).permutation(30)

cen = LogisticRegression(max_iter=2000).fit(Xtr, ytr).score(Xte, yte)
print(f"centralizado: {cen:.3f}")
print(f"{'K':>3}{'atributos por parte':>34}{'acurácia VFL':>14}")
for K in (1, 2, 3, 5, 10):
    colunas = np.array_split(ordem, K)
    w, b, _ = vfl_treinar([Xtr[:, c] for c in colunas], ytr)
    acc = (vfl_prever([Xte[:, c] for c in colunas], w, b) == yte).mean()
    print(f"{K:>3}{str([len(c) for c in colunas]):>34}{acc:>14.3f}")
```

SAÍDA

```
centralizado: 0.959
  K               atributos por parte  acurácia VFL
  1                              [30]         0.965
  2                          [15, 15]         0.965
  3                      [10, 10, 10]         0.965
  5                   [6, 6, 6, 6, 6]         0.965
 10    [3, 3, 3, 3, 3, 3, 3, 3, 3, 3]         0.965
```

EXEMPLO **O que observar**

A acurácia praticamente não depende de K: a soma das parcelas z_k reproduz exatamente o modelo logístico com todas as colunas, independentemente de como elas são repartidas. O treino vertical é matematicamente equivalente ao centralizado (a diferença vem só da otimização). O custo muda: mais participantes, mais mensagens por passo.

Exercício 2 — Ruído no resíduo

PYTHON

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from arquiteturas import *
```

```
X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
esc = StandardScaler().fit(Xtr)
Xtr, Xte = esc.transform(Xtr), esc.transform(Xte)
colunas = np.array_split(np.random.default_rng(0).permutation(30), 3)
Xp = [Xtr[:, c] for c in colunas]; Xtp = [Xte[:, c] for c in colunas]

def vfl_ruido(Xp, y, sigma, epocas=300, eta=0.5, seed=0):
    rng = np.random.default_rng(seed)
    n = len(y); w = [np.zeros(Xk.shape[1]) for Xk in Xp]; b = 0.0
    for _ in range(epocas):
        p = sigmoide(sum(Xk @ wk for Xk, wk in zip(Xp, w)) + b)
        r = p - y
        for k, Xk in enumerate(Xp):
            rk = r if k == 0 else r + rng.normal(0, sigma, len(r))      # só as passivas recebem ruído
            w[k] -= eta * Xk.T @ rk / n
        b -= eta * r.mean()
    return w, b

print(f"{'σ':>5}{'acurácia':>10}{'rótulos recuperados':>22}")
for sigma in (0, 0.1, 0.5, 1, 2):
    acc, rec = [], []
    for s in range(5):
        w, b = vfl_ruido(Xp, ytr, sigma, seed=s)
        acc.append((vfl_prever(Xtp, w, b) == yte).mean())
        r0 = 0.5 - ytr                                                  # resíduo inicial (p = 0,5)
        ruidoso = r0 + np.random.default_rng(s).normal(0, sigma, len(ytr))
        rec.append(((ruidoso < 0).astype(int) == ytr).mean())
    print(f"{sigma:>5}{np.mean(acc):>10.3f}{np.mean(rec):>22.3f}")
```

SAÍDA

```
    σ  acurácia   rótulos recuperados
    0     0.965                 1.000
  0.1     0.961                 1.000
  0.5     0.953                 0.851
    1     0.951                 0.689
    2     0.951                 0.604
```

EXEMPLO O que observar

Com σ pequeno (0,1), o vazamento continua total. Com σ = 0,5, cerca de 85% dos rótulos ainda são recuperados. Com σ = 2, a recuperação cai a cerca de 60% (perto do acaso, que é 63% para a classe majoritária neste conjunto), e a acurácia do modelo cai só de 0,965 para 0,951. É um exemplo de compromisso: ruído suficiente reduz o vazamento a um custo moderado. Ruído sem garantia formal não é privacidade diferencial.

Exercício 3 — Comunicação: split learning contra FedAvg

PYTHON

```
n_in, n_oculto, n_classes = 64, 16, 10
n_param_total = n_in * n_oculto + n_oculto * n_classes          # sem vieses, para simplificar

def bytes_split_por_epoca(n_exemplos, bits=32):
    return 2 * (bits // 8) * n_exemplos * n_oculto               # ativações (↑) + gradientes (↓)
```

```python
def bytes_fedavg_por_rodada(bits=32):
    return 2 * (bits // 8) * n_param_total                    # modelo (↓) + atualização (↑)

print(f"parâmetros do modelo completo: {n_param_total}")
print(f"{'exemplos locais':>16}{'split (KB/época)':>19}{'FedAvg (KB/rodada)':>21}{'razão':>8}")
for n in (100, 1347, 10_000, 100_000):
    s, f = bytes_split_por_epoca(n) / 1024, bytes_fedavg_por_rodada() / 1024
    print(f"{n:>16}{s:>19.1f}{f:>21.1f}{s / f:>8.1f}")
```

SAÍDA

```
parâmetros do modelo completo: 1184
 exemplos locais   split (KB/época)   FedAvg (KB/rodada)   razão
             100               12.5                  9.2     1.4
            1347              168.4                  9.2    18.2
           10000             1250.0                  9.2   135.1
          100000            12500.0                  9.2  1351.4
```

EXEMPLO **O que observar**

O custo do split learning cresce com o número de exemplos locais (as ativações de cada exemplo viajam a cada época), enquanto o do FedAvg só depende do tamanho do modelo. Com 100 exemplos o split é mais barato, mas com 1.347 já é cerca de 18 vezes mais caro, e com 100.000, mais de mil vezes. Em contrapartida, o split exige menos cálculo no cliente.

Exercício 4 — Velocidade do consenso no anel

PYTHON

```python
import numpy as np
from arquiteturas import *

rng = np.random.default_rng(0)
print(f"{'n':>4}{'λ₂':>9}{'rodadas previstas':>20}{'rodadas medidas':>18}")
for n in (10, 20, 50):
    A = matriz_mistura("anel", n)
    autovalores = np.sort(np.abs(np.linalg.eigvalsh(A)))[::-1]
    lam2 = autovalores[1]
    previstas = np.log(0.01) / np.log(lam2)
    valores = rng.normal(0, 1, (n, 5))
    erros = consenso(valores, A, 3000)
    medidas = next(t for t, e in enumerate(erros) if e <= 0.01 * erros[0])
    print(f"{n:>4}{lam2:>9.4f}{previstas:>20.1f}{medidas:>18}")
```

SAÍDA

```
   n       λ₂   rodadas previstas   rodadas medidas
  10   0.8727               33.8                30
  20   0.9674              138.8                87
  50   0.9947              873.7               493
```

EXEMPLO O que observar

O segundo autovalor se aproxima de 1 quando n cresce, e o número de rodadas cresce aproximadamente com n^2 no anel: com 50 nós, são necessárias centenas de rodadas. A previsão pelo autovalor acompanha o valor medido (de mesma ordem de grandeza). Topologias mais conectadas têm λ_2 menor e convergem muito mais depressa.

Capítulo 11 — Fronteiras e Prática

Exercício 1 — Escolhendo k no K-means federado

PYTHON

```python
import numpy as np
from sklearn.cluster import KMeans
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from fronteiras import *

X, y = load_wine(return_X_y=True)
X = StandardScaler().fit_transform(X)
partes = particao_dirichlet(y, 5, 0.3, seed=0)
rng = np.random.default_rng(0)

def sse_federado(partes, X, C):
    total = 0.0
    for p in partes:                                        # cada cliente calcula o seu SSE local
        d = ((X[p][:, None, :] - C[None]) ** 2).sum(-1)
        total += d.min(1).sum()
    return total                                            # o servidor soma

print(f"{'k':>3}{'SSE federado':>15}{'inertia sklearn':>18}")
for k in range(1, 7):
    C0 = X[rng.choice(len(X), k, replace=False)]
    Cf = kmeans_federado(partes, X, C0, iteracoes=100)
    km = KMeans(k, init=C0, n_init=1, max_iter=300, tol=0, algorithm="lloyd").fit(X)
    print(f"{k:>3}{sse_federado(partes, X, Cf):>15.2f}{km.inertia_:>18.2f}")
```

SAÍDA

```
  k   SSE federado   inertia sklearn
  1        2314.00           2314.00
  2        1659.95           1659.95
  3        1277.93           1277.93
  4        1216.81           1216.81
  5        1107.08           1107.08
  6        1066.64           1066.64
```

EXEMPLO **O que observar**

Os valores federados e centralizados coincidem (a mesma inicialização produz a mesma solução). A queda do SSE é forte até k = 3 e depois suaviza, sugerindo k = 3 (o "cotovelo"), que corresponde às três classes de vinho. Observe que SSE sempre diminui com k, e por isso o cotovelo é um critério visual, e não exato.

Exercício 2 — PCA federado nos dígitos

PYTHON

```python
import numpy as np
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
```

```
from fronteiras import *

X, y = load_digits(return_X_y=True)
X = StandardScaler().fit_transform(X)
X = X[:, X.std(axis=0) > 1e-8]                # remove colunas constantes
partes = particao_dirichlet(y, 10, 0.5, seed=0)

mu, valores, vetores = pca_federado(partes, X, X.shape[1])
total = np.var(X, axis=0, ddof=1).sum()
acum_fed = np.cumsum(valores) / total
pca = PCA().fit(X)
acum_cen = np.cumsum(pca.explained_variance_ratio_)

print(f"{'componentes':>12}{'federado':>12}{'centralizado':>14}")
for k in (5, 10, 20, 30):
    print(f"{k:>12}{acum_fed[k - 1]:>12.4f}{acum_cen[k - 1]:>14.4f}")
print("componentes para 90% da variância (federado)    :", int(np.searchsorted(acum_fed, 0.90) + 1),
"de", X.shape[1])
print("componentes para 90% da variância (centralizado):", int(np.searchsorted(acum_cen, 0.90) + 1))
```

SAÍDA

```
 componentes    federado  centralizado
           5      0.4140        0.4140
          10      0.5887        0.5887
          20      0.7931        0.7931
          30      0.8932        0.8932
componentes para 90% da variância (federado)    : 31 de 61
componentes para 90% da variância (centralizado): 31
```

EXEMPLO **O que observar**

Os valores coincidem (a matriz de covariância federada é exatamente a centralizada). Cerca de 31 componentes (de 61 atributos não constantes) explicam 90% da variância, ou seja, metade da dimensão. Isto serve para economizar comunicação em etapas posteriores, desde que a redução seja feita de modo federado e com os devidos cuidados de privacidade.

Exercício 3 — Regressão linear federada exata

PYTHON

```
import numpy as np
from sklearn.datasets import load_diabetes
from sklearn.linear_model import LinearRegression, Ridge

X, y = load_diabetes(return_X_y=True)
n = len(y)
rng = np.random.default_rng(0)
idx = rng.permutation(n)
clientes = np.split(idx, [30, 90, 160, 250, 350])            # 6 clientes de tamanhos diferentes
print("tamanhos:", [len(c) for c in clientes])

d = X.shape[1] + 1
XtX, Xty = np.zeros((d, d)), np.zeros(d)
for c in clientes:                                            # cada cliente envia só estas somas
    Xk = np.hstack([X[c], np.ones((len(c), 1))])
```

```
    XtX += Xk.T @ Xk
    Xty += Xk.T @ y[c]

beta = np.linalg.solve(XtX, Xty)                                # servidor: equações normais
ref = LinearRegression().fit(X, y)
print("OLS: diferença máxima nos coeficientes :", float(np.abs(beta[:-1] - ref.coef_).max()))
print("OLS: diferença no intercepto           :", float(abs(beta[-1] - ref.intercept_)))

# Ridge (sem penalizar o intercepto)
lam = 1.0
P = np.eye(d) * lam; P[-1, -1] = 0
beta_r = np.linalg.solve(XtX + P, Xty)
ref_r = Ridge(alpha=lam).fit(X, y)
print("Ridge: diferença máxima nos coeficientes:", float(np.abs(beta_r[:-1] - ref_r.coef_).max()))
```

SAÍDA

```
tamanhos: [30, 60, 70, 90, 100, 92]
OLS: diferença máxima nos coeficientes : 1.5006662579253316e-11
OLS: diferença no intercepto           : 2.842170943040401e-14
Ridge: diferença máxima nos coeficientes: 5.684341886080801e-13
```

EXEMPLO **O que observar**

A diferença fica na casa dos erros de arredondamento. As somas X^TX e X^Ty são **somas sobre exemplos**, e portanto se combinam exatamente entre clientes. A regularização de Ridge só acrescenta uma matriz no servidor. Atenção: X^TX e X^Ty também revelam informação sobre os dados (por exemplo, em clientes muito pequenos); combine com agregação segura e DP quando necessário.

Exercício 4 — Quanto posto o adaptador precisa?

PYTHON

```python
import numpy as np
from fronteiras import *

Xtr, Xte, ytr, yte = carregar_digits()
W0 = np.zeros((65, 10))
for _ in range(300):
    W0 -= 0.5 * gradiente(W0, Xtr, ytr)

def tarefa(trocas):
    perm = np.arange(10)
    for a, b in trocas:
        perm[a], perm[b] = b, a
    return perm[ytr], perm[yte]

tarefas = {"1 troca": [(3, 5)], "2 trocas": [(3, 5), (4, 9)], "3 trocas": [(3, 5), (4, 9), (1, 7)]}
print(f"{'tarefa':<10}" + "".join(f"{'r = ' + str(r):>9}" for r in (1, 2, 4, 8)))
for nome, trocas in tarefas.items():
    ytr2, yte2 = tarefa(trocas)
    linha = []
    for posto in (1, 2, 4, 8):
        accs = []
        for s in range(2):
            partes = particao_dirichlet(ytr, 20, 0.5, seed=s)
```

```
            h, _ = fedavg_adaptador(partes, Xtr, ytr2, Xte, yte2, W0, posto, seed=s)
            accs.append(h[-1])
        linha.append(np.mean(accs))
    print(f"{nome:<10}" + "".join(f"{a:>9.3f}" for a in linha))
```

SAÍDA

```
tarefa        r = 1    r = 2    r = 4    r = 8
1 troca       0.968    0.969    0.969    0.969
2 trocas      0.790    0.957    0.962    0.964
3 trocas      0.517    0.817    0.946    0.950
```

EXEMPLO **O que observar**

Cada troca de duas classes corresponde a uma correção de posto 2 (trocam-se duas colunas dos pesos). Com uma troca, o posto 1 basta (0,97); com duas, é preciso posto 2 ou mais (0,96); com três, o posto 2 já não basta (0,82), e os postos 4 e 8 chegam a cerca de 0,95. A capacidade do adaptador precisa crescer com a complexidade da mudança.

Apêndice E — Soluções de referência dos projetos integradores

Estas são **soluções de referência**: caminhos possíveis, e não os únicos. Compare com o seu trabalho, observe as decisões e discuta as diferenças. A avaliação de um projeto deve considerar também a qualidade da argumentação, e não apenas o código. Os critérios de avaliação de cada projeto aparecem antes do código.

Capítulo 1 — Bem-vindo ao Aprendizado Federado

Rede de Clínicas: vale a pena colaborar?

Critérios de avaliação

- Padronização e separação de teste feitas corretamente (sem vazar o teste).
- Modelos locais, média dos modelos e centralizado comparados na mesma base de teste.
- Repetição com várias sementes e relato de média e desvio padrão.
- Recomendação clara, com ao menos dois limites (dados IID simulados, uma só rodada, escalonamento global).

PYTHON

```python
import numpy as np
from sklearn.datasets import load_wine
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X, y = load_wine(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
esc = StandardScaler().fit(Xtr)
Xtr, Xte = esc.transform(Xtr), esc.transform(Xte)

def media_de_modelos(modelos, pesos):
    m = LogisticRegression(max_iter=2000)
    p = np.array(pesos) / np.sum(pesos)
    m.coef_ = np.tensordot(p, [k.coef_ for k in modelos], axes=1)
    m.intercept_ = np.tensordot(p, [k.intercept_ for k in modelos], axes=1)
    m.classes_ = modelos[0].classes_
    return m

res = {"local": [], "federado": [], "centralizado": []}
for semente in range(20):
    rng = np.random.default_rng(semente)
    idx = rng.permutation(len(ytr))
    partes = np.array_split(idx, 5)
    modelos = []
    for p in partes:
        modelos.append(LogisticRegression(max_iter=2000).fit(Xtr[p], ytr[p]))
    res["local"].append(np.mean([m.score(Xte, yte) for m in modelos]))
    res["federado"].append(media_de_modelos(modelos, [len(p) for p in partes]).score(Xte, yte))
    res["centralizado"].append(LogisticRegression(max_iter=2000).fit(Xtr, ytr).score(Xte, yte))

print(f"{'estratégia':<14}{'média':>8}{'desvio':>9}")
for k, v in res.items():
    print(f"{k:<14}{np.mean(v):8.3f}{np.std(v):9.3f}")
```

SAÍDA

```
estratégia       média   desvio
local            0.955    0.016
federado         0.982    0.004
centralizado     1.000    0.000
```

EXEMPLO **Discussão dos resultados**

Na simulação, o modelo federado (média dos modelos) supera o local médio (cerca de 0,98 contra 0,95) e fica perto do centralizado (1,00, usando todo o treino). O centralizado aqui usa **todos** os dados de treino, então é um teto otimista. Mas lembre dos limites: os dados foram divididos ao acaso (IID), a padronização usou o treino inteiro e houve apenas um encontro com o servidor. A recomendação deve ser cautelosa: o resultado sugere que colaborar vale a pena, mas deve ser validado com dados reais e heterogêneos.

Capítulo 2 — Preparando o Terreno

Triagem com poucos dados: do zero ao modelo

Critérios de avaliação

- Divisão estratificada em treino, validação e teste, e padronização ajustada apenas no treino.
- Uso correto de `partial_fit` e registro das curvas de treino e validação.
- Escolha do `eta0` feita pela validação, não pelo teste.
- Métricas completas no teste, avaliado uma única vez, e explicação clara do motivo.

PYTHON

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import SGDClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xtmp, ytr, ytmp = train_test_split(X, y, test_size=0.4, random_state=0, stratify=y)
Xva, Xte, yva, yte = train_test_split(Xtmp, ytmp, test_size=0.5, random_state=0, stratify=ytmp)
esc = StandardScaler().fit(Xtr)
Xtr, Xva, Xte = esc.transform(Xtr), esc.transform(Xva), esc.transform(Xte)

def treinar(eta0, epocas=30):
    m = SGDClassifier(loss="log_loss", learning_rate="constant", eta0=eta0, random_state=0)
    hist = []
    for _ in range(epocas):
        m.partial_fit(Xtr, ytr, classes=[0, 1])
        hist.append((m.score(Xtr, ytr), m.score(Xva, yva)))
    return m, hist

melhor, melhor_val = None, -1
print("eta0  | acc treino | acc validação (última época)")
for eta0 in [0.001, 0.01, 0.1]:
    m, h = treinar(eta0)
    print(f"{eta0:<6}| {h[-1][0]:10.3f} | {h[-1][1]:.3f}")
    if h[-1][1] > melhor_val:
        melhor, melhor_val, eta_escolhido = m, h[-1][1], eta0

pred = melhor.predict(Xte)
print("\nEscolhido eta0 =", eta_escolhido)
print(f"Teste: acc = {accuracy_score(yte, pred):.3f} | prec = {precision_score(yte, pred):.3f} | "
      f"rev = {recall_score(yte, pred):.3f} | F1 = {f1_score(yte, pred):.3f}")
print(confusion_matrix(yte, pred))
```

SAÍDA

```
eta0  | acc treino | acc validação (última época)
0.001 |      0.982 | 0.939
0.01  |      0.988 | 0.947
0.1   |      0.991 | 0.965

Escolhido eta0 = 0.1
```

```
Teste: acc = 0.974 | prec = 0.986 | rev = 0.972 | F1 = 0.979
[[42  1]
 [ 2 69]]
```

EXEMPLO Discussão dos resultados

O melhor `eta0` foi o maior (0,1): o menor aprende devagar demais em 30 épocas. O teste só entrou no final para dar uma estimativa honesta. Se você ajustar o modelo olhando o teste, o resultado deixa de ser uma estimativa de desempenho em dados novos.

Capítulo 3 — Anatomia de um Sistema Federado

Rede de Hospitais: do central ao federado

Critérios de avaliação

- Referência centralizada calculada com todos os dados de treino.
- Rodada de cruzamento do limiar identificada corretamente para FedAvg e FedSGD.
- Tráfego calculado com a fórmula $2 \times 4 \times |w| \times$ clientes $\times$ rodadas.
- Recomendação em linguagem acessível e com ressalvas (dados IID simulados, modelo simples).

PYTHON

```python
import numpy as np
from sklearn.linear_model import LogisticRegression
from fedlib import *

Xtr, Xte, ytr, yte = carregar_digits()
referencia = LogisticRegression(max_iter=3000).fit(Xtr[:, :-1], ytr).score(Xte[:, :-1], yte)
limiar = referencia - 0.01
print(f"referência centralizada = {referencia:.3f} | limiar (1 ponto abaixo) = {limiar:.3f}")

partes = particao_iid(ytr, 10, seed=0)
n_param = Xtr.shape[1] * 10
bytes_rodada = 2 * 4 * n_param * 10

configs = {
    "FedAvg (E=5)": dict(epocas=5, eta=0.1, lote=32),
    "FedSGD (1 passo)": dict(epocas=1, eta=0.5, lote=10 ** 6),
}
print(f"{'método':<18}{'1ª rodada >= limiar':>22}{'tráfego até lá':>18}")
for nome, cfg in configs.items():
    _, hist = fedavg(partes, Xtr, ytr, Xte, yte, rodadas=40, seed=0, **cfg)
    ok = [i + 1 for i, a in enumerate(hist) if a >= limiar]
    if ok:
        print(f"{nome:<18}{ok[0]:>22}{ok[0] * bytes_rodada / 1024:>15.1f} KB")
    else:
        print(f"{nome:<18}{'não atingiu em 40':>22}{40 * bytes_rodada / 1024:>15.1f} KB")
```

SAÍDA

```
referência centralizada = 0.969 | limiar (1 ponto abaixo) = 0.959
método              1ª rodada >= limiar    tráfego até lá
FedAvg (E=5)                         22         1117.2 KB
FedSGD (1 passo)       não atingiu em 40         2031.2 KB
```

EXEMPLO **Discussão dos resultados**

O FedAvg cruza o limiar na rodada 22, com cerca de 1,1 MB de tráfego total, enquanto o FedSGD não o atinge nas 40 rodadas (cerca de 2 MB gastos). Em valores absolutos o custo é trivial para este modelo minúsculo. A recomendação deve ressalvar que os dados foram divididos ao acaso e que o modelo é pequeno; com dados reais e modelos grandes, o orçamento de comunicação será crucial.

Capítulo 4 — Quando os Dados Não São Iguais

Escolhendo o algoritmo: mapa de heterogeneidade

Critérios de avaliação

- TV médio calculado e relacionado ao α.
- Três métodos comparados nas mesmas partições, com 3 sementes.
- Tabela com acurácia nas rodadas 10 e 40 e desvio.
- Recomendação que considere o custo extra do SCAFFOLD (2× comunicação) e cite limites (modelo simples, 3 sementes, dados simulados).

PYTHON

```python
import numpy as np
from fedlib import *
from scaffold import scaffold

Xtr, Xte, ytr, yte = carregar_digits()
global_p = np.bincount(ytr, minlength=10) / len(ytr)
def tv_medio(partes):
    return np.mean([0.5 * np.abs(np.bincount(ytr[p], minlength=10) / len(p) - global_p).sum() for p in
partes if len(p)])

cfg = dict(rodadas=40, epocas=10, eta=0.1, frac=0.25)
print(f"{'α':>5}{'TV':>6} | {'método':<10}{'rod. 10':>9}{'rod. 40':>9}{'dp':>7}")
for alfa in (1.0, 0.3, 0.1):
    res = {"FedAvg": [], "FedProx": [], "SCAFFOLD": []}
    tvs = []
    for s in range(3):
        partes = particao_dirichlet(ytr, 20, alfa, seed=s)
        tvs.append(tv_medio(partes))
        res["FedAvg"].append(fedavg(partes, Xtr, ytr, Xte, yte, seed=s, **cfg)[1])
        res["FedProx"].append(fedavg(partes, Xtr, ytr, Xte, yte, mu=0.1, seed=s, **cfg)[1])
        res["SCAFFOLD"].append(scaffold(partes, Xtr, ytr, Xte, yte, seed=s, **cfg)[1])
    for nome, hs in res.items():
        h = np.array(hs)
        print(f"{alfa:>5}{np.mean(tvs):>6.2f} | {nome:<10}{h[:, 9].mean():>9.3f}{h[:, -1].mean():>9.3f}
{h[:, -1].std():>7.3f}")
```

SAÍDA

```
    α    TV | método      rod. 10  rod. 40     dp
  1.0  0.34 | FedAvg        0.949    0.965  0.005
  1.0  0.34 | FedProx       0.947    0.964  0.003
  1.0  0.34 | SCAFFOLD      0.953    0.967  0.002
  0.3  0.54 | FedAvg        0.942    0.967  0.004
  0.3  0.54 | FedProx       0.943    0.967  0.003
  0.3  0.54 | SCAFFOLD      0.947    0.964  0.001
  0.1  0.70 | FedAvg        0.924    0.959  0.005
  0.1  0.70 | FedProx       0.917    0.959  0.008
  0.1  0.70 | SCAFFOLD      0.927    0.963  0.009
```

EXEMPLO **Discussão dos resultados**

Em heterogeneidade moderada (α = 1), os três métodos são equivalentes; o custo extra do SCAFFOLD não se justifica. Com α = 0,1, o SCAFFOLD fica ligeiramente à frente (0,963 contra 0,959 no fim), mas a diferença é do tamanho do desvio entre sementes, e o FedProx fica no mesmo nível do FedAvg. Em todos os casos as diferenças finais são pequenas. Para este cenário, **não** vale pagar o dobro de comunicação do SCAFFOLD; o ganho aparece só em heterogeneidade extrema (veja a Figura 4.5). Ressalvas: modelo convexo simples, poucas sementes e partições simuladas.

Capítulo 5 — Falando Menos

Orçamento de comunicação: qual configuração escolher?

Critérios de avaliação

- Tráfego por rodada calculado corretamente para cada método.
- Rodada máxima dentro do orçamento e acurácia nessa rodada registradas.
- Melhor configuração identificada para cada orçamento, com tabela clara.
- Recomendação que discuta o compromisso entre bytes por rodada, número de rodadas e épocas locais, e cite limites do estudo.

```python
import numpy as np
from compressao import *

Xtr, Xte, ytr, yte = carregar_digits()
metodos = [("float32", dict()),
           ("8 bits", dict(metodo="quantizar", param=8)),
           ("2 bits", dict(metodo="quantizar", param=2)),
           ("top-10%+EF", dict(metodo="topk", param=0.1, erro_acumulado=True)),
           ("top-1%+EF", dict(metodo="topk", param=0.01, erro_acumulado=True))]
orcamentos = {"100 KB": 100 * 1024, "300 KB": 300 * 1024}
RODADAS = 60

resultados = {}
for nome, kw in metodos:
    for E in (1, 5):
        hs, bpr = [], []
        for s in range(2):
            partes = particao_dirichlet(ytr, 20, 1.0, seed=s)
            _, h, t = fedavg_comprimido(partes, Xtr, ytr, Xte, yte, rodadas=RODADAS, epocas=E,
frac=0.5, seed=s, **kw)
            hs.append(h); bpr.append(t / RODADAS)
        resultados[(nome, E)] = (np.mean(hs, axis=0), np.mean(bpr))

print(f"{'configuração':<18}{'bytes/rodada':>13} | " + " | ".join(f"{o:>16}" for o in orcamentos))
melhores = {o: (None, -1) for o in orcamentos}
for (nome, E), (h, bpr) in resultados.items():
    linha = f"{nome + ', E=' + str(E):<18}{bpr:>13.0f} | "
    cel = []
    for o, limite in orcamentos.items():
        r = int(limite // bpr)
        if r < 1:
            cel.append(f"{'não cabe':>16}")
        else:
            r = min(r, RODADAS); acc = h[r - 1]
            cel.append(f"{acc:>8.3f} (r={r:>2})".rjust(16))
            if acc > melhores[o][1]:
                melhores[o] = (f"{nome}, E={E}", acc)
    print(linha + " | ".join(cel))
for o, (cfg, acc) in melhores.items():
    print(f"melhor para {o}: {cfg} (acurácia {acc:.3f})")
```

SAÍDA

```
configuração       bytes/rodada |           100 KB |           300 KB
float32, E=1              26000 |     0.863 (r= 3) |     0.908 (r=11)
float32, E=5              26000 |     0.913 (r= 3) |     0.942 (r=11)
8 bits, E=1                6580 |     0.917 (r=15) |     0.937 (r=46)
8 bits, E=5                6580 |     0.946 (r=15) |     0.962 (r=46)
2 bits, E=1                1710 |     0.944 (r=59) |     0.944 (r=60)
2 bits, E=5                1710 |     0.962 (r=59) |     0.963 (r=60)
top-10%+EF, E=1            3900 |     0.924 (r=26) |     0.944 (r=60)
top-10%+EF, E=5            3900 |     0.957 (r=26) |     0.967 (r=60)
top-1%+EF, E=1              360 |     0.923 (r=60) |     0.923 (r=60)
top-1%+EF, E=5              360 |     0.953 (r=60) |     0.953 (r=60)
melhor para 100 KB: 2 bits, E=5 (acurácia 0.962)
melhor para 300 KB: top-10%+EF, E=5 (acurácia 0.967)
```

EXEMPLO **Discussão dos resultados**

Com orçamento pequeno, métodos que enviam poucos bytes por rodada (quantização forte, top-k com memória de erro) permitem **muito mais rodadas** dentro do limite e vencem o float32, que só cabe em poucas rodadas. Com mais orçamento, a vantagem diminui. Mais épocas locais (E = 5) também ajudam, pois cada rodada rende mais. Note que o top-1% com memória de erro gasta só cerca de 22 KB nas 60 rodadas e **não aproveita** o orçamento extra: não adianta economizar demais se sobra banda. Limites: modelo pequeno, duas sementes e contagem simplificada de bytes.

Capítulo 6 — Cada Um do Seu Jeito

Chegou um cliente novo: o que fazer com poucos dados?

Critérios de avaliação

- Clientes novos fora do treino federado, corretamente separados.
- Quatro estratégias comparadas para os três valores de n, com 3 sementes.
- Uso da perda nos poucos exemplos para escolher o grupo (IFCA), sem ver o teste.
- Recomendação que discuta o efeito do número de exemplos e as ressalvas (grupos limpos, laboratório, K conhecido).

PYTHON

```python
import numpy as np
from personalizacao import *

Xtr, Xte, ytr0, yte0 = carregar_digits()
print(f"{'n':>4}{'global':>9}{'só local':>10}{'ajuste fino':>13}{'grupo IFCA':>12}")
for n in (5, 20, 50):
    saida = {"global": [], "local": [], "ajuste": [], "ifca": []}
    for s in range(3):
        partes, y, grupo = particao_conceitos(ytr0, 12, 3, s)
        treino, teste = dividir_local(partes, 0.3, s)
        novos = [0, 1, 2]                                  # um cliente de cada grupo
        resto = [i for i in range(12) if i not in novos]
        tr_resto = [treino[i] for i in resto]
        Wg, _ = fedavg(tr_resto, Xtr, y, Xte, yte0, rodadas=30, epocas=5, eta=0.1, seed=s)
        modelos, _ = ifca(tr_resto, Xtr, y, K=3, seed=s)
        for i in novos:
            p, t = treino[i][:n], teste[i]
            saida["global"].append(acuracia(Wg, Xtr[t], y[t]))
            saida["local"].append(acuracia(treino_local(np.zeros_like(Wg), Xtr[p], y[p], 30, 0.1, 32,
0.0, i), Xtr[t], y[t]))
            saida["ajuste"].append(acuracia(treino_local(Wg, Xtr[p], y[p], 5, 0.1, 32, 0.0, i), Xtr[t],
y[t]))
            j = int(np.argmin([perda(W, Xtr[p], y[p]) for W in modelos]))
            saida["ifca"].append(acuracia(modelos[j], Xtr[t], y[t]))
    print(f"{n:>4}{np.mean(saida['global']):>9.3f}{np.mean(saida['local']):>10.3f}"
          f"{np.mean(saida['ajuste']):>13.3f}{np.mean(saida['ifca']):>12.3f}")
```

SAÍDA

```
   n   global  só local  ajuste fino  grupo IFCA
   5    0.366     0.333        0.497       0.935
  20    0.366     0.641        0.605       0.935
  50    0.366     0.837        0.771       0.935
```

EXEMPLO **Discussão dos resultados**

O modelo do grupo (IFCA) já atende bem o cliente novo com apenas 5 exemplos (cerca de 0,93), pois **só precisa de poucos dados para escolher o grupo certo**. O global falha (cerca de 0,37), e o modelo só local precisa de muito mais exemplos (só chega a cerca de 0,84 com 50). O ajuste fino melhora com n, mas fica atrás. Ressalvas: os grupos do experimento são limpos e K é conhecido; em dados reais, a atribuição de grupo é menos clara.

Capítulo 7 — O Que Pode Vazar

Auditoria de privacidade: qual configuração aprovar?

Critérios de avaliação

- Oito configurações avaliadas, com cinco sementes e AUC média e desvio.
- Critério de aprovação aplicado corretamente (AUC < 0,58).
- Escolha da melhor configuração entre as aprovadas, por acurácia de teste.
- Parecer com ao menos duas limitações e sem afirmar que o modelo é "seguro" sem garantias formais.

PYTHON

```python
import numpy as np
from itertools import product
from sklearn.metrics import roc_auc_score
from fedlib import *

Xtr, Xte, ytr, yte = carregar_digits()
def perdas(W, X, y):
    P = softmax(X @ W)
    return -np.log(P[np.arange(len(y)), y] + 1e-12)

LIMITE = 0.58
linhas = []
for n, rodadas, epocas in product((150, 600), (10, 50), (2, 20)):
    aucs, a_tr, a_te = [], [], []
    for s in range(5):
        idx = np.random.default_rng(s).permutation(len(ytr))[:n]
        partes = [idx[p] for p in particao_iid(ytr[idx], 6, s)]
        W, _ = fedavg(partes, Xtr, ytr, Xte, yte, rodadas=rodadas, epocas=epocas, eta=0.1, seed=s)
        lm, ln = perdas(W, Xtr[idx], ytr[idx]), perdas(W, Xte, yte)
        aucs.append(roc_auc_score(np.r_[np.ones(len(lm)), np.zeros(len(ln))], -np.r_[lm, ln]))
        a_tr.append(acuracia(W, Xtr[idx], ytr[idx])); a_te.append(acuracia(W, Xte, yte))
    linhas.append((n, rodadas, epocas, np.mean(a_tr), np.mean(a_te), np.mean(aucs), np.std(aucs)))

print(f"{'n':>5}{'rod.':>6}{'ép.':>5}{'acc tr':>8}{'acc te':>8}{'AUC':>7}{'dp':>6}  status")
for n, r, e, atr, ate, auc, dp in linhas:
    print(f"{n:>5}{r:>6}{e:>5}{atr:>8.3f}{ate:>8.3f}{auc:>7.3f}{dp:>6.3f}  {'aprovada' if auc < LIMITE else 'reprovada'}")
aprov = [l for l in linhas if l[5] < LIMITE]
melhor = max(aprov, key=lambda l: l[4])
print(f"\nMelhor aprovada: n={melhor[0]}, rodadas={melhor[1]}, épocas={melhor[2]} (acurácia de teste {melhor[4]:.3f}, AUC {melhor[5]:.3f})")
```

SAÍDA

```
    n  rod.  ép.  acc tr  acc te    AUC    dp  status
  150    10    2   0.941   0.872  0.600 0.024  reprovada
  150    10   20   0.995   0.907  0.624 0.024  reprovada
  150    50    2   0.988   0.910  0.621 0.025  reprovada
  150    50   20   1.000   0.911  0.641 0.023  reprovada
  600    10    2   0.957   0.928  0.527 0.012  aprovada
  600    10   20   0.990   0.948  0.533 0.012  aprovada
  600    50    2   0.989   0.947  0.532 0.013  aprovada
  600    50   20   0.999   0.952  0.536 0.011  aprovada
```

```
Melhor aprovada: n=600, rodadas=50, épocas=20 (acurácia de teste 0.952, AUC 0.536)
```

EXEMPLO Discussão dos resultados

Configurações com 600 exemplos tendem a passar no critério e ainda entregam a maior acurácia de teste. Com 150 exemplos, treinar mais aumenta a AUC e leva à reprovação. O parecer deve ser prudente: o ataque usado é o mais simples e passar nele **não prova** privacidade. Para garantias formais, é preciso privacidade diferencial (Capítulo 8).

Capítulo 8 — Protegendo os Dados

Orçamento de privacidade da clínica

Critérios de avaliação

- Dois cenários, quatro níveis de ruído e cinco sementes, com ε, acurácia e AUC.
- Interpretação correta do compromisso: mais ruído, menos acurácia, e AUC cai pouco ou nada em modelos que já vazam pouco.
- Parecer que reconheça que ε baixo exige muito mais clientes e dados, e que nenhum nível testado oferece, ao mesmo tempo, ε baixo e acurácia alta.
- Menção a limitações: contabilidade conservadora, ataque simples, dados simulados.

PYTHON

```python
import numpy as np
from sklearn.metrics import roc_auc_score
from privacidade import *

Xtr, Xte, ytr, yte = carregar_digits()
def perdas(W, X, y):
    P = softmax(X @ W)
    return -np.log(P[np.arange(len(y)), y] + 1e-12)

print(f"{'cenário':<22}{'z':>5}{'ε (50 rod.)':>13}{'acc teste':>11}{'AUC ataque':>12}")
for nome, n, K in (("A: 150 pac., 30 cli.", 150, 30), ("B: 300 pac., 50 cli.", 300, 50)):
    for z in (0, 1, 3, 5.8):
        aucs, accs = [], []
        for s in range(5):
            idx = np.random.default_rng(s).permutation(len(ytr))[:n]
            partes = [idx[p] for p in particao_iid(ytr[idx], K, s)]
            W, h = fedavg_dp(partes, Xtr, ytr, Xte, yte, z=z, S=0.5, rodadas=50, epocas=10, seed=s)
            lm, ln = perdas(W, Xtr[idx], ytr[idx]), perdas(W, Xte, yte)
            aucs.append(roc_auc_score(np.r_[np.ones(len(lm)), np.zeros(len(ln))], -np.r_[lm, ln]))
            accs.append(h[-1])
        eps = "∞" if z == 0 else f"{epsilon_rdp(z, 50):.1f}"
        print(f"{nome:<22}{z:>5}{eps:>13}{np.mean(accs):>11.3f}{np.mean(aucs):>12.3f}")
```

SAÍDA

```
cenário                  z  ε (50 rod.)  acc teste  AUC ataque
A: 150 pac., 30 cli.     0            ∞      0.909       0.627
A: 150 pac., 30 cli.     1         58.9      0.900       0.626
A: 150 pac., 30 cli.     3         14.1      0.786       0.608
A: 150 pac., 30 cli.   5.8          6.6      0.591       0.575
B: 300 pac., 50 cli.     0            ∞      0.930       0.557
B: 300 pac., 50 cli.     1         58.9      0.928       0.558
B: 300 pac., 50 cli.     3         14.1      0.893       0.558
B: 300 pac., 50 cli.   5.8          6.6      0.802       0.547
```

EXEMPLO **Discussão dos resultados**

No cenário A (30 clientes), z = 5,8 reduz a AUC do ataque (de cerca de 0,63 para 0,58), mas a acurácia cai de cerca de 0,91 para 0,59. No cenário B (50 clientes), a acurácia sofre menos (cerca de 0,93 para 0,80), e a AUC já é baixa (cerca de 0,56). Em ambos, o ε obtido (6,6 a 59) está longe de valores considerados fortes (< 1). Conclusão: com tão poucos clientes e dados, proteger formalmente custa muito. Seria preciso muito mais clientes e dados, ou aceitar um ε maior, com decisão do comitê.

Capítulo 9 — Quando Alguém Trapaceia

Parecer de segurança: qual agregador adotar?

Critérios de avaliação

- Cinco agregadores avaliados nos três cenários, com 3 sementes.
- Critério de aprovação aplicado corretamente e escolha justificada.
- Parecer que mencione o custo da robustez em dados heterogêneos e a tensão com a agregação segura.
- Pelo menos duas limitações (ataques simples, modelo simples, dados simulados, fração de maliciosos conhecida).

PYTHON

```python
import numpy as np
from robustez import *

Xtr, Xte, ytr, yte = carregar_digits()
K = 20
agregadores = {
    "média":            agg_media,
    "mediana":          agg_mediana,
    "média aparada":    lambda D: agg_media_aparada(D, 0.25),
    "Multi-Krum":       lambda D: agg_krum(D, f=6, multi=8),
    "corte de norma":   lambda D: agg_corte_norma(D, 0.5),
}
print(f"{'agregador':<16}{'(a) acc α=0,3':>15}{'(b) acc sinal 20%':>19}{'(c) acc backdoor':>18}{'(c) ASR':>9}  status")
resultados = {}
for nome, ag in agregadores.items():
    a, b, c_acc, c_asr = [], [], [], []
    for s in range(3):
        a.append(fedavg_robusto(particao_dirichlet(ytr, K, 0.3, s), Xtr, ytr, Xte, yte, agregador=ag, seed=s)[1][-1])
        partes = particao_iid(ytr, K, s)
        b.append(fedavg_robusto(partes, Xtr, ytr, Xte, yte, agregador=ag, maliciosos=range(4), ataque="sinal", seed=s)[1][-1])
        _, h, r = fedavg_robusto(partes, Xtr, ytr, Xte, yte, agregador=ag, maliciosos=range(2), ataque="backdoor", gama=1, seed=s)
        c_acc.append(h[-1]); c_asr.append(r[-1])
    a, b, c_acc, c_asr = map(np.mean, (a, b, c_acc, c_asr))
    ok = a >= 0.90 and b >= 0.90 and c_asr <= 0.10
    resultados[nome] = (a, b, ok)
    print(f"{nome:<16}{a:>15.3f}{b:>19.3f}{c_acc:>18.3f}{c_asr:>9.2f}  {'aprovado' if ok else 'reprovado'}")

aprovados = {n: (v[0] + v[1]) / 2 for n, v in resultados.items() if v[2]}
if aprovados:
    melhor = max(aprovados, key=aprovados.get)
    print(f"\nRecomendado: {melhor} (média das acurácias a e b = {aprovados[melhor]:.3f})")
else:
    print("\nNenhum agregador atendeu ao critério.")
```

SAÍDA

```
agregador         (a) acc α=0,3  (b) acc sinal 20%  (c) acc backdoor  (c) ASR  status
```

```
média                  0.959          0.481          0.953     0.77  reprovado
mediana                0.948          0.950          0.960     0.00  aprovado
média aparada          0.951          0.946          0.959     0.00  aprovado
Multi-Krum             0.905          0.952          0.949     0.00  aprovado
corte de norma         0.956          0.933          0.953     0.70  reprovado

Recomendado: mediana (média das acurácias a e b = 0.949)
```

EXEMPLO Discussão dos resultados

A média e o corte de norma são reprovados (ASR alto e/ou colapso com sinal invertido). Mediana e média aparada passam: mantêm a acurácia perto de 0,95 no cenário não-IID moderado ($\alpha = 0{,}3$), resistem ao sinal invertido e anulam o backdoor. O Multi-Krum passa nos ataques, mas perde acurácia em $\alpha = 0{,}3$ (cerca de 0,90). Entre os aprovados, a mediana ou a média aparada têm as maiores acurácias; a diferença é pequena. Limitações: ataques simples (não adaptativos), modelo convexo e participação total. Em uso real, avalie ataques adaptativos e considere a tensão com a agregação segura.

Capítulo 10 — Outros Desenhos

Banco e loja: quando o aprendizado vertical compensa?

Critérios de avaliação

- Colunas repartidas e treinos (a), (b) e (c) comparados nas mesmas condições.
- Identificação do ρ a partir do qual o VFL deixa de compensar e explicação (menos exemplos alinhados).
- Resultado do ruído no resíduo interpretado como compromisso, sem alegar privacidade formal.
- Recomendação que cite a necessidade de alinhar muitos clientes e proteger o resíduo.

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from arquiteturas import *

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
esc = StandardScaler().fit(Xtr)
Xtr, Xte = esc.transform(Xtr), esc.transform(Xte)
cA, cB = np.array_split(np.random.default_rng(1).permutation(30), 2)

print(f"{'ρ':>5}{'só A':>8}{'só B':>8}{'VFL':>8}  VFL > melhor sozinho?")
for rho in (1, 0.5, 0.2, 0.1):
    a, b, v = [], [], []
    for s in range(5):
        idx = np.random.default_rng(s).permutation(len(ytr))[:int(rho * len(ytr))]
        a.append(LogisticRegression(max_iter=2000).fit(Xtr[:, cA], ytr).score(Xte[:, cA], yte))
        b.append(LogisticRegression(max_iter=2000).fit(Xtr[:, cB], ytr).score(Xte[:, cB], yte))
        w, bias, _ = vfl_treinar([Xtr[idx][:, cA], Xtr[idx][:, cB]], ytr[idx])
        v.append((vfl_prever([Xte[:, cA], Xte[:, cB]], w, bias) == yte).mean())
    print(f"{rho:>5}{np.mean(a):>8.3f}{np.mean(b):>8.3f}{np.mean(v):>8.3f}  {'sim' if np.mean(v) >
max(np.mean(a), np.mean(b)) else 'não'}")

# resíduo com ruído (ρ = 1)
def vfl_ruido(Xp, y, sigma, epocas=300, eta=0.5, seed=0):
    rng = np.random.default_rng(seed); n = len(y); w = [np.zeros(Xk.shape[1]) for Xk in Xp]; bias = 0.0
    for _ in range(epocas):
        r = sigmoide(sum(Xk @ wk for Xk, wk in zip(Xp, w)) + bias) - y
        for k, Xk in enumerate(Xp):
            w[k] -= eta * Xk.T @ (r if k == 0 else r + rng.normal(0, sigma, len(r))) / n
        bias -= eta * r.mean()
    return w, bias

acc, rec = [], []
for s in range(5):
    w, bias = vfl_ruido([Xtr[:, cA], Xtr[:, cB]], ytr, 1.0, seed=s)
    acc.append((vfl_prever([Xte[:, cA], Xte[:, cB]], w, bias) == yte).mean())
    rec.append((((0.5 - ytr) + np.random.default_rng(s).normal(0, 1.0, len(ytr)) < 0).astype(int) ==
ytr).mean())
print(f"\nρ = 1 com ruído σ = 1: acurácia = {np.mean(acc):.3f} | rótulos recuperados pelo sinal =
{np.mean(rec):.3f}")
```

SAÍDA

```
    ρ    só A    só B     VFL  VFL > melhor sozinho?
    1   0.947   0.953   0.965  sim
  0.5   0.947   0.953   0.956  sim
  0.2   0.947   0.953   0.944  não
  0.1   0.947   0.953   0.933  não

ρ = 1 com ruído σ = 1: acurácia = 0.953 | rótulos recuperados pelo sinal = 0.689
```

EXEMPLO **Discussão dos resultados**

Com todos alinhados (ρ = 1), o VFL supera cada parte isolada (cerca de 0,965 contra 0,95). Com 50%, ainda há ganho pequeno; com 20% ou menos, o VFL fica abaixo de um participante isolado, pois treina com poucos exemplos. Ruído σ = 1 no resíduo reduz a recuperação de rótulos para cerca de 0,69 (de 1,0), com acurácia de aproximadamente 0,95. Recomendação: só vale investir no VFL se uma fração grande dos clientes puder ser alinhada, e o resíduo deve ser protegido com técnicas formais (homomórfica ou DP).

Capítulo 11 — Fronteiras e Prática

Perfil de clientes sem juntar os cadastros

Critérios de avaliação

- Padronização, PCA e K-means federados implementados com somas locais.
- Comparação com o centralizado mostrando equivalência (diferença próxima de zero e ARI = 1).
- Experimento com ruído, com interpretação do compromisso entre privacidade e qualidade.
- Recomendação que mencione a necessidade de agregação segura e DP, e os limites (ruído sem garantia formal; dados de vinho como substituto).

```python
import numpy as np
from sklearn.cluster import KMeans
from sklearn.datasets import load_wine
from sklearn.decomposition import PCA
from sklearn.metrics import adjusted_rand_score
from fronteiras import *

X, y = load_wine(return_X_y=True)
partes = particao_dirichlet(y, 5, 0.3, seed=0)

# 1) padronização federada
n = sum(len(p) for p in partes)
mu = sum(X[p].sum(0) for p in partes) / n
sigma = np.sqrt(sum((X[p] ** 2).sum(0) for p in partes) / n - mu ** 2)
Xs = (X - mu) / sigma

# 2) PCA federado (2 componentes) e projeção
mu_p, val, vet = pca_federado(partes, Xs, 2)
Z = (Xs - mu_p) @ vet

# 3) K-means federado
C0 = Z[np.random.default_rng(0).choice(len(Z), 3, replace=False)]
Cf = kmeans_federado(partes, Z, C0, iteracoes=50)
rot_f = ((Z[:, None, :] - Cf[None]) ** 2).sum(-1).argmin(1)

# 4) pipeline centralizado equivalente: mesma padronização, PCA e K-means do scikit-learn,
#    com os mesmos exemplos como centroides iniciais
Xc = (X - X.mean(0)) / X.std(0)
Zc = PCA(2).fit_transform(Xc)
indices = np.random.default_rng(0).choice(len(Z), 3, replace=False)
kmC = KMeans(3, init=Zc[indices], n_init=1, max_iter=300, tol=0, algorithm="lloyd").fit(Zc)
print("padronização: diferença máxima na média  :", float(np.abs(mu - X.mean(0)).max()))
print("PCA: |cos| entre componentes federadas e centralizadas:", np.round([abs(vet[:, i] @
PCA(2).fit(Xc).components_[i]) for i in range(2)], 6))
print("K-means: ARI federado x centralizado      :", adjusted_rand_score(rot_f, kmC.labels_))
print("K-means: ARI do federado contra as classes:", round(adjusted_rand_score(y, rot_f), 3))

# 6) privacidade: ruído nas somas e contagens agregadas
def kmeans_federado_ruidoso(partes, X, C0, sigma, iteracoes=50, seed=0):
    rng = np.random.default_rng(seed)
    C = C0.copy()
    for _ in range(iteracoes):
```

```
        S, N = np.zeros_like(C), np.zeros(len(C))
        for p in partes:
            a = ((X[p][:, None, :] - C[None]) ** 2).sum(-1).argmin(1)
            for j in range(len(C)):
                S[j] += X[p][a == j].sum(0); N[j] += (a == j).sum()
        S = S + rng.normal(0, sigma, S.shape); N = np.maximum(N + rng.normal(0, sigma, N.shape), 1)
        C = S / N[:, None]
    return C

print(f"\n{'σ':>5}{'ARI vs. sem ruído (média de 10 sementes)':>45}")
for sg in (0, 1, 5, 20):
    aris = []
    for s in range(10):
        Cr = kmeans_federado_ruidoso(partes, Z, C0, sg, seed=s)
        rr = ((Z[:, None, :] - Cr[None]) ** 2).sum(-1).argmin(1)
        aris.append(adjusted_rand_score(rot_f, rr))
    print(f"{sg:>5}{np.mean(aris):>45.3f}")
```

SAÍDA

```
padronização: diferença máxima na média  : 8.881784197001252e-15
PCA: |cos| entre componentes federadas e centralizadas: [1. 1.]
K-means: ARI federado x centralizado      : 1.0
K-means: ARI do federado contra as classes: 0.846

    σ     ARI vs. sem ruído (média de 10 sementes)
    0                                        1.000
    1                                        0.941
    5                                        0.906
   20                                        0.751
```

EXEMPLO Discussão dos resultados

O pipeline federado coincide com o centralizado (componentes principais iguais até o sinal e ARI = 1 entre os agrupamentos). O ARI contra as classes mostra que os grupos recuperam bem os perfis ocultos. Com ruído pequeno (σ = 1), o agrupamento praticamente não muda; com σ = 20, as somas ficam muito ruidosas e o ARI cai. Isto ilustra o preço da privacidade, mas esse ruído, por si só, não oferece garantia formal: seria preciso calibrá-lo à sensibilidade (corte dos dados) e contabilizar ε.

Capítulo 12 — Projeto Integrador Final

Federa Saúde: a decisão final

Critérios de avaliação

- Diagnóstico de heterogeneidade feito e interpretado.
- Seis configurações avaliadas em três sementes, com todas as métricas (global, ASR, KB, média e percentil 10 por cliente).
- Critério de aprovação aplicado corretamente e escolha justificada pelo menor tráfego.
- Relatório de uma página, claro, com riscos e limitações explícitos (ataque simples, modelo simples, dados substitutos, DP sem contabilidade completa) e sem alegar privacidade formal sem garantia.
- Extensão opcional: AUC de pertencimento e sensibilidade a α, se feita, bem interpretada.

```python
import numpy as np
from projeto import *

Xtr, Xte, ytr, yte = carregar_digits()
K, ALFA = 20, 0.3
global_p = np.bincount(ytr, minlength=10) / len(ytr)

# 1) diagnóstico (semente 0)
partes0 = particao_dirichlet(ytr, K, ALFA, 0)
tv = [0.5 * np.abs(np.bincount(ytr[p], minlength=10) / len(p) - global_p).sum() for p in partes0 if
len(p)]
print("tamanhos dos clientes:", sorted(len(p) for p in partes0))
print(f"distância de variação total média: {np.mean(tv):.2f}\n")

# 2) configurações
configs = [
    ("1 base (média)",         dict(agregador="media"),                False),
    ("2 mediana",              dict(agregador="mediana"),              False),
    ("3 mediana + 8 bits",     dict(agregador="mediana", bits=8),      False),
    ("4 mediana + 4 bits",     dict(agregador="mediana", bits=4),      False),
    ("5 DP (z=1)",             dict(dp_z=1.0),                         False),
    ("6 mediana+4b+ajuste",    dict(agregador="mediana", bits=4),      True),
]

print(f"{'configuração':<22}{'acc glob':>9}{'ASR':>7}{'KB':>8}{'cli média':>11}{'cli p10':>9}  status")
tabela = {}
for nome, kw, ajustar in configs:
    linhas = []
    for s in range(3):
        partes = particao_dirichlet(ytr, K, ALFA, s)
        treino, teste = dividir_local(partes, 0.3, s)
        W, acc, asr, bytes_ = fedavg_integrado(treino, Xtr, ytr, Xte, yte, maliciosos=(0, 1), seed=s,
**kw)
        ok = [i for i in range(K) if len(teste[i]) >= 5]
        modelos = [treino_local(W, Xtr[treino[i]], ytr[treino[i]], 3, 0.1, 32, 0.0, i) if ajustar else
W for i in ok]
        por_cliente = acuracias_locais(modelos, [teste[i] for i in ok], Xtr, ytr)
        linhas.append((acc, asr, bytes_ / 1024, por_cliente.mean(), np.percentile(por_cliente, 10)))
    m = np.mean(linhas, axis=0)
```

```
    aprovada = m[1] <= 0.10 and m[0] >= 0.90 and m[4] >= 0.84
    tabela[nome] = (m, aprovada)
    print(f"{nome:<22}{m[0]:>9.3f}{m[1]:>7.2f}{m[2]:>8.0f}{m[3]:>11.3f}{m[4]:>9.3f}  {'aprovada' if
aprovada else 'reprovada'}")

aprovadas = {n: v[0] for n, v in tabela.items() if v[1]}
melhor = min(aprovadas, key=lambda n: (round(aprovadas[n][2]), -aprovadas[n][3]))      # menor KB;
empate: maior média por cliente
print(f"\nRecomendação: {melhor} (tráfego {aprovadas[melhor][2]:.0f} KB; média por cliente
{aprovadas[melhor][3]:.3f})")
```

SAÍDA

```
tamanhos dos clientes: [37, 38, 39, 41, 43, 50, 56, 56, 61, 71, 73, 75, 77, 80, 83, 84, 87, 91, 95,
110]
distância de variação total média: 0.52

configuração           acc glob    ASR      KB  cli média  cli p10  status
1 base (média)            0.950   0.70    1523      0.939    0.878  reprovada
2 mediana                 0.939   0.00    1523      0.918    0.847  aprovada
3 mediana + 8 bits        0.938   0.00     386      0.918    0.846  aprovada
4 mediana + 4 bits        0.941   0.00     195      0.916    0.850  aprovada
5 DP (z=1)                0.927   0.54    1523      0.922    0.870  reprovada
6 mediana+4b+ajuste       0.941   0.00     195      0.940    0.886  aprovada

Recomendação: 6 mediana+4b+ajuste (tráfego 195 KB; média por cliente 0.940)
```

EXEMPLO **Discussão dos resultados**

A base (média) tem ASR alto (cerca de 0,70): o backdoor funciona e a acurácia global quase não revela o problema. A mediana elimina o backdoor (ASR ≈ 0) com custo pequeno em acurácia (cerca de 1 ponto global e 2 pontos por cliente). A quantização a 8 e 4 bits reduz o tráfego em 4 e 8 vezes sem piorar os resultados. A DP com z = 1 **não** impede o backdoor (ASR em torno de 0,5): o corte e o ruído limitam a influência de cada cliente, mas não distinguem o malicioso do honesto; e ainda custa acurácia. O ajuste fino local melhora a média e o percentil 10 por cliente. Entre as configurações aprovadas, duas empatam no tráfego mínimo (195 KB): a **mediana com 4 bits** e a mesma com **ajuste fino local**; pelo critério de desempate, a recomendada é a segunda, que eleva a média por cliente de 0,92 para 0,94 sem custo de comunicação (a clínica precisa guardar um modelo próprio). Limites: o backdoor é simples (não adaptativo), o modelo é minúsculo, os dados são substitutos, e a DP não tem contabilidade completa nem protege contra servidor malicioso.

Referências

Lista de artigos de periódicos e conferências de referência, além de livros reconhecidos, citados ao longo do texto.

ABADI, M.; CHU, A.; GOODFELLOW, I.; MCMAHAN, H. B.; MIRONOV, I.; TALWAR, K.; ZHANG, L. Deep learning with differential privacy. In: ACM SIGSAC Conference on Computer and Communications Security (CCS), 2016. p. 308-318.

ALISTARH, D.; GRUBIC, D.; LI, J.; TOMIOKA, R.; VOJNOVIC, M. QSGD: communication-efficient SGD via gradient quantization and encoding. In: Advances in Neural Information Processing Systems (NeurIPS), v. 30, 2017.

BAGDASARYAN, E.; VEIT, A.; HUA, Y.; ESTRIN, D.; SHMATIKOV, V. How to backdoor federated learning. In: International Conference on Artificial Intelligence and Statistics (AISTATS), 2020. Proceedings of Machine Learning Research, v. 108.

BARUCH, G.; BARUCH, M.; GOLDBERG, Y. A little is enough: circumventing defenses for distributed learning. In: Advances in Neural Information Processing Systems (NeurIPS), v. 32, 2019.

BERNSTEIN, J.; WANG, Y.-X.; AZIZZADENESHELI, K.; ANANDKUMAR, A. signSGD: compressed optimisation for non-convex problems. In: International Conference on Machine Learning (ICML), 2018. Proceedings of Machine Learning Research, v. 80.

BHAGOJI, A. N.; CHAKRABORTY, S.; MITTAL, P.; CALO, S. Analyzing federated learning through an adversarial lens. In: International Conference on Machine Learning (ICML), 2019. Proceedings of Machine Learning Research, v. 97.

BLANCHARD, P.; EL MHAMDI, E. M.; GUERRAOUI, R.; STAINER, J. Machine learning with adversaries: Byzantine tolerant gradient descent. In: Advances in Neural Information Processing Systems (NeurIPS), v. 30, 2017.

BOENISCH, F.; DZIEDZIC, A.; SCHUSTER, R.; SHAMSABADI, A. S.; SHUMAILOV, I.; PAPERNOT, N. When the curious abandon honesty: federated learning is not private. In: IEEE European Symposium on Security and Privacy (EuroS&P), 2023.

BONAWITZ, K.; EICHNER, H.; GRIESKAMP, W.; HUBA, D.; INGERMAN, A.; IVANOV, V.; KIDDON, C.; KONEČNÝ, J.; MAZZOCCHI, S.; MCMAHAN, H. B.; VAN OVERVELDT, T.; PETROU, D.; RAMAGE, D.; ROSELANDER, J. Towards federated learning at scale: system design. In: Conference on Machine Learning and Systems (MLSys), 2019. Proceedings of Machine Learning and Systems, v. 1.

BONAWITZ, K.; IVANOV, V.; KREUTER, B.; MARCEDONE, A.; MCMAHAN, H. B.; PATEL, S.; RAMAGE, D.; SEGAL, A.; SETH, K. Practical secure aggregation for privacy-preserving machine learning. In: ACM SIGSAC Conference on Computer and Communications Security (CCS), 2017. p. 1175-1191.

BOYD, S.; GHOSH, A.; PRABHAKAR, B.; SHAH, D. Randomized gossip algorithms. IEEE Transactions on Information Theory, v. 52, n. 6, p. 2508-2530, 2006.

CARLINI, N.; CHIEN, S.; NASR, M.; SONG, S.; TERZIS, A.; TRAMÈR, F. Membership inference attacks from first principles. In: IEEE Symposium on Security and Privacy (S&P), 2022.

CARLINI, N.; TRAMÈR, F.; WALLACE, E.; JAGIELSKI, M.; HERBERT-VOSS, A.; LEE, K.; ROBERTS, A.; BROWN, T.; SONG, D.; ERLINGSSON, Ú.; OPREA, A.; RAFFEL, C. Extracting training data from large language models. In: USENIX Security Symposium, 2021.

COLLINS, L.; HASSANI, H.; MOKHTARI, A.; SHAKKOTTAI, S. Exploiting shared representations for personalized federated learning. In: International Conference on Machine Learning (ICML), 2021. Proceedings of Machine Learning Research, v. 139.

DAYAN, I. et al. Federated learning for predicting clinical outcomes in patients with COVID-19. Nature Medicine, v. 27, p. 1735-1743, 2021.

DUA, D.; GRAFF, C. UCI Machine Learning Repository. Irvine: University of California, School of Information and Computer Science, 2019. Disponível em: https://archive.ics.uci.edu.

DWORK, C.; MCSHERRY, F.; NISSIM, K.; SMITH, A. Calibrating noise to sensitivity in private data analysis. In: Theory of Cryptography Conference (TCC), 2006. Lecture Notes in Computer Science, v. 3876, p. 265-284.

DWORK, C.; ROTH, A. The algorithmic foundations of differential privacy. Foundations and Trends in Theoretical Computer Science, v. 9, n. 3-4, p. 211-407, 2014.

EL MHAMDI, E. M.; GUERRAOUI, R.; ROUAULT, S. The hidden vulnerability of distributed learning in Byzantium. In: International Conference on Machine Learning (ICML), 2018. Proceedings of Machine Learning Research, v. 80.

FALLAH, A.; MOKHTARI, A.; OZDAGLAR, A. Personalized federated learning with theoretical guarantees: a model-agnostic meta-learning approach. In: Advances in Neural Information Processing Systems (NeurIPS), v. 33, 2020.

FANG, M.; CAO, X.; JIA, J.; GONG, N. Z. Local model poisoning attacks to Byzantine-robust federated learning. In: USENIX Security Symposium, 2020.

FOWL, L.; GEIPING, J.; CZAJA, W.; GOLDBLUM, M.; GOLDSTEIN, T. Robbing the Fed: directly obtaining private data in federated learning with modified models. In: International Conference on Learning Representations (ICLR), 2022.

GEIPING, J.; BAUERMEISTER, H.; DRÖGE, H.; MOELLER, M. Inverting gradients: how easy is it to break privacy in federated learning? In: Advances in Neural Information Processing Systems (NeurIPS), v. 33, 2020.

GENTRY, C. Fully homomorphic encryption using ideal lattices. In: ACM Symposium on Theory of Computing (STOC), 2009. p. 169-178.

GHOSH, A.; CHUNG, J.; YIN, D.; RAMCHANDRAN, K. An efficient framework for clustered federated learning. In: Advances in Neural Information Processing Systems (NeurIPS), v. 33, 2020.

GUPTA, O.; RASKAR, R. Distributed learning of deep neural network over multiple agents. Journal of Network and Computer Applications, v. 116, p. 1-8, 2018.

HADDADPOUR, F.; KAMANI, M. M.; MOKHTARI, A.; MAHDAVI, M. Federated learning with compression: unified analysis and sharp guarantees. In: International Conference on Artificial Intelligence and Statistics (AISTATS), 2021. Proceedings of Machine Learning Research, v. 130.

HASTIE, T.; TIBSHIRANI, R.; FRIEDMAN, J. The Elements of Statistical Learning: Data Mining, Inference, and Prediction. 2. ed. New York: Springer, 2009.

HOULSBY, N.; GIURGIU, A.; JASTRZEBSKI, S.; MORRONE, B.; DE LAROUSSILHE, Q.; GESMUNDO, A.; ATTARIYAN, M.; GELLY, S. Parameter-efficient transfer learning for NLP. In: International Conference on Machine Learning (ICML), 2019. Proceedings of Machine Learning Research, v. 97.

HU, E. J.; SHEN, Y.; WALLIS, P.; ALLEN-ZHU, Z.; LI, Y.; WANG, S.; WANG, L.; CHEN, W. LoRA: low-rank adaptation of large language models. In: International Conference on Learning Representations (ICLR), 2022.

HUANG, Y.; GUPTA, S.; SONG, Z.; LI, K.; ARORA, S. Evaluating gradient inversion attacks and defenses in federated learning. In: Advances in Neural Information Processing Systems (NeurIPS), v. 34, 2021.

KAIROUZ, P. et al. Advances and open problems in federated learning. Foundations and Trends in Machine Learning, v. 14, n. 1-2, p. 1-210, 2021.

KAIROUZ, P.; LIU, Z.; STEINKE, T. The distributed discrete Gaussian mechanism for federated learning with secure aggregation. In: International Conference on Machine Learning (ICML), 2021. Proceedings of Machine Learning Research, v. 139.

KARIMIREDDY, S. P.; KALE, S.; MOHRI, M.; REDDI, S. J.; STICH, S. U.; SURESH, A. T. SCAFFOLD: stochastic controlled averaging for federated learning. In: International Conference on Machine Learning (ICML), 2020. Proceedings of Machine Learning Research, v. 119.

KARIMIREDDY, S. P.; REBJOCK, Q.; STICH, S. U.; JAGGI, M. Error feedback fixes SignSGD and other gradient compression schemes. In: International Conference on Machine Learning (ICML), 2019. Proceedings of Machine Learning Research, v. 97.

KOLOSKOVA, A.; LOIZOU, N.; BOREIRI, S.; JAGGI, M.; STICH, S. U. A unified theory of decentralized SGD with changing topology and local updates. In: International Conference on Machine Learning (ICML), 2020. Proceedings of Machine Learning Research, v. 119.

KUANG, W. et al. FederatedScope-LLM: a comprehensive package for fine-tuning large language models in federated learning. In: ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), 2024.

LAMPORT, L.; SHOSTAK, R.; PEASE, M. The Byzantine generals problem. ACM Transactions on Programming Languages and Systems, v. 4, n. 3, p. 382-401, 1982.

LI, Q.; DIAO, Y.; CHEN, Q.; HE, B. Federated learning on non-IID data silos: an experimental study. In: IEEE International Conference on Data Engineering (ICDE), 2022. p. 965-978.

LI, T.; HU, S.; BEIRAMI, A.; SMITH, V. Ditto: fair and robust federated learning through personalization. In: International Conference on Machine Learning (ICML), 2021. Proceedings of Machine Learning Research, v. 139.

LI, T.; SAHU, A. K.; TALWALKAR, A.; SMITH, V. Federated learning: challenges, methods, and future directions. IEEE Signal Processing Magazine, v. 37, n. 3, p. 50-60, 2020b.

LI, T.; SAHU, A. K.; ZAHEER, M.; SANJABI, M.; TALWALKAR, A.; SMITH, V. Federated optimization in heterogeneous networks. In: Conference on Machine Learning and Systems (MLSys), 2020c. Proceedings of Machine Learning and Systems, v. 2.

LI, T.; SANJABI, M.; BEIRAMI, A.; SMITH, V. Fair resource allocation in federated learning. In: International Conference on Learning Representations (ICLR), 2020a.

LIAN, X.; ZHANG, C.; ZHANG, H.; HSIEH, C.-J.; ZHANG, W.; LIU, J. Can decentralized algorithms outperform centralized algorithms? A case study for decentralized parallel stochastic gradient descent. In: Advances in Neural Information Processing Systems (NeurIPS), v. 30, 2017.

LIN, Y.; HAN, S.; MAO, H.; WANG, Y.; DALLY, W. J. Deep gradient compression: reducing the communication bandwidth for distributed training. In: International Conference on Learning Representations (ICLR), 2018.

LIU, Y.; KANG, Y.; XING, C.; CHEN, T.; YANG, Q. A secure federated transfer learning framework. IEEE Intelligent Systems, v. 35, n. 4, p. 70-82, 2020.

MCMAHAN, H. B.; MOORE, E.; RAMAGE, D.; HAMPSON, S.; AGÜERA Y ARCAS, B. Communication-efficient learning of deep networks from decentralized data. In: International Conference on Artificial Intelligence and Statistics (AISTATS), 2017. Proceedings of Machine Learning Research, v. 54, p. 1273-1282.

MCMAHAN, H. B.; RAMAGE, D.; TALWAR, K.; ZHANG, L. Learning differentially private recurrent language models. In: International Conference on Learning Representations (ICLR), 2018.

MELIS, L.; SONG, C.; DE CRISTOFARO, E.; SHMATIKOV, V. Exploiting unintended feature leakage in collaborative learning. In: IEEE Symposium on Security and Privacy (S&P), 2019.

MIRONOV, I. Rényi differential privacy. In: IEEE Computer Security Foundations Symposium (CSF), 2017. p. 263-275.

MOHRI, M.; SIVEK, G.; SURESH, A. T. Agnostic federated learning. In: International Conference on Machine Learning (ICML), 2019. Proceedings of Machine Learning Research, v. 97.

NASR, M.; SHOKRI, R.; HOUMANSADR, A. Comprehensive privacy analysis of deep learning: passive and active white-box inference attacks against centralized and federated learning. In: IEEE Symposium on Security and Privacy (S&P), 2019.

NGUYEN, D. C. et al. Federated learning for Internet of Things: a comprehensive survey. IEEE Communications Surveys & Tutorials, v. 23, n. 3, p. 1622-1658, 2021.

PAILLIER, P. Public-key cryptosystems based on composite degree residuosity classes. In: International Conference on the Theory and Application of Cryptographic Techniques (EUROCRYPT), 1999. Lecture Notes in Computer Science, v. 1592, p. 223-238.

PEDREGOSA, F. et al. Scikit-learn: machine learning in Python. Journal of Machine Learning Research, v. 12, p. 2825-2830, 2011.

PILLUTLA, K.; KAKADE, S. M.; HARCHAOUI, Z. Robust aggregation for federated learning. IEEE Transactions on Signal Processing, v. 70, p. 1142-1154, 2022.

REDDI, S. J. et al. Adaptive federated optimization. In: International Conference on Learning Representations (ICLR), 2021.

REISIZADEH, A.; MOKHTARI, A.; HASSANI, H.; JADBABAIE, A.; PEDARSANI, R. FedPAQ: a communication-efficient federated learning method with periodic averaging and quantization. In: International Conference on Artificial Intelligence and Statistics (AISTATS), 2020. Proceedings of Machine Learning Research, v. 108.

RIEKE, N. et al. The future of digital health with federated learning. npj Digital Medicine, v. 3, art. 119, 2020.

SATTLER, F.; MÜLLER, K.-R.; SAMEK, W. Clustered federated learning: model-agnostic distributed multitask optimization under privacy constraints. IEEE Transactions on Neural Networks and Learning Systems, v. 32, n. 8, p. 3710-3722, 2021.

SATTLER, F.; WIEDEMANN, S.; MÜLLER, K.-R.; SAMEK, W. Robust and communication-efficient federated learning from non-IID data. IEEE Transactions on Neural Networks and Learning Systems, v. 31, n. 9, p. 3400-3413, 2020.

SHAMIR, A. How to share a secret. Communications of the ACM, v. 22, n. 11, p. 612-613, 1979.

SHELLER, M. J. et al. Federated learning in medicine: facilitating multi-institutional collaborations without sharing patient data. Scientific Reports, v. 10, art. 12598, 2020.

SHOKRI, R.; STRONATI, M.; SONG, C.; SHMATIKOV, V. Membership inference attacks against machine learning models. In: IEEE Symposium on Security and Privacy (S&P), 2017. p. 3-18.

SMITH, V.; CHIANG, C.-K.; SANJABI, M.; TALWALKAR, A. Federated multi-task learning. In: Advances in Neural Information Processing Systems (NeurIPS), v. 30, 2017.

STICH, S. U.; CORDONNIER, J.-B.; JAGGI, M. Sparsified SGD with memory. In: Advances in Neural Information Processing Systems (NeurIPS), v. 31, 2018.

T DINH, C.; TRAN, N. H.; NGUYEN, T. D. Personalized federated learning with Moreau envelopes. In: Advances in Neural Information Processing Systems (NeurIPS), v. 33, 2020.

TAN, A. Z.; YU, H.; CUI, L.; YANG, Q. Towards personalized federated learning. IEEE Transactions on Neural Networks and Learning Systems, 2022. Publicação antecipada (early access).

WANG, H.; SREENIVASAN, K.; RAJPUT, S.; VISHWAKARMA, H.; AGARWAL, S.; SOHN, J.-Y.; LEE, K.; PAPAILIOPOULOS, D. Attack of the tails: yes, you really can backdoor federated learning. In: Advances in Neural Information Processing Systems (NeurIPS), v. 33, 2020a.

WANG, J.; LIU, Q.; LIANG, H.; JOSHI, G.; POOR, H. V. Tackling the objective inconsistency problem in heterogeneous federated optimization. In: Advances in Neural Information Processing Systems (NeurIPS), v. 33, 2020b.

WARNER, S. L. Randomized response: a survey technique for eliminating evasive answer bias. Journal of the American Statistical Association, v. 60, n. 309, p. 63-69, 1965.

XU, J.; GLICKSBERG, B. S.; SU, C.; WALKER, P.; BIAN, J.; WANG, F. Federated learning for healthcare informatics. Journal of Healthcare Informatics Research, v. 5, p. 1-19, 2021.

YANG, Q.; LIU, Y.; CHEN, T.; TONG, Y. Federated machine learning: concept and applications. ACM Transactions on Intelligent Systems and Technology, v. 10, n. 2, art. 12, 2019.

YE, R. et al. OpenFedLLM: training large language models on decentralized private data via federated learning. In: ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), 2024.

YEOM, S.; GIACOMELLI, I.; FREDRIKSON, M.; JHA, S. Privacy risk in machine learning: analyzing the connection to overfitting. In: IEEE Computer Security Foundations Symposium (CSF), 2018. p. 268-282.

YIN, D.; CHEN, Y.; RAMCHANDRAN, K.; BARTLETT, P. Byzantine-robust distributed learning: towards optimal statistical rates. In: International Conference on Machine Learning (ICML), 2018. Proceedings of Machine Learning Research, v. 80.

ZHU, L.; LIU, Z.; HAN, S. Deep leakage from gradients. In: Advances in Neural Information Processing Systems (NeurIPS), v. 32, 2019.